
HTML Development Framework



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: 24 de julio de 2003.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

Introducción

■ Acerca de este manual	1
Propósito	1
Audiencia	1
Requisitos	2
Estructura del contenido	2

I Información básica para el desarrollo en cliente Web

1 Metodología

■ Acerca de este capítulo	7
■ Estándares Web	8
Uso del código Java	8
Uso de JavaScript	8
Uso de XHTML	11
Uso de la hoja de estilos o CSS	13
■ Estructura de los productos y las páginas	15
Componentes básicos	15
Estructura de los productos	20
■ Configuración de la aplicación para un producto Web	28
Configuración de un portal para un producto Web	28
Configuración de la página de inicio de sesión	33
Configuración página de desconexión	35
Configuración página de error de inicio de sesión	35
■ Uso de Servicios Web desde JSP	36
Registros y excepciones	36
Tarea inicial	37
Acceso al servicio de datos	37
Presentación	40
■ Meta4Objects de productos	41
Meta4Objects comunes	41
Reutilización de Meta4Objects existentes	41

Prestaciones de los Meta4Objects típicos utilizados en productos Web	41
■ Presentaciones y acceso a objetos del cliente	43
Separación de la lógica empresarial y el cliente	43
Reglas para las librerías JavaScript.	44
Plantillas de presentación	44
Diseño y navegación	46
■ Desarrollo y ciclo de vida	47
Gestión de la configuración	47
Herramientas de desarrollo	48
■ Formatos de fechas, de números y de monedas	49
Personalización de formatos	49
Fechas	50
Números y monedas	52
■ Validaciones de código JavaScript.	54
Implementación de traducciones de código.	54

2 Plantillas

■ Acerca de este capítulo	59
■ Plantilla de mantenimiento	60
Tablas de mantenimiento	60
Listas	61
Listas multiselección	62
Filtros tipo lista	62
Meta4Objects	63
Páginas JSP	68
■ Plantilla para asistentes: <i>wizard</i>	70
Meta4Objects	71
Páginas JSP	75
■ Plantilla del asistente de procesos.	76
Creación de un nuevo Meta4Object del asistente de procesos.	76
Integración del Planificador de tareas	83
Creación de páginas JSP	86
■ Plantilla de información	87
Meta4Objects	87
Páginas JSP	92
■ Plantilla para tablas de valores	95
Valores fijos	96
Valores incidencias	98
Incidencias masivas.	99
Selección de componentes de cálculo.	100

Valores por defecto	101
Edición por clave primaria	102
Históricos a fecha de pago y fecha de imputación.	102
Tramos.	103
Cambios respecto a Windows	105
Meta4Objects.	105
Páginas JSP	108
■ Plantilla de filtro sobre datos cargados	116
Meta4Objects.	116
Páginas JSP	119
■ Gestión del ciclo de vida	120
Introducción	120
Modificación de funcionalidad	120
Implantación del ciclo de vida	121
■ Casística de las plantillas	124
Modificar el comportamiento del botón limpiar en los wizard.	124

3 Generación automática de páginas del cliente Web

■ Introducción	127
■ Visión general de la generación automática de páginas	128
Requisitos previos	128
Resumen de la generación automática	130
■ Uso de la opción Mantenimientos y Listas	130
Seleccionar un Meta4Object nuevo o existente.	130
Introducir datos del Meta4Object.	131
Introducir la información de la página JSP	132
Seleccionar campos para el filtro de la página JSP de tipo lista	133
Seleccionar campos para la tabla de la página JSP de tipo lista.	134
Seleccionar campos de retorno para la página JSP de tipo lista.	135
Definir etiquetas de texto para la página JSP de tipo mantenimiento	135
Seleccionar campos para la tabla de mantenimiento	136
Definir los campos de edición de la página JSP de mantenimiento	137
Finalización del proceso	141
■ Uso de la opción Asistentes	144
Seleccionar un Meta4Object nuevo o existente.	145
Introducir datos del Meta4Object.	145
Introducir datos del proceso guiado.	147
Seleccionar campos para filtrar el asistente	148
Definir el filtro tipo lista.	149
Definir etiquetas de texto para la página JSP	150

Definir los pasos del asistente	150
Definir un paso guiado del asistente	151
Definir el filtro del paso guiado del asistente	153
Seleccionar los campos para la tabla del asistente	153
Definir los campos de edición de la página JSP del paso	153
Finalización del proceso	153

4 Hojas de estilos: CSS

■ Acerca de este capítulo	155
■ Hojas de estilos del producto estándar	156
■ Gestión de hojas de estilos	156
Hoja de estilos general estándar	156
Creación y configuración de una nueva hoja de estilos general	157
Creación y configuración de la hoja de estilos de conexión	158
■ Cambio de hoja de estilo del usuario	160

5 Parámetros del webservices

■ Acerca de este capítulo	161
■ Acceso a parámetros desde páginas jsp	162
■ Acceso a parámetros desde Meta4Objects	163
■ Parámetros del webservices	164
Configuración del cliente Web	164
Opciones generales	165

6 Gestión de menús

■ Acerca de este capítulo	167
■ Funcionamiento de los menús	167
■ Configuración de las opciones de menú	168
Clasificación de los menús	168
Gestión de niveles	169
Gestión de menús dependientes de los menús de nivel	171
Opciones de menú no modificables	173
■ Seguridad	175
Seguridad por <i>outsourcing</i>	175
Seguridad por tarea	175
■ Generación dinámica de las páginas de menús de cada módulo	176
Definición del código de la página jsp	177

Edición del archivo shco_portal_idioma.properties	177
Imágenes	179

7 Configuración Funcional

■ Introducción	181
■ Agregar un proceso a la Lista de tareas	181

8 Página de moneda

■ Acerca de este capítulo	183
■ Funcionalidad de la página de moneda	183
■ Forma de invocación a la página	184
Obtención del listado de monedas y tipos de cambio	185

9 Temporalidad: control de fechas

■ Acerca de este capítulo	187
Listas y filtros tipo lista	188
En los asistentes	189
En las consultas de información	189

10 Gestión de caracteres especiales (HTML, JAVASCRIPT)

■ Acerca de este capítulo	191
■ Atributos de formateo	191
Uso de atributo htmlspecialchars	192
Uso de atributo javascript	192
Casos especiales	194
Funciones equivalentes en java	194
■ Ejemplos de uso	195
Código javascript puro	195
Código javascript embebido en HTML	195
Código HTML sin javascript	196
Input recogiendo el valor de variable java	196

11 Páginas JSP multiidioma

■ Acerca de este capítulo	197
---------------------------------	-----

■ Crear archivos de literales traducidos	198
■ Crear la página de carga de las traducciones	199
■ Organización de páginas multidioma	200
Métodos de traducción de literales	200
Organización de páginas multidioma	200

12 Gestión de errores

■ Acerca de este capítulo	201
■ Gestión de errores en servidor	201
Proceso	202
■ Gestión de errores en cliente	203
Política de mantenimiento de los archivos de error	203
Organización de los archivos de error	204

13 Filtros en ejecución

■ Acerca de este capítulo	209
■ Introducción	209
■ Cómo llamar al filtro dinámico	210
Función de invocación al filtro: <i>m4dynfilter()</i>	210
■ Cómo usar el filtro dinámico	213
■ Ejemplos de uso	214
<i>sretmode</i> = 1	214
<i>sretmode</i> = 2	214
<i>sretmode</i> = 3	215
<i>sretmode</i> = 4	215

14 Ejecución y planificación de informes

■ Acerca de este capítulo	217
■ Introducción	218
■ Cómo ejecutar o planificar la ejecución de un informe	219
Transferencia de parámetros a través de un formulario	219
Transferencia de parámetros a través de un Meta4Object	220
■ Cómo configurar la ejecución de un informe	223
Ejecución de un informe	223
Planificación de un informe	225
Resultados de la ejecución del informe	226
Ejecución de informes con grupos	227

■ Configuración del sistema	229
Parámetros	229
Reglas de validación de parámetros	238
Restricciones	240

15 Compilación e instanciación de páginas JSP

■ Acerca de este capítulo	241
Requisitos	241
■ Proceso general de compilación e instanciación	242
■ Compilación e instanciación en JRun 3.1	243
Definir y compilar las páginas JSP	243
Instanciar las clases de las páginas JSP	247
■ Compilación e instanciación en JRun 4.0	251
Definir y compilar las páginas JSP	251
Instanciar las clases de las páginas JSP	254

II Desarrollo avanzado en cliente WEB

16 Gestión del límite de código compilado JAVA

■ Acerca de este capítulo	261
■ Comprobar el tamaño del código	262
■ Reducción del tamaño del código JAVA	262
Uso de <jsp:include>	263
Tags Outputdef y Move	268
Etiquetas genéricas	269
Etiquetas y valores de items propios de la página.	270
Reducción de Tags	272

17 Depuración de páginas JSP

■ Acerca de este capítulo	275
■ Depuración de una página JSP	276
Comprobar errores de sintaxis	276
Revisar archivos de traza del servidor Web	277
Revisar archivos de trazas del servidor de aplicaciones	283

Revisar el Meta4Object en el cliente Windows	284
Añadir más información de depuración desde las páginas JSP	285
■ Configuración del sistema de trazas de los servicios Web	286
■ Errores más frecuentes	287

18 Operaciones automáticas sobre páginas JSP: Robot M4robotjsp2k2

■ Acerca de este capítulo	289
■ Procesos implementados en el robot.	290
Inyección de dobles comillas en los atributos de los tags de meta4	290
Gestión de caracteres especiales para html y javascript.	290
Actualización de tópicos de ayuda.	292
Modificar imágenes para la gestión multiproducto.	293
Establecer estándares en el tag <m4:input> y en expresiones JAVA	293
Incluir validaciones de datos en las cajas de texto	294
■ Cómo instalar el robot	298
■ Uso del robot	299
■ Interpretación de los resultados al ejecutar el robot sobre páginas JSP	302
Paso de procesos en modo testeo	302
Paso de procesos guardando cambios	303

19 Ayuda en línea

■ Acerca de este capítulo	305
■ Descripción funcional	306
Acceso a las ayudas en línea	306
Parametrización de las ayudas en línea	307
■ Ayudas para nuevas páginas funcionales jsp	308
■ Ayudas en las páginas de menú genéricas	310
■ Tablas de valor	312
■ Páginas de ayuda en el servidor.	313

20 Configuración de la conexión unificada

■ Acerca de este capítulo	315
■ Proceso para utilizar la conexión unificada.	316
Parámetros de aplicación	316
Opciones de menú con acceso directo a Web.	316
Opciones de menú con acceso combinado Windows/Web.	317
Diseño de una página JSP con enlaces entre sitios	319

21 Desarrollo de componentes que difieren del diseño estándar del development framework

■ Acerca de este capítulo	321
■ Desarrollo de componentes modificables	322
Identificación del producto	322
Elementos relacionados con el aspecto	323
Disclaimer	327
Ayuda en línea	328
Asistentes: estándar y de procesos	329
■ Uso más frecuente	331

III Apéndices

A Apéndice: Librerías API Javascript

■ Acerca de este capítulo	335
■ Manipulación de objetos Javascript	337
Funciones genéricas	337
Objeto <INPUT Type=RADIO> (m4gen.js)	347
Objeto <SELECT> (m4gen.js)	348
■ Manipulación de datos	351
Cadenas (m4val.js)	351
Números (m4gen.js)	352
Fechas/Horas (m4gen.js)	357
■ Validaciones	364
Funciones de validación (m4val.js)	364
Personalizar el mensaje de error (m4_err_lang.js)	371
Ejemplos de utilización	371
■ Trabajando con monedas (m4gen.js)	373
m4currency	373
m4currencysetdefaultvalues	374
■ Gestión de calendarios (m4gen.js)	376
m4calendar	376
■ Gestión de ventanas (m4gen.js)	378
m4window	378
m4returnvalues	380
m4window_extension	380

m4window_extension_return	381
m4windowcallback	382
m4navegar	382
■ Trabajando con plantillas de mantenimiento (m4gen_mt.js)	384
m4eliminar	385
m4filtro	385
m4filtrocallback	386
m4filtro_wargs	387
m4remonte	388
m4ordenar	390
m4setPKA	390
m4setm4tit	390
m4lim	391
m4com2	391
m4edit	391
m4cambiofondo	391
■ Trabajando con plantillas de listado (m4gen_list.js)	392
m4buscar	392
m4filtrar()	393
m4filtro_wargs	393
m4ordenar	394
Valores	394
■ Trabajando con Plantillas de listado con CheckBox (m4gen_list_ck.js)	395
m4acceptall	395
m4cancelall	396
m4retorno	397
■ Trabajando con Plantillas de asistente (m4gen_wz.js)	398
m4cambioestado	398
m4comwz	398
m4editwz	398
m4limwz	399
m4navwz	400
m4setm4titwz	400
m4wzeliminar	401
m4wzins	401
■ Trabajando con Plantillas de asistente de procesos (m4gen_param_wz.js)	403
m4wzins	403
m4wzsave	404
m4wzins_term	405
m4wzins_dyn	405
m4after_filter	406
m4generate_merge_word	407

■ Trabajando con dual-list (m4duallist.js)	407
m4funoptionselected	409
m4moveoption	409
m4updownoption	411
■ Gestión de errores y excepciones	412
Excepciones (m4gen_excep.js)	412
Excepciones (Ejemplo de uso)	416
Funciones genéricas	416

B Apéndice: Archivos no modificables

■ Acerca de este capítulo	421
■ Archivos	422
files_gif	423
Images	427
library	429
shco_g0	431
shco_py	434
shco_py_sse	436
style	436
translation	437
■ Repositorio	439
META4OBJECT	439
BUSINESS PROCESS	440
LOGICAL TABLE	441
APP_ROLE	441
MENU OPTION	441
PRESENTATION	442
APP PARAM	442

C Apéndice: Recursos adicionales

■ Acerca de este capítulo	443
■ Referencias disponibles en Internet	444

D Apéndice: Código de páginas JSP

■ Acerca de este apéndice	447
■ Tipos de páginas JSP	447
Páginas de mantenimiento	448

Páginas de listas y de filtros tipo lista	455
Página lista multiselección.....	461
Plantillas para asistentes: wizard.....	472
Mantenimiento de N registros	480
Plantilla del asistente de procesos.....	487

Introducción

Acerca de este manual

Propósito

El manual está pensado como un conjunto de capítulos que sirven de guía para dar solución al desarrollo en cliente Web de forma puntual, mientras se está desarrollando con el producto a la vez. Está basado en la tecnología empleada para el desarrollo de funcionalidades sobre el producto estándar en el cliente Web. Su contenido es muy práctico, por tanto, no se recomienda leer el manual de forma continua, sino apoyarse en él para resolver problemas concretos.

Se recomienda su uso por varios motivos:

- Todos los desarrollos seguirán el mismo esquema que el producto estándar de Meta4.
- Se facilita así las actualizaciones.
- Se reduce el tiempo y coste de implementación de desarrollos de nuevas funcionalidades en el cliente Web.

Audiencia

Este manual está dirigido a personas involucradas en la implantación y desarrollo de funcionalidades en el cliente Web de aplicaciones basadas en Meta4 PeopleNet Ksystem.

Requisitos

Para un correcto uso y entendimiento del contenido del manual es necesario tener conocimientos de algunos conceptos tecnológicos: arquitectura Meta4Objects, herencia, desarrollo funcional en cliente Web (Meta4tags). Para ello, puede consultar los manuales correspondientes. También es necesario conocimientos de HTML y javascript.

Además, existen una serie de manuales de configuración y de desarrollo que le pueden ayudar a profundizar en aspectos concretos, se recomienda su lectura:

- Desarrollo de funcionalidad para el cliente HTML
- Administración del Servidor de Aplicaciones

Para el correcto uso y aprovechamiento de este manual se recomienda utilizar la versión 4.0 de Adobe Acrobat Reader o superior para una correcta visualización del contenido.

Estructura del contenido

A pesar de que el manual sea práctico, la recomendación es antes de comenzar un proyecto de cliente Web:

- leer el capítulo de metodología en primer lugar
- consultar el capítulo de plantillas en las que se deben basar todos los desarrollos y por último
- repasar el contenido del manual para saber donde ir a buscar cuando surjan problemas específicos.

Se divide el contenido en dos partes atendiendo a si el capítulo es imprescindible para todos los desarrollos (Básico) y aquéllos que sólo se usarán en funcionalidades más específicas (Avanzado).

- Básico
 - Metodología de desarrollo en cliente Web
 - Plantillas que se han creado para diferentes funciones. Así, existe una plantilla que se usará cuando el desarrollador tenga que diseñar formularios para introducir datos, otra para realizar el mantenimiento de esos datos, otra para generar filtros, etcétera
 - Herramientas de cliente Web
 - Hoja de estilo CSS
 - Parámetros de desarrollo
 - Gestión de menús

-
- Filtros en ejecución
 - Ejecución y planificación de informes
 - Configuración funcional
 - Multimoneda
 - Temporalidad
 - Caracteres especiales
 - Multilenguaje
 - Gestión de errores
 - Compilador de páginas JSP e instanciación de clases
 - Avanzado
 - Limites Java
 - Depuración aplicación: errores más frecuentes
 - Online help
 - Conexión unificada
 - Apéndices
 - Librerías Javascript
 - Páginas no modificables

PARTE

I

INFORMACIÓN BÁSICA PARA EL DESARROLLO EN CLIENTE WEB

En esta primera parte del manual se explica la metodología básica de desarrollo en cliente Web y aquellos elementos que permiten al desarrollador comenzar a crear nuevas páginas o a parametrizar otras existentes en el producto estándar.

Como ya se ha indicado en la introducción, se recomienda comenzar leyendo el capítulo de metodología para adquirir los conceptos básicos. Después, revisar el capítulo de plantillas para ver que funcionalidades se tratan y que facilitan el trabajo de desarrollo. El resto de capítulos se consultarán para resolver y aclarar dudas puntuales y concretas.

Metodología

1

Acerca de este capítulo

En este capítulo se definen las reglas generales y las prácticas de desarrollo recomendables para crear y personalizar los productos Web de Meta4. Esta información permitirá a los desarrolladores de software tomar las decisiones adecuadas al diseñar productos para los clientes HTML.

En este capítulo se tratan los siguientes temas:

En la Parte I se tratan cuestiones generales y técnicas como:

- Qué estándares Web deben seguir los productos.
- Qué estructura de directorios deberían tener las páginas de productos en el servidor Web y qué componentes básicos no son modificables.
- Cómo configurar el portal y la página de *login* de la aplicación.
- Qué debería tenerse en cuenta al usar la tecnología de los servicios Web.

En la Parte II se tratan cuestiones de desarrollo funcional como:

- Cómo deben estructurarse los Meta4Objects.
- Cómo deben estructurarse las presentaciones.
- Qué herramientas debería utilizar y qué reglas seguir.

En la Parte III se tratan las siguientes cuestiones:

- Qué formato deben tener las fechas y los números.
- Cómo validar el código JavaScript.

Estándares Web

En este apartado se indican los principales elementos Web estándar que se utilizan en el desarrollo del cliente Web y cómo gestionarlos.

- Java
- JavaScript
- XHTML
- CSS

Uso del código Java

No debería utilizarse código Java en las páginas JSP a menos que sea necesario. El código en los scripts puede dificultar la localización de los errores de sintaxis, y si hay demasiado código en las páginas, puede ser difícil mantener la aplicación Web. Si realmente necesita utilizar código Java, debería documentarlo cuidadosamente, incluyendo comentarios sobre las páginas.

Uso de JavaScript

Objetos y el Modelo de objetos del documento

En JavaScript puede crear sus propios objetos para almacenar datos. No obstante, lo más frecuente es que utilice los numerosos objetos incorporados que le permitirán trabajar, manipular y acceder a la página Web y al navegador Web. Este conjunto de objetos pre-existentes se conoce como el Modelo de objetos del documento (DOM), una jerarquía de todos los objetos incorporados en JavaScript.

La mayoría de estos objetos están relacionados directamente con las características de la página o el navegador Web. El motivo por el cual utilizamos la palabra “incorporado” es porque el DOM no forma parte técnicamente del propio JavaScript. Es decir, la especificación del lenguaje JavaScript, estandarizado por la ECMA, no especifica realmente la naturaleza ni las características específicas del DOM. En consecuencia, Netscape y Microsoft han desarrollado sus propios DOM individuales, que no son totalmente compatibles. Además, el DOM se distingue de JavaScript porque en teoría también otros lenguajes de script podrían acceder a él. En resumen, lo que denominamos “JavaScript” se compone en realidad de JavaScript, el lenguaje y el DOM o modelo de objetos al que puede acceder JavaScript.

Los productos de Meta4 utilizan DOM2: <http://www.w3.org/TR/DOM-Level-2-Core>.

Librerías: metodología desarrollo de código javascript en páginas JSP

Normalmente no debería escribir sus propias funciones JavaScript en páginas JSP. Esto significa que debería intentar utilizar las funciones definidas en las librerías. Las librerías están centralizadas y controladas. Puede escribir algunas funciones en las páginas, pero si desarrolla una función JavaScript, es posible que también pueda utilizarla en otro sitio.



No se recomienda utilizar directamente el DOM JavaScript en las páginas JSP.

Cuando sea necesario utilizar funciones de javascript propias en las páginas JSP para gestionar correctamente acciones de usuario, es necesario seguir estrictamente la metodología que se explica a continuación.

El objetivo es evitar un impacto importante en el rendimiento de la ejecución de las páginas que requieran de estas funciones javascript extra. Por este motivo hay que evitar embeber las funciones javascript en las propias páginas JSP porque incrementa el tamaño del código HTML que viaja entre el cliente y el servidor web y por consiguiente el rendimiento de la página es más bajo.

El objetivo se consigue ubicando las funciones javascript en archivos independiente con extensión .js, dado que este tipo de archivos el navegador los cachea automáticamente en la máquina cliente.

Los archivos js se incluyen directamente por las páginas JSP que necesiten hacer uso de dichas funciones a través de llamadas del tipo `<script type="text/javascript" src="fichero.js"></script>`.

Las normas básicas para el desarrollo javascript son:

- En primer lugar es necesario conocer la tipología de funciones existentes en las distintas librerías existentes en la tecnología y que están documentadas en el Apéndice: Librerías API Javascript. Se deben usar estas funciones siempre que sea posible porque entre otras cosas se evitan posibles tareas de mantenimiento en futuros desarrollos o productos.
- Sólo si no existe ninguna función estándar que responda a su objetivo proceda a crear una nueva. Para ello, debe tener en cuenta el alcance de la función:
 - si el código de control javascript que se quiere añadir es genérico, común para distintas funcionalidades de una misma tarea, debe crear un archivo js con el nombre de la tarea.

- Si el código es específico, propio de una funcionalidad de una tarea: debe crear un archivo js con el nombre del componente.

Aquí se incluirán todas las funciones y código javascript que sea posible. Estas librerías deben localizarse bajo el directorio default, en una carpeta de librerías por producto que tiene que crear. Dentro de esta carpeta se crearán más carpetas asociadas a las tareas del producto.

EJEMPLO

Si dentro del producto Peoplenet se tuviera que desarrollar funcionalidad genérica para las tablas de valores, tarea shco_py, y específica para una tabla de valor en concreto, por ejemplo para la tabla de incidencias masivas, debe crear la siguiente estructura de directorios bajo default:



- Una vez que ha creado las carpetas, librerías y funciones oportunas, incluya la librería que corresponda en la cabecera de las páginas JSP que la utilicen. En el ejemplo:

```
<script type="text/javascript" src="/library_pn/shco_py/
shco_py.js"></script>
<script type="text/javascript" src="/library_pn/ shco_py/
shco_incid_mas.js"></script>
```

No hace falta utilizar el atributo del lenguaje de la etiqueta del script para aclarar que las librerías siguen el ECMAScript 262 rev 3 (JavaScript1.5, JScript 5.1). Sin embargo, si especifica algo, la cadena debería ser "javascript1.5":

```
<script type="text/javascript" src="../a/b.js"
language="javascript1.5">
</script>
```

Portabilidad

No se permite el uso de ningún método JavaScript que no sea estándar (es decir, no portable). Sin embargo, no hay ningún validador para ECMAScript, por lo que tendrá que depender de su conocimiento de este estándar para crear

librerías compatibles.

Uso de XHTML

Las páginas JSP compiladas deben producir documentos *XHTML 1.0 Transitional* válidos y deben incluir siempre el encabezado correspondiente. Se acepta HTML 4.01 para los prototipos. En el futuro el desarrollo podría enfocarse hacia XHTML 1.0 Strict (no están permitidos los elementos de presentación). Sin embargo, para generar dinámicamente el contenido, el documento original JSP debe ser compatible con XML antes de la compilación. En este manual se recomienda hacer esto, aunque no es obligatorio.

```
<!DOCTYPE html
PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"DTD/xhtml1-transitional.dtd">
```

Valide el HTML de todas las páginas JSP con el validador W3 (<http://validator.w3.org>) o utilice un editor HTML con validación W3C como el HTML-Kit de Chami.com. Las discrepancias existentes con el estándar *XHTML 1.0 Transitional* puede borrarlas manualmente (se recomienda para páginas con altas prestaciones de diseño) o utilizando HTML Tidy, un editor HTML que convierte automáticamente a XHTML (sólo se recomienda para páginas con pocas funciones de presentación y que no sean XML). Para obtener más información, vea el apartado Convertir HTML a XHTML en el *Apéndice: Recursos adicionales*.



Validación de documentos HTML con XHTML 1.0 Transitional

Recuerde las principales diferencias entre HTML4 y XHTML1:

- Los nombres de elementos y atributos deben estar siempre en minúscula.
- No se puede omitir los elementos 'head' y 'body'.
- Los elementos 'title' deben ser los primeros del 'head'.
- Puede anidar elementos, pero no solaparlos.
- Para los elementos que no estén vacíos, deben usarse etiquetas de fin.
- Los nombres de atributos no se pueden usar sin valores: XML no permite minimizar los atributos.
- Los elementos vacíos deben tener una etiqueta de fin o la etiqueta inicial debe terminar con />.

- Los documentos deben utilizar el atributo 'id' para los elementos *a*, *applet*, *form*, *frame*, *iframe*, *img* y *map*, que históricamente también tenían un atributo de nombre.
- En el texto de páginas HTML se debe utilizar los caracteres "´" en lugar de "á".



No utilice nunca caracteres acentuados, en su lugar, utilice la forma &code; HTML.

Encabezados HTML y directivas Meta

Cuando escriba páginas en idiomas latinos, debe utilizar la directiva Meta Content Type con atributo http-equiv para especificar la codificación de caracteres ISO-8859-1 (idiomas europeos de 8 bits) en todas las páginas.

```
<meta http-equiv="Content-Type" content="text/html";
charset="iso-8859-1">
```

Para idiomas como el árabe, la directiva Content-Type con atributo http-equiv debe ser UTF-8.

```
<meta http-equiv="Content-Type" content="text/html";
charset="utf-8">
```

Para obtener más información, , vea el apartado Directivas meta en el *Apéndice: Recursos adicionales*.

Almacenamiento en caché del navegador

Las opciones de configuración del explorador relacionadas con los archivos temporales de Internet suelen ser importantes para el funcionamiento y el rendimiento de los productos Web.

Las páginas deberían funcionar con cualquier opción de configuración de Internet Explorer en la ruta **Tools | Internet Options | General | Settings | Check For Newer Versions of Stored Pages**. El código no debería depender de las opciones. Su significado en Internet Explorer es el siguiente:

- **Cada vez que se visita la página:** Internet Explorer (IE) no utiliza los archivos temporales de Internet almacenados en la caché.
- **Cada vez que se inicia Internet Explorer:** si visita un sitio que visitó anteriormente en la misma sesión, IE utiliza los archivos temporales de Internet almacenados en la caché, a menos que pulse F5 o haga clic en **Actualizar**.

- **Automáticamente** (sólo IE 5.x): funciona como la opción anterior, pero IE crea un historial de la página de sesiones anteriores. Si IE determina que la página no cambia frecuentemente, no comprueba si hay nuevas páginas al comienzo de cada sesión.
- **Nunca**: Internet Explorer no comprueba en el servidor Web si hay contenido nuevo.

Debería utilizar la directiva "expires" pero no "no-cache" (consulte el artículo de Microsoft [http://support.microsoft.com/default.aspx?scid=kb;\[LN\];Q234067](http://support.microsoft.com/default.aspx?scid=kb;[LN];Q234067)):

- Cuando una página está marcada con la directiva "no-cache", no se almacena en la caché, lo que significa que no se podrá volver a ella haciendo clic en el botón **Atrás** en determinadas circunstancias (y cuando la página procede de una operación POST, habrá que volver a ejecutar la operación).

```
<% // NOT recommended
response.setHeader("Cache-Control","no-cache"); //HTTP 1.1
response.setHeader("Pragma","no-cache"); //HTTP 1.0 to:
%>
```

- Por otro lado, cuando una página está marcada con una fecha de expiración pasada (por ejemplo, -1), la próxima vez que Internet Explorer la cargue, se ejecutará el código JSP, pero seguirá estando disponible desde el botón **Atrás**.

```
<% // Recommended
response.setHeader("Expires", -1);
%>
```

Uso de la hoja de estilos o CSS

Utilice una hoja de estilos (CSS) para describir características como los colores de las fuentes y de fondo. Puede utilizar el CSS Style-O-Matic o una herramienta similar si necesita ayuda para convertir HTML a CSS. Para obtener más información, vea el apartado Herramientas de validación CSS en el *Apéndice: Recursos adicionales*.

Las hojas de estilos deberían ser archivos almacenados externamente enlazados a páginas HTML. Si se usan hojas de estilos de archivos externos, una única hoja de estilo puede describir varias páginas. Esto permite realizar fácilmente cambios en toda la página e incluso en el sitio.



No defina nunca fuentes o colores en el HTML escrito para una página JSP.

Nivel de CSS

Las hojas de estilos deben ser CSS-1 (<http://www.w3.org/pub/WWW/TR/REC-CSS1>). CSS-2 se basa en CSS-1 y, con muy pocas excepciones, todas las hojas de estilo CSS-1 válidas son también válidas en CSS-2 (<http://www.w3.org/TR/REC-CSS2>).

Puede utilizar la herramienta CSSCheck (<http://www.htmlhelp.com/tools/csscheck>) para comprobar la compatibilidad con CSS-1. Opcionalmente, puede utilizar el validador W3C para comprobarlo con el estándar CSS-2 (<http://jigsaw.w3.org/css-validator>). Para obtener más información, vea el apartado Herramientas de validación CSS en el *Apéndice: Recursos adicionales*.



Valide el CSS con el CSS-1 u, opcionalmente, con el CSS-2.

Estructura de los productos y las páginas

En este apartado se describen los siguientes temas:

- Conceptos básicos
- Estructura de los productos

Componentes básicos

Se ofrece una completa descripción de los principales componentes del desarrollo del cliente:

- Componentes del producto, como tipos de página y tipos de carpeta.
- Componentes básicos no modificables
- Estructura de las páginas e *includes*

Componentes del producto

Un producto Meta4 Web está compuesto por distintos componentes. Hay cuatro páginas básicas (inicio, fin, portal y error) y ocho tipos básicos de carpetas de contenido (común, genérico, tareas, imágenes, plantillas, librerías, estilos, ayuda y propiedades).

Figura 1. Diseño de la carpeta del producto Web

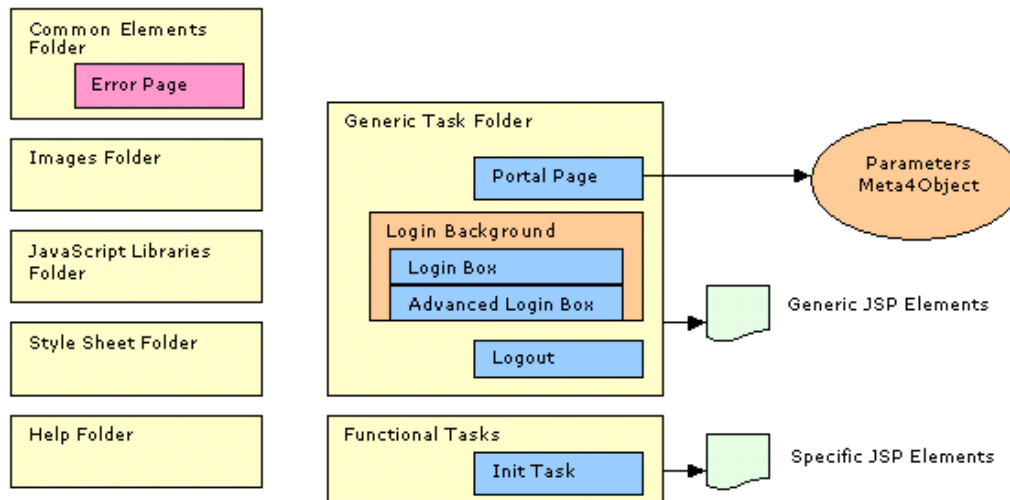


Tabla 1. Componentes de los productos Meta4 HTML

Página de inicio de sesión	Página JSP para que introduzca su nombre de usuario y su contraseña.
Página de inicio de sesión avanzada	Página JSP distinta para que introduzca su nombre de usuario y su contraseña, con casillas de verificación avanzadas opcionales para activar las opciones de sociedad, rol, moneda y fechas al iniciar sesión. Tanto la página de inicio de sesión sencilla como la avanzada pueden incluir un inicio de sesión por defecto o un inicio de sesión con opciones.
Página de inicio de sesión en segundo plano	Elementos adicionales informativos que se añaden como fondo para el cuadro en el que se introduce el nombre de usuario y la contraseña al iniciar sesión. Los clientes pueden eliminar este fondo para colocar el cuadro de inicio de sesión en la Intranet.
Página del portal	Primera página que ve después de iniciar la sesión. Presenta toda la funcionalidad del producto en forma de portal. Entre la página de inicio de sesión y la página del portal puede haber pasos adicionales que le obliguen a seleccionar el puesto o la empresa, o que inicien el sistema.
Carpeta de elementos comunes	Contiene elementos variados que no pertenecen a tareas, no son JavaScript y son comunes a todos los productos, como la página de error.
Página de error	Lee el atributo de error de la solicitud y muestra el texto del error localizado.
Elementos JSP genéricos	Las páginas JSP que no pertenecen a un único módulo funcional, sino a todos. La página de inicio y la página del portal son elementos genéricos, puesto que el encabezado y el pie de todas las páginas son elementos genéricos para todas las páginas.
Carpeta de tareas genéricas y del portal	Contiene elementos JSP genéricos de cada producto.
Módulo funcional HTML	Cada uno de los elementos de un producto que contienen funcionalidades comunes. Un producto tiene n módulos funcionales. No se pueden separar de los elementos genéricos de todos los módulos funcionales. No corresponde específicamente a una tarea: puede tener un módulo que incluya varias tareas.

Tareas	Entidades de la base de datos que se utilizan para agrupar funcionalidades y permisos.
Carpetas de tareas	Carpetas que contienen páginas JSP funcionales.
Carpeta de imágenes	Contiene iconos, así como otras imágenes, junto con elementos auxiliares para tratarlas.
Carpeta de librerías	Contiene los archivos de las librerías JavaScript
Carpeta de estilos	Contiene archivos CSS.
Carpeta de ayuda	Contiene páginas JSP y otros archivos auxiliares para la ayuda en línea.
Carpeta de propiedades	Contiene archivos de propiedades que se leen desde Java.
Página de fin	Debe proporcionar una forma de cerrar la sesión (por ejemplo, para iniciar otra con un usuario distinto) sin cerrar el navegador.

Componentes básicos no modificables

El HTML Development Framework presenta un entorno completo para el desarrollo de un producto Web sobre la tecnología de Meta4.

De forma general, el HTML Development Framework suministra la tecnología necesaria para establecer una sesión y acceder a un portal, ofreciendo acceso al resto de la funcionalidad a través de menús. Además soportará y proveerá la metodología para el desarrollo de nueva funcionalidad basada en plantillas, así como gestión de páginas multidioma, etcétera.

Finalmente incluye una serie de herramientas genéricas como la publicación de *queries*, filtros dinámicos, *To do list*, ejecución de informes, planificador de tareas.

Un producto Web que se desarrolla con el HTML Development Framework, como PeopleNet Ksystem, está compuesto de los siguientes elementos como partes básicas:

- Páginas básicas:
 - Páginas de inicio y fin de sesión.

- Página de portal que da acceso a través de menús al resto de la funcionalidad.
- Página de error de inicio de sesión y de web services.
- Contenido básico:
 - Carpetas genéricas: contienen los elementos *jsp* genéricos de un producto que se pueden utilizar en todas las páginas de la aplicación. Entre ellas se incluyen las plantillas (patrones de presentación que se usan varias veces en una aplicación) y las páginas especiales como la página de moneda o el calendario. Además proporciona toda la funcionalidad necesaria para la gestión de menús, la personalización del aspecto de las páginas a través de las hojas de estilo, gestión de páginas multiidioma, gestión de errores, sistema de ayuda en línea, etcétera.
 - Carpetas de imágenes.
 - Carpetas de librerías javascript.
 - Carpetas de estilos: Hojas de estilos (css y files_gif indirecciones jsp a imágenes). La referencia al fichero css se hace desde la página *shco_gen_css.jsp*.
 - Carpetas con los contenidos de la ayuda en línea.
 - Carpetas con los mensajes de error en cliente y literales.
 - Herramientas de uso general: *To Do List*, publicación de *queries*, filtros Html, planificación de tareas, ejecución de informes, etcétera.
 - Soportando todo lo anterior están los componentes del servidor de aplicaciones de Meta4, como son, las tareas, Meta4Objects, parámetros de aplicación, etcétera.
- Carpetas específicas del producto Web:

Módulos funcionales compuestos por páginas *jsp* y los correspondientes objetos específicos del servidor de aplicaciones que componen la funcionalidad básica que provee el producto Web.

Los módulos funcionales se desarrollan utilizando los componentes básicos que el HTML Development Framework suministra. La característica fundamental para el desarrollo de nuevos módulos es la herencia de objetos: el Framework proporciona al funcionalidad básica que será extendida y sobrescrita en los nuevos desarrollos.

En principio se identifican sólo como elementos modificables por el usuario o desarrollador:

- Hojas de estilos e imágenes.
- Páginas de Inicio de sesión y portal : enlaza con la funcionalidad específica de los módulos.

- Mensajes de error y literales: deberá incluir mensajes nuevos para sus desarrollos específicos.



El resto es parte de la tecnología y es modificable por el usuario.

Para más información sobre aquellos contenidos no modificables consulte el *Apéndice: Archivos no modificables*.

Estructuras de las páginas e includes

Al escribir el código, puede ser conveniente modularizar el desarrollo dividiendo las páginas JSP en trozos que contenga código de distintos ámbitos y enlazarlos usando las directivas `<%@ include file="filename.ext" %>`.



No confunda las directivas `include` con las acciones `<jsp:include>`.

No obstante, una de las desventajas de las directivas *include* es que pueden provocar un estilo de programación demasiado complejo. Para evitar los problemas de mantenimiento, los desarrolladores que utilizan *includes* deben comentar cuidadosamente su diseño. Además, incluir código innecesario puede provocar el problema de los 64K (hay un límite de tamaño para el método compilado en un archivo de clase Java).

Por eso, es necesario alcanzar un compromiso entre la conveniencia de dividir el código por ámbitos usando includes y la claridad del código en una sola página. Debería distinguir las páginas que no se pueden utilizar solas, por ejemplo usando una convención de nomenclatura distinta.



Es recomendable utilizar la extensión *jspx* para indicar las páginas que sólo se pueden utilizar como *includes* y no se pueden utilizar solas.



Puede identificar los trozos de código JSP que se repiten y seleccionarlos para crear un módulo con un nombre diferenciable, para que se pueda utilizar con un *include*. Aún así, tenga cuidado de no abusar de las directivas *"include"*.



Puede identificar las páginas a las que no vaya a acceder ni se vayan a compilar en lotes usando nombres especiales.

Los ámbitos se pueden dividir en:

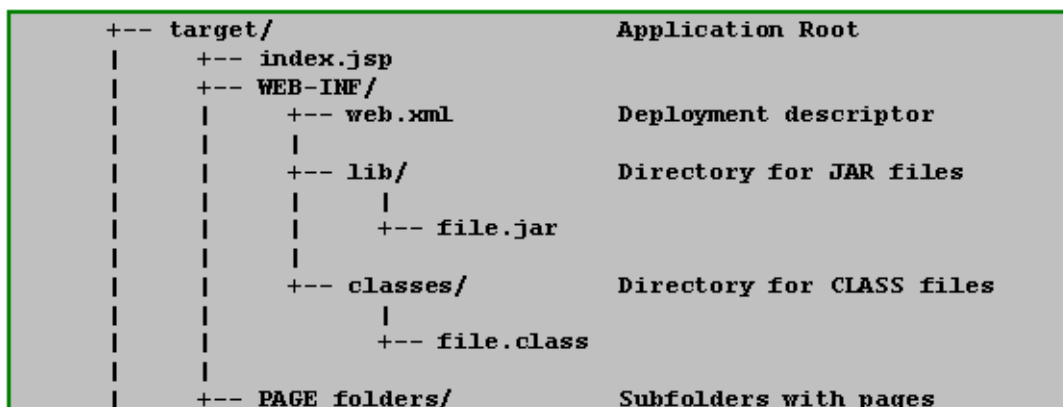
- Ámbito 1 (específico) para el código específico de una única página.
- Ámbito 2 (común) para el código común a varias páginas de un módulo funcional.
- Ámbito 3 (genérico) para el código común a todas las páginas del producto.
- Ámbito 4 (todo el producto) para el código proporcionado por la tecnología y que no debería ser (aunque puede) sustituido por los productos.

Estructura de los productos

Estructura de la aplicación Web estándar *Servlet 2.2*

Con respecto a la estructura de la aplicación Web, se debe respetar el estándar *Servlet 2.2* siempre que sea posible. De acuerdo con esta norma, una aplicación Web se estructura del siguiente modo:

Figura 2. Estructura Web



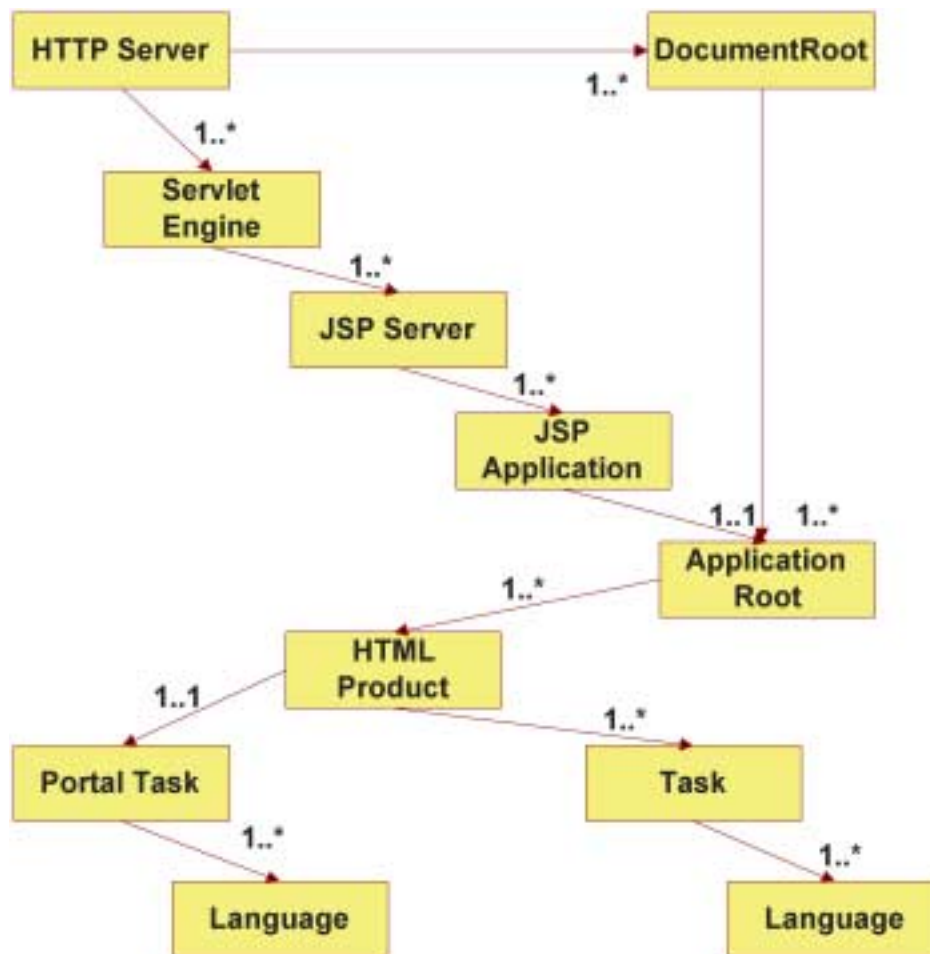
Esto significa que las páginas no tienen que estar necesariamente en la raíz del servidor Web.

Varios productos

Un servidor contiene 1...n motores de *servlets*, que contienen 1...n servidores JSP, que contienen 1...n aplicaciones JSP (esto es parte de la norma JSP) y

cada una tiene una raíz de aplicación. Es obligatorio usar distintos servidores JSP para implantar productos usando distintos repositorios, puesto que son instancias distintas de la misma tecnología (clases de Java).

Figura 3.



Puesto que la norma y la separación de los repositorios no le permite introducir dos productos en el mismo motor de *servlets*, el código compilado de Java se reproducirá en todos los servidores JSP (es decir, debe instalar la tecnología una vez para cada producto que quiera tener).



Como norma, no puede tener dos productos juntos en el mismo servidor de un motor de *servlets* por defecto.

Puesto que todos los productos Meta4 utilizan el contexto raíz, como se indica a continuación, si quiere utilizar dos productos, debe conectar dos servidores

Web distintos a dos instancias de su motor de *servlets*.

Directorios raíz

La carpeta raíz de la aplicación o carpeta “destino” será la que contenga directamente las páginas. Con respecto a los desarrollos Meta4 más antiguos, esto significa que ya no es necesario el `m4thinclientroot` repetido.

Por motivos de compatibilidad, el identificador de la aplicación que utilizarán todos los productos es “/” (para que al escribir “/carpeta” en el URL se abra directamente la carpeta destino).



El directorio con carpetas de tareas debe coincidir con el contexto raíz (“/”).

Teniendo en cuenta, además, la recomendación de que la carpeta destino esté fuera de la raíz del servidor Web, esto haría a la aplicación Web incompatible con otro contexto estático en el directorio de los documentos del servidor Web. Por eso, es recomendable que las páginas utilicen rutas relativas a los *includes* y los archivos de librerías e imágenes para cumplir la obligación de usar un identificación de aplicación Web que no sea “/” cuando sea necesario.



Es recomendable que se haga referencia a todos los *includes* y los archivos de librerías e imágenes con URL “relativas” para poder abandonar el contexto “/” a favor de una aplicación Web “nombrada” por petición del cliente.

Estructura de directorios: carpetas de páginas

Las carpetas de páginas de la aplicación Web deberían tener el siguiente diseño para los productos identificados como prod. En todos los casos, el contenido debe colocarse en carpetas específicas para cada idioma cuando esté traducido, de lo contrario, puede ponerse fuera.

Figura 4.

```
+-- target/
|
| [...] Public Task
|
| Folders
|
|   +-- prod g0 / Generic Folder
|   |   +-- espanol/
|   |   +-- english/
|   +-- prod g1 /
|   |   +-- espanol/
|   |   +-- english/
|   +-- prod py /
|   |   +-- espanol/
|   |   +-- english/
|
| [...] Utility Folders
|
|   +-- common/
|   |   +-- espanol/
|   |   +-- english/
|   +-- images/
|   |   +-- espanol/
|   |   +-- english/
|   +-- library/
|   |   +-- espanol/
|   |   +-- english/
|   +-- style/
|   |   +-- espanol/
|   |   +-- english/
|
| [...]
```

Ésta es la descripción de todas las carpetas:

- Carpeta Generic:
 - Carpeta especial de tareas (prod_g0) debajo de la carpeta raíz, denominada carpeta de tareas genéricas o del portal.
 - El sufijo de la carpeta de tareas genéricas es g0.
 - Las tareas en la base de datos deben estar definidas por su nombre completo, pero en mayúsculas, por ejemplo, PROD_G0.
 - En esta carpeta se almacenan tres tipos distintos de contenidos: la página del portal, la página de inicio y los *includes* JSP genéricas o trozos de páginas JSP que pueden utilizar todas las páginas de la aplicación.
- Otras carpetas de tareas:
 - Otras carpetas de tareas justo debajo de la carpeta raíz (prod_gl, prod_py).
 - Las tareas que pertenecen a distintos productos tienen un prefijo de carpeta diferente.

- Common (no se proporciona):
 - Contiene la página de error personalizada proporcionada con la tecnología.
 - No es aconsejable modificar la página de error que se incluye con la tecnología, aunque se pueden hacer cambios en la presentación para mejorar el diseño. Por eso está aquí y no en la carpeta de tareas genéricas.
- Help:
 - Contiene todo lo que necesita un sistema de ayuda en línea, distribuido en carpetas por idiomas.
- Image, style, library:
 - Distintas carpetas que contienen imágenes, hojas de estilos y JavaScript, cada una de ellas con subcarpetas para cada idioma.
- Temp:
 - Directorio vacío para archivos temporales.

Estilos y contenido

Includes de imágenes

Junto con el uso de CSS, la funcionalidad de las *includes* de imágenes es muy útil para separar la funcionalidad y el diseño de las páginas de productos.

Esta funcionalidad se basa en usar una página JSP auxiliar para almacenar el nombre y el tamaño del icono. Cuando se utiliza una etiqueta HTML *img*, no contiene los atributos *src*, de altura o anchura. Debe tener la etiqueta *alt*, que se puede traducir. La etiqueta *img* señalará a una JSP incluida que contiene los atributos *src*, altura y anchura de la etiqueta *img*. La página auxiliar se almacena en la carpeta de librerías y la imagen en el directorio de imágenes.

EJEMPLO

En la página JSP, incluya la JSP auxiliar de forma relativa:

```
<img <%include file="../../../library/accept.jsp" alt="txt"/>
```

En la página auxiliar *accept.jsp*, incluya el icono de forma absoluta:

```
Src="/images/accept_20x40.jpg" height="20" width="40"
```

Hay una etiqueta `<m4:img>` para realizar el proceso descrito anteriormente pero usando una sintaxis XML. Si la compatibilidad con XML es importante para usted, utilice el enfoque `<m4:img>`.

En cualquier caso, es importante asegurarse de que el producto utiliza todos los iconos que se están distribuyendo. Debe comprobarse periódicamente que no se están distribuyendo iconos con el producto que no se utilizan. Esto es aplicable a iconos de distintos tamaños.

Apariencias predeterminadas

Puede personalizar la imagen del producto, elegir entre varios diseños predefinidos. Cada uno de estos diseños se llamará “apariencia predeterminada”. Para que esto sea posible, elegirá el tipo de diseño predeterminado, representado por un entero, que se almacenará en el Meta4Object de parámetros. Al iniciar una sesión se cargará el Meta4Object de parámetros y se definirá una variable Java que permitirá seleccionar los iconos y los archivos CSS apropiados. Esto impone restricciones sobre las convenciones de nomenclatura, tal y como se explicará en los apartados siguientes.

Es posible que los productos sólo incluyan un diseño predeterminado conforme a los estándares de nomenclatura y que incluyan la documentación necesaria para que los clientes hagan sus propias apariencias personalizadas.

Convenciones de nomenclatura en el servidor Web

Convenciones de nomenclatura para tareas, carpetas y páginas

No utilice caracteres no ASCII 7 ni espacios en los nombres de las carpetas de tareas ni las páginas.

No utilice mayúsculas en los nombres de las carpetas de tareas ni las páginas.



Los nombres de las carpetas de tareas deben estar siempre en minúsculas y no pueden incluir caracteres de juegos de caracteres ampliados.

- Escriba siempre en inglés los nombres de las carpetas de tareas y las páginas (usted decide si le conviene más utilizar un nombre largo para la página, en inglés, en lugar de un identificador que no signifique nada, derivado de raíces inglesas).
- Las tareas tecnológicas (tareas comunes a todos los productos) no incluirán las partes <producto> y g_<id>, sino que tendrán un <nombre>. Esto le ayudará a distinguir las tareas que forman parte del producto y las que forman parte de la tecnología común.

- Utilice la convención de nomenclatura `<producto>_<módulo>_<nombrearchivojsp>.jsp` para los archivos JSP. Los módulos son más pequeños que las tareas (por ejemplo, el módulo de formación, el módulo de solicitud de vacaciones).
- Los archivos JSP tecnológicos no incluirán el `<producto>` ni el `<módulo>`, sino el `<nombrearchivojsp>.jsp`.

Convenciones de nomenclatura para iconos e imágenes

- Utilice la siguiente notación para los nombres de archivos de imágenes, incluyendo el nombre, la anchura, la altura, el identificador de la apariencia y la extensión
`<tipoimg>_<nombreimg>_<ancho>_<alto>_<idapariciencia>.<ext>`; por ejemplo, `icono_aceptar_34x36_0.gif`. Si no va a utilizar apariencias predeterminadas, termine todos los nombres en 0 y utilice el `Meta4Object` de parámetros para almacenar siempre 0.
- Todos los iconos se almacenan en la misma carpeta: "images". Sólo el diseñador necesita acceder a esta carpeta.

Convenciones de nomenclatura para las hojas de estilo

- Todas las hojas de estilo se almacenan en la misma carpeta: "estilos".
- Se denominan `<producto>_<nombrearchivocss>_<id>.css` donde ID es el entero que identifica el tipo de apariencia predeterminada.

Gestión de la jerarquía de páginas

Includes relativas

- Includes JSP: relativas.
- Inclusión de imágenes: relativa (aunque los diseñadores quizá prefieran trabajar con rutas absolutas)
- Inclusión de CSS: relativa (aunque los diseñadores quizá prefieran trabajar con rutas absolutas)
- Inclusión de JavaScript: ahora son absolutas (pero lo ideal es que fueran relativas).

Las inclusiones absolutas impiden adaptar el diseño a las necesidades de otras aplicaciones Web. No obstante, facilitan el trabajo a los diseñadores HTML, por lo que se pueden utilizar en la fase de desarrollo.

Comprobaciones de seguridad

En la arquitectura actual del cliente web, debe seleccionar un usuario y, opcionalmente, definir el puesto y la organización. Es necesario iniciar sesión para acceder al servicio de datos y utilizar los URL de *CheckSecurity* para los permisos. Aunque no quiera beneficiarse del esquema de permisos de los servicios Web, debe enlazar y utilizar los formularios referentes a los URL de *CheckSecurity* si contienen la etiqueta de declaración de la página de Meta4 (*page* o *startpage*, véase el apartado siguiente). Cualquier página que contenga esta etiqueta y a la que no se acceda a través de *CheckSecurity* le redirigirá a una página de error.

Por eso, no puede utilizar herramientas automáticas para comprobar si hay enlaces rotos en las páginas.

Tareas no dependientes del idioma

Cuando se pide una página de cliente web desde el explorador:

```
.../servlet/CheckSecurity/tarea/pagina.jsp
```

el *servlet*, una vez comprobada la seguridad, concatena a la *url* final el idioma con el que el usuario está logado, quedando la *url*:

```
.../tarea/idioma/pagina.jsp
```

Debido a que el *servlet* *CheckSecurity* siempre concatena el idioma, la páginas se distribuyen en las carpetas por idioma que únicamente redirigen a las genéricas. Para evitar esta repetición de páginas es el *servlet* el que tiene que aplicar el idioma o no.

Para ello, en la definición de una tarea de cliente ligero se pueden añadir propiedades. Esto se hace desde la ventana **Editar código** del diseñador de tareas.

Se añade una nueva propiedad, **_LANGUAGE** que permite definir si se aplica el idioma de la sesión o no.

Los valores posibles son:

- si la propiedad no existe se concatena el idioma de la sesión a la *url*.
- si **_LANGUAGE=default** se concatena el idioma de la sesión a la *url*.
- si **_LANGUAGE=none** no se concatena el idioma.
- si **_LANGUAGE=anyothervalue** se concatena *anyothervalue* a la *url*.



Recuerde que las propiedades en el diseñador de tareas se escriben separadas por un punto y coma, y finalizadas en punto y coma.

Configuración de la aplicación para un producto Web

Además de añadir el contenido propio de la aplicación es necesario personalizar las páginas básicas de inicio de sesión (*login*) y el portal. Las páginas de desconexión y de error de inicio de sesión son genéricas y no tendrán que personalizarse.

Para permitirlo hay que establecer un identificador para su aplicación. Este valor se utiliza para localizar en una nueva carpeta las páginas que se tienen que configurar para dotar de un aspecto personal el portal.

El identificador de producto se puede establecer desde:

- La página de conexión añadiendo el atributo `_PROD`:

```
String zurl = "/servlet/CheckSecurity/JSP/shco_g0_pn/
shco_gen_portal.jsp...
<% request.setAttribute("LANG", zlang);
// Identificador de producto
request.setAttribute("_PROD", "PN");
request.setAttribute("URL", zurl);...
<jsp:include page="/shco_g0/shco_gen_login_box.jsp" flush="false"
/>
```

Es importante que la identificación se haga antes del `jsp:include`

```
<jsp:include page="/shco_g0/shco_gen_login_box.jsp" flush="false" />
```

puesto que esta página necesita el valor del atributo `_PROD`.

- A través de la etiqueta `MainProductID` del `configclient.xml`. Recuerde que este fichero se modifica a través de las páginas de configuración del producto. Si la identificación no consta en la página de conexión, se toma por defecto el definido en este fichero.

Configuración de un portal para un producto Web

Configuración del Meta4Object

El proceso es el siguiente:

1. Cree un `Meta4Object` que herede de **SHCO_GN_PORTAL_TEC**. Proporciona la carga de menús y los parámetros globales básicos.
2. Sobreescriba el nodo **SHCO_GN_ROOT** y cree una nueva estructura de nodo para añadir las nuevas variables o parámetros globales.

El nodo **SHCO_GN_ROOT** sólo tiene los siguientes métodos:

- SHCO_CONS_CURR
- SHCO_LOAD
- SHCO_LOAD_MENUS
- SHCO_LOAD_OTHER_PARAMS
- SHCO_PARAM

Cualquier otro que se necesite tendrá que ser añadido.

3. Edite el código LN4 del método **SHCO_LOAD_OTHER_PARAMS** y sobrescriba su regla, añadiendo la carga de los parámetros que se deseen. Esta operación requiere la creación de elementos propiedad para el almacenamiento temporal de los valores.

Configuración de las páginas JSP de portal

Las páginas que sirven como referencia y que salen como ejemplo de portal están en:

`tctools/espanol/tc_portal.jsp` (si se necesita multiidioma, copiar también `tctools/english/tc_portal.jsp` y `tctools/francais/tc_portal.jsp`)

`tctools/tc_portal.jsp`

`tctools/tc_portal_code.jsp`

Posiblemente también necesite crear una página jsp que cargue los literales traducibles que apareceN en el portal, pues el portal de tecnología no carga ninguno.

1. Copie esas páginas sobre la carpeta donde va a residir el portal (en el ejemplo, *shco_pn*) y renómbrelas según la nomenclatura que utilice, por ejemplo:

`shco_pn/espanol/shco_gen_portal.jsp`

`shco_pn/shco_gen_portal.jsp`

`shco_pn/shco_gen_portal_code.jsp`



Es recomendable crear estas páginas en otra carpeta para no mezclar con el resto de componentes de *shco_g0*. Por ejemplo, se podría crear la carpeta *shco_pn* para almacenar las nuevas páginas de login y portal. En este caso debe crear una tarea tipo cliente ligero con ID de tarea *SHCO_PN* para poder ejecutar páginas jsp.

2. Cree la página para cargar las traducciones, por ejemplo:

`shco_pn/shco_gen_portal_trans.jsp`

así como los archivos con los literales en la carpeta *translations*:

```
translations/scho_portal_es.properties
translations/scho_portal_en.properties
translations/scho_portal_fr.properties
```

3. Edite *shco_pn/espanol/shco_gen_portal.jsp* y *shco_pn/shco_gen_portal.jsp* y cambie las referencias a los nuevos nombres de los archivos:

[en *shco_pn/espanol/shco_gen_portal.jsp*]

```
<%@ include file="../../shco_gen_portal.jsp" %>
```

[y en *shco_pn/shco_gen_portal.jsp*]

```
[=====] --%>
<%@ include file="../../shco_g0/shco_gen_portal_code_begin.jspf" %>
<%-- [-----] --%>
<%--                                     begin: include here your portal
page                                     --%>
<%-- [-----] --%>
<%@ include file="shco_gen_portal_code.jsp" %>
<%-- [-----] --%>
<%--                                     end: include here your portal page
--%>
<%-- [-----] --%>
<%@ include file="../../shco_g0/shco_gen_portal_code_end.jspf" %>
```

4. A continuación, edite el archivo *shco_gen_portal_code.jsp* y sobrescriba las siguientes líneas para indicar el nombre del Meta4Object que soporta el portal.

```
<%
// Declaración del M4Object:
String zsubsesion = "SHCO_GN_PORTAL_TEST";
String zm4object = "SHCO_GN_PORTAL_TEST";
String znodoraiz = "SHCO_GN_ROOT";
String znodo = "SHCO_GN_ROOT";
%>
```

5. Elimine la línea que muestra la ayuda tecnológica:

```
//Para mostrar la ayuda tecnológica, quitar si se quiere mostrar la
funcional
z_gHelpFolder="help_tec";
```

6. Añada la línea donde se incluye la carga del nuevo archivo de traducciones *shco_pn/shco_gen_portal_trans.jsp*:

```
<% // include here your *_trans file= %>
<%@ include file="../../shco_pn/shco_portal_trans.jsp" %>
```

El archivo que carga las traducciones debe hacer:

```

<%
    java.util.Properties Tran_shco_portal = new Properties();
    Tran_shco_portal.load(application.getResourceAsStream("/
translations/shco_portal_" + zlanguser + ".properties"));
%>

```

7. Añada la personalización del *desktop* donde se indica a continuación:

```

<div id="capa_cuerpo2" style="position:relative; left:21%;
top:140px; width:60%; height:70%; z-index:1">
<% // include here your page code, links, ... %>
</div>

```

Gestión de nuevos parámetros de producto

El proceso normal para usar nuevos parámetros es el siguiente:

1. Cárguelas en el Meta4Object del portal en el método **SHCO_LOAD_OTHER_PARAMS** del nodo **SHCO_GN_ROOT**.
2. En la página de variables del portal, deje el valor cargado en la bolsa de propiedades.
3. Dejar en una variable java el valor del parámetro accesible para todas las páginas.

Configuración de la página JSP de carga de parámetros

En la página */shco_g0/shco_gen_portal_bag.jspf* el sistema carga una página específica de cada producto:

```

<%
    request.setAttribute("zNODE", znodo);
    request.setAttribute("zM4OBJECT", zm4object);
%>
<jsp:include page='<%= "/shco_g0_" + lp_zappprod + "/"
shco_gen_portal_bag.jsp' %>' flush="false" />

```

Por tanto, si tiene que añadir nuevos parámetros o variables globales, copie los 4 archivos de la carpeta *shco_g0_tec* en la carpeta que corresponde a tu producto, por ejemplo *shco_g0_pn*.

Edita la página *shco_gen_portal_bag.jsp* y meta en la bolsa de propiedades el nuevo valor, por ejemplo, para el parámetro **STANDAR**:

```

<%- -
    @(#)FileVersion: 601.000.003
    @(#)FileDescription: Variables de la bolsa en el portal
para producto

```

```

        @(#)CompanyName: Meta4 Spain, S.A.
        @(#)LegalCopyright: (c)1998
        @(#)ProductName: Meta4 PeopleNet Ksystem
        @(#)ProductVersion: 6.0
        @(#)InternalName: shco_gen_portal_bag.jsp
        @(#)Date: 21/02/2002
    --%>
<%@ include file="../shco_g0/shco_gen_taglib.jsp" %>
<%
String znodo = (String) request.getAttribute("zNODE");
String zm4object = (String) request.getAttribute("zM4OBJECT");
String zSHCO_STANDAR = "";
M4Operations Introduccion2 = new M4Operations(request);
zSHCO_STANDAR =
Introduccion2.getItem(znodo, zm4object, znodo, "", "SHCO_STANDAR");
M4SessionCl zsesion2 = M4Context.getM4SessionCl(request);
zsesion2.putBagEntries("SHCO_STANDAR", zSHCO_STANDAR);
%>

```

Configuración de la página de parámetros del portal

Para que el nuevo parámetro esté disponible para todas las páginas del producto, la página */shco_g0/shco_gen_bag.jsp* incluye una línea que carga una página propia de producto:

```

<jsp:include page='<%= "/shco_g0_" + zappprod + "/shco_gen_bag.jsp"
%>' flush="false" />

```

Por tanto, edite la página *shco_go_pn/shco_gen_bag.jsp* y cargue en variables java los nuevos parámetros, por ejemplo:

```

<%- -
        @(#)FileVersion: 601.000.003
        @(#)FileDescription: parámetros de la bolsa de producto
        @(#)CompanyName: Meta4 Spain, S.A.
        @(#)LegalCopyright: (c)1998
        @(#)ProductName: Meta4 PeopleNet Ksystem
        @(#)ProductVersion: 6.0
        @(#)InternalName: shco_gen_bag.jsp
        @(#)Date: 21/02/2002
    --%>
<%@ include file="../shco_g0/shco_gen_taglib.jsp" %>
<%
M4SessionCl zsesion = M4Context.getM4SessionCl(request);

```

```
String zstandar = zsesion.getBagEntries("SHCO_STANDAR");
%>
```

Configuración de la página de inicio de sesión

Crear la página JSP

El proceso es el siguiente:

1. Copie las páginas jsp que conforman el ejemplo, y que se localizan en

tctools/espanol/tc_login.jsp

tctools/tc_login.jsp

tctools/tc_login_trans.jsp,

en la carpeta donde vayas a ubicar el nuevo login, por ejemplo, *shco_pn*.

2. Renombre los archivos a sus nombres definitivos y edítelos para cambiar sus referencias internas, por ejemplo:

shco_pn/espanol/shco_gen_login.jsp

shco_pn/shco_gen_login.jsp

shco_pn/shco_login_trans.jsp

3. En *shco_pn/shco_gen_login.jsp* se sobrescribe las siguientes líneas:

- La ruta al archivo que carga los recursos

```
<%@ include file="/shco_pn/shco_login_trans.jsp" %>
```

también debe cambiar los usos de la variable **Tran_tc_login** por la que haya definido en su archivo *_trans.jsp*; por ejemplo, si este era

```
<%
```

```
    java.util.Properties Tran_shco_login = new Properties();
```

```
    Tran_shco_login.load(application.getResourceAsStream("/  
translations/shco_login_" + zlanguser + ".properties"));
```

```
%>
```

debe cambiarla por **Tran_shco_login**.

- Modifique la línea donde se establece el atributo **_PROD** del request y establezca el valor PN para dicho atributo:

```
request.setAttribute ("_PROD", "PN");
```

Esta página, así como las de *login* avanzado y la página de error, utilizan la hoja de estilos definida en el *configclient.xml* en entrada **THINCLIENT/STYLESHEET**

```
<THINCLIENT>
```

```
<UPLOADDIR>c:\temp</UPLOADDIR>
```

```
<MAXFILESIZE>50000</MAXFILESIZE>
```

```
<DEFAULTPAGE>/servlet/CheckSecurity/JSP/tctools/tc_portal.jsp</
DEFAULTPAGE>

<STYLESHEET>tc_login.css</STYLESHEET>
```



Si su aplicación utiliza otra, no olvide sobrescribir esa entrada.

El único requisito en este punto es que la hoja de estilos utilizada contenga, al menos, las siguientes clases:

```
.tablalink
{
}
.fuentelinkcampo
{
}
```

Para levantar la ayuda funcional al pulsar el *link* **Ayuda** debe cambiar:

```
<a class="dis" id="AdvanceLink" name="AdvanceLink"
title="<%=Tran_tc_login.getProperty("login.ToolTipGoAdvance")%>"
href="javascript:showHideLayer('ADVANCED');">
<%=Tran_tc_login.getProperty("login.5")%></
a>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<a
class="dis" id="Help" name="Help"
title="<%=Tran_tc_login.getProperty("login.Help")%>"
href="javascript:m4help('<%=zLangFolder%>', 'help_tec');">
<%=Tran_tc_login.getProperty("login.Help")%></a>
```

eliminando el segundo argumento `.'help_tec'` de la llamada `m4help`.

Configuración

La página de *login* creada tiene obligatoriamente que configurarse en la entrada del *configclient.xml* `<login>`:

```
<UPLOADDIR>c:\temp</UPLOADDIR>
<MAXFILESIZE>50000</MAXFILESIZE>
<DEFAULTPAGE>/servlet/CheckSecurity/JSP/tctools/tc_portal.jsp</
DEFAULTPAGE>
<THINCLIENTROOT></THINCLIENTROOT>
<Espanyol>
<login>/shco_g0/espanol/shco_gen_login.jsp</login>
```

La razón es que la página genérica que hace la desconexión (*shco_g0/shco_gen_logout.jsp*) así como la página que permite el cambio de opciones de conexión (*shco_gen_portal_code_begin.jspf*) redirigen al *login* tras la desconexión y tras mostrar errores respectivamente.

Configuración de la fecha GMT en el SSE/SSM

Para establecer la fecha GMT en el SSE/SSM añada el siguiente código en `generic_login.jsp` (en la cabecera):

```
<script type="text/javascript">
    function setGMTOffsetCookie ()
    {
        var today = new Date();
        var offset = -(today.getTimezoneOffset()/60);
        var newCookie = "COOKIE_GMT_OFFSET" + '=' + escape(offset)
+ "; path=/";
        document.cookie = newCookie;
    }
</script>
setGMTOffsetCookie()
```

Configuración página de desconexión

Para que la página de desconexión genérica funcione bien, es necesario configurar adecuadamente la página del *login* en el **configclient.xml**. Para más información consulte el apartado *Configuración*.

Configuración página de error de inicio de sesión

Para utilizar una página propia de error de inicio de sesión debe establecer en el *configclient.xml* su propia página:

```
<Espanyol>
<login>/tctools/espanol/tc_login.jsp</login>
<Error>/shco_g0/espanol/shco_show_error.jsp</Error>
<Change_Password>/tctools/espanol/change_password.jsp</
Change_Password>
<Folder>espanol</Folder>
<Locale>es</Locale>
</Espanyol>
```

Uso de Servicios Web desde JSP

Antes de empezar a desarrollar debería leer y entender todos los conceptos tratados en este capítulo. Si no le queda claro alguno de los conceptos aquí tratados, pida una aclaración.

El cliente Web es un conjunto de capas para las comunicaciones XML entre el cliente y el servidor. A veces la capa superior no ofrece la funcionalidad suficiente para sus necesidades empresariales. Sólo en ese caso debe utilizar directamente las API de niveles inferiores (como Meta4Operations). Si mezcla etiquetas con código Java sin motivo, será muy difícil mantener las páginas. Cuantas más cosas pueda hacer con etiquetas, mejor. Documente siempre lo que haga aparte de las etiquetas.

Al utilizar las clases Java auxiliares sólo debería importar unas pocas de los paquetes públicos: por ejemplo, `com.meta4.operations.M4Operations`. Existe otro documento en el que se describe qué parte de la API es privada en cada versión. El resto de la API del cliente Web debería considerarse privada y está sujeta a cambios. Esto incluye varias prestaciones que se han utilizado poco o la API para leer el archivo de configuración.

Registros y excepciones

Si quiere utilizar *logs* en sus páginas, no utilice `System.out` para escribir en los *logs* del servidor Web o en la consola. Puede utilizar el sistema de registro empleado en el resto de los servicios Web declarando una categoría `log4j` al comienzo de la página del servidor Java.

EJEMPLO

Declare una categoría e indique las excepciones en el archivo `jsp.log`:

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<%@ page import="java.io.*, java.util.*,
com.meta4.organization.*"%>
<%@ page
import="com.meta4.common.utils.logsystem.M4i18nCategory"%>
<%@ page import="org.apache.log4j.Category" %>
<% M4i18nCategory m_log =
(M4i18nCategory)M4i18nCategory.getInstance("com.meta4.jsp");
try{ListOrganization oListOrganization =new
ListOrganization(request);Hashtable hOrganization =
oListOrganization.getListOrganization();
m_log.trace("Found "+hOrganization.size()+" organizations);
```

```
        } catch(Exception e) {m_log.error("Exception reading  
organization list: " +e.toString());  
    }
```

```
%>
```

Este ejemplo mostrará un rastro en el archivo jsp.log como:

```
Sep 20 18:32:13  
[jrun__tctools__espanol__lu_organizations2ejsp25.java(68) -  
172.17.14.136]: Encontró 9 organizaciones
```

Utilice el sistema de *log* para generar un mensaje cuando se produzcan excepciones. El sistema de *log* debería abrirlo siempre un bloque *catch*.

Tarea inicial

Las tareas deben tener siempre una página “central” (parámetro `_INITTASK`) cuando las defina. Esto dará la sensación de producto acabado cuando escriba un URL incompleto que incluya *CheckSecurity* y vea las páginas por defecto de la tarea.

Acceso al servicio de datos

Acerca de las etiquetas del cliente Web

La principal función de las etiquetas del cliente Web es dar acceso al servicio de datos. Se ha diseñado toda una nueva generación de etiquetas de cliente Web con los siguientes objetivos en mente:

Compatibilidad con XML: es decir, la posibilidad de utilizar sólo una etiqueta `<m4:page></m4:page>` en lugar de dos `<m4:startpage/><m4:endpage/>`.

Distintos argumentos para especificar un nodo, elemento, método o filtro para las etiquetas de acceso a datos. Es decir, en lugar de usar `ALIAS!NODE[RANGE]` al definir un `outputdef` dentro de una etiqueta `<m4:param>`, puede definir `ALIAS`, `NODE` y `RANGE` como argumentos distintos de la etiqueta `outputdef`.

Al ser la nueva sintaxis más sencilla y permitir procesar más fácilmente los archivos JSP originales como XML, es recomendable que cambie gradualmente las páginas para utilizar la nueva generación de etiquetas de cliente Web. La sintaxis antigua seguirá funcionando. En cualquier caso, debería evitar mezclar los dos tipos de sintaxis en el mismo producto.

EJEMPLO

Si quiere utilizar la nueva generación de etiquetas, así es como debe hacer un outputdef. Para un nuevo desarrollo, utilice esta sintaxis, que es más sencilla.

```
<m4:outputdef
  m4alias="RANGO"
  m4object="PERSON"
  node="STC_MT_TEST_PERSON"
  records="[1-3]">
</m4:outputdef>
```

Ésta es la sintaxis antigua. Es más complicada y es incorrecta:

```
<m4:outputdef m4alias="RANGO">
<m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[1-3]" />
</m4:outputdef>
```

Alias con variables Java

En vez de utilizar los nombres reales de Meta4Objects, nodos, elementos y registros y crear alias para ellos dentro de las etiquetas de los servicios Web, declare todos sus nombres como variables de Java y después utilice estas variables como los alias dentro de las etiquetas. Lógicamente, los nombres de variables deberían tener el mayor significado posible.

EJEMPLO

Declare la subsesión, el Meta4Object, el nodo, etc.

```
<%
  Cadena stSubsession = "THIS_SUBSESSION"
  Cadena stM4Object= "CXX_THISM4OBJECT"
  Cadena stNode = "CXX_THISNODE"
%>
```

Después incluya una página que utilice stSubsession, stM4Object y stNode para realizar algunas operaciones sobre ese nodo:

```
<% @ include file = "../..task/page_that_loads.jsp"
```

Los demás desarrolladores no sabrán que page_that_loads.jsp no puede verse sola, así que tendrá que documentar detalladamente todas las páginas que no sean autónomas.

Esto evita los errores que suelen surgir al utilizar alias. También permite reutilizar el código en directivas include. Por ejemplo: si siempre utiliza los mismos nombres de variables en las transacciones típicas que comprenden 3 nodos, puede reciclar la parte de las etiquetas en todas las páginas que usen 3 nodos.



Se recomienda utilizar variables Java para los alias del cliente Web. Declárelas antes que las etiquetas.

Uso de etiquetas de definición de transacciones (página y tarea)

No debe utilizar nunca una etiqueta `<m4:page/>` (o `<m4:startpage/>`) en la página de inicio. No funciona.

Sólo puede haber una etiqueta `<m4:page/>` (o `<m4:startpage/>`) por página y debe utilizarla para seleccionar el nombre de subsesión.

Las etiquetas `<m4:page/>` (o `<m4:startpage/>`) requieren que acceda a ellas a través del *servlet* `CheckSecurity`.

Es importante seleccionar una subsesión adecuada en función del uso que haga de ella. Esto puede causar un gran impacto en la memoria y el rendimiento.

El nivel de preservación es un parámetro opcional, `subsessionid` para la etiqueta `<m4:page>` (o `m4preservelevel` para `<m4:startpage/>`), que se puede definir para que las subsesiones no se almacenen en el servidor de aplicaciones. Si se hace con cuidado, definir las subsesiones como “no almacenables” (*not persistent*) puede ahorrar memoria. Sin embargo, abusar de subsesiones “no almacenables” puede ralentizar la interacción del cliente Web con el servidor de aplicaciones.

En este capítulo se ve el uso de nombres de subsesiones y de niveles de preservación.

Navegación en el cliente y en el servidor

Si utiliza la etiqueta `<m4:move/>` entre un `<m4:startjob/>` y un `<m4:endjob/>`, se está moviendo por los registros del servidor de aplicaciones.

Si la utiliza después de la etiqueta `<m4:endjob/>`, está pasando por el XML en la parte del servidor Web. Por lo tanto, no puede modificar las posiciones.

Lo mismo ocurre con la variable `$COUNT$`. Puede obtener el recuento en el cliente y en el servidor.

Debe navegar al primer registro del cliente si quiere conseguir la etiqueta de un nodo usando la etiqueta <m4:label>.

Llamadas de nivel 2

Las llamadas de nivel 2 se utilizan mucho en módulos como la búsqueda de RH. Para hacer llamadas de nivel 2 desde una página JSP, puede utilizar el parámetro m4find="TRUE" para la etiqueta <m4:datadef/>, como se describe en el Manual del desarrollador del cliente web. Si el Meta4Object A llama al B usando llamadas de nivel 2, asegúrese de que sólo haya una instancia del Meta4Object A (m4find="true" equivale a "no volver a cargar").

Las llamadas de nivel 2 se pueden realizar de dos maneras:

- Con una declaración explícita de alias: debe definir la instancia mientras hace el diseño. Además, para que el servidor de aplicaciones tenga la instancia, debe realizar una transacción antes de invocar la instancia.
- Uso de LN4 para definir la instancia: DefineInstance crea una JIT donde se crea la instancia, por lo que ya no es necesario definir el nombre de la instancia al realizar el diseño y se evita tener que recurrir a la transacción previa.

Presentación

Caracteres especiales

Debe utilizar los atributos htmsafe/jsafe en las etiquetas para evitar las cadenas HTML y JavaScript no seguras.

Lectura de parámetros en los formularios

Si utiliza formularios HTML, todos los parámetros que pase a través de la petición deben ser codificados y decodificados si no pueden contener caracteres especiales o espacios.

Recuerde que si pasa simultáneamente un archivo y parámetros en un formulario con varias partes o de datos, la página de lectura de parámetros debe leerlos como atributos del pageContext, no como parámetros de la respuesta.

Meta4Objects de productos

Meta4Objects comunes

Cuando se define un nuevo Meta4Object para utilizarse en un producto Web, debe definirse como:

- Separable: No
- Atrás: Sí
- Con datos de varias empresas: No
- Metadatos almacenables en caché: Sí

Reutilización de Meta4Objects existentes

Si va a cambiar de un desarrollo Meta4 existente, tenga en cuenta lo siguiente:

- No existe ningún comportamiento OLT/Proxy (se utiliza el XML Executor), aunque utilice llamadas de nivel 2.
- No existe la carga parcial: usted introduce un número máximo de registros en el servidor de aplicaciones y después los recupera en “ventanas” del servidor Web. No es recomendable que escriba su propia implementación de carga parcial.
- Puede dejar los Meta4Objects como estaban (con padres e hijos), pero debe añadir algunas funcionalidades para aligerar la lógica del cliente. En el siguiente apartado se explica cómo se han diseñado los Meta4Objects que se suelen utilizar en los productos Web de Meta4.

Prestaciones de los Meta4Objects típicos utilizados en productos Web

Nodo principal

Los Meta4Objects típicos tienen un nodo principal sin ninguna tabla física asociada (ROOT_TABLE). La estructura de nodo (EN) principal sólo tiene métodos y propiedades que permiten utilizar otras estructuras de nodo reutilizables que se comentan a continuación.

Carga del nodo principal

El nodo principal tiene un método de carga especial que acepta un argumento. Por defecto, este argumento es NORMAL. Estos Meta4Objects no tienen el conector libre con el ROOTLOAD, por lo que debe eliminarse cuando se crea un nuevo Meta4Object.

Estructuras de nodo reutilizables

Todas las estructuras de nodo del Meta4Object, salvo la estructura de nodo principal, deben ser utilizables; es decir, independientes del Meta4Object y disponibles para todos los Meta4Objects.

La estructura de nodo principal utiliza las EN reutilizables para realizar operaciones. La EN principal no tiene que ser reutilizable.

Estructuras de nodo compartidas

Los Meta4Objects típicos utilizan dos EN auxiliares:

- EN TOOLS: para métodos LN4 auxiliares, común a toda la aplicación.
- EN COMMUNICATION: para los elementos de tipo propiedad que gestionan la información que pasan los Meta4Objects a las páginas. El Meta4Object decide lo que ha de enviarse en función de los resultados de una operación y después la EN de comunicación formatea los resultados de un modo u otro. Por ejemplo, deben mostrarse mensajes de error para todos los Meta4Objects, por lo que tiene sentido tratarlos aquí.
- LABEL_TI: para elementos de tipo propiedad que gestionan etiquetas genéricas
- LOG_TI: para elementos de tipo propiedad que gestionan mensajes de log

El uso de estas estructuras de nodo compartidas es una gran ventaja porque obliga a reutilizar el código.

Evitar repetir cargas

La carga de nodos puede ser una operación costosa si se hace repetidas veces y de forma innecesaria. Por lo tanto, antes de cargar un nodo:

- Compruebe el recuento de nodos.
- Si no hay nada, cargue el nodo y envíe un mensaje con la carga del nodo.

Esto permitirá controlar después qué nodos se están cargando en la aplicación.

Número máximo de registros

Debe evitar la posibilidad de cargar un número desconocido de registros. Esto provoca siempre un mal funcionamiento. Para controlarlo, establezca el número máximo de registros del nodo de carga del Meta4Object en 300. Si necesita recuperar más de 300 registros de la base de datos, modifique la condición de filtrado.

Errores

Utilice habitualmente el LN4 Setlog para comprobar errores. Antes de hacer nada en la lógica del Meta4Object que pueda causar un error, debe prever la condición del error y emitir un mensaje adecuado. Los errores deberían dividirse en módulos y submódulos funcionales para poder aplicar filtros y ver los errores correspondientes a sólo uno de ellos.

Presentaciones y acceso a objetos del cliente

Separación de la lógica empresarial y el cliente

La regla de oro del cliente web es proporcionar toda la lógica posible a los objetos. No es recomendable utilizar Meta4Objects simples y páginas complejas.

En el cliente web, los métodos están escritos en JavaScript. Aunque puede parecer obvio, no tiene sentido utilizar métodos de cliente en el Meta4Object.

Paginación

Cuando se espera mostrar un alto número de registros (un máximo de 300), el usuario no quiere verlos a la vez. No hace falta proporcionar una solución a nivel de Meta4Object (una carga parcial simulada se considera demasiado costosa). El problema se soluciona a nivel de cliente, asegurando que:

- Hay un filtro inicial.
- Se cumple el tope máximo de 300.
- Aporta funciones JavaScript que recuperan resultados del servidor de aplicaciones en “ventanas” (por ejemplo, 40 cada vez).

Serialización

La serialización es un mecanismo para insertar o eliminar registros en un Meta4Object al que se le ha dado una cadena con valores de caracteres delimitados (como: \$nodo\$EON\$elemento\$EOI\$registro\$EOR\$acción). Esta forma de realizar inserciones y actualizaciones puede ser útil cuando haya cambiado muchos valores en una página que pertenece a distintos nodos y elementos. En la mayoría de los casos es preferible utilizar este método que un `<m4:setitems/>` para cada elemento.

Puede proporcionar métodos JavaScript para crear la cadena y después hacer las inserciones una a una.

Reglas para las librerías JavaScript

Errores en las librerías

Los errores del código JavaScript se gestionan usando excepciones.

Está prohibido utilizar `alert()` en el producto final para indicar errores.

Ámbitos

Se pueden distinguir tres niveles: (1) librerías aplicables a un solo fin, (2) librerías aplicables a más de una página y (3) librerías aplicables al resto de la aplicación. Las librerías de ámbito 4 (herramientas como el calendario), que pueden ser utilizadas por cualquier producto, todavía no están previstas.

Debe publicarse una lista con todas las librerías JavaScript para que todos los desarrolladores puedan usarlas.

Plantillas de presentación

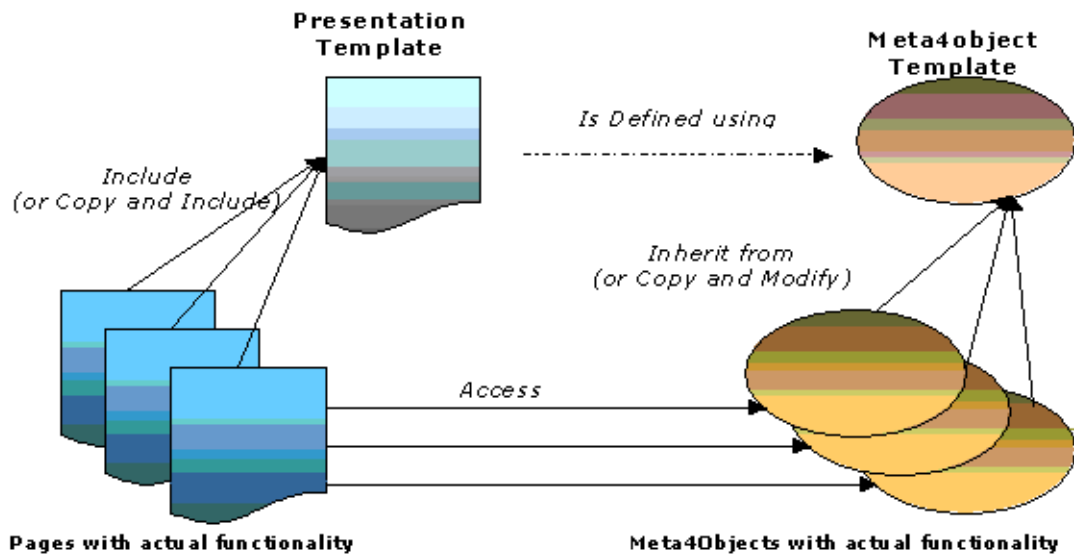
Es útil identificar “patrones” de presentación (acceso a objetos del cliente) que se utilizan varias veces en una aplicación. Cuando haya identificado los patrones básicos debería definir una colección de “plantillas” para facilitar el desarrollo.

Las plantillas de presentación estarán incluidas en las páginas que implementen ese acceso al cliente. Por ejemplo, puede definir la plantilla para una tabla de mantenimiento (TM) y varias páginas funcionales donde utilice la plantilla de la TM, como una pantalla donde introducir unidades organizativas.

La plantilla de presentación tendrá asociada una plantilla de Meta4Object de presentación. Los Meta4Objects que implementen esa funcionalidad ampliarán (mediante la herencia) la plantilla del Meta4Object de presentación.

A veces puede ser necesario reescribir la plantilla (la página, el Meta4Object o ambos) porque el patrón no cubre sus necesidades. En esos caso puede copiar la plantilla y adaptarla a sus necesidades. Sin embargo, no es recomendable hacerlo; debe intentar evitar personalizar las plantillas.

Figura 5. Plantilla de presentación



Ciclo de vida de la plantilla

Para cada plantilla de presentación habrá una "Funcionalidad de referencia". No tiene sentido gestionar las plantillas por separado de una Funcionalidad de referencia, puesto que las abstracciones suelen "olvidarse" (falta de mantenimiento y corrección de errores).

Las plantillas y su funcionalidad de referencia deben estar bien documentadas para que los clientes puedan partir de esas plantillas e implementar su propio acceso a los objetos.

Puede proponer nuevas plantillas si se utiliza el mismo patrón de acceso a objetos del cliente en algunas de sus páginas.

Ejemplos de plantillas

Para una aplicación de RH se han identificado varios patrones de acceso a objetos del cliente. Estos patrones se enumeran a continuación para que conozca el concepto de plantilla. Su aplicación puede utilizar patrones completamente diferentes: aquí sólo se indican los nombres como referencia.

- Plantillas de formulario:
 - Formulario de filtro o lista: el formulario de filtro o lista es una pantalla en la que puede introducir criterios para buscar en un número de registros, en los que puede hacer clic para actualizarlos (*zoom* en una tabla de mantenimiento).
 - Tabla de mantenimiento (TM): la Tabla de mantenimiento (TM) es una pantalla en la que puede insertar o actualizar datos mientras examina otros registros de la misma tabla. Si se utiliza una TM, el número de campos de la tabla debe ser pequeño (de 5 a 10 campos), de lo contrario la plantilla de acceso del cliente debería ser un asistente.
 - Asistente: el asistente es un formulario para un proceso largo de inserción o actualización, por ejemplo, uno que comprenda un gran número de campos (muchos nodos). Estos campos se agruparán en pasos. Los pasos pueden imponer condiciones en otros pasos. Para garantizar que pueda dejar el largo proceso del asistente sin tener que volver a empezar desde el principio, se pueden usar tablas temporales para almacenar la información incompleta.
 - Tablas de valor: las tablas de valor muestran una matriz tridimensional de datos que permite visualizarlos o modificarlos. El ejemplo típico es una tabla para conceptos de nómina, empleados y turnos. Puede optar por modificar el empleado y ver todos los conceptos de nómina, o modificar el concepto y ver las cantidades por empleado, etc.
- Plantillas de sólo lectura:
 - Árboles: los árboles muestran información como el árbol de la empresa, al que se puede acceder seleccionando la raíz y la fecha de validez.
 - Páginas descriptivas (gráficos/informes): las páginas descriptivas contienen gráficos o informes como un resumen de datos.

Diseño y navegación

Las reglas de diseño para los productos Meta4 se almacenan en un documento interno distinto (imagen, reglas de usabilidad, fuentes, imagen de la empresa, reglas sobre las imágenes y los colores, etc.). Algunas de ellas se indican aquí como referencia rápida:

- Uso de HTML de tamaño fijo o modificable: los tamaños fijos adaptados para un navegador de 800x600 son más fáciles de diseñar y dan más control al diseñador. Sin embargo, es más difícil colocar estas páginas en el portal de una empresa con otras capas fijas. Las capas cuyo tamaño es modificable dan menos control al diseñador pero son más fáciles de incrustar en una Intranet. Hay que intentar que el tamaño de las piezas pequeñas se pueda modificar pero que el de los generales sea fijo.
- Ancho y largo: Para un usuario típico de 800x600, las páginas de tamaño fijo no deberían superar dos veces el tamaño vertical de la pantalla y exactamente el horizontal.
- Validaciones: cuando hay un campo obligatorio y envía un formulario sin completarlo, debe marcarse el recuadro (no el texto) con un color llamativo y emitir un mensaje en el que se indique que el campo es obligatorio. El texto de los campos obligatorios debe estar precedido de un asterisco.
- Presentaciones y acceso a objetos del cliente

Desarrollo y ciclo de vida

Gestión de la configuración

Identificación de las páginas

Debería incluir menciones de copyright y versiones del “código” en todas las páginas JSP. Lo ideal es que estas versiones del “código” se actualicen con herramientas automáticas. Si no se usan, puede dejarlas fijas.

EJEMPLO

Éste sería un ejemplo de un encabezado de versión:

```
<%- - [=====]
@(#)FileVersion: 510.000.000
@(#)FileDescription: Recupera la lista de tareas
@(#)CompanyName: Meta4 Spain, S.A.
@(#)LegalCopyright: (c) 1998
@(#)ProductName: Meta4Mind Set
@(#)ProductVersion: 5.0+
@(#)InternalName: load_list_of_tasks.jsp
@(#)Date: 08/07/2001
[=====] - -%>
```

Control de versiones

Es importante controlar las versiones de las páginas desarrolladas. Los desarrolladores deben compartir archivos a través de un sistema de control de versiones, como por ejemplo, Visual Source Safe. Para obtener más información, vea el apartado Herramientas generales de otros fabricantes en el *Apéndice: Recursos adicionales*.

Herramientas de desarrollo

Edición

Se recomienda un editor de texto desde el que se pueda tener una vista previa del HTML y personalizar la sintaxis resaltada (como EditPlus, que es shareware, o el HTML-Kit de Chami.com, que realiza la verificación de XHTML). También puede utilizar Microsoft FrontPage 2000, que añade más prestaciones de edición de HTML y la posibilidad de automatizar algunas tareas de creación de plantillas usando la herramienta de JSP que incorpora. Esta herramienta no sustituye a las prestaciones del cliente Web o JSP (estamos buscando un editor JSP que compile páginas y una todas las *includes* para mostrar una vista previa sin tener que implantar las páginas en el servidor). Para obtener más información, vea el apartado Herramientas generales de otros fabricantes en el *Apéndice: Recursos adicionales*.

Depuración

En general, la herramienta de depuración más potente procede de la salida XML del servidor de aplicaciones que se almacena en el archivo connect.log si se habilita el nivel de registro en el apéndice m4connect del archivo de configuración del sistema de registro m4logsystem.cdf.

Las páginas JSP se depuran usando el registro del motor de *servlets*. Puede añadir registros a las propias páginas JSP para seguir el flujo del código Java en las páginas. Los depuradores de JSP trabajan con Tomcat, lo que facilita el uso de un servidor Web distinto para la depuración y la generación. Si no le importa tener que implementar los servicios Web manualmente en Tomcat, puede utilizar cualquiera de los IDE Java que ofrecen depuración, como Jbuilder Enterprise Edition de Borland o Forte For Java Internet Edition de Sun. Para obtener más información, vea el apartado Lenguaje Java e IDE en el *Apéndice: Recursos adicionales*.

Las librerías JavaScript se depuran usando mensajes de alerta.

Formatos de fechas, de números y de monedas

Este apartado describe cómo tratar el formateo de datos en la aplicación. Tiene la posibilidad de elegir diferentes formatos para las fechas, los números y las monedas.

Personalización de formatos

La aplicación Web hace posible personalizar los formatos de fechas y números a través de la página de parámetros de usuario (*shco_tl_param.jsp*):

Figura 6. Personalización de formato de fechas y números

PARÁMETROS DE USUARIO			
Visualización de la fecha		dd-MM-yyyy	
Formato de la fecha		dd-MM-yyyy	
Separador de la fecha		-	
Separador decimal		.	
Visualización de los números		1234.56	
Separador de miles		☐	
Número de decimales		5	
Visualización de las monedas		\$ 1234.56000000	
Separador de miles		☒	
Número de decimales		5	
Número de decimales fijos		5	
Hoja de estilos en cascada		100	
Desactivar advertencias e informaciones		☐	
Confirmar navegación en asistentes		☒	

Existen tres operaciones de tratamiento de datos en los que se deben tener en cuenta los formatos:

- **Mostrar datos en página que provienen de un Meta4Object:**
 - si se usa el tag `<m4:item>`, es el propio tag quien formatea los datos en función del tipo de item.
 - si se usa la `M4Operations` es necesario formatear los datos obtenidos mediante funciones específicas java.

EJEMPLO

```
<%  
M4SessionManager oSess = (new M4Context()).getSession(request);  
com.meta4.format.M4Format oFmt = oSess.getM4Format();
```

```
String zValOrFax = "";
try{
zValOrFax =
x.getItem(znodoview,zsubsesion,znodoview,"","STD_OR_PHONE_FAX");
}catch(Exception e) {}
zValOrFax=oFmt.outFormat(oSess, zValOrFax,
com.meta4.format.M4Format.NUMBER);
%>
```

- **Tratamiento de datos vía JavaScript:** JavaScript requiere un formato fijo en sus operaciones. Por tanto es necesario transformar los datos con el formato del usuario al formato JavaScript y volver a transformarlos al formato del usuario para mostrarlos. Para ello se utilizan las funciones genéricas de JavaScript de Meta4 (m4gen.js).
- **Trabajar con datos en un Meta4Object que vienen de una página JSP:** Se requiere la transformación de los datos de formato usuario al formato de base de datos. Esto se realiza a través de métodos genéricos de Meta4Object utilizados para formatear.

Fechas

Puede seleccionar tanto el formato de fecha como el separador de fecha con el que va a trabajar.

Los formatos de fecha posible son:

- MM-dd-yyyy
- MM-yyyy-dd
- dd-MM-yyyy
- dd-yyyy-MM
- yyyy-MM-dd
- yyyy-MM-dd

Los posibles separadores de fecha son el punto (.), el guión (-) y la barra diagonal (/).

Existen tres operaciones de tratamiento de datos en los que se deben tener en cuenta los formatos:

-
- **Mostrar datos en página que provienen de un Meta4Object:**
 - Con el *tag* <m4:item> el formateo es automático.
 - Si usa la *m4operation* es necesario formatear como en el ejemplo:

EJEMPLO

```
<%
M4SessionManager oSess = (new M4Context()).getSession(request);
com.meta4.format.M4Format oFmt = oSess.getM4Format();
String zValDate = "";
try{
zValDate =
x.getItem(znodoview, zsubsesion, znodoview, "", "STD_DATE");
}catch(Exception e) {}
if (zValDate.equals("4000-01-01 00:00:00")){ zValDate
="" };else{zlcoNode12_3 = oFmt.outFormat(oSess, zValDate,
com.meta4.format.M4Format.DATE);}
%>
```

Tratamiento de datos vía JavaScript: se utilizan las funciones genéricas de la librería Fechas/Horas (m4gen.js) descritas en el *Apéndice: Librerías API Javascript*.

- para trabajar con las fechas. Se tienen automáticamente los formatos.
- **Trabajar con datos en un Meta4Object que vienen de una página JSP:**
Es necesario utilizar el método SHCO_GET_DATE que está en la estructura de nodo SHCO_GN_TOOLS para transformar los datos con formato de usuario a una fecha con formato de base de datos. Este proceso es automático.

Números y monedas

Puede elegir el formato de números y monedas con el que quiere trabajar. Estos formatos tienen en común el separador de decimal y de dígito.



El separador de dígitos es optativo para ambos tipos, y pueden o no tenerlo. Si lo tienen será el mismo.

Separador decimal	.	Separador de miles	,
Visualización de los números	123456		
Separador de miles	☐	Número de decimales	1
Visualización de las monedas	1234560000		
Separador de miles	☒	Número de decimales	1
		Número de decimales fijos	9

Los separadores de miles y de decimales pueden ser el punto (.) y la coma (,). Además puede parametrizar si desea o no tener el separador, el número de decimales posibles y el número de decimales fijos que serán rellenos por ceros si el número no tiene ningún decimal.

En el caso de las monedas, el valor del **Número de decimales fijos** puede verse reducido en función del número de decimales definidos para la moneda con la que se esté trabajando.

Existen tres operaciones de tratamiento de datos en los que se deben tener en cuenta los formatos:

- **Mostrar datos en página que provienen de un Meta4Object:**
 - Si utiliza el *tag* <m4:item> formatea los valores automáticamente según el formato del usuario.
 - Si utiliza la *m4operation* es necesario formatear después de obtener el valor.

EJEMPLO

```
<%
M4SessionManager oSess = (new M4Context()).getSession(request);
com.meta4.format.M4Format oFmt = oSess.getM4Format();
String zValNum = "";
String zValCurrency = "";
try{
zValNum = x.getItem(znodoview,zsubsesion,znodoview,"","STD_NUM")
zValCurrency =
x.getItem(znodoview,zsubsesion,znodoview,"","STD_CURRENCY");
```

```
}catch(Exception e) {}  
zValNum = oFmt.outFormat(oSess, zValNum,  
com.meta4.format.M4Format.NUMBER);  
zValCurrency = oFmt.outFormat(oSess, zValCurrency,  
com.meta4.format.M4Format.CURRENCY);  
%>
```

- **Tratamiento de datos vía JavaScript:** sabiendo que las operaciones en Javascript no permiten separador de dígitos y que el separador de decimal debe ser un punto, para trabajar con datos numéricos en la aplicación se deben tener en cuenta los siguientes puntos:
 - En general, el uso de las funciones estándar, *parseInt* y *parseFloat*, fallará siempre que se utilice sobre los valores que no vengan de operaciones javascript y, por tanto, puedan estar formateados. Para estos casos use las funciones equivalentes *m4parseInt* y *m4parseFloat* parametrizadas adecuadamente descritas en el *Apéndice: Librerías API Javascript*.
 - Todos los valores devueltos por esas funciones deben ser formateados con los formatos del usuario antes de mostrarse en pantalla. Para ello utiliza la función *m4formatNum* descrita en el *Apéndice: Librerías API Javascript*.
 - Toda operación que utilice un *m4parseInt*/*m4parseFloat* sobre un valor que no sea Javascript, es decir, que tenga aplicados los formatos, debe ser indicado por parámetro.

Para más información sobre estas funciones consulte *Números (m4gen.js)* del *Apéndice: Librerías API Javascript*.

Existen otras funciones de ayuda a este tipo de operaciones:
m4changeDecimalSepToPoint, *m4changePointToDecimalSep*,
m4takeoutsepdig descritas en el mismo apéndice.

- **Trabajar con datos en un *Meta4Object* que vienen de una página JSP:** automáticamente, el método **SHCO_GET_NUM**, tanto para el caso de los números como en el caso de las monedas, que están en la estructura de nodo **SHCO_GN_TOOLS**, transforma los datos con formato de usuario a un número o una moneda con formato de base de datos.

Validaciones de código JavaScript

En cliente web, las páginas disponen de la tecnología necesaria para editar códigos y su correspondiente traducción. Esto se realiza mediante una ventana de lista en la que se escoge un registro. Con los datos de este registro se rellenan tanto el código como la traducción.

El objetivo de este apartado es explicar cómo validar el código después de editarlo. En principio, si el código es válido la traducción se rellena de forma automática, y si no lo es, se abre la ventana de lista correspondiente.

Los datos de las listas con los códigos y las traducciones están en formato XML.



El esquema de XML es el mismo que se utiliza para el servidor de aplicaciones. De este modo, la carga de proceso del servidor web será mínima.

Para recoger las listas con los datos y analizarlas se utilizan funciones Javascript que realizan las peticiones *http* y analizan los archivos en formato XML.

Los datos de las listas que se utilizan para traducir los códigos se almacenarán en los mecanismos de caché del navegador.

Implementación de traducciones de código

El objetivo es describir cómo implementar manualmente las traducciones de códigos en las actuales páginas JSP. La validación se puede realizar también de otros modos:

- El robot M4robotjsp2k2 permite insertar automáticamente las validaciones de datos en páginas JSP desarrolladas con anterioridad al SP3 tecnológico. Más información, Incluir validaciones de datos en las cajas de texto del capítulo Operaciones automáticas sobre páginas JSP: Robot M4robotjsp2k2.
- A partir de el SP3 la funcionalidad se ha implementado directamente en las Herramientas de desarrollo | Herramientas de cliente ligero. Más información, Introducir la información de la página JSP del capítulo Generación automática de páginas del cliente web.

Página de introducción de código

Para realizar la traducción de códigos es necesario disponer del Meta4Object utilizado para listar.

Igual que se crea una página para las listas, es necesaria una página para traducir códigos: una página por cada campo a traducir.

Para ello puede utilizar el siguiente ejemplo como base para sus páginas:

EJEMPLO

Una página para traducir códigos de países es del estilo:

```
<%@ page import="com.meta4.validate.XMLValidate"%>
<%
    // Response Headers...
    //response.setDateHeader("Expires", -1);
    response.setHeader("Cache-Control", "max-age=3600,public");
    XMLValidate valobject = new XMLValidate();
    valobject.setM4object("SHCO_TK_MT_COUNTRY");
    valobject.setNodeData("SHCO_TK_MT_COUNTRY");
    valobject.setNodeRoot("SHCO_GN_ROOT");
    valobject.setInputFields(new String[]{"STD_ID_COUNTRY"});
    valobject.setInputTypes(new String[]{"4"}); // "4" cadenas,
    "10" fechas y "11" números
    valobject.setOutputFields(new String[]{"STD_ID_COUNTRY",
    "STD_N_COUNTRY"});
    valobject.process(request, response);
%>
```

La primera línea indica al navegador como tratar con esta página cuando la guarde en su caché.

La línea *response.setDateHeader("Expires", -1)* indica, al navegador o *proxy*, que no guarde la página en la caché y, por tanto, se pide de nuevo al servidor cada vez que el usuario solicite una traducción.

La línea *response.setHeader("Cache-Control", "max-age=3600,public")* indica, al navegador o *proxy*, que guarde la página en la caché durante una hora a menos que no la refresque explícitamente. Si pasado ese periodo realiza una traducción la página se solicita de nuevo al servidor y no se toma de la caché.

Los métodos que invocan al objeto de tipo XMLValidate tienen el siguiente significado:

- **setM4Object**: identificador del Meta4Object encargado de la traducción de códigos. Es el mismo que el encargado de listar valores.
- **setNodeData**: identificador del nodo donde se carga la lista de valores.
- **setNodeRoot**: identificador del nodo raíz donde se busca el método de filtrado y carga del Meta4Object.
- **setInputFields**: lista de los campos de entrada, los campos por los que se filtrará.
- **setInputTypes**: lista con los tipos correspondientes a los campos de entrada; "4" para cadenas, "10" para fechas y "11" para números.
- **setOutputFields**: lista de los campos de salida, los campos de retorno que se mostrarán en la página que invoca la traducción de códigos.

Invocación de la traducción de códigos

- **Inclusión de funciones javascript**: toda página que invoque a las traducciones de códigos utilizando esta tecnología debe contener la siguiente línea después del tag <body> que incluye las funciones necesarias para realizar la traducción de códigos:

```
<script type="text/javascript" src="/library/m4valdata.js"></script>
```

- **Invocación de la traducción**: existen dos funciones con los mismos parámetros y mismo funcionamiento: *m4translatelist* y *m4translatepick*.

La diferencia estriba en que *m4translatelist* trae a la máquina cliente la lista con todos los valores. La página se cachea en cliente y no es necesario hacer una nueva transacción para realizar otra traducción. Esta es la llamada indicada si la lista de valores no es muy grande.

La función *m4translatepick* sólo trae el valor que se quiere traducir y por tanto cada nueva traducción implica una nueva transacción. Esta llamada es indicada si la lista de valores es grande.

El prototipo de las funciones es el siguiente:

```
m4translatelist o m4translatepick (
    <página_de_traducción>,
    <página_de_lista>,
    new Array(<lista_de_campos_de_entrada>),
    new Array(<lista_de_campos_de_salida>)
);
```

Donde los parámetros son:

- **Página de traducción**: la página creada previamente para la traducción de códigos.
- **Página de lista**.

-
- Lista de campos de entrada: la lista de los campos por los que vamos a filtrar.
 - Lista de campos de salida: la lista de los campos de retorno.

El momento de invocar esta función suele ser en el evento *onblur* de la caja que queremos traducir.

EJEMPLO

Una llamada que invoca a la traducción de países, tal y como la hemos definido en el apartado anterior, sería del estilo:

```
<input id="%=zFieldX_Std_id_country%"
onblur="m4translatelist('shco_py/shco_tk_xml_country.jsp',
'shco_py/shco_tk_list_country.jsp' , new
Array('%=zFieldX_Std_id_country%'), new
Array('%=zFieldX_Std_id_country%', '%=zFieldX_Std_n_country%'))
;"
value="%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(
zParam_Std_id_country)%">/>
```

Refresco de la lista de los valores de traducción

La lista de valores es cacheada en la máquina cliente con la suposición de que esta lista de valores no va a tener muchas modificaciones. No obstante, es imprescindible disponer de un mecanismo que permita refrescar esta lista desde el cliente para disponer de una lista con los valores actualizados.

Para refrescar la lista de traducciones se aprovecha el momento de refresco de la lista asociada, es decir, cuando pulse F5 en su navegador mientras visualiza la página de lista asociada.

Para realizar esta acción incluya la siguiente línea en la página de lista:

```
<iframe style="border: none; width: 0px; height: 0px;"
src="<página_de_traducción>"></iframe>
```


Acerca de este capítulo

El cliente Web incorpora una serie de plantillas que resuelven de forma común el desarrollo (principalmente páginas JSP) de funcionalidades básicas como:

- Mantenimiento básico de datos
- Asistentes (*wizard*): un conjunto de pasos guiados para completar una funcionalidad.
- Asistentes de procesos
- Páginas para mostrar información resultante de consultas
- Tablas de valores
- Filtros

En cada plantilla se define:

- Su funcionalidad y objetivo principal
- Estructura y características básicas de los Meta4Objects implicados
- Definición de la página jsp asociada

Este capítulo describe cada una de estas plantillas así como su funcionalidad y estructura básica. Asimismo, detalla, por cada plantilla, los pasos a seguir para crear páginas JSP de manera manual.

Actualmente las herramientas de Cliente Web para Mantenimiento y listas, y Asistentes automatizan la generación automática de páginas JSP de tipo Lista y Mantenimiento, así como Asistentes. Véase el capítulo 3 para mayor información acerca de estas opciones.

Plantilla de mantenimiento

Esta plantilla es válida para resolver el desarrollo relacionado con tareas de mantenimiento de la información. Así, se describen los procesos para desarrollar:

- Tablas de mantenimiento
- Listas
- Listas multiselección*
- Filtros tipo lista



Las listas multiselección es el único tipo de página de mantenimiento que actualmente no puede generarse automáticamente a través de las herramientas de Cliente Web, Mantenimiento y Listas descritas en el capítulo 3.

A continuación se detallan las características básicas de cada uno de ellos.

Tablas de mantenimiento

Estas tablas tienen como objetivo el mantenimiento ya sea a nivel de inserción, actualización o borrado de la información. Un claro ejemplo son las tablas de referencia.

Figura 7. Tabla de mantenimiento: contiene un sector de entrada de datos y un sector de lista

The screenshot shows a web interface for maintaining civil status types. At the top, there is a breadcrumb trail: Inicio | Tablas de la persona | Tablas médicas | Tablas del CV | Tablas del periodo | Tablas rol. Below this is the title 'MANTENIMIENTO DE TIPOS DE ESTADO CIVIL' with a help icon. A 'Nuevo registro' (New record) form is present, with fields for '* Id. Estado civil' (containing a dropdown menu) and '* Estado civil' (containing a text input field). Below the form is a table with the following data:

Id. Estado civil	Estado civil	
01	Soltero/a	
02	Casado/a	
03	Viudo/a	
04	Divorciado/a	
05	Pareja de hecho	
06	Separado/a	

At the bottom of the table, it indicates '1 - 6' and '1-6 de 6'.

Estas tablas permiten:

- Insertar, actualizar y borrar información.
- Navegar mediante remotes y listas
- Ordenar la información por campos
- Pagar la información

Listas

Las listas son aquellas páginas que facilitan la selección de datos con el objetivo de cumplimentar campos. Se despliegan normalmente desde los botones de lista asociados a los campos.

Las listas pueden configurarse para que al invocarlas inicialmente vengan sin datos. También se pueden mantener en el caché automáticamente para los sucesivos accesos a la información dentro de la sesión actual de la aplicación.

Figura 8. Ejemplo de lista: contiene un sector de filtro y un sector de lista

MOTIVOS DE CAMBIO

Filtro avanzado

Id. Motivo cambio Empieza por

Id. Motivo cambio Empieza por

1 2 3 4 5 6 7 8

Id. Motivo cambio	Motivo cambio
1	TRASLADO
2	PROMOCIÓN
3	REORGANIZACIÓN
4	CONTRATADO
5	DEFUNCIÓN
6	NUEVA ENTRADA
7	CANCELACIÓN
8	Despido

1 - 8

Estas listas permiten:

- Filtrar la información que aparece en la parte inferior de la página utilizando hasta dos condiciones. Estas condiciones se establecen con los campos que se parametricen. Las opciones de filtrado se pueden multiplicar si en el Meta4Object correspondiente se activa el enlace **Filtro avanzado**.
- Controlar automáticamente los tipos de campos.
- Facilitar el acceso a la inserción de nuevos valores si lo requiere la funcionalidad asociada.
- Navegar mediante la selección de registros.
- Ordenar la información por campos

- Pagar la información

Listas multiselección

Las listas multiselección son un tipo especial de lista que devuelve n registros en lugar de uno como las listas genéricas.

Figura 9. Ejemplo de lista multiselección (Nómina, Valores empresa)

EMPRESAS

Filtro avanzado

Marcar todos

Desmarcar todos

Aceptar

	Id. Empresa	Empresa	Inicio	Fin	
☐	rsu	wrtuy	27-06-2002		
☐	testee	testee	10-11-1999		
☐	tets	test	28-10-1999		
☐	YYY	YYY	10-11-1999	27-06-2002	
☐	0001	Technology for Medicine S.A.	01-02-1990		
☐	0002	prueba	01-01-1800		
1 - 6					

Estas listas permiten además de las funciones de las listas genéricas:

- Seleccionar uno o varios registros a través de los casillas de verificación
- Seleccionar todos los registros en una sola operación con el botón **Marcar todos**.
- Anular la selección de los registros en una sola operación con el botón **Desmarcar todos**.



A pesar de poderse generar automáticamente las páginas de tipo Lista, véase capítulo 3, este tipo de lista no está actualmente disponible.

Filtros tipo lista

Los filtros tipo lista son un tipo especial de lista que actúan como filtros y no como medio de facilitar la selección e inserción de la información.

Figura 10. Ejemplo de Filtros tipo lista

EMPRESAS

Filtro avanzado

Id. Empresa

Empieza por

Id. Empresa

Empieza por



 Id. Empresa	 Empresa	 Inicio	 Fin	1-6 de 6
rsu	wrtuy	27-06-2002		
testeee	testeee	10-11-1999		
tets	test	28-10-1999		
YYY	YYY	10-11-1999	27-06-2002	
0001	Technology for Medicine S.A.	01-02-1990		
0002	prueba	01-01-1800		
1 - 6				

Los filtros tipo lista permiten:

- Filtrar la información que aparece en la parte inferior de la página utilizando hasta dos condiciones. Estas condiciones se establecen con los campos que se parametricen.
- Navegar mediante la selección de registros.
- Ordenar la información por campos
- Pagar la información

Meta4Objects

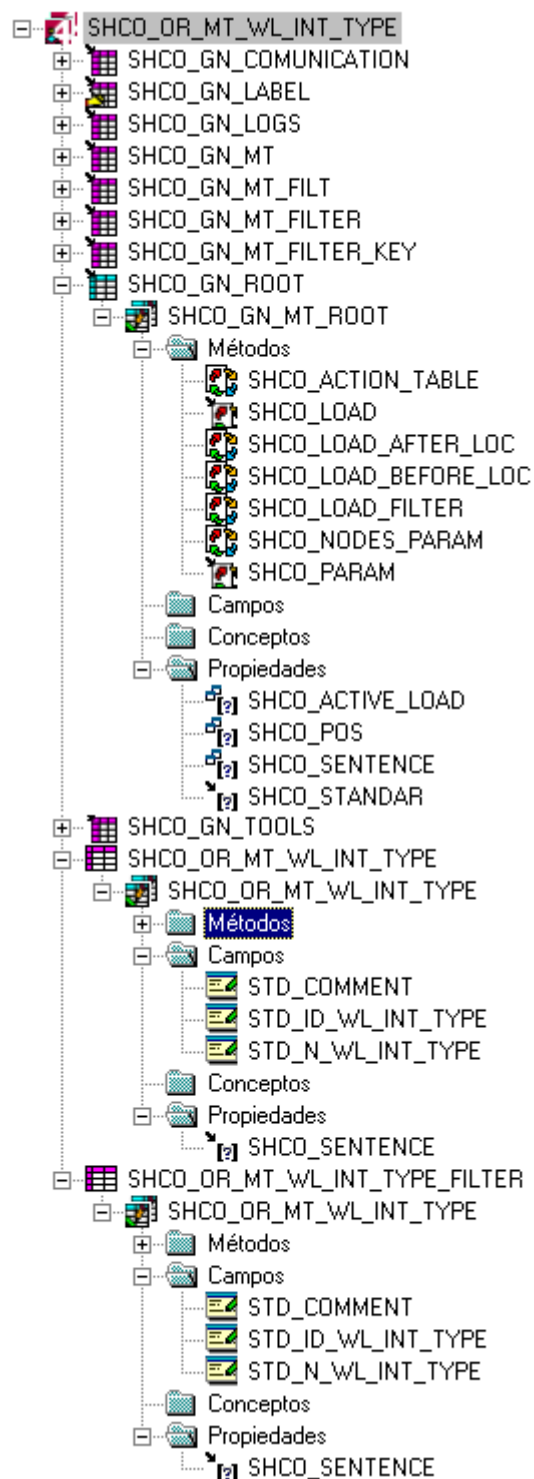
El Meta4Object asociado a esta plantilla es el SHCO_GN_MT. Todos los Meta4Objects que se creen heredarán de éste.

El proceso de generación del Meta4Object será más o menos complejo en función de si se incluyen en la tabla remotes y listas asociadas. Así se explica:

- La generación de un Meta4Object sin FK: todos los campos utilizados pertenecen a la misma tabla.
- La generación de un Meta4Object con FK: los campos utilizados pertenecen a varias tablas.
- La activación de un Meta4Object en el caché para poder acceder a los datos contenidos frecuentemente sin recurrir al servidor de aplicaciones. Típicamente, se activa los Meta4Objects del tipo Lista, sobre todo si contienen información relativamente estática.

A continuación, se muestra el Meta4Object que servirá como ejemplo en la explicación del proceso de creación de Meta4Objects de mantenimiento. Se ha utilizado el Meta4Object que resuelve la tabla de mantenimiento Tipo interno de lugar de trabajo.

Figura 11. Ejemplo de Meta4Object de mantenimiento



Meta4Object sin FK

Se propone una alternativa rápida y eficaz al proceso manual que se describe a continuación mediante la opción Mantenimiento y Listas. Esta opción contiene un asistente que le guía paso a paso en el proceso de generación automática. Para mayor información véase el capítulo 3 del presente manual.

Los pasos a seguir para crear un Meta4Object son:

1. Crear un nuevo Meta4Object que herede del Meta4Object de esta plantilla: SHCO_GN_MT. En el ejemplo: SHCO_OR_MT_WL_INT_TYPE.
2. Añadir un nuevo nodo libre, y modificar la propiedad de máximo número de registros de base de datos dándole un valor de 300 registros.

El ID del nodo libre, por defecto, debería ser igual al del Meta4Object. En el ejemplo SHCO_OR_MT_WL_INT_TYPE. Se hereda para este nodo la estructura de nodo SHCO_GN_MT.

Se añade una TI en el nodo. En esta sentencia se leen todos los campos de la tabla. Es importante reseñar que es obligatorio que la sentencia tenga una ordenación funcional.

3. Añadir un nuevo nodo libre. Por defecto, el ID será igual al del Meta4Object + "_FILTER". En el ejemplo: SHCO_OR_MT_WL_INT_TYPE_FILTER. Este nodo usará la misma TI que el nodo anterior (en el ejemplo SHCO_OR_MT_WL_INT_TYPE).

Si los nombres de los nodos, tanto en este paso como en el anterior, no son los que se indican por defecto, es necesario sobrescribir el método SHCO_NODES_PARAM. Para ello se sobrescribe el nodo SHCO_GN_ROOT e indicamos los nombres correctos:

```
If MsParam = "NODE" Then
    MvValor = "NUEVO_NOMBRE_NODO"
ElseIf MsParam = "FILTER_NODE" Then
    MvValor = "NUEVO_NOMBRE_NODO_FILTRO"
EndIf
Return(MvValor)
```

4. Sobreescibir el nodo SHCO_GN_LABEL y crear dos nuevas propiedades de ámbito bloque:
 - SHCO_LB_TITLE: etiqueta para el título de la lista
 - SHCO_LB_TITLE_ROOT: etiqueta para el título de las mt
5. En el caso de necesitar listas, acudir a la opción de menú **Configuración funcional de cliente ligero | Parametrización de filtros** y rellenar la tabla M4SCO_FILTER con los campos por los que se desea filtrar en la página jsp de lista. Los nombres de los ítems asociados en la TI será lo que se muestre en la caja de filtro. En el ejemplo:

```

SCO_ID_M4OBJECT= SHCO_OR_MT_WL_INT_TYPE
SCO_ID_FIELD= STD_ID_WL_INT_TYPE
SCO_ID_TYPE      = 0 (para String)
                  = 1 (para numéricos)
                  = 2 ( para fechas)
SCO_ID_TABLE = STD_LU_WL_INTERNAL_TYPE
SCO_ID_ORDEN= 0 (Orden de aparición de los campos en el filtro de
la lista).

```

Si en la lista es necesario incluir el enlace **Filtro avanzado** que permita al usuario establecer una sentencia más refinada existen dos opciones:

- En el nodo libre cuyo ID es igual al del Meta4Object, en el ejemplo SHCO_OR_MT_WL_INT_TYPE, indicar en sus propiedades avanzadas el valor Sí para Filtro dinámico.
- En el nodo libre cuyo ID es igual al del Meta4Object, en el ejemplo SHCO_OR_MT_WL_INT_TYPE, añadir una propiedad con las siguientes características:
 - Ámbito bloque
 - En Avanzado | Propiedades lógicas su ID Tipo Interno debe ser igual a 34.

Meta4Object con FK

Los pasos a seguir para crear un Meta4Object que incorpore campos de varias tablas son:

1. Se realizan todos los pasos del apartado anterior: Meta4Object sin remotes y listas.
2. Añadir el *join* en la sentencia de la TI del nodo libre que se creó en el apartado anterior (SHCO_OR_MT_WL_INT_TYPE) para mostrar el nombre del campo de remonte (FK). Normalmente al haber añadido el campo con el Editor de sentencias de estructuras de nodo del Diseñador de Meta4Objects, la sentencia se completa automáticamente.

En nuestro ejemplo, se quiere dar la posibilidad de filtrar por el nombre del tipo interno de lugar de trabajo. Se escribiría la siguiente sentencia en la estructura de nodo:

```

SELECT @STD_ID_WL_TYPE = A.STD_ID_WL_TYPE, @STD_N_WL_TYPE =
A.STD_N_WL_TYPE, @STD_ID_WL_INT_TYPE = A.STD_ID_WL_INT_TYPE,
@STD_COMMENT = A.STD_COMMENT, @STD_N_WL_INT_TYPE =
B.STD_N_WL_INT_TYPE
FROM &STD_LU_WL_TYPE A, STD_LU_WL_INTERNAL_TYPE B
WHERE A.ID_ORGANIZATION = B.ID_ORGANIZATION(+) AND
A.STD_ID_WL_INT_TYPE = B.STD_ID_WL_INT_TYPE(+)
ORDER BY A.STD_ID_WL_TYPE ASC

```


3. Sobrecribir el método SHCO_PARAM. Para ello, se sobrecribe el nodo **SHCO_GN_ROOT**.

Añadir la condición de que si esa variable es el nombre del Meta4Object actual, igualamos MvValor (lo que devuelve el método) a un *string* que es el *from* y el *where* que copiamos de nuestra estructura de nodo. Esto solo es necesario cuando pretendemos filtrar por los campos de la otra tabla.

```
If MsParam = "SHCO_OR_MT_WL_TYPE" Then MvValor = "FROM  
&STD_LU_WL_TYPE A, STD_LU_WL_INTERNAL_TYPE B WHERE  
A.ID_ORGANIZATION = B.ID_ORGANIZATION(+) AND A.STD_ID_WL_INT_TYPE  
= B.STD_ID_WL_INT_TYPE(+)"
```

Para cada campo por el que queremos filtrar que no este en la tabla base hacemos lo mismo, igualando MvValor al alias que tiene su tabla en nuestra TI:

```
ElseIf MsParam = "STD_N_WL_INT_TYPE" Then  
MvValor = "B"
```



Recordar que esto sólo es necesario si vamos a filtrar por campos de más de una tabla. En el caso de que se inserten registros en esta tabla por otro procedimiento que no sea una página de Mantenimiento se tiene que sobrecribir el método SHCO_PARAM y modificar:

```
If MsParam = "TR" Then  
MvValor = "0"  
EndIf  
Por  
If MsParam = "TR" Then  
MvValor = "1"  
EndIf
```

Un ejemplo de este caso es cuando son necesarias unas listas pero para éstas no hay un mantenimiento desde cliente Web. Ocurre mucho en el módulo Organización.

Si genera una lista en la que es necesario incluir el enlace **Filtro avanzado** que permita al usuario establecer una sentencia más refinada siga los pasos indicados en el apartado Meta4Object sin FK .

Configuración de la carga inicial de la lista

Ya que las listas se pueden configurar para que al invocarlas inicialmente vengan sin datos, los pasos a seguir son:

1. Desde el Diseñador de Meta4Objects, abra el Meta4Object Lista a modificar.

2. Localice el método SHCO_PARAM y edite su regla, sobrescribiendo si fuera necesario.
3. Localice en la regla el parámetro FIRST_LOAD y modifique su valor, cambiándole a 0:

EJEMPLO

Sustituya lo siguiente:

```
ElseIf MsParam = "FIRST_LOAD" Then
    MvValor = "1"
EndIf
```

Por:

```
ElseIf MsParam = "FIRST_LOAD" Then
    MvValor = "0"
EndIf
```



Recuerde que habrá que sobrescribir el nodo correspondiente si no se ha hecho anteriormente.

Activación del Meta4Object en el caché

Los pasos a seguir para cachear un Meta4Object son:

1. Desde el Diseñador de Meta4Objects, abra el Meta4Object a modificar.
2. Modifique el valor de la propiedad Avanzada del Meta4Object llamado **Guardar en caché** a Sí.

Páginas JSP

La generación de páginas JSP varía en función de los elementos que se incorporen a ella:

- Las páginas más simples son las que están asociadas a un MT sin claves primarias, porque significan que no habrán ni remotes ni listas asociadas.
- Las páginas con remotes y listas asociadas se encuentran en las del tipo Tablas de mantenimiento y Lista típicamente.
- Una vez creadas estas páginas relacionadas con el mantenimiento de datos, hay que considerar cómo se llaman desde las páginas JSP que las utilizan.

-
- Además existe una serie de consideraciones a tomar en cuenta cuando se edita un registro y algunos campos se quieren deshabilitar.

Para explicaciones detalladas mediante unos ejemplos, véase el *Apéndice: Código de páginas JSP* y concretamente los siguientes apartados:

- Páginas de mantenimiento
- Páginas de listas y de filtros tipo lista
- Mantenimiento de N registros
- Página lista multiselección

Plantilla para asistentes: *wizard*

Plantilla válida para todos aquellos procesos guiados a través de varios pasos. En particular se utiliza para todos los procesos de mantenimiento anidados (MT's de varios nodos).

Figura 12. Ejemplo de *wizard*

Un *wizard* divide la pantalla en dos partes bien diferenciadas, una, a la izquierda, con todos los pasos del proceso, y la otra, centrada, con la funcionalidad de cada paso.

Un *wizard* está provisto de una navegación guiada por todo el proceso funcional que se desea realizar. La navegación puede ser:

- secuencial: paso tras paso, desde el principio hasta el fin o
- directa a un paso en concreto.

Los casos funcionales soportados por la plantilla son los siguientes:

- Temporalidad respecto a la manera de actualizar la información en BBDD:
 - Proceso atemporal: cualquier acción realizada se actualiza directamente sobre BBDD (*Insert*, *Update*, *Delete*). En particular, el mantenimiento de MT's de varios nodos son siempre atemporales.
 - Proceso temporal: cualquier acción realizada se actualiza sobre una tabla temporal. El usuario debe decidir si el proceso queda finalizado o se cancela. Esto permite mantener abierto un proceso durante varias sesiones del usuario. En particular, los procesos guiados con un gran número de pasos o de información serán temporales, por ejemplo, el alta o la petición de vacantes.
- Obligatoriedad: se refiere a la existencia de pasos obligatorios dentro de la funcionalidad. Afecta a la navegación entre pasos del *wizard*.

-
- No existen obligatorios. La navegación es libre, es decir, se puede realizar de manera secuencial o directa sin orden obligatorio alguno.
 - Pasos obligatorios ordenados. La navegación no es libre, hay que pasar por los pasos obligatorios primero. Además, entre los obligatorios también se establece un orden que se tiene que seguir para poder continuar.

Lo habitual es que un paso prioritario se encargue de cargar datos en el resto de los pasos del wizard.

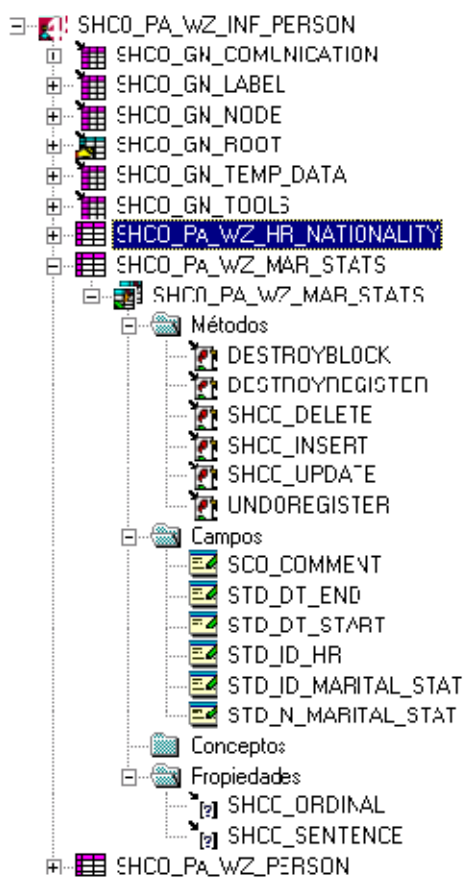
- Limpieza de datos. El botón **Limpiar** permite eliminar los datos introducidos por el usuario en un paso y dejarlos en blanco. Si al hacer clic en el botón **Limpiar** desea conservar los valores por defecto para aquellos campos que tengan esta característica vea el apartado Modificar el comportamiento del botón limpiar en los wizard.

Meta4Objects

El Meta4Object asociado a esta plantilla es el SHCO_GN_WIZ. Todos los Meta4Objects que se creen heredarán de éste.

Tomaremos como ejemplo la creación de un Meta4Object válido para el mantenimiento de datos de la persona cuyo wizard se muestra en la figura anterior.

Figura 13. Estructura del Meta4Object para el wizard Mantenimiento de la persona



Los pasos a seguir para crear un Meta4Object válido para un asistente son:

1. Crear un nuevo Meta4Object que herede del Meta4Object de plantilla de asistentes (SHCO_GN_WIZ). En el ejemplo, **SHCO_PA_WZ_INF_PERSON**.
2. Crear un nuevo nodo libre para cada uno de los pasos del asistente:
 1. Modificar la propiedad de máximo número de registros a 300.
 2. Crear una estructura de nodo con el mismo identificador que el nodo. En el ejemplo: **SHCO_PA_WZ_MAR_STATS**.
 3. Sobre la nueva estructura de nodo, heredar la estructura de nodo SHCO_GN_NODE. Esta acción nos proporciona los métodos y propiedades requeridos por paso.
 4. Sobrecribir el nodo y añadir los items (campos, métodos, etcétera) necesarios.
 5. Guardar el Meta4Object.
3. Parametrizar el Meta4Object. Existen dos niveles de parametrización.

- Parametrización global

1. En el método SHCO_PARAM del nodo **SHCO_GN_ROOT** se parametrizan datos que afectan a todo el proceso. Prototipo: valorParametro = SHCO_PARAM (ID Parámetro). El método SHCO_PARAM debe contener obligatoriamente en la regla asociada los siguientes parámetros por proceso:
 - TEMP_TABLE. Determina la temporalidad del proceso. Valor por defecto: No temporal. (No modificar los valores por defecto).
 - BUTTS_STATE. Parámetro requerido en un proceso temporal. El estado inicial y valor por defecto de los dos botones del proceso temporal es el siguiente:
 - Aceptar el proceso: deshabilitado
 - Cancelar el proceso: habilitado (valor por defecto es 1 de habilitado)Si el Wizard tiene pasos obligatorios el valor del botón Aceptar según este parámetro es habilitado, se habilitará o no en función de si cumple el requisito de obligatoriedad.
 - ID_TR: campos que vienen del filtro. Si son varios van separados por {
 - ID_TR_FIELD: Si el campo de la base de datos no es igual que el de ID_TR.

```
ElseIf MsParam = "ID_TR" Then
    MvValor="SHCO_ID_PERSON"
ElseIf MsParam = "ID_TR_FIELD" Then
    'Identificador del campo fisico del filtro (TR) de
    acceso al wizard.
```

MvValor="STD_ID_PERSON"

- ORDER_STEP: orden de visualización de los pasos. Se genera mediante la concatenación de los pasos separados entre #. Hay que tener en cuenta que el primer paso es siempre fijo.

EJEMPLO

MvValor = "1#3#2#"

En este caso, aparecerían 3 pasos en el siguiente orden:

- Primero el paso 1
- Segundo el paso 3
- Tercero el paso 2

Todos estos pasos deben encontrarse definidos en la fórmula del método SHCO_WZ_PARAM_STEP_LOC.

Si existen más pasos definidos, pero no se encuentran en el orden aquí descrito, no aparecerán en el *wizard*.

- RELOAD_STEPS: indica si cuando se refresca un paso, se vuelve a evaluar el resto de los pasos obligatorios (por posibles dependencias) o solo el actual. Por defecto, vale 0 (no se vuelve a evaluar).
- Parametrización local
 1. En el método SHCO_WZ_PARAM_STEP_LOC se define y parametriza cada uno de los pasos del asistente. Este método debe definir obligatoriamente los siguientes parámetros para cada paso:
 - OBLIGATORY_STEP: indica si un paso es obligatorio o no. Valor por defecto: No obligatorio.
 - VISUALIZATION_NODE: relaciona cada paso con un nodo de visualización.
 - JSP_LINK: indica el *path* asociado a este paso (directorio/pagina.jsp). El valor del parámetro se concatena internamente con "/servlet/CheckSecurity/JSP/".
 4. Los métodos modificables en el nodo raíz (**SHCO_GN_ROOT**) contienen el sufijo LOC. Para modificar estos métodos es necesario sobrescribir sus respectivas reglas.
 - SHCO_WZ_PARAM_STEP_LOC: definición de pasos.
 - SHCO_WZ_LOAD_BEFORE_LOC: acción previa a la carga de cada paso.
 - SHCO_WZ_LOAD_AFTER_LOC: acción posterior a la carga de cada paso.

-
- SHCO_WZ_CONTROL_OBLG_STEP_LOC: definición de la obligatoriedad de cada paso. Por defecto es que el nodo tenga al menos un registro.
 - SHCO_WZ_HEADER_LOC: método que contiene la primera operación que quiere que se ejecute nada más abrir un wizard. Es un método abierto y varía en función de la funcionalidad que se desarrolle.
5. Métodos modificables en cada paso (nodo):
- SHCO_UPDATE: acción previa a la inserción.
 - SHCO_INSERT: acción previa a la actualización.
 - SHCO_DELETE: acción previa a la inserción.
6. Sobreescribir el nodo SHCO_GN_LABEL y crear la propiedad de ámbito bloque:
- SHCO_LB_TITLE: etiqueta para el título de la lista

Páginas JSP

Duplicando la página más similar al caso a resolver se hacen los retoques necesarios. Dos casos comunes son las páginas JSP para:

- Mantenimiento de 1 registros
- Mantenimiento de n registros

Para explicaciones detalladas mediante unos ejemplos, Plantillas para asistentes: wizard en el *Apéndice: Código de páginas JSP* .

Plantilla del asistente de procesos

Un asistente de procesos recopila datos, por lo general a lo largo de una serie de pasos, y los utiliza para ejecutar un *proceso*. La plantilla existente permite, por ejemplo, la creación de procesos que recopilan los datos para ejecutar los informes, las combinaciones de correspondencia (*mail merge*) y la integración del filtro dinámico en el proceso.

El asistente de procesos cuenta con muchas de las características de los *wizards*, por ejemplo:

- Estructura de las páginas. A la izquierda de la página, se muestran los pasos como enlaces, mientras que, a la derecha, se introducen los datos en campos.
- Pasos obligatorios y no obligatorios. Los pasos de un proceso se pueden definir como obligatorios, de modo que sea necesaria la introducción de los datos para poder ejecutar el proceso final (es decir, un informe).
- Limpieza de datos en cada paso y restablecimiento de los valores por defecto.
- Parámetros globales y locales. La parametrización es necesaria tanto a nivel de proceso (global) como a nivel de paso (local).

No obstante, entre el *wizard* y el asistente de procesos, también existen las siguientes diferencias:

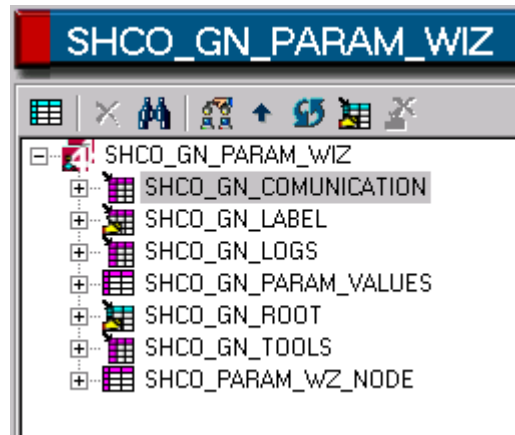
- Los datos no se guardan en las tablas permanentes.
- Al finalizar un asistente de procesos, se suele iniciar otro proceso.

En este apartado del capítulo se explica cómo crear un Meta4Object del asistente de procesos, y se incluyen ejemplos comentados del código de la página JSP para ayudarle a desarrollar las correspondientes páginas del cliente Web.

Creación de un nuevo Meta4Object del asistente de procesos

Cree un nuevo Meta4Object del asistente de procesos que esté heredado del Meta4Object de la plantilla **SHCO_GN_PARAM_WIZ**. Todos los Meta4Objects del asistente que se creen heredarán de éste.

Figura 14. Estructura interna del Meta4Object de la plantilla del asistente de procesos



Una vez heredado el Meta4Object nuevo del objeto de la plantilla para asistentes (**SHCO_GN_PARAM_WIZ**), lleve a cabo este proceso para personalizar el asistente nuevo. El proceso se divide en cuatro fases:

- Fase 1: Cree los nodos para los pasos del asistente.
- Fase 2: Configure los parámetros globales y locales.
- Fase 3: Edite los métodos SHCO_GN_ROOT.
- Fase 4: Configure el paso de los parámetros (opcional)

Fase 1: Cree los nodos para los pasos del asistente.

Debe seguir estos pasos para cada uno de los incluidos en su asistente.

1. Cree un nuevo nodo libre.
2. Cree una estructura de nodo con el mismo ID que el nodo.
3. En esta estructura de nodo, herede la estructura de nodo de la plantilla **SHCO_PARAM_WZ_NODE**. De este modo, se obtienen los métodos y las propiedades necesarias para cada paso del asistente.
4. Sobrescriba el nodo y agregue los elementos (campos, métodos, etc.) necesarios.

Los elementos que van a recibir los parámetros deben tener propiedades de ámbito bloque si se trabaja con un sólo registro/bloque (REG_NUMBER="1"). Si se trabaja con varios serán de ámbito registro (REG_NUMBER="N").

5. Guarde el Meta4Object.

- Sobrescriba el nodo **SHCO_GN_LABEL** y cree las propiedades en ámbito bloque, **SHCO_LB_TITLE** y **SHCO_LB_TITLE_ROOT**. De ellos cuelgan el título de la página que aparece en la barra de título del explorador y el nombre del asistente que se muestra en la página.

Fase 2: configure los parámetros globales y locales.

Hay dos niveles de configuración: global y local. Cada uno consta de una serie de parámetros que se describen en los apartados siguientes.

Parametrización global

En el método **SHCO_PARAM** se parametrizan datos que afectan a todo el proceso del *wizard*. Por ejemplo:

```
valorParametro = SHCO_PARAM (ID Parámetro)
```

El método **SHCO_PARAM** debe incluir los siguientes parámetros de proceso:

Parámetro	Finalidad
RET_PAGE	Redirección al finalizar el proceso.
TYPE_PAGE	Indica cómo finaliza el proceso: 0 : abre una página en otra ventana del explorador. 1 : abre una página en la misma ventana del explorador. 2 : inicia una combinación de correspondencia. En este caso, hay que configurar los parámetros ID_DATA_SOURCE y ID_TEMPLATE .
ID_DATA_SOURCE	Nombre del archivo de datos utilizado para la combinación de correspondencia. Este parámetro se utiliza solamente cuando TYPE_PAGE=2 .
ID_TEMPLATE	Nombre de la plantilla que se utiliza para la combinación de correspondencia. Este parámetro se utiliza solamente cuando TYPE_PAGE=2 .
BUTTS_STATE	Estado de los botones de activación del proceso. Por defecto, el botón Aceptar el proceso está desactivado (0) y el botón Cancelar el proceso está activado (1). Si el asistente tiene pasos obligatorios y hay un botón Aceptar el proceso, sólo se activará si se incluyen los datos obligatorios.
EXEC_METHOD	Método que se ejecuta al hacer clic en el botón Aplicar cuando finaliza el proceso.

Parámetro	Finalidad
ORDER_STEP	Orden en que se muestran los pasos. Para establecer el orden, haga una lista de los números de los pasos y sepárelos mediante el signo #. Por ejemplo: MvValor="3#2#" Esto significa que el paso 1 aparecerá en primer lugar, después el paso 3 y, finalmente, el paso 2. Nota importante: Estos pasos se deben haber definido previamente en el método SHCO_P_WZ_PARAM_STEP_LOC. Este parámetro debe configurar todos los pasos definidos en dicho método para que aparezcan en el asistente.

Parametrización local

La parametrización local afecta a un determinado paso del proceso. Por tanto, tendrá que realizar esta parametrización para cada uno de los pasos (de cada nodo) que haya definido en el asistente.

Los siguientes parámetros de parametrización local se encuentran en el método SHCO_P_WZ_PARAM_STEP_LOC.

Parámetro	Finalidad
OBLIGATORY_STEP	Indica si el paso es obligatorio o no. El valor por defecto de este parámetro es <i>No obligatorio</i> .
VISUALIZATION_NODE	Código identificativo del nodo de visualización asociado al paso.
LOAD	Indica si en este paso se cargan datos. Sólo se aplica a los pasos asociados a nodos de visualización.
JSP_LINK	Ruta de acceso asociada a este paso. Su expresión es: <i>directorio/página .jsp</i> El valor de este parámetro se concatena internamente con la ruta de acceso: <i>/servlet/CheckSecurity/JSP/</i>
REG_NUMBER	El valor de este parámetro indica el número de registros con los que se trabaja por paso. Por defecto tiene valor 1 para indicar que se trabaja con un sólo registro o bloque. El valor N indica que se trabajan con varios registros.

Fase 3: Modifique los métodos SHCO_GN_ROOT.

Los métodos modificables se identifican en el nodo raíz (**SHCO_GN_ROOT**) porque su código identificativo termina en **_LOC**. Para modificar estos métodos es necesario sobrescribir sus respectivas reglas.

Método	Finalidad
SHCO_P_WZ_PARAM_STEP_LOC	Definición de los pasos.
SHCO_P_WZ_LOAD_BEFORE_LOC	Acción que precede a la carga de cada paso.
SHCO_P_WZ_LOAD_AFTER_LOC	Acción que sigue a la carga de cada paso.
SHCO_P_WZ_CONTROL_OBLG_STEP_LOC	Definición de la obligatoriedad de cada paso. Por defecto es que el nodo tenga al menos un registro.

Fase 3: Edite el método del nodo SHCO_CONTROL_PARAM.

El método **SHCO_CONTROL_PARAM** aplica los controles que se llevan a cabo antes de aceptar los valores de los datos de un paso como posibles parámetros. Si este método devuelve el error *m4_error*, aparece la página de error. Utilice este método para definir los posibles errores mediante la creación de un *setlog* para cada error. Se debe editar en el nodo de cada paso del asistente.

Fase 4: Configure el paso de los parámetros.

Esta fase es opcional. Al finalizar el proceso del asistente, podrá ejecutar un proceso específico. En este apartado se incluyen las indicaciones necesarias para configurar el paso de los parámetros para generar un informe, llevar a cabo una combinación de correspondencia o integrar el filtro dinámico.

Sin embargo, éstas no son las únicas posibilidades. Puede utilizar los parámetros existentes para definir la ejecución de otros procesos.

Para generar un informe

1. Al sobrescribir el método SHCO_PARAM, defina los siguientes parámetros:
 - RET_PAGE: "shco_rp/shco_m4throw.jsp"
 - EXEC_METHOD: SHCO_EXEC_REPORT
2. Sobrescriba el método SHCO_PARAM_REPORT_LOC. Aquí se configuran los parámetros del informe. Consulte el capítulo *Ejecución y planificación de informes* de este manual.
3. Sobrescriba el método SHCO_PARAM_REPORT_MACHING_LOC. Aquí se configura el paso de los valores del Meta4Object al proceso de ejecución del informe.

Por ejemplo:

```
valorParametro = SHCO_PARAM_REPORT_MACHING_LOC
```

Otros valores posibles son:

- NUM_NODES: Número de nodos en el Meta4Object del informe que van a recibir parámetros.

Para cada nodo del informe que va a recibir parámetros:

- NODE_ID: Código identificativo del nodo en el informe.
- NUM_PARAM: Número de parámetros que se pasarán a este nodo.
- Para cada parámetro:

```
PARAM_REPORT | NODE_PARAMETERS | PARAM_PARAMETERS
```

Donde **PARAM_REPORT** es el ID del informe, **NODE_PARAMETERS** es el ID del nodo en el Meta4Object de parámetros, y **PARAM_PARAMETERS** es el ID del elemento del nodo.

Para ejecutar una combinación de correspondencia en Word

Para llevarlo a cabo, debe conocer el nombre de la plantilla que se va a usar y el nombre del archivo que contiene los datos.

1. Sobrescriba el método SHCO_PARAM con los valores siguientes:
 - TYPE_PAGE: Se debe establecer en **2**.
 - ID_DATA_SOURCE: Nombre del archivo del servidor en el que se guardan los datos.
 - ID_TEMPLATE: Nombre de la plantilla que se va a utilizar.

Por ejemplo:

```
TYPE_PAGE=2  
ID_DATA_SOURCE=DATA.TXT  
ID_TEMPLATE=TEMPLATE.DOC
```

2. La propiedad SHCO_LONG3 del nodo **SHCO_GN_COMUNICACION** indica la ruta de acceso en el servidor Web en la que se guardará el archivo de datos. Un método genera el archivo de datos en el servidor de aplicaciones y después transfiere el archivo al servidor Web.
3. Para ello, lo primero que hay que hacer es configurar la generación del archivo en el servidor de aplicaciones.

' Variables

```
sTempDir = "" ' Indique el archivo que se va a crear en el
directorio temporal sFileExt = ".txt" ' Indique la extensión del
archivo
```

```
' Genere el archivo en el directorio temporal del servidor de
aplicaciones
```

```
PROP_BLOB_FILE.SysCreateBlobFile(sTempDir, sFileExt)
```

```
' Recupere el nombre del archivo generado
```

```
sTempFileName = ToString(PROP_BLOB_FILE)
```

4. El método M4CopyFileGD del recurso M4DOC transfiere el archivo de datos desde el servidor de aplicaciones hasta el servidor Web. Este método tiene tres parámetros:

- Origen_AppServer: El valor de este parámetro devuelve la ejecución del informe.
- Destino_WebServer: Ubicación destino en el servidor Web. Se debe asignar un valor a PATH_TEMP_MAPPING (introducido por el usuario) y el nombre del archivo de datos.

Por ejemplo:

```
' Recupere el nombre del archivo generado
```

```
Origen_AppServer = sTempFileName
```

```
Destino_WebServer = Obtenga este valor de la propiedad SHCO_LONG3
en la estructura de nodo SHCO_GN_COMUNICACION.
```

- Sobrescriba el indicador.

5. Una vez indicados los lugares de origen y destino, se ejecuta el método M4CopyFileGD. A continuación, el archivo se borra del servidor de aplicaciones con el método DESTROYFILE(sTempFileName r). Si el archivo de texto se generó mediante un elemento *BLOB*, esto será suficiente para eliminar el elemento. En este caso, *asegúrese de que el elemento tiene ámbito bloque* y no ámbito nodo; de lo contrario, al eliminar el elemento se eliminaría también el nodo.

Para integrar el filtro dinámico en el proceso

Si quiere que en el botón de ejecutar levante el filtro dinámico siga estos pasos:

1. Sobrescriba el método SHCO_PARAM con los siguientes valores:
 - DYN_FILTER: con valor 0 si no hay filtro ó, 1 si hay filtro.
 - DYN_FILTER_M4OBJECT: Meta4Object sobre el que se va a aplicar el filtro.
 - DYN_FILTER_ACTION: indica qué acción realizará el filtro:
 - En el caso de los informes el valor es 1. Devuelve el filtro en formato *nodo item valor*.
 - En el resto de caso el valor es 3. Indica que el filtro se va a aplicar sobre el Meta4Object indicado en DYN_FILTER_M4OBJECT.

Integración del Planificador de tareas

Para llevar a cabo esta integración es necesario conocer la tarea a planificar y el valor de los parámetros que tenga asociados.

Después de ejecutar la invocación se abre el planificación de tareas, de la misma forma que si se realizara una nueva planificación desde la opción de menú pero con las siguientes diferencias:

- Aparecen los datos que se pasen por parámetro
- No se permite modificar:
 - La tarea planificada
 - Los valores de los parámetros de la tarea (Ocultos).
 - El rol de ejecución.

La selección de la sociedad/es de ejecución se puede bloquear (ver parámetros)

Invocación del planificador

Esta invocación en el caso de los asistentes de procesos, se hace desde la página configurada como página de retorno del asistente (Parámetro "RET_PAGE").

Los pasos a seguir para realizar la invocación al planificador son los siguientes:

1. Incluir la página: shco_gen_schedule_include.jsp
2. Asignar valores a los parámetros. (Ver siguiente apartado)

3. Si el modo de retorno del wizard es (retPagemode = 0), es decir con un window.open, y no se ve bien el planificador, se puede modificar el tamaño de la ventana usando la función javascript: window.resizeTo(800,600)
4. Invocación al planificador usando la función javascript m4JobScheduler(). Esta función se conecta al planificador y le pasa los parámetros necesarios

Parámetros de invocación

Tanto el identificador del formulario como el de los parámetros que contiene son variables java, con lo cual los valores deben usar la sintaxis '<%=.. %>'.
 Identificador del formulario: <%=zfrmscheduleparams%>

ID Parámetro	Descripción	Valores
<%=zsh_TaskGroupName%>	Nombre del grupo de tarea	<u>Defecto:</u> ""
<%=zsh_TaskGroupDesc%>	Descripción del grupo de tarea	<u>Defecto:</u> ""
<%=zsh_IdTask%>	Tarea de ejecución	<u>Defecto:</u> "" Obligatorio
<%=zsh_OrgList%>	Lista de sociedades para las que se planifica.	<u>Defecto:</u> "" <u>Sintaxis:</u> _(Indicar primero el número de sociedades entre asteriscos y luego las sociedades separadas por la barra del 7) *num_soc*001/..002 <u>Ejemplos:</u> *1*0001 *2*0001/0002
<%=zsh_OrgMultiselection%>	Pemitir o no seleccionar múltiples sociedades	<u>Valores:</u> "1": Pemitir multiselección resto: no pemitir <u>Defecto:</u> "1"
<%=zsh_OrgEditable%>	Pemitir o no modificar la/s sociedad/es	<u>Valores:</u> "1": Editable resto: no editable <u>Defecto:</u> "1"

<code><%=zsh_ParamNameList%></code>	Lista de los identificadores de los parámetros de la tarea <code><%=zsh_IdTask%></code>	Defecto: "" Obligatorio si la tarea tiene parámetros <u>Sintaxis:</u> (Separar por +. + los distintos valores) P1+. +. +. +Parn <u>Ejemplo:</u> PARAM1 PARAM1+. +. +PARAM2
<code><%=zsh_ParamValueList%></code>	Lista de los valores de los parámetros de la tarea <code><%=zsh_IdTask%></code> . Cada valor se asigna al parámetro que venga en la misma posición en la lista <code><%=zsh_ParamNameList%></code> .	Defecto: "" Obligatorio si la tarea tiene parámetros <u>Sintaxis:</u> (Separar por +. + los distintos valores) PVal1+. +. +. +PValn <u>Ejemplo:</u> VALUE1 VALUE1+. +. +VALUE2
<code><%=zsh_ret_page%></code>	Página de retorno tras la planificación	Lista de planificaciones

EJEMPLO

```

<%@ include file="/shco_g0/shco_gen_schedule_include.jsp" %>

script type="text/javascript">
m4valor(' <%=zfrmscheduleparams%> ', ' <%=zsh_TaskGroupName%> ', 'TASK_NAME', 'se
m4valor(' <%=zfrmscheduleparams%> ', ' <%=zsh_TaskGroupDesc%> ', 'TASK_DESC', 's
m4valor(' <%=zfrmscheduleparams%> ', ' <%=zsh_IdTask%> ', 'TASK1', 'set');
m4valor(' <%=zfrmscheduleparams%> ', ' <%=zsh_OrgList%> ', '1*0001', 'set');
m4valor(' <%=zfrmscheduleparams%> ', ' <%=zsh_ParamNameList%> ', 'PARAM1', 'set')
m4valor(' <%=zfrmscheduleparams%> ', ' <%=zsh_ParamValueList%> ', 'VALUE1', 'set')

m4valor(' <%=zfrmscheduleparams%> ', ' <%=zsh_ret_page%> ', 'shco_rp/shco_m4thro
.jsp?zretmode=js', 'set');

window.resizeTo(800,600);
m4JobScheduler();

</script>

```

Creación de páginas JSP

Para explicaciones detalladas mediante unos ejemplos, Plantilla del asistente de procesos en el *Apéndice: Código de páginas JSP*.

Plantilla de información

Esta plantilla es válida para desarrollar páginas JSP que muestran información resultante de consultas. Por ejemplo, para consultar información básica sobre una empresa se ha creado:

- Un Meta4Object que sustente dicha funcionalidad
- Una página JSP que muestre la información en cliente Web tal y como se aprecia en la figura.

Figura 15. Página de información



The screenshot shows a web application interface for 'Meta4 PeopleNet'. The header includes a navigation bar with links: Inicio | Organigramas | Información | Compensación salarial. Below the header, the main content area displays the company name '0001 - Technology for Medicine S.A.' and its start date 'Inicio: 01-02-1990'. The data is organized into two tables. The first table, 'Banco empresa asociado', lists associated banks with columns for 'Inicio - Fin', 'ID Banco empresa', 'Nombre banco empresa', 'Información Bancaria', and 'Banco por defecto'. The second table, 'Dirección empresa', lists company addresses with columns for 'Inicio - Fin' and 'Dirección completa'.

Banco empresa asociado				
Inicio - Fin	ID Banco empresa	Nombre banco empresa	Información Bancaria	Banco por defecto
01-02-1990 -	BANK_01	BANK_01	BANK_01 893453	No
01-02-1990 -	BANK_03	BANK_03	BANK_03 0065679	Si

Dirección empresa	
Inicio - Fin	Dirección completa
01-02-1990 -	Avenida de la Poblaciones Edificio Capital Plantas 22-23 España Madrid Madrid Madrid Norte
01-02-1990 -	72-76 rue Laugier Paris XVII France Paris Paris 17e 75017

Se toma este ejemplo como guía para describir los pasos que se siguen para crear el Meta4Object y página JSP correspondiente.

Meta4Objects

El Meta4Object asociado a esta plantilla es el SHCO_GN_ROOT. Este Meta4Object es el genérico y base que se utiliza en todos aquellos casos de desarrollo que no están contemplados con el resto de plantillas. Todos los Meta4Objects que se creen heredarán de éste.

Se utiliza como ejemplo la creación de un Meta4Object válido para visualizar información de Empresas en el módulo Organización.

Los pasos a seguir para crear un Meta4Object son:

1. Crear un nuevo Meta4Object de herencia (**SHCO_OR_LEG_ENT_F**), que herede del Meta4Object de esta plantilla SHCO_GN_ROOT.

2. Crear nodos funcionales. Para ello, se consulta el Meta4Object que usa la presentación Windows y se replica esta estructura.

Por ejemplo: un nodo padre, **SHCO_OR_LEG_ENT_F** que tiene dos nodos hijos: **SHCO_OR_LEG_ENT_ADDRESS** y **SHCO_OR_LEG_ENT_COMPBANK**.

Incluir los campos a utilizar y en su sentencia incluir filtro para mostrar sólo la información válida en la fecha contenida en SHCO_AP_DATE y opcionalmente una ordenación:

```
El parámetro SHCO_AP_DATE>=El campo SCO_DT_START de la tabla
SCO_COMPANY_BANK And ( El parámetro SHCO_AP_DATE <= El campo
SCO_DT_END de la tabla SCO_COMPANY_BANK Or El Campo SCO_DT_END de
la tabla SCO_COMPANY_BANK IS NULL)
```

Añadir en cada nodo:

- el método DESTROYBLOCK
- la propiedad SHCO_AP_DATE. Recoge la fecha de corte. Esta fecha de corte se calcula teniendo en cuenta los siguientes aclaraciones:
 - Para el nodo raíz, aquel que carga la información principal, no se filtrará por ninguna fecha. Por ejemplo, si se quiere mostrar la ficha de puestos de una persona, se mostrarán todos sus puestos independientemente de las fechas con las que se trabaja.
 - Para visualizar los datos correspondientes al histórico (puesto en el ejemplo) si que se utilizará una fecha de corte que se calcula llamando al SHCO_FILTER_DATE del nodo SHCO_GN_TOOLS. Este método recibe dos argumentos, una fecha de inicio y una fecha de fin. Si el nodo raíz tiene fechas se le pasan éstas, en caso contrario se pasarán vacíos y el método toma las del sistema. En el ejemplo el histórico de puestos tiene fecha de inicio y fin. La fecha de corte será:
 - Fecha de hoy si Hoy está comprendido entre las fechas de inicio y fin
 - Fecha fin si Hoy es mayor o igual que la fecha fin
 - Fecha inicio si Hoy es menor o igual que la fecha de inicio

Añadir en el nodo o nodos padre funcionales:

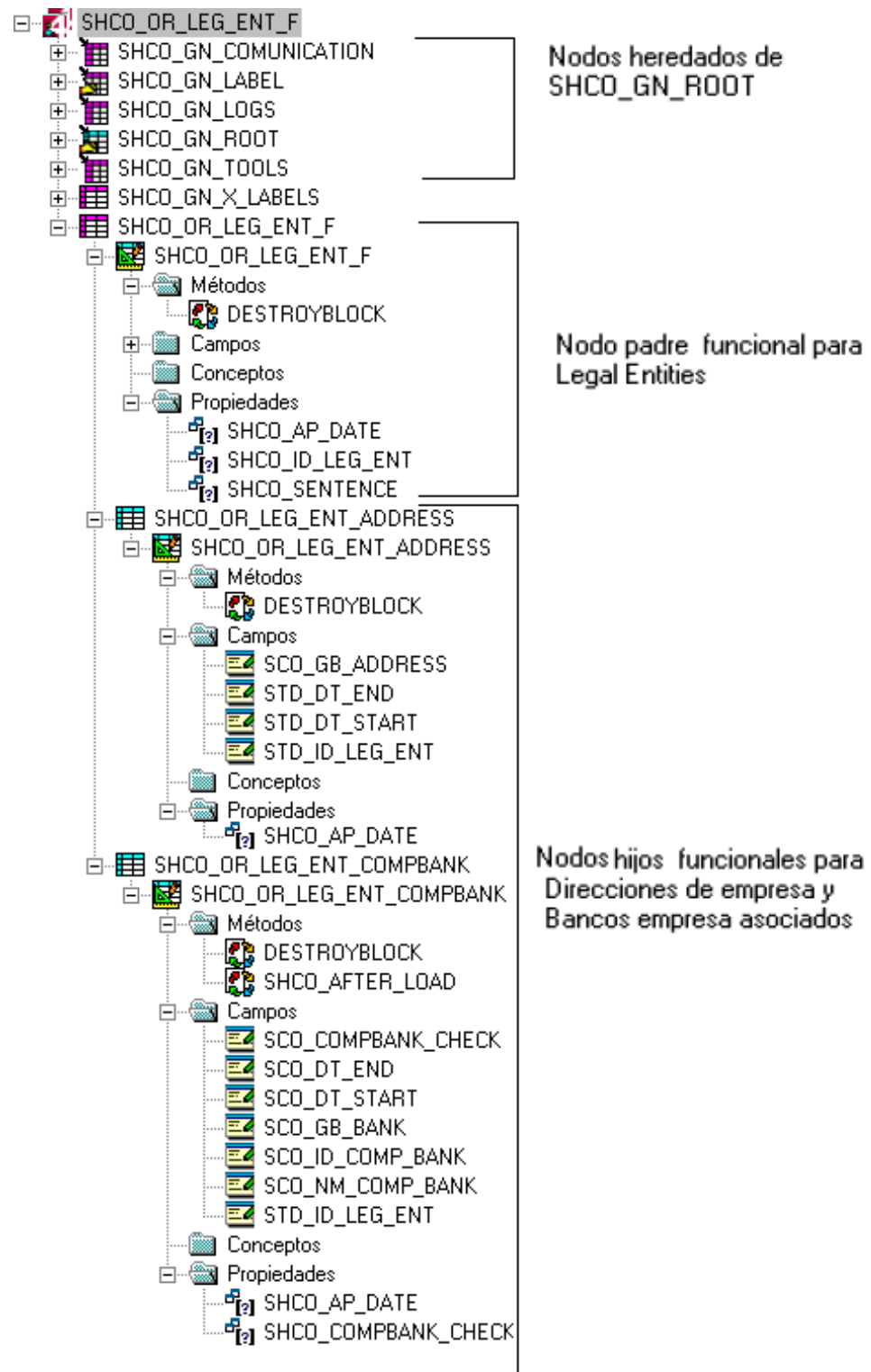
- La propiedad SHCO_SENTENCE (Tipo interno 20)
- La propiedad ámbito nodo correspondiente al identificador del registro a recuperar, que en nuestro caso es: SHCO_ID_LEG_ENT.

Conectar nodos hijos con el nodo padre utilizando el identificador correspondiente en nuestro caso STD_ID_LEG_ENT.

En el atributo "Nombre" de cada nodo su descripción funcional será pintada en la página.

Como norma establecemos la propiedad "Máximo número de registros" a 300 salvo excepciones.

Figura 16. Meta4Object creado



3. Sobrecribir el nodo **SHCO_GN_LABEL**

Creamos dos propiedades de ámbito bloque SHCO_LB_TITLE y SHCO_LB_TITLE_ROOT que contienen en su atributo "Nombre" el literal para el título de la página html y el título funcional de la Consulta.

4. Sobrecribir el nodo **SHCO_GN_ROOT**

Sobrecribir la regla del método SHCO_LOAD donde se genera la carga de los nodos funcionales creando el filtro para la carga del nodo/nodos padre y lanzando la carga de los hijos:

```

*****
' Metodo:  SHCO_GN_LOAD
' Fecha:   11/06/2002
' Version: 1.0
*****
' Carga el m4object.
'
'          LOAD_TYPE_ARG      String  Id de la parametrización

MsLoadType = LOAD_TYPE_ARG
If MsLoadType ="NORMAL" Then

    MsIdLeg = STD_ID_LEG_ENT
    If MsIdLeg = "" or MsIdLeg= NullValue() Then
        Setlog(-1,61,2,1)
        Call("SHCO_GN_TOOLS","SHCO_DEBUG_MAKER")
        Return(M4_Error)
    EndIf

    MdApDate = SHCO_AP_DATE
    If MdApDate = NullValue() Then MdApDate = Today()

    SHCO_OR_LEG_ENT_F.DESTROYBLOCK( )
    SHCO_OR_LEG_ENT_F.SHCO_ID_LEG_ENT= MsIdLeg
    MsEmpresa = "FROM &STD_LEG_ENT A WHERE (A.STD_ID_LEG_ENT =
@SHCO_ID_LEG_ENT)"
    SHCO_OR_LEG_ENT_F.SHCO_SENTENCE = MsEmpresa
    SHCO_OR_LEG_ENT_F.load_blk()
    Setlog(0,61,1,7,"SHCO_OR_LEG_ENT_F")

    MnCount = Call("SHCO_OR_LEG_ENT_F","Count")
    If MnCount > 0 Then
        Call("SHCO_OR_LEG_ENT_COMPBANK", "Load_Blk")

```



```

Setlog(0,61,1,7,"SHCO_OR_LEG_ENT_COMPBANK")
Call("SHCO_OR_LEG_ENT_ADDRESS", "Load_Blk")
Setlog(0,61,1,7,"SHCO_OR_LEG_ENT_ADDRESS")
EndIf

```

```
EndIf
```

Incluir la propiedad SHCO_AP_DATE.

Añadir en este nodo tantas propiedades como campos necesitamos para filtrar el registro (normalmente los campos de la PK). Son propiedades de ámbito nodo y visible con el mismo id que el campo por el que filtraremos el nodo padre funcional. En nuestro caso STD_ID_LEG_ENT.

Crear el método SHCO_LOAD_BEFORE_LOC y SHCO_LOAD_AFTER_LOC donde cada país pondrá el código a ejecutar antes y después de la carga respectivamente.

5. Traducir valores

En el caso de campos que contienen un valor numérico del que queremos representar su significado utilizaremos la estructura del nodo **SCO_X_LABELS**.

En nuestro caso el nodo **SHCO_OR_LEG_ENT_COMPBANK** contiene el campo SCO_COMPBANK_CHECK con valores 0 ó 1. En realidad pintaremos "No" o "Sí" respectivamente. Para lograr esto creamos en este nodo una propiedad ámbito registro que contendrá el nombre para el valor SHCO_COMPBANK_CHECK.

Incluir un nuevo nodo libre en el Meta4Object con identificador **SHCO_GN_X_LABELS** y reutilizamos la estructura de nodo homónima existente ya en la aplicación.

Figura 17. Nodo libre **SHCO_GN_X_LABELS**



Creamos en nuestro nodo funcional un nuevo método SHCO_AFTER_LOAD ámbito nodo con el código que recorre los registros que hay cargados en este encontrando el literal para cada valor llamando al método SHCO_GET_NAME de SHCO_GN_X_LABELS y asignándolo a la nueva propiedad.

```

*****
' Metodo:  SHCO_AFTER_LOAD
' Fecha:   11/06/2002
' Version: 1.0
*****

' Asocia la etiqueta traducida a los campos de tipo check
'

MnCount = Count()
For i = 0 to MnCount - 1
    MoveTo(i)
    SHCO_COMPBANK_CHECK =
SHCO_GN_X_LABELS.SHCO_GET_NAME(SCO_COMPBANK_CHECK)
Next

Begin()

```

Con un Conector Libre disparamos este método después del método de sistema LOAD_BLK de nuestro nodo.

Páginas JSP

Se recomienda duplicar la página que más se asemeje al caso a resolver y hacer sobre la copia los retoques necesarios. A continuación se describen estos ajustes:

shco_or_leg_ent_f.jsp

Esta es la página base de la plantilla a partir de la cual se llama al resto de páginas implicadas.

La parametrización de esta página se centra en los siguientes puntos:

- Dar valor a la variable zcarril para recoger los parámetros que nos pasa la lista previa a la consulta para filtrar el dato que queremos. En nuestro ejemplo:


```
zcarril="/servlet/CheckSecurity/JSP/shco_g1_sse/
shco_or_list_leg_ent.jsp?zpag=shco_g1_sse/shco_or_leg_ent_f.jsp";
String zdireccion = "shco_or_leg_ent_f.jsp";
```
- Cambiar o añadir valores para znodo, znodo1, znodo2; así como para zoutputdef, zoutputdef1, zoutputdef2.

-
- La ruta en `zraiz1, ...` contiene la referencia que utilizaremos dentro de la Consulta para nodos que sólo muestran un registro y sus etiquetas o para las etiquetas de los que muestran más de uno.
 - La ruta en `zcomun1, ...` contiene la referencia para los nodos que muestran más de un registro, por tanto serán utilizados dentro de bucles `<m4:loop>`.
 - Para cada nodo se definen variables `zidcampo` con la referencia a los campos, nombre de nodos (sólo hijos) y etiquetas que se obtienen del literal del campo con `<m4:label>`
 - Si el nodo hijo sólo presenta un registro `zidcampo` nos servirá para pintar tanto el valor como la etiqueta.
 - Si el nodo hijo presenta múltiples registros generamos `zidcampo` para pintar el valor y `zlidcampo` para pintar la etiqueta.
 - En los *includes* cambiamos la página a incluir en cada caso. Aquí se hacen los *includes* del resto de páginas implicadas y que se detallan a continuación.

shco_or_leg_ent_f_m4def.jsp

La parametrización de esta página se centra en los siguientes puntos:

- En esta página toman valor las variables que van a pasar los datos al `Meta4Object` para que filtre los datos que ha solicitado el usuario a través de la lista. Cambiamos los identificadores con los nombres de los campos de los que reciben valor del `Meta4Object` Lista correspondiente. En nuestro caso `STD_ID_LEG_ENT`.
- Cambiar los valores para el nombre de la página jsp de lista para nuestra Consulta en `zcarril`.
- Poner nuestros valores en `zdireccion`, `zsubsesion` y `zmeta4object`. Para la subsesión se recomienda utilizar el mismo id que el usado para el `Meta4Object`.

shco_or_leg_ent_f_bexec.jsp

La parametrización de esta página se centra en los siguientes puntos:

- Cambiamos el id de la variable java en la que guardamos el valor del parámetro/s que recibimos en la URL, así como el id de este parámetro/s que hemos definido en el `Meta4Object`. En nuestro caso `STD_ID_LEG_ENT`. Es el que se utiliza para la carga filtrada del nodo funcional padre.

shco_or_leg_ent_f_outputdef.jsp

La parametrización de esta página se centra en los siguientes puntos:

- Cambiar o añadir los outputdefs para cada nodo utilizando las variables java que habíamos creado en shco_or_leg_ent_f.jsp
- Además en el código java contamos el número de registros cargados en cada nodo hijo y lo asignamos a las variables zcount1, zcount2, zcountv1 y zcountv2. Las primeras son tipo entero y las segundas tipo cadena conteniendo el mismo valor.

shco_or_leg_ent_f_cabec.jsp

La parametrización de esta página se centra en los siguientes puntos:

- Utilizando las variables anteriormente creadas para los valores del nodo padre funcional y sus etiquetas pintar esta información siguiendo el estilo definido para las Consultas.
- En versiones anteriores existían dos enlaces: **Informes** y **Análisis** que debemos eliminar si existen.

shco_or_leg_ent_f_block1.jsp/shco_or_leg_ent_f_block2.jsp

La parametrización de esta página se centra en los siguientes puntos:

- Utilizando las variables creadas para los valores cada nodo hijo y sus etiquetas pintamos esta información en la Consulta.
- Utilizamos zraiz1, zraiz2 como ruta para los valores de nodos que sólo muestran un registro y sus etiquetas; así como las etiquetas de nodos que presentan múltiples registros.
- Usamos zcomun1, zcomun2 como ruta para los valores de nodos que presentan múltiples registros. Por tanto van incluidos dentro de un bucle.
- Para las etiquetas usamos las variables con el prefijo "zl" mientras que para los valores las variables con el prefijo "z".
- Cambiamos el contador para cada nodo hijo: zcountv1, zcountv2.

shco_or_leg_ent_f_fnodes.jsp

Se incluye vacío a nivel corporativo para que se incluya la definición de nodos y elementos propios de cada país.

shco_or_leg_ent_f_oblock.jsp

Se incluye vacío a nivel corporativo para que se incluyan los bloques propios de cada país.

Filtro tipo lista asociado

Casi todas las consultas de información, dependiendo de la funcionalidad, llevan asociada una lista/filtro que aparece antes para que el usuario pueda elegir el registro del que quiere ver la información. La llamada a la lista se realiza al dar de alta la opción de menú según se explica en el capítulo de *Gestión de Menús*, incluyendo en el campo HTTP lo correspondiente. En nuestro ejemplo:

```
var parametros=['zpag','znw'];var valores=['shco_g1_sse/  
shco_or_leg_ent_f.jsp','1'];m4navegar('shco_g1_sse/  
shco_or_list_leg_ent.jsp',parametros,valores);
```

Plantilla para tablas de valores

Este apartado describe la configuración para cada una de las tablas de valores que queramos editar con las páginas de edición de valores HTML.

Esta plantilla es la base sobre la que implementar las funcionalidades de valores fijos, valores incidencias y entrada masiva de incidencias. Para valores fijos e incidencias, se podrá acceder a la pantalla principal utilizando una lista, una lista de multiselección o la búsqueda básica. Hay que tener en cuenta que, si bien mediante las listas se elige directamente una población, al utilizar la búsqueda lo que se define es un criterio, y la población será aquella que cumpla ese criterio en las fechas en las que se vayan a aplicar los valores.



Dependiendo de la versión de producto adquirida puede que no tenga disponible la búsqueda básica. Por tanto, considere, que el acceso a la pantalla principal se realizará exclusivamente a través de una lista o de una lista de multiselección.

Es decir, no existe una única página para la edición de valores, sino que difiere por cada tipo de tabla de valor. Así tendremos diferentes páginas en función de si son:

- Valores fijos
- Valores incidencias
- Incidencias masivas

Estas páginas son las principales. Además, se utilizan una serie de pantallas auxiliares tales como:

- Selección de conceptos
- Valores por defecto
- Edición por clave primaria
- Histórico a fecha de pago y a fecha de imputación
- Tramos

A continuación se describe la funcionalidad de cada una de las pantallas.

Valores fijos

La función de esta página es facilitar la asignación de valores para periodos superiores al de una paga.

Figura 18. Edición de valores fijos

Meta4 PeopleNet

Inicio | Gestión de tiempo | Valores del periodo del RH | Valores del rol | Valores generales

EDICIÓN DE VALORES DEL ROL

Fechas de filtro

* Imputacion: 01-01-2003

* Pago: 01-01-2003

Nuevos valores
Cambio de grupo
Valores por defecto
Mostrar/Ocultar detalles

	2020 - Cantidad	2001 - Bono del rol del RH	696969 - Número de días festivos	456789 - Número de días de tiempo teórico	888888 - Número de horas festivas
M10169 - 2 Gárdenas Guevara, Ke					

La pantalla se puede dividir en dos zonas: la parte superior es la zona de filtro y la inferior es la zona de valores:

- En la zona de filtro se encuentran las fechas por las que se filtrarán los valores; la primera vez que se entra en la pantalla, aparece el primer día de la primera paga abierta no bloqueada.

Si el standard es UK y los valores son del periodo o del rol, la paga será la correspondiente a la frecuencia que se haya seleccionado en la pantalla anterior.

Cuando se cambie alguna de las dos fechas, hay que pulsar el botón **Aplicar** para que el cambio tenga efecto.

Desde esta parte de la pantalla también se pueden realizar las siguientes acciones:

- Nuevos valores: deja vacías las celdas de la zona de edición, de esta forma se pueden introducir nuevos valores en celdas que ya tenían algún valor.
- Cambio de grupo: muestra la pantalla **Selección de conceptos** que se utiliza para filtrar los conceptos que se quieren visualizar en la zona de edición.
- Valores por defecto: muestra la pantalla **Valores por defecto**; se utiliza para cambiar los valores que se aplicarán si al insertar un nuevo registro se dejan vacíos.
- Mostrar/Ocultar detalles: muestra/oculta la caja de texto de la moneda en los conceptos cuyo valor sea de tipo Moneda.

- En la zona de valores se muestran los valores válidos para las fechas de filtro; están organizados en formato tabla: en las columnas aparecen los conceptos y en las filas la población. Por cada valor de población se puede acceder a la pantalla de Edición por clave primaria. En cada celda aparece el botón **Detalles**, que muestra una capa donde se puede asignar las fechas de inicio y fin (en caso de no querer utilizar las fechas por defecto), se puede editar el valor y se puede acceder a las pantallas de los históricos a fecha de pago y fecha de imputación.

Existe una tabla (Grupos por defecto) mediante la cual se puede asociar un grupo de conceptos a una tabla de valor: si la tabla de valor tiene asociado un grupo, sólo se mostrarán los conceptos correspondientes a ese grupo; si no tiene ningún grupo asociado se mostrarán los conceptos del periodo o del rol (dependiendo del nivel de la tabla de valor). Se recomienda definir grupos de valores funcionales para cada tabla, de esta forma las tablas serán más manejables. Se pueden dar valores a esta tabla mediante la opción de menú de windows **Herramientas de desarrollo | Configuración funcional cliente ligero | Asociar grupos a tablas de valor**.

Las acciones que se pueden realizar en la zona de valores son inserción, modificación y borrado:

- Para insertar un registro en una celda que esté vacía, no hay más que insertar el valor, modificar los valores de las fechas en caso de no querer las fechas por defecto y pulsar el botón de guardar.
- Si se quiere insertar un registro y la celda ya está ocupada por un valor, hay que pulsar el enlace **Nuevos valores** e insertar el registro. En este último caso, si la fecha de inicio del nuevo registro está dentro del periodo del registro anterior, se cerrará el registro anterior con $\text{fecha_fin_viejo} = \text{fecha_inicio_nuevo} - 1$; pero si coinciden las fechas de inicio y la fecha de pago también es la misma, dará un error porque se considera que es el mismo registro. En este caso, para cambiar el valor lo que hay que hacer es modificarlo, no insertar uno nuevo.
- Para modificar un registro, basta con cambiar los datos y guardar.

Si la grabación de un valor ha resultado ser errónea, se muestra el valor en amarillo, y aparece en la parte inferior de la pantalla un mensaje explicativo. Si se ha intentado guardar varios valores al mismo tiempo y algunos han sido correctos y otros no, los correctos se grabarán y los erróneos se marcarán como tales.

Recomendaciones de uso:

- Primero hay que decidir las fechas de imputación y pago, y aplicarlas.
 - Para valores que afecten a pagas normales, conviene poner como fecha de pago y fecha de imputación la fecha de inicio del valor.
 - Si son valores para pagas retroactivas, hay que poner como fecha de imputación la fecha de inicio de validez del dato; y como fecha de pago, la fecha de inicio de la paga en la que se va a lanzar la nómina.
- También conviene cambiar los valores por defecto si la mayoría de los registros tienen valores distintos de los que se cargan por defecto (como fecha de inicio la fecha de imputación, como fecha de fin 1/1/400, prioridad media-baja y sin motivo de cambio)

Valores incidencias

Hay dos diferencias principales entre esta pantalla y la de valores fijos:

- En lugar de tener dos fechas por las que filtrar, ahora aparecen dos pagas, que por defecto se corresponden con la primera paga abierta no bloqueada. Para UK, esta paga vendrá determinada por la frecuencia seleccionada en la pantalla de filtro o de búsqueda.
- En incidencias no se filtra a una fecha determinada, sino entre las fechas de inicio y fin de una paga; así que se puede dar el caso en que haya más de un registro en cada celda (cambios de valor a lo largo de la paga). Para insertar, editar y borrar este tipo de registros, se utiliza la pantalla de tramos (descrita más adelante), a la cual se accede mediante un botón contiguo al

de **Detalles**. Si al cargar los valores hay una celda en la que hay más de un registro, se muestra sólo el primer registro y cambia la forma del icono (de un lápiz a dos) para indicar que hay más información de la que se muestra en la pantalla.

Figura 19. Edición de valores incidencias

Meta4
PeopleNet

Inicio | Gestión de tiempo | Valores del periodo del RH | Valores del rol | Valores generales |

EDICIÓN DE INCIDENCIAS DEL ROL

Fechas de filtro

* Imputacion 27-01-2003 004 Primera paga Mensual 2003-01

* Pago 27-01-2003 004 Primera paga Mensual 2003-01

Nuevos valores
Cambio de grupo
Valores por defecto
Mostrar/Ocultar detalles

	2020 - Cantidad	2001 - Bono del rol del RH	696969 - Número de días festivos	456789 - Número de días de tiempo teórico	888888 - Número de horas festivas
M10169 - 3 Cárdenas Guevara, Ro.					

1 - 1

Incidencias masivas

Esta pantalla está pensada para introducir valores rápidamente para una paga de imputación, una de pago y un concepto. No se realiza ninguna carga porque el usuario sabe a quién quiere asignar los valores, y si quiere modificar en vez de insertar, debe utilizar las pantallas de incidencias normales. Cuando los valores se han insertado correctamente, la zona de valores se queda vacía para que el usuario pueda seguir insertando; pero si algún valor es erróneo, se quedan los valores en la pantalla, los erróneos aparecen en amarillo y los correctos aparecen deshabilitados, de esta forma el usuario puede ver cuáles debe corregir.

Desde esta pantalla no se accede a ninguna de las pantalla comunes a los otros tipos de pantallas.

Figura 20. Edición de incidencias masivas

Inicio | Gestión de tiempo | Valores del periodo del RH | Valores del rol | Valores generales |

EDICIÓN DE INCIDENCIAS

Valores por defecto

* Imputacion: 27-01-2003 004 Primera paga Mensual 2003-01

* Pago: 27-01-2003 004 Primera paga Mensual 2003-01

Componente de cálculo: [] [] []

Fecha de inicio: 01-01-2003 Fecha de fin: 31-01-2003

* Prioridad: Medie baja ID Motivo cambio: []

Nuevos Valores
Mostrar/Ocultar detalles

ID RH	Núm. rol	Valor
1	[]	[] [] []
2	[]	[] [] []
3	[]	[] [] []

Selección de componentes de cálculo

En esta pantalla, se puede seleccionar los componentes que se van a visualizar en la pantalla principal. Los componentes son los correspondientes al grupo de componentes del periodo o del rol dependiendo del nivel de cada tabla de valor. Se permite la elección de otros grupos más selectivos utilizando el filtro.

Figura 21. Selección de componentes de cálculo

The screenshot shows a web browser window titled 'Selección de componentes de cálculo - Microsoft Internet Explorer'. The main content area has a header 'SELECCIÓN DE COMPONENTES DE CÁLCULO' with a help icon. Below this is a form with three tabs: 'Conceptos finales' (selected), 'Variables', and 'Totales'. The 'Conceptos finales' tab contains several dropdown menus: 'Nombre de concepto final', 'ID Componente de cálculo', and 'Nombre corto'. To the right of these are three 'Empieza por' dropdowns. Below these are two more dropdowns labeled 'Clasificaciones' and 'Subclasificaciones'. A magnifying glass icon is centered below the form. At the bottom left, there is a tree view showing a hierarchy: 'PERICO' (with a folder icon), '1001 - Bono del periodo del RH' (with a document icon), '1002 - Cantidad fija para familia' (with a document icon), '1017 - Plus con base como incidencia' (with a document icon), and 'Neto' (with a folder icon).

Valores por defecto

En esta pantalla se definen los valores por defecto que se asignarán al insertar registros nuevos. Sólo se aplican si el nuevo registro tiene vacíos estos valores en la celda.

Figura 22. Selección de valores por defecto

The screenshot shows a web browser window titled 'Valores por defecto - Microsoft Internet Explorer'. The main content area has a header 'Valores por defecto'. Below this is a form with four fields: '* Prioridad' with a dropdown menu showing 'Media baja'; 'Motivo de cambio' with a dropdown menu; '* Fecha de inicio' with a text input showing '01-01-2003' and a calendar icon; and 'Fecha de fin' with a text input showing '01-01-4000' and a calendar icon. At the bottom of the form are two circular buttons: a green checkmark and a red 'X'.

Edición por clave primaria

Muestra los valores de todos los conceptos correspondientes a un ejemplar de la población. Permite insertar, modificar y borrar. El botón de la doble flecha permite ir a la pantalla principal de valores refrescando los últimos cambios que se hayan hecho en ésta.

Figura 23. Edición de valores por clave primaria

Inicio | Gestión de tiempo | Valores del periodo del RH | Valores del rol | Valores generales |

 PROYECCIÓN POR ROL 

Fechas de filtro

Imputacion	27-01-2003	004	Primera paga Mensual 2003-01		Nuevos valores Valores por defecto Mostrar/Ocultar detalles
Pago	27-01-2003	004	Primera paga Mensual 2003-01		

M10169 - 3 Cárdenas Guevara, Kenny	
2020 - Cantidad	 
2001 - Bono del rol del RH	 
696969 - Número de días festivos	 
456789 - Número de días de tiempo teórico	 

Históricos a fecha de pago y fecha de imputación

Muestra la evolución a lo largo del tiempo del valor correspondiente a un concepto y a un valor de la población. Esta evolución puede ser tanto por pago como por imputación. Sólo se permite la visualización de los valores.

Figura 24. Históricos de valores por fecha de imputación y por fecha de pago

Meta4
PeopleNet

Inicio | Gestión de tiempo | Valores del periodo del RH | Valores del rol | Valores generales |

HISTÓRICO DE VALORES

Fechas de filtro

Imputación: 01-01-2003

Rol: M10199-1

Componente: 2020- Cantidad

Desde	Fecha de inicio	Fecha de fin	Valor
18-12-2002	18-12-2002	08-01-2003	129500.000000 EUR

Botón de doble flecha

Tramos

A esta pantalla sólo se puede acceder desde incidencias. La parte superior, muestra los datos que vienen de la página de incidencias, en la parte inferior aparece los registros correspondientes y en el centro está el formulario para insertar y editar datos. Su comportamiento es similar a una pantalla de mantenimiento. El botón de la doble flecha permite ir a la pantalla principal de valores refrescando los últimos cambios que se hayan hecho en ésta.

Figura 25. Edición de tramos

 EDICIÓN DE INCIDENCIAS DEL ROL 

Imputación	27-01-2003	004	Primera paga Mensual 2003
Pago	27-01-2003	004	Primera paga Mensual 2003
ID RH	M10169	Núm. rol	3
Componente de cálculo	2000	SCO_AMOUNT	Cantidad

Nuevo registro			
*Inicio		*ID Prioridad	Máxima
Fin		ID Motivo cambio	
*Valor		Comentario	



Inicio	Fin	Valor	ID Prioridad	ID Motivo cambio	1-0 de 0
					

Cambios respecto a Windows

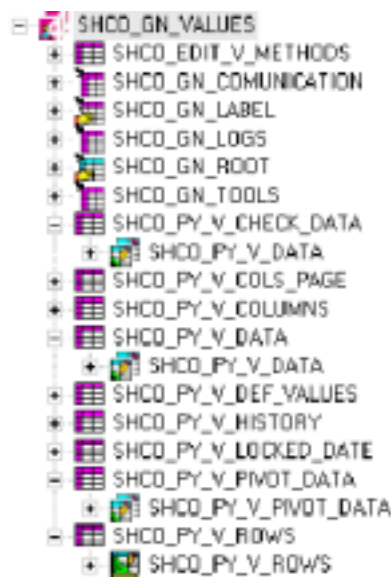
Las diferencias respecto al sistema de tablas de valor en windows son:

- Se utiliza una única pantalla para pagas bloqueadas y no bloqueadas. No hay fecha de efectividad.
- Desaparece la fecha de bloqueo: la pantalla aparece habilitada o no dependiendo de la fecha de pago.
- Se cambia la forma de generar la fecha de bloqueo: ahora es el máximo entre las fechas de fin de:
 - las pagas cerradas.
 - las pagas abiertas cuya fecha de corte sea menor que hoy.
- Si se inserta un valor cuyas fechas se solapan con otro valor anterior, se cierra este último y nunca se crean proyecciones.
- No se pueden insertar correcciones a futuro.

Meta4Objects

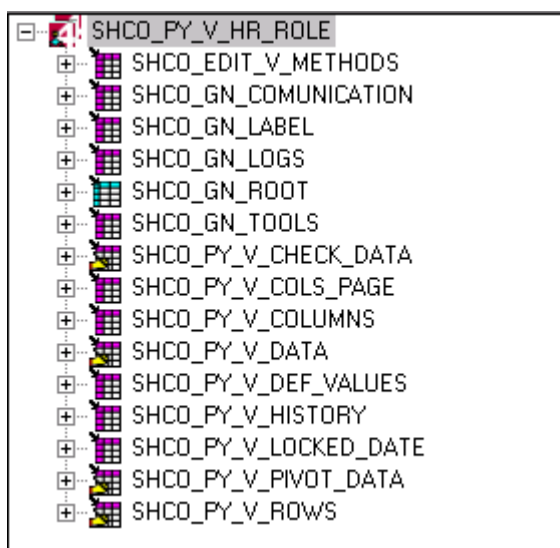
El nuevo Meta4Object será heredado del Meta4Object genérico **SHCO_GN_VALUES**:

Figura 26. Meta4Object genérico para la edición de valores



A continuación se detalla el proceso para crear un Meta4Object de una nueva tabla de valor. Como ejemplo se crea el **SHCO_PY_V_HR_ROLE**.

Figura 27. Creación de un Meta4Object para introducir valores



Para crear el Meta4Object de una nueva tabla de valor:

1. Se crea un nuevo Meta4Object de herencia **SHCO_PY_V_HR_ROLE**, heredando el m4object de plantilla tablas de valor (**SHCO_GN_VALUES**).
2. Se sobrescribe el nodo **SHCO_PY_V_ROWS** con una estructura de nodo nueva, que debe leer de la tabla de población correspondiente a la tabla de valor (en este caso **SCO_HR_ROLE**). Los ids de la PK deben ser iguales a los ids de la PK de la tabla de valor. Si la página de entrada es la búsqueda, se sobrescribe el método **SHCO_SET_SEARCH_PROPERTIES** y se asignan las propiedades de la búsqueda **SHCO_FILTER_MODE** y **SHCO_ID_OPERATION**. Si las tablas van sobre el periodo o el rol, también se sobrescribe la regla del concepto **SHCO_SYSENTENCE_FREQ** para que filtre los datos por la frecuencia de pago.
3. Se sobrescribe el nodo **SHCO_PY_V_DATA** con una estructura de nodo nueva, que debe leer de la tabla de valor (**SCO_ROLE_VALUES**). Esta estructura de nodo se reutiliza para sobrescribir el nodo **SHCO_PY_V_CHECK_DATA**. Para incidencias, se sobrescribe la regla del concepto **SCO_INCIDENCES** de tal forma que devuelva un 1 si son incidencias masivas y un 2 si son incidencias normales; por defecto (valores fijos) devuelve 0.

El código identificativo de los campos de la tabla de valor debe coincidir con el código identificativo de los campos que se señalan en la siguiente tabla.

Tabla 2. Listado de códigos identificativos correctos

CÓDIGOS de CAMPOS
SCO_ID_CURRENCY
SCO_ID_REASON_CHANGE
EX_TYPE
SCO_DT_EXCHANGE
DT_COR_END
DT_COR_START
DT_END
DT_START
SCO_VALUE
ID_DMD
ID_DMD_COMPONENT
ID_M4_TYPE
ID_PRIORITY
SCO_COMMENT

Si el código de algún campo no coincide con el listado, por ejemplo que la tabla tenga un campo con el código identificativo **CHANGE_REASON** en lugar de **SCO_ID_REASON_CHANGE**, al seleccionarlo en la sentencia de la estructura de nodo puede cambiarle el nombre en la pestaña **Selección | ID Columna** del **Editor de sentencias**.

Si el caso es que en su tabla de valor no existe alguno de los campos, por ejemplo el **SCO_COMMENT**, debe crearse una propiedad que se llame igual que el campo que le falta en el nodo **SHCO_PY_V_DATA**.

- Se sobrescribe el nodo **SHCO_PY_V_PIVOT_DATA DATA** con una estructura de nodo nueva, que debe leer de la tabla de valor (en este caso **SCO_ROLE_VALUES**).
- Si la tabla es de valores incidencias, se crea un nuevo nodo **SHCO_PY_V_DATA_SLICE** con una estructura de nodo nueva que herede de **SHCO_PY_V_PIVOT_DATA** y que debe leer de la tabla de valor. Se utiliza para la pantalla de tramos.

Páginas JSP

Existen dos tipos de páginas en la gestión de los valores:

- Páginas tecnológicas que no hay que modificar.
- Páginas de configuración que son las que hay que crear cuando se quiere añadir una nueva tabla de valor. Se detalla el proceso completo de generación de éstas.

Páginas tecnológicas

Se describen aquellas páginas implicadas en la asignación de valores pero que no hay que modificar.

Búsqueda básica

Antes de la página de asignación de valores se filtrará la población que se verá afectada con los nuevos valores. Dependerá de la clase de la población sobre la que queramos editar los valores.

Esta página es común para toda la aplicación.

Puede no existir este paso previo. Se utilizan siempre que se asignen valores incidencias y fijos para un grupo.

Página de edición de valores (`shco_py_values.jsp?module=[módulo]`)

Es la página principal. La que muestra los valores para su edición filtrados por imputación y pago.

A esta página se accede desde el paso de búsqueda básica para filtrado de la población sobre la que trabajar

Desde esta página se accede a las páginas de pivotado por clave primaria y de histórico del valor.

También desde esta página se muestran las páginas de cambio de grupo y de modificación de valores por defecto.

Parámetros de la página

Esta página permite los siguientes parámetros para poder servir como detalle (*Zoom*) por clave primaria de una población fija que se quiera indicar:

-
- streamпка, streamпkb, streamпkc, streamпd: secuencia de valores correspondientes a la clave primaria de la población cuya tabla de valores se quiere editar. Vendrá especificado dependiendo del tipo de población correspondiente: empresa, unidad organizativa, etcétera.

Página de edición por clave primaria (shco_proyectпk.jsp)

En esta página se muestran todos los valores para un individuo para facilitar su la edición.

Página de histórico (shco_historyvalues.jsp)

En esta página se muestran todos los valores por imputación o por pago para un individuo y un concepto.

Diálogo de cambio de grupo (shco_py_list_payroll_concepts.jsp)

El usuario elige los conceptos que quiere que aparezcan en las columnas de la página de edición de valores.

Diálogo de valores por defecto (shco_defaultvalues.jsp)

Se eligen los valores por defecto para los campos correspondientes a los nuevos valores que se editen o inserten.

Páginas de configuración

Para la configuración de una nueva tabla de valores se generan cinco páginas jsp: una página principal que contendrá la referencia al resto de páginas de configuración.

A continuación, se detalla el proceso de creación de una página de configuración principal. Y dentro de este proceso se referencia a cada una de las páginas secundarias de configuración.

La configuración de la página principal para una nueva tabla de valores implica:

- Generar la página de configuración
- Verificar la estructura de los archivos

Generación de la página de configuración

Se crea la página de configuración principal que será la que se pase a las páginas de valores shco_py_values.jsp?module=[модуло] para la edición de cada tabla de valores concreta.

Se entiende por módulo cada una de estas páginas de configuración. El

nombre de este módulo será el nombre de la página principal de configuración sin la extensión jsp.

La nomenclatura a seguir es siempre la misma:

```
shco_class_py[funcionallity]params
```

sustituyendo *funcionallity* por las cadena *role*, *hr*, *incrole*, *legent*, etcétera

La página se almacena en el servidor en la carpeta [m4webservices-root]/shco_py.

EJEMPLO

Se toma como ejemplo para la configuración de un módulo la edición de valores del rol. Para ello ya se ha creado el Meta4Object apropiado con el identificador: **"SHCO_PY_V_ROLE"**.

Para crear un nuevo módulo para editar los valores del rol, en principio, es suficiente con decidir el nombre **"shco_class_pyroleparams"**. Así, la dirección URL de acceso a la página principal de edición de valores del rol sería: **"http://[servidor]:[puerto]/servlet/CheckSecurity/JSP/shco_py/shco_pyvalues.jsp?module=shco_class_pyroleparams"**.

La nomenclatura a seguir es siempre la misma, cambiando simplemente la cadena role por: hr, incrole, legent, etcétera.

Con el nombre decidido en el paso anterior se crea la página **shco_class_pyroleparams.jsp** y la situamos en la carpeta del servidor [m4webservices-root]/shco_py

La página debe tener el aspecto siguiente:

```
<%
// Configuración del modulo Tabla de valores del Rol
request.setAttribute("PAGESIZE", new Integer(10));
request.setAttribute("Channel.ID", "SHCO_PY_V_ROLE");
// jsp que añade el atributo correspondiente al DMD Group
pageContext.include("/shco_py/shco_dmd_period.jsp");
request.setAttribute("Values.PKA.Item", "SCO_ID_HR");
request.setAttribute("Values.PKA.Type", "2");
request.setAttribute("Values.PKB.Item", "SCO_OR_HR_ROLE");
request.setAttribute("Values.PKB.Type", "6");
request.setAttribute("Values.PKC.Item", "");
request.setAttribute("Values.PKC.Type", "");
request.setAttribute("Values.PKD.Type", "");
// jsp que pinta la celda que hace de cabecera de las filas
request.setAttribute("Page.HRows", "/shco_py/
shco_class_pyrolecellhrow.jsp");
// jsp que pinta el tooltip en la celda que hace de cabecera de las
filas.
request.setAttribute("Page.HRowsName",
    "/shco_py/shco_class_pyrolecellhrowname.jsp");
// jsp que pinta la celda cabecera de las columnas
// en la página de proyección por clave primaria.
request.setAttribute("Page.HCols", "/shco_py/
shco_class_pyrolecellhcol.jsp");
// literales que aparecen en las páginas.
request.setAttribute("Translations.Bundle",
    "shco_class_pyroletrans");
//página de ayuda
request.setAttribute("Help.Page", "shco_pyvalues_1.htm");
%>
```

Los parámetros de la página de configuración son los siguientes:

Genéricos:

■ PAGESIZE

Es el número de individuos por página que aparecerán en la página principal. En nuestro caso mostraremos 10 individuos por página.

■ Channel.ID

Identificador del Meta4Object al que haremos referencia para la edición de los valores. Y que debe cumplir la especificación requerida para que las páginas HTML de edición de valores puedan funcionar correctamente. En nuestro caso el Meta4Object tiene como identificador **"SHCO_PY_V_ROLE"**.

- Channel.DMDGroup

Identificador del grupo de conceptos (Periodo o Rol) al que están asociados los conceptos que se van a editar en esta página de valores. En esta página se indirecciona al archivo **"shco_dmd_role.jsp"** que contiene el atributo correspondiente al DMD component *DMD_GROUP1* con la siguiente sintaxis:

```
<%
    request.setAttribute("Channel.DMDGroup", "DMD_GROUP1");
%>
```

En la aplicación estándar dispone de dos páginas para este fin, la **shco_dmd_role.jsp** y la **shco_dmd_period.jsp**. Cada una de ellas tiene un componente de dominio diferente. Si dispone de otros DMD debe modificar estas páginas con el valor correspondiente.

- Values.PKA.Item, Values.PKB.Item, Values.PKC.Item, Values.PKD.Item

Nombre de los identificadores correspondientes a la clave primaria de los individuos de cuyos valores estamos editando.

- Values.PKA.Type, Values.PKB.Type, Values.PKC.Type, Values.PKD.Type

Tipo M4 del item correspondiente.

- Help.Page

Indica el tópico de ayuda correspondiente a la página de valores que queremos editar.

Por regla general hay que poner:

- para las páginas de valores fijos **"shco_pyvalues_1.htm"**
- para las páginas de incidencias **"shco_pyvalues_2.htm"**
- para las páginas de valores generales (empresa, lugar de trabajo, unidad organizativa, etcétera) **"shco_pyvalues_3.htm"**

Inclusión de las páginas de configuración secundarias:

Para las páginas secundarias se sigue esta nomenclatura:

```
shco_class_py[functionality]cellhcol.jsp,
shco_class_py[functionality]cellhrow.jsp,
shco_class_py[functionality]cellhrowname.jsp y
shco_class_py[functionality]trans_es.properties (más fr y en)
```

sustituyendo *funcionallity* por las cadena *role*, *hr*, *incrole*, *legent*, etcétera

- Page.Hrows

Página JSP que pinta la celda que hace de cabecera de las filas en la página de edición de valores, es decir la que identifica a cada individuo. Como modelo, mirar el archivo **`/shco_py/shco_class_pyrolecellhrow.jsp`**.

- Page.HrowsName

Página JSP que pinta el *tooltip* en la celda que hace de cabecera de las filas en la página de edición de valores. Como modelo mirar el archivo **`/shco_py/shco_class_pyrolecellhrowname.jsp`**.

- Page.Hcols

Página JSP que pinta la celda cabecera de las columnas en la página de edición por clave primaria. Como modelo mirar el archivo **`/shco_py/shco_class_pyrolecellhcol.jsp`**.

- Translations.Bundle

Archivos de propiedades localizados para los literales que deben aparecer en las páginas HTML. Por ejemplo, en nuestro caso si se pone **`"shco_class_pyroletrans"`**, quiere decir que debemos tener los archivos **`/translations/shco_class_pyroletrans_es.properties`** para los literales en español, **`/translations/shco_class_pyroletrans_en.properties`** para los literales en inglés, etcétera.

EJEMPLO

Por ejemplo en nuestro caso el archivo **`/translations/shco_class_pyroletrans.properties`** tiene el aspecto siguiente:

```
# Literales en español
PYValuesPage.Header=Edici&oacute;n de valores del rol
PYValuesPage.Title=Edici&oacute;n de valores del rol
PYHistoryPage.Title=Hist&oacute;rico de valores
PYHistoryPage.Header=Hist&oacute;rico de valores
Filter.PK=Rol
PYProjectPKPage.Title=Proyecci&oacute;n por rol
PYProjectPKPage.Header=Proyecci&oacute;n por rol
```

Y cada identificador del archivo de literales corresponde con los siguientes elementos de las páginas de edición de valores:

Figura 28. Literales en la página de edición de valores del rol

Inicio | Gestión de tiempo | Valores del periodo del RH | Valores del rol | Valores generales |

EDICIÓN DE VALORES DEL ROL
PYValuesPage.Title
Edición de valores del rol

Fechas de filtro

* Imputación: 01-01-2003

* Pago: 01-01-2003

Nuevos valores
 Cambio de grupo
 Valores por defecto
 Mostrar/Ocultar detalles

	2020 - Cantidad	2001 - Bono del rol del RH	696969 - Número de días festivos	456789 - Número de días de tiempo teórico	888888 - Número de horas festivas
M10140 - 1 TTT, TTT				123	
M10199 - 1 Quevedo, Francisco	129500.00				
M10200 - 1 Quevedo, Francisco	6155.0600				
M10206 - 1 Canoyra, Juan Carlos					

Figura 29. Literales en la página de proyección del rol

Inicio | Gestión de tiempo | Valores del periodo del RH | Valores del rol | Valores generales |

PROYECCIÓN POR ROL
PYProyectPKPage.Title
Proyección por rol

Fechas de filtro

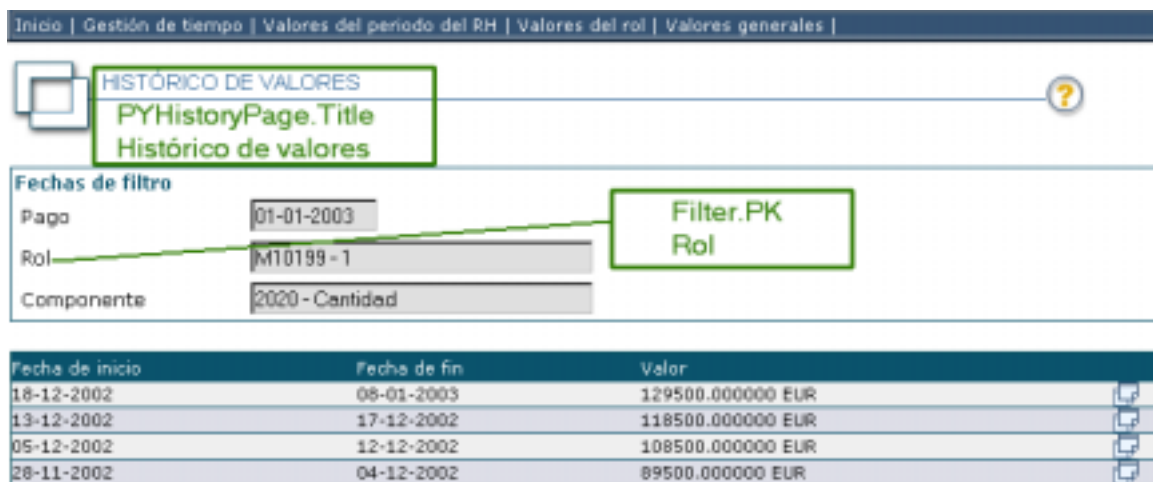
Imputación: 01-01-2003

Pago: 01-01-2003

Nuevos valores
 Valores por defecto
 Mostrar/Ocultar detalles

2020 - Cantidad	M10199 - 1 Quevedo, Francisco 129500.00
2001 - Bono del rol del RH	
696969 - Número de días festivos	
456789 - Número de días de tiempo teórico	
888888 - Número de horas festivas	

Figura 30. Literales en las páginas de **Históricos por imputación y pago para un rol**



Verificar la estructura de archivos

Finalizada la configuración para la edición de valores del rol, la estructura de los archivos quedaría como sigue:

Figura 31. Archivos en shco_py

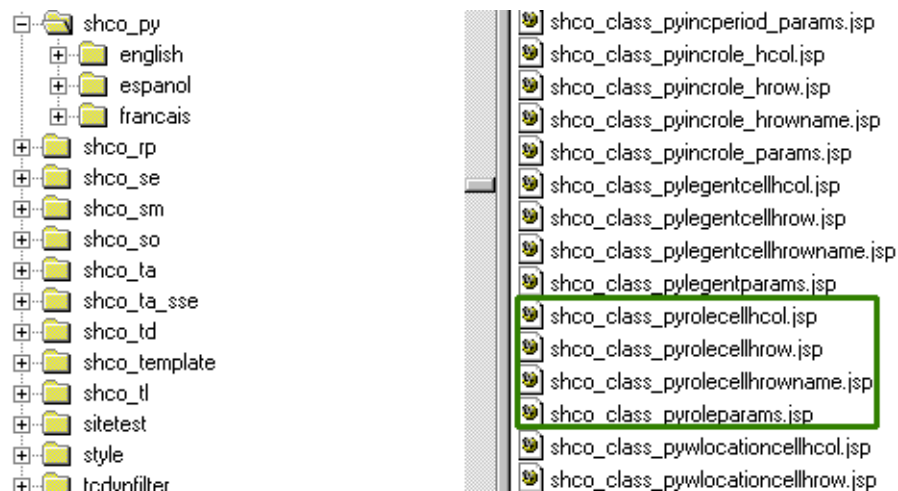


Figura 32. Archivos en translations

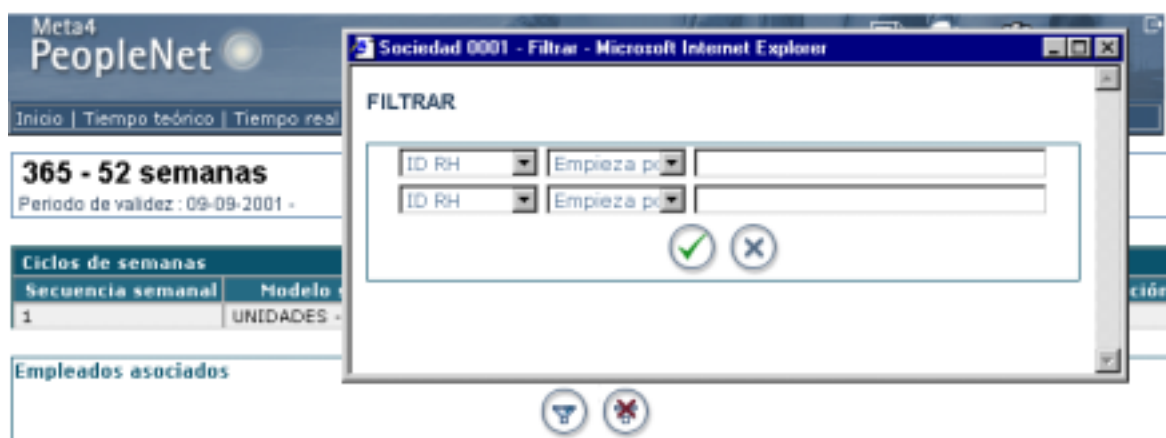


Plantilla de filtro sobre datos cargados

Plantilla válida para todos aquellos Meta4Objects que generen un filtro sobre datos ya existentes en otra página.

El filtro se abre desde la página sobre la que se va a aplicar el filtro. Una vez seleccionadas las condiciones del filtro, se cierra la página refrescando la página madre.

Figura 33. Ejemplo de filtro



Meta4Objects

La configuración de un filtro afecta a dos Meta4Objects:

- El nuevo Meta4Object que genera el filtro
- Modificación del Meta4Object al que afecta el filtro

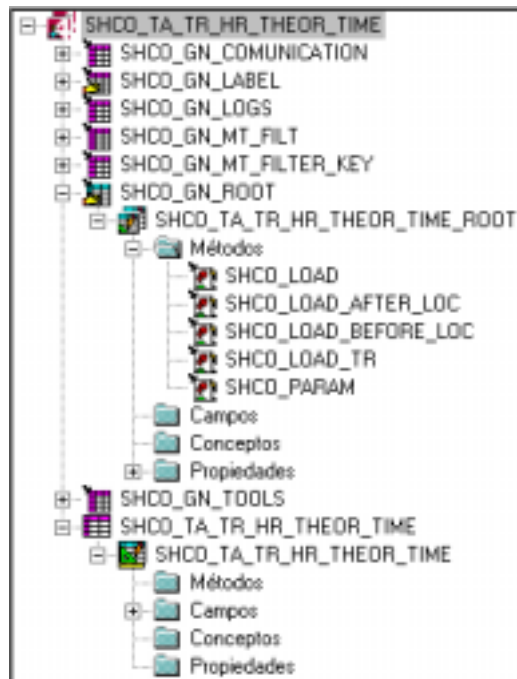
En primer lugar se explica como generar el Meta4Object de filtro, y a continuación, se detalla cómo afecta al Meta4Object que se filtra.

Generación del Meta4Object de filtro

Los pasos a seguir son los siguientes:

1. Se crea un nuevo Meta4Object de herencia (por ejemplo: **SHCO_TA_TR_HR_THEOR_TIME**), heredando el Meta4Object de plantilla de TR's: **SHCO_GN_TR**.

Figura 34. Meta4Object de filtro



2. Se añade un nuevo nodo libre, con un id. igual al del Meta4Object. En el ejemplo **SHCO_TA_TR_HR_THEOR_TIME**.
3. Se añade una estructura de nodo. En esta sentencia se leen todos los campos que queremos que aparezcan en el filtro.
4. En la opción de menú **Configuración funcional de cliente ligero | Parametrización de filtros** se rellena la tabla **M4SCO_FILTER** con los campos por los que se desea filtrar.

EJEMPLO

SCO_ID_M4OBJECT= SHCO_TA_TR_HR_THEOR_TIME

SCO_ID_FIELD= STD_ID_HR

SCO_ID_TYPE

Ejemplo: = 0 (para *String*)

Ejemplo: = 1 (para numéricos)

Ejemplo: = 2 (para fechas)

SCO_ID_TABLE = SCO_HR_THEOR_TIME

SCO_ID_ORDEN= 0 (Orden de aparición de los campos en el filtro de la lista)

5. Se sobrescribe el nodo **SHCO_GN_LABEL** y se crean dos nuevas propiedades de ámbito bloque.

- **SHCO_LB_TITLE**: etiqueta para el título de la lista
- **SHCO_LB_TITLE_ROOT**: etiqueta para el título de las listas

6. Para poder filtrar por campos de otras tablas:

- Se añade el *join* en la sentencia de la TI del paso 3 si se desea mostrar el nombre de la FK. Normalmente al haber añadido el campo con el editor de sentencias de estructuras de nodo del diseñador de Meta4Objects, la sentencia se completa automáticamente.

En el ejemplo, se quiere dar la posibilidad de filtrar por el nombre del identificador de la *legal entity*, por el nombre de la *legal entity*, y por el nombre global de empleado. La sentencia de la estructura de nodo será:

```
SELECT @STD_ID_HR = A.STD_ID_HR, @SCO_GB_NAME = E.SCO_GB_NAME,
@SCO_ID_LEG_ENT = C.SCO_ID_LEG_ENT, @STD_N_LEG_ENT =
D.STD_N_LEG_ENT, @STD_OR_HR_PERIOD = A.STD_OR_HR_PERIOD,
@DT_START = A.DT_START, @DT_END = A.DT_END
FROM &SCO_HR_THEOR_TIME A, STD_HR_PERIOD B,
SCO_H_HR_LEGAL_ENTITY C, STD_LEG_ENT D, STD_PERSON E
WHERE A.ID_ORGANIZATION = B.ID_ORGANIZATION(+) AND A.STD_ID_HR =
B.STD_ID_HR(+) AND A.STD_OR_HR_PERIOD = B.STD_OR_HR_PERIOD(+)
AND C.ID_ORGANIZATION(+) = B.ID_ORGANIZATION AND C.SCO_ID_HR(+)
= B.STD_ID_HR AND C.SCO_OR_HR_PERIOD(+) = B.STD_OR_HR_PERIOD AND
C.ID_ORGANIZATION = D.ID_ORGANIZATION(+) AND C.SCO_ID_LEG_ENT =
D.STD_ID_LEG_ENT(+) AND A.STD_ID_HR = E.STD_ID_PERSON(+)
```

- Se sobrescribe el método **SHCO_PARAM** (para lo cual se sobrescribe el nodo **SHCO_GN_ROOT**). Se añade la condición de que si esa variable es el nombre de nuestro Meta4Object se iguala MvValor (lo que devuelve el método) a un *string* que es el *from* y el *where* que copiamos de nuestra estructura de nodo.

```
If MsParam = "SHCO_TA_TR_HR_THEOR_TIME" Then
    MvValor = "FROM &SCO_HR_THEOR_TIME A, STD_HR_PERIOD B,
SCO_H_HR_LEGAL_ENTITY C, STD_LEG_ENT D, STD_PERSON E WHERE
A.ID_ORGANIZATION = B.ID_ORGANIZATION(+) AND A.STD_ID_HR =
B.STD_ID_HR(+) AND A.STD_OR_HR_PERIOD = B.STD_OR_HR_PERIOD(+)
AND C.ID_ORGANIZATION(+) = B.ID_ORGANIZATION AND
C.SCO_ID_HR(+) = B.STD_ID_HR AND C.SCO_OR_HR_PERIOD(+) =
B.STD_OR_HR_PERIOD AND C.ID_ORGANIZATION =
D.ID_ORGANIZATION(+) AND C.SCO_ID_LEG_ENT =
D.STD_ID_LEG_ENT(+) AND A.STD_ID_HR = E.STD_ID_PERSON(+)"
ElseIf MsParam = "SCO_ID_LEG_ENT" Then
    MvValor = "C"
ElseIf MsParam = "STD_N_LEG_ENT" Then
    MvValor = "D"
ElseIf MsParam = "SCO_GB_NAME" Then
    MvValor = "E"
```

```
EndIf  
Return(MvValor)
```

Modificación del Meta4Object al que afecta el filtro

Se detallan las condiciones que debe tener el Meta4Object al que se aplica el filtro:

- EL nodo sobre el cual voy a aplicar el filtro tiene que tener la misma tabla base que el Meta4Object nuevo que se ha creado.
- Hay que crear una propiedad en el nodo raíz **SHCO_GN_ROOT** llamada **SHCO_SENTENCE** de ámbito bloque y tipo básico *Long*, que luego se usa para filtrar.

Páginas JSP

Para usar este filtro no es necesario crear ninguna página, se llama a una genérica. La forma de llamarla es:

```
m4window('filtro','/servlet/CheckSecurity/JSP/shco_g0/  
shco_gen_tr.jsp?zsub='+ '<%=zsubsesion%>'+'&zC=SHCO_TA_TR_HR_TH  
EOR_TIME',' ',' ','500','200');
```

Donde zsubsesion es el nombre de la subsesión de la página madre y **SHCO_TA_TR_HR_THEOR_TIME** es el nombre del nuevo Meta4Object.

En la página madre hay que tener la función JavaScript m4fil_ret() que se ejecuta desde la hija y es la que se usa para refrescar la página dependiendo de cada funcionalidad.

Gestión del ciclo de vida

Introducción

La gestión del ciclo de vida es la gestión de los cambios a la funcionalidad que se realiza entre versiones.

Cuando se crea una funcionalidad de HTML con el asistente de plantillas, sea para Mantenimientos, Listas o Asistentes, se generan un bloque de información, un Meta4Object y un conjunto de páginas JSP.

La gestión del ciclo de vida afecta sobre todo los datos, los Meta4Objects y las páginas JSPs que se generan con las diferentes herramientas de cliente ligero. Toda la complejidad del ciclo de vida viene dada por solucionar la localización de los Meta4Objects y de las páginas JSPs.

Antes para modificar la funcionalidad HTML se podía hacer de las siguientes formas:

- Si era necesario modificar la funcionalidad de las páginas JSP simplemente se abrían con un editor y se modificaban.
- Para modificar la funcionalidad del Meta4Object se podía hacer de dos maneras: o abriendo el Meta4Object con el diseñador de Meta4Object y realizando las modificaciones oportunas o creando un Meta4Object nuevo que heredara del Meta4Object cuya funcionalidad se quisiera modificar.

Actualmente las herramientas de cliente ligero incluyen los asistentes de plantillas de Mantenimientos y Listas, y Asistentes que soportan la modificación de funcionalidad de las formas que se ha venido haciendo hasta ahora y además siguen la misma metodología, heredando el Meta4Object cuando sea requerido por el desarrollador de funcionalidad.

Modificación de funcionalidad

El problema de la localización y parametrización viene dado por la complejidad que existe al realizar cambios y que esos cambios lleguen a todos los posibles usuarios de los mismos, teniendo en cuenta las modificaciones que han podido sufrir los objetos en el destino.

Actualmente existe tres niveles de desarrollo que afecta el ciclo de vida del desarrollo en cliente Web:

- Corporativo, es quien desarrolla la funcionalidad genérica que en principio valdría para todos los países y los clientes.

-
- País X, es quien parametriza la funcionalidad desarrollada para adaptarla a un país concreto.
 - Cliente Y, es donde se dan los últimos retoques para que la funcionalidad sea la esperada por un cliente en concreto.

Un cliente sólo tendrá que considerar los cambios realizados por un País X entre versiones junto con sus propios cambios en su ciclo de vida de desarrollo.

Funcionalidad nueva del producto estándar

Si se ofrece una funcionalidad nueva, tanto genérica como específico para un país, en el producto estándar de PeopleNet Ksystem, el cliente puede personalizarla para sus necesidades particulares, mediante la herencia. Este es el caso más sencillo ya que la herencia resuelve sin ningún problema.

Funcionalidad modificada del producto estándar

Si se ofrece una funcionalidad modificada, tanto genérica como específico para un país, en una nueva versión del producto estándar de PeopleNet Ksystem, el cliente puede conservar estas modificaciones en el Meta4Object y la actualización de las páginas también mediante la herencia. Sin embargo, en caso de que el cliente haya realizado una parametrización, hay que tener en cuenta que pueden encontrarse inconsistencias que habrán que tratar y se pueden dar casos en los que con posterioridad el usuario tenga que tocar la funcionalidad a mano.

Funcionalidad modificada de la parametrización del cliente

En este caso, el cliente realiza un cambio local para personalizar la aplicación a sus necesidades y a continuación puede volver a modificar dicha parametrización local. Con solo conservar las modificaciones realizadas en el Meta4Object local y la actualización de las páginas locales se puede resolver el ciclo de vida del desarrollo local.

Implantación del ciclo de vida

Se utilizan las herramientas del cliente ligero proporcionadas por Meta4 para este tipo de tareas y una gestión más eficaz del ciclo de vida del desarrollo. Se pueden dar tres casos a la hora de utilizar estas herramientas:

- Nuevo: En el caso de crear un Mantenimiento, una Lista o un Asistente nuevo, seguirá comportándose como hasta ahora.

- Nuevo heredado: se describe a continuación en el apartado Nueva funcionalidad heredada.
- Existente: se describe a continuación en el apartado Modificar existente.

Nueva funcionalidad heredada

Este es el caso del cliente que desea añadir funcionalidad local a una funcionalidad proporcionado por el producto estándar de un país.

Utilizando las herramientas, se selecciona la opción **Nuevo heredado** que crea un nuevo bloque de información que hereda del que el desarrollador indique, es decir un nuevo Meta4Object que hereda del Meta4Object generado por el bloque de información del que se hereda, y nuevas páginas JSP. De esta forma siempre se podrá tener los últimos cambios del tipo corporativo o de un país.

La herramienta mostrará la información que se hereda y la que se puede sobrescribir la inferirá bien de la información del original.

Los datos que vienen heredados se visualizarán de forma especial para que el usuario sepa que son heredados.

Solo se podrán añadir objetos u elementos nuevos, no se podrán borrar ni modificar los que vengan heredados.

Modificar existente

En el caso de editar una funcionalidad existente, que sea heredada de otra, la cual ha podido sufrir cambios habrá que tener en cuenta que puede haber alguna situación inconsistente a resolver con la herramienta correspondiente.

La herramienta mostrará la información de forma que los elementos u objetos nuevos vayan a continuación de los heredados, es decir, si en la información del original (Lista, Mantenimiento o Asistente) existe 3 campos y en la nueva información 2 campos, en caso de que con posterioridad se añade uno mas a la información del original, los 2 campos de la información del nuevo siempre van a ir al final.

Para resolver las inconsistencias en la pantalla de edición, se indicarán como actuar en caso de conflicto.

En el caso de editar uno existente que ha tenido cambios manuales en el Meta4Object:

- Se permite mantener los cambios que se hayan realizado en métodos o propiedades del usuario.

-
- Se avisan los cambios que se hayan realizado en métodos o propiedades del usuario con el fin de permitir mantenerlos o ignorarlos. No se considerarán o mantendrán los cambios que no tengan que ver con la EN existente (como la creación de nuevos nodos, nuevas EN o la asociación de otras EN a los nodos).
 - Los cambios que se mantienen son a nivel global, es decir, todos los métodos y todas las propiedades o ninguna, no se ofrece la opción de elegir unos si y otros no.
 - Cuando se mantiene la información de los métodos que ha modificado el usuario a mano, estos pueden dejar de funcionar por múltiples causas, solo se indica el error al compilar la regla.

En el caso de editar uno existente que haya tenido cambios manuales en las páginas:

- Se respetarán dentro de lo posible los posibles cambios realizados dentro de las páginas JSPs previamente generadas.
- Se avisan los cambios que se hayan realizado en las páginas JSPs con el fin de permitir mantenerlos o ignorarlos.
- Se utiliza el mecanismo del CVS para realizar la actualización de la página nueva con los cambios de la modificada.
- Los cambios que se mantienen son a nivel global, es decir, todos los cambios o ninguno, no se ofrece la opción de elegir unos si y otros no.
- El proceso automático del combinar entre páginas puede requerir un proceso manual posterior en caso de que alguna inconsistencia se quede sin resolver porque el CVS no sepa como comportarse.

Modificar funcionalidad directamente con la herramienta del cliente ligero

Este caso consiste en añadir funcionalidad, corregir errores etcétera usando las herramientas del cliente ligero.

Es en este caso, a la hora de generar el Meta4Object y las páginas JSP debe comprobar las posibles modificaciones directas que se hayan realizado sobre el Meta4Object o las páginas JSP y mantener estas modificaciones.

Casuística de las plantillas

En este apartado se detallan los procesos a seguir para modificar algunos comportamientos estándar en caso de necesidad.

Modificar el comportamiento del botón limpiar en los wizard

El objetivo del botón limpiar es eliminar todos los datos de un paso de un wizard dejando los campos vacíos incluidos aquellos que tienen un valor por defecto. Si necesita en algún momento que este botón limpie sólo aquellos valores introducidos por el usuario y que vuelva a establecer los valores por defecto en aquellos campos que estén definidos así, lea atentamente las instrucciones que se detallan.

Figura 35. Comportamiento del botón limpiar

Imagine que desea borrar el contenido de los campos y establecer el valor por defecto de las casillas de verificación (marcada la primera y desmarcada la segunda). Con el comportamiento actual la aplicación estándar limpiaría todos los valores. Para modificar esta característica siga estas recomendaciones:

- En el Meta4object correspondiente, para cada campo que tenga un valor por defecto se creará una propiedad. Estas propiedades se leerán en los inputs ocultos de la página jsp de tal forma que el valor de éstos no se modifiquen nunca en pantalla y por tanto mantengan siempre los valores por defecto.
- En la página jsp es necesario hacer los siguientes cambios:
 - Los valores por defecto irán en inputs ocultos.
 - En el link de limpiar, tras llamar a la `m4clearform()`, invocar a una nueva función javascript llamada, por ejemplo, `initvalues()`. Esta función restablecerá los valores por defecto en aquellos input que así lo requieran.

EJEMPLO

```
//link del botón limpiar
<th><a title=""<%=zSHCOLBCLEAN_val%>"
href="javascript:m4clearform('NombreFormulario');limp();m4limwz
();initvalues ();"><img alt="<%=zSHCOLBCLEAN_val%>" <%@ include
file="../../../files_gif/ic_des.jsp" %> /></a></th>
```

Un ejemplo del contenido de esta función sería el siguiente:

```
function initvalues(){
//Repetir para cada campo que requiera reestablecer para el valor
m4valor('NombreFormulario','input1',m4valor('NombreFormulario','in
put1_default','','get'),'set');
}
```

El código de la función se insertará en el lugar apropiado siguiendo la metodología señalada en el apartado Librerías: metodología desarrollo de código javascript en páginas JSP del capítulo Metodología.

Generación automática de páginas del cliente Web

3

Introducción

Existen herramientas para automatizar el proceso de creación y modificación de las páginas del cliente Web. Estas herramientas se encuentran dentro de las herramientas de desarrollo y se puede acceder a ellas desde esta ruta de menú:

Acceso

Herramientas de desarrollo | Herramientas de cliente ligero

Estas herramientas le proporcionan un modo rápido e intuitivo de crear o modificar diferentes tipos de páginas.



Para obtener una descripción completa del formato y el uso de este tipo de páginas, consulte los apartados *Plantilla de mantenimiento* y *Plantilla de asistentes* del capítulo *Plantillas* de este manual.

En próximas versiones se incluirán más herramientas para automatizar la generación de Meta4Objects y los distintos tipos de páginas JSP.

Visión general de la generación automática de páginas

La opción Mantenimientos y Listas le proporciona una herramienta eficaz e intuitiva que permite crear y mantener las páginas JSP de tipo lista y mantenimiento y la opción Asistentes permite crear y mantener las páginas JSP de asistentes.

Las herramientas en sí son asistentes que le guían en los procesos de:

- Creación de nuevos Meta4Objects y sus correspondientes páginas JSP de tipo lista y mantenimiento, y de asistentes.
- Modificación y regeneración de Meta4Objects existentes y sus páginas JSP

Tras asistirle en la definición, automáticamente las herramientas generan las páginas JSP y los Meta4Object correspondientes para las Listas, Mantenimientos y los Asistentes.



Las herramientas, tanto Mantenimientos y Listas como Asistentes, sólo deben usarse para crear Meta4Objects nuevos o modificar otros que *inicialmente se crearon con esta herramienta*. Los Meta4Objects y páginas que se hayan creado manualmente sólo se podrán modificar manualmente. Consulte en el Capítulo 2 de este manual las instrucciones para modificar manualmente las páginas.

Requisitos previos

Para que esta función se ejecute correctamente y pueda probar las páginas JSP que usted genera, debe cerciorarse de que los parámetros de la aplicación relacionados con el *SingleSignOn (conexión unificada)* y las *Plantillas JSP* estén bien configurados. Estos parámetros se definen inicialmente con valores predeterminados, pero si es necesario, se pueden personalizar. Para revisar dichos parámetros, utilice la siguiente ruta de menú.

Acceso

Herramientas de desarrollo | Configuración de aplicación | Parámetros de aplicación

La configuración del *SingleSignOn (conexión unificada)* se encuentra en el grupo FORMS y se describe en el capítulo *SingleSignOn* de la segunda parte de este manual.

En la siguiente tabla se describen los parámetros de los distintos apartados del grupo Plantillas JSP.

Parámetro	Descripción
PARÁMETROS JSP	
RETURN	Cuando el número de registros excede el valor <i>WINDOW</i> , éste es el número de grupos de registros que quiere que aparezcan en una fila debajo de la lista. El valor por defecto es 5 .
WINDOW	El número de registros que quiere que aparezca en la lista de una página. El valor por defecto es 20 .
PARÁMETROS DE META4OBJECT	
META4OBJECT_INHERIT	El identificador del Meta4Object del que va a heredar para los Meta4Objects de mantenimiento. El valor por defecto es SHCO_GN_MT .
METHOD_OVERWRITE_NODEPARAM METHOD_OVERWRITE_PARAM NODE_OVERWRITE_LABEL NODE_OVERWRITE_PARAM RULE_OVERWRITE_NODEPARAM RULE_OVERWRITE_PARAM	Elementos de META4OBJECT_INHERIT que se utilizan para sobrescribir valores.
TI_INHERIT	El identificador del Meta4Object del que va a heredar para los Meta4Objects de mantenimiento. El valor por defecto es SHCO_GN_MT .
PARÁMETROS DE PRUEBA	
PORTAL	El URL de la página de inicio del servidor Web.

Resumen de la generación automática

La primera fase consta de tres pasos en los que:

- Se indica si se está creando un Meta4Object nuevo o se está trabajando con uno ya existente.
- Se definen o vuelven a definir las características principales del Meta4Object.
- Se especifica si se va a generar de modo simultáneo una página JSP correspondiente de tipo lista, de tipo mantenimiento o ambas.

La segunda fase del proceso le guía a través de la definición de la página o páginas que se van a generar. Esta fase consta de tres o seis pasos, en función de si se crean uno o ambos tipos de páginas.

Uso de la opción Mantenimientos y Listas

En este apartado del capítulo se detallan todos los pasos del proceso de creación o modificación.

Para empezar, seleccione la siguiente opción de menú:

Acceso

Herramientas de desarrollo | Herramientas de cliente ligero | Mantenimientos y Listas

Seleccionar un Meta4Object nuevo o existente

La opción Mantenimientos y Listas le permite crear un Meta4Object nuevo o modificar la definición de uno existente y volverlo a generar. Al seleccionar esta opción se abre la ventana **Seleccionar un Meta4Object nuevo o existente**.

Si está creando un objeto nuevo, deberá seleccionar la tabla de datos que se va a utilizar como base del objeto y, mediante el Editor de sentencias, redactar la sentencia SQL que le proporcionará los campos necesarios.

Si está heredando un objeto ya existente, introduzca el código identificativo del Meta4Object existente en el campo correspondiente. Recuerde que, para que el Meta4Object se procese correctamente, debe haberse creado con esta herramienta. Además habrá metadatos no modificables en este Meta4Object heredado que viene del Meta4Object padre.

Si está regenerando un objeto ya existente, introduzca el código identificativo del Meta4Object existente en el campo correspondiente. Recuerde que, para que el Meta4Object se procese correctamente, debe haberse creado con esta herramienta. Si, por el contrario, se ha creado manualmente, no debe modificarse de este modo, sino de forma manual.

Además puede ver los elementos por identificador o nombre en las siguientes pantallas.

Consulte el capítulo 2 de este manual para obtener información sobre la modificación manual de estos Meta4Objects.

Introducir datos del Meta4Object

En el primer paso, se definen o actualizan las características principales del Meta4Object en la ventana Introducir datos del Meta4Object.



Al crear un Meta4Object nuevo, algunos campos de esta ventana se rellenan automáticamente mediante reglas de denominación y valores predeterminados estándar; no obstante, se pueden modificar.

Rellene las propiedades básicas de la pestaña Información Meta4Object de la ventana teniendo en cuenta que:

- Al introducir el **ID Meta4Object** único el sistema permite regenerar los nombres por defecto de algunas propiedades avanzadas.
- Al introducir el **Nombre Meta4Object** intuitivo, automáticamente se rellenan los campos del **Título de la página** y la **Cabecera de la página** con el mismo texto por defecto. Estos campos aparecen como títulos y cabeceras de ventana tanto en las páginas de tipo lista como en las de tipo mantenimiento.

En Propiedades avanzadas, generalmente no se modifica los parámetros:

- El **SQL lógico** corresponde a la tabla ya seleccionada para el nodo raíz de este Meta4Object. Haga clic en el botón de remonte situado junto a la sentencia SQL lógico para abrir el Editor de sentencias, donde podrá modificar la sentencia.
- El **ID Nodo** e **ID Estructura de nodo** se rellenan automáticamente al introducir el ID Meta4Object.
- El **Proyecto** se utiliza para asociar este Meta4Object a un proyecto determinado. Consulte el Explorador de proyectos en la ayuda en línea.
- El **Indicador de propietario** es un valor numérico que sale por defecto con el que depende de la configuración de la aplicación.

- La casilla de verificación **Generar Meta4Object** indica si, al finalizar el proceso, se debe regenerar el Meta4Object junto con las páginas JSP. Desactive esta casilla de verificación si está utilizando la herramienta para hacer cambios sólo en las páginas JSP y va a dejar intacto el Meta4Object.



Cualquier cambio al Meta4Object requiere automáticamente la regeneración del Meta4Object. Por ejemplo, si modifica el ID o el nombre del Meta4Object, la casilla de verificación Generar Meta4Object se activa pero se deshabilita para asegurarse de que el Meta4Object se regenere automáticamente.

Si hereda el Meta4Object, se podrá agregar a la sentencia SQL lógico, pero no se podrá eliminar ninguna parte de la definición heredada desde el Editor de sentencias.

Además, si hereda un Meta4Object, los parámetros **ID Nodo**, **Título de la página** y **Cabecera de la página** no son editables porque corresponden a los metadatos del Meta4Object padre.

En la pestaña Información herencia, especifique el Meta4Object y otros elementos de los cuales el nuevo Meta4Object heredará su estructura. El valor por defecto hace referencia a un Meta4Object de mantenimiento estándar que prácticamente *no es necesario modificar*.

Introducir la información de la página JSP

En este paso, se indican qué página o páginas se van a generar, los nombres de sus archivos y dónde se van a guardar. Active la casilla de verificación **JSP de Lista**, la casilla **JSP de Mantenimiento** o ambas.



El número de pasos de este proceso depende de si genera ambos tipos de páginas o sólo una. Si decide generar ambas páginas, tendrá que realizar todos los pasos.

Rellene el resto de los campos de la ventana teniendo en cuenta que:

- La **Ruta de las páginas** es la ruta de acceso en el servidor a los directorios en los que se encuentran las páginas JSP correspondientes a las tareas.
- Los **ID Tarea** son los nombres de los directorios en los que se deben agrupar las páginas en el servidor.
- Al crear páginas nuevas, los nombres de los archivos JSP se escriben automáticamente mediante reglas de denominación, pero siguen siendo modificables (página de mantenimiento, lista y validación)

- El **Nivel de menú** debe reflejar el nivel en el que desea que aparezca la página o páginas dentro de la estructura de menús.
- Para los JSP de lista:
 - La página de validación es la encargada de comprobar que los datos que introduzca el usuario son correctos. Por ejemplo, en un formulario en el que se debe indicar el país, después de introducir el código identificativo del país se recupera automáticamente el nombre correspondiente. Se valida que el código introducido sea correcto y en caso contrario invoca la página de la lista para que el usuario pueda seleccionar el valor adecuado. Más información, Definición de un elemento nuevo en el área de edición (controles de cajas de texto).
 - Active la casilla de verificación **Botón de nuevo** si la lista va a permitir a los usuarios crear registros nuevos.
 - Si en una lista desea mostrar el enlace **Filtro avanzado** active la casilla de verificación **Filtro dinámico**. Este enlace permitirá al usuario establecer sentencias de filtrado más complejas a través de las páginas **Filtro dinámico** y **Filtro html**.
 - Desactive la casilla de verificación **Cargar datos**, si no quiere que la lista inicialmente venga con datos.
- Para los JSP de mantenimiento, introduzca el nombre del archivo de ayuda correspondiente a la página que está creando. Normalmente el nombre es igual al nombre de la página JSP pero con extensión MT. Introduzca aquí sólo el nombre del archivo, no la ruta de acceso.

Si hereda un Meta4Object y sus páginas, no podrá editar el ID tarea o el nombre de la página correspondientes, tanto para las páginas JSP de Lista como las de Mantenimiento.

Seleccionar campos para el filtro de la página JSP de tipo lista

En la ventana Seleccionar campos para el filtro de la página JSP de tipo lista, utilice los botones de flecha para mover los campos entre los cuadros de lista hasta que el cuadro **Campos del filtro** contenga sólo los campos que desea incluir en el filtro de la página de lista. Por defecto, los campos de clave primaria aparecen en el cuadro **Campos del filtro**. En este cuadro pueden aparecer dos tipos de campos filtro:

-  Campos estándares
-  Campos heredados en color gris claro

Al heredar un Meta4Object y sus páginas, por lógica, los campos de filtro heredados no se pueden modificar y no se pueden quitar de la lista de campos filtro. En este caso, sólo se podrá agregar más campos del Meta4Object al filtro.

Haga clic en el botón de remonte asociado a **Sociedad** para abrir el Listado de sociedades para seleccionar todas las sociedades que deben incluirse en el filtro. En esta ventana seleccione todas las sociedades a las que se debe propagar el filtro. Si desea seleccionar al mismo tiempo más de una sociedad, mantenga pulsada la tecla Mayúsculas o ALT, y active con el ratón todas las sociedades que desee.

Seleccionar campos para la tabla de la página JSP de tipo lista

En la ventana Seleccionar campos para la tabla de la página JSP de tipo lista, utilice los botones de flecha para mover los campos que quiera incluir en el filtro de la página de tipo lista desde el cuadro **Campos del Meta4Object** al cuadro **Campos de la tabla**.

En este cuadro pueden aparecer tres tipos de elementos:

-  Campos estándares
-  Campos heredados en color gris claro
- —o—o—o—o—o—o— Separadores

En el cuadro **Campos de la tabla**, los separadores representan los bordes de las columnas. Seleccione un campo o un separador y utilice los botones de flecha arriba y abajo para ordenar los elementos de la columna. Si desea incluir varios valores de campo en una columna, haga doble clic sobre el separador o use la flecha para eliminar el separador que hay entre ellos. Si necesita más separadores, simplemente seleccione el separador en el cuadro Campos del Meta4Object y mediante los botones de flecha pase una copia al cuadro de **Campos de la tabla**.

Si desea que los usuarios puedan ordenar el contenido de la tabla por un campo determinado, active la casilla de verificación que aparece junto a dicho campo. Debe seleccionar al menos un campo para ordenar la tabla. El campo indicado en **Campo filtro ordenación** determina el modo en que el campo aparecerá ordenado por defecto.

Al heredar un Meta4Object y sus páginas, por lógica, los campos heredados no se pueden quitar de la lista de campos de la tabla. En este caso, sólo se podrá agregar más campos del Meta4Object a la tabla de la página de tipo Lista. Sin embargo, se puede cambiar el orden de los campos heredados y se puede ordenar por uno de estos campos.

Además, se podrá ocultar los campos heredados de la página JSP que se está creando. Seleccione el campo heredado a ocultar y haga clic en el botón conmutador . Si quiere que el campo heredado oculto vuelva a estar visible en la página que se está creando, haga clic en el botón conmutador .

Seleccionar campos de retorno para la página JSP de tipo lista

Un campo de retorno es meramente un campo que se devuelve o se muestra en la página JSP que ve el usuario.

En este paso, identifique los campos de retorno que deben tener comportamiento de modo lista y los que deben tener comportamiento de modo filtro.



Consulte la descripción de las páginas de tipo lista frente a las páginas de tipo filtro tipo lista en el apartado *Tablas de mantenimiento* del capítulo "Plantillas".

Mediante los botones de flecha, desplace los campos de un cuadro de lista a otro hasta que el cuadro **Campos de retorno en modo lista** contenga sólo los campos que deben tener comportamiento de modo lista y el cuadro **Campos de retorno en modo lista** contenga sólo los campos que deben tener comportamiento de modo lista.

En este cuadro pueden aparecer dos tipos de campos:

-  Campos estándares
-  Campos heredados en color gris claro

Al heredar un Meta4Object y sus páginas, por lógica, los campos heredados no se pueden modificar y no se pueden quitar de la lista de campos de retorno. En este caso, sólo se podrá agregar más campos del Meta4Object los campos de retorno.

Los botones de flecha arriba y abajo le permiten ordenar los campos en los cuadros para garantizar que los valores se obtienen en el mismo orden de los campos correspondientes.

Definir etiquetas de texto para la página JSP de tipo mantenimiento

Las etiquetas de texto le permiten definir hasta 30 caracteres de texto fijo para insertar en la zona de entrada de datos de la página JSP.

Puede definir varias etiquetas de texto para utilizarlas en una única página. El sistema genera un código alfanumérico en el campo **ID etiqueta de texto**.

Para crear una etiqueta nueva:

1. Haga clic en el botón **Nuevo** e introduzca el texto en el campo **Texto**.

Si se ha equivocado al introducir el texto, haga clic en el botón **Anular** para vaciar el campo y volver a introducir texto.

2. Haga clic en el botón **Aplicar**.

Para modificar una etiqueta existente:

1. Seleccione la etiqueta a cambiar e introduzca el texto nuevo en el campo **Texto**.

Si se ha equivocado al introducir el texto, haga clic en el botón **Anular** para recuperar el texto original y volver a introducir el texto.

2. Haga clic en el botón **Aplicar**.

Para borrar un ID etiqueta:

1. Seleccione la etiqueta.
2. Haga clic en **Borrar** para eliminarla.

Al heredar un Meta4Object y sus páginas, por lógica, las etiquetas heredadas no se pueden ni modificar ni borrar. Sin embargo, se puede agregar nuevas etiquetas a la tabla de la página correspondiente.

Seleccionar campos para la tabla de mantenimiento

En la ventana Seleccionar campos para la tabla de mantenimiento, utilice los botones de flecha para mover los campos entre los cuadros de lista hasta que el cuadro **Campos de la tabla** contenga sólo los campos con los que desea formar las columnas de la tabla en la lista de registros de la página de mantenimiento.

En este cuadro pueden aparecer tres tipos de elementos:

-  Campos estándares
-  Campos heredados en color gris claro
- —o—o—o—o—o—o— Separadores

En el cuadro **Campos de la tabla**, los separadores representan los bordes de las columnas. Seleccione un campo o un separador y utilice los botones de flecha arriba y abajo para ordenar los elementos de la columna. Si desea incluir varios valores de campo en una columna, haga doble clic sobre el separador o use la flecha para eliminar el separador que hay entre ellos. Si necesita más separadores, simplemente seleccione el separador en el cuadro Campos del Meta4Object y mediante los botones de flecha pase una copia al cuadro de **Campos de la tabla**.

Si desea que los usuarios puedan ordenar el contenido de la tabla por un campo determinado, active la casilla de verificación que aparece junto a dicho campo. Debe seleccionar al menos un campo para ordenar la tabla. El campo indicado en **Campo filtro ordenación** determina el modo en que el campo aparecerá ordenado por defecto.

Al heredar un Meta4Object y sus páginas, por lógica, los campos heredados no se pueden quitar de la lista de campos de la tabla. En este caso, sólo se podrá agregar más campos del Meta4Object a la tabla de la página del tipo Mantenimiento. Sin embargo, se puede cambiar el orden de los campos heredados y se puede ordenar por uno de estos campos.

Además, se podrá ocultar los campos heredados de la página JSP que se está creando. Seleccione el campo heredado a ocultar y haga clic en el botón conmutador . Si quiere que el campo heredado oculto vuelva a estar visible en la página que se está creando, haga clic en el botón conmutador .

Definir los campos de edición de la página JSP de mantenimiento

Esta ventana, que constituye el último paso en la definición de una página JSP (del tipo mantenimiento o de un paso guiado determinado, por ejemplo), es un editor que le permite especificar los elementos que van a aparecer en la zona de entrada de datos de la página JSP. Cada elemento se sitúa en una celda que refleja aproximadamente la ubicación de los elementos en la página JSP final. Puede agregar o eliminar elementos de la página, y moverlos de una celda a otra.

Al crear una nueva página JSP, el asistente coloca automáticamente los campos clave en la cuadrícula de diseño por defecto. Revise este diseño y modifíquelo si es necesario. Para crear o borrar celdas, utilice los botones **Nuevo** y **Borrar**. Para mover un elemento situado dentro de la cuadrícula, emplee los comandos estándar de copiar y pegar de Windows®.

Al heredar un Meta4Object y sus páginas, los campos heredados de edición de la página JSP no se pueden modificar ni borrar. Aparecen desactivado en color gris claro y con el icono  en la fila correspondiente y las celdas que no se pueden modificar tienen un borde en color gris claro. En este caso, sólo se podrá agregar más campos de edición del Meta4Object para la página JSP correspondiente.

Sin embargo se puede representar o no representar estos campos de distintas maneras in la página JSP:

- En la cuadrícula, haga clic con el botón derecho sobre alguna celda para mostrar un menú contextual desde el cual puede cortar, copiar, pegar, agregar un campo nuevo o borrar el campo. Sólo puede copiar campos heredados, que se puede pegar en una celda de una fila nueva no heredada.
- Fuera de la cuadrícula, haga clic con el botón derecho sobre el ID de una fila para mostrar un menú contextual desde el cual puede insertar, borrar, subir, bajar, cortar, copiar y pegar filas, además de deshacer el borrado de una fila. Se puede hacer de todo con las filas heredadas, salvo pegar sobre una.

Utilizar estos menús contextuales permite un mayor rediseño de la página a pesar de la herencia.

Definición de un elemento nuevo en el área de edición

Agregue etiquetas de campo, campos, botones y casillas de verificación a la página de mantenimiento mediante este procedimiento, en el que tendrá que realizar el paso final de la opción Mantenimientos y Listas, Definir los campos de edición de la página JSP de mantenimiento.

1. Haga clic para resaltar la celda vacía en la que desea colocar el elemento en el formulario.
2. Haga clic en el botón **Nuevo**. Los campos situados por encima de la cuadrícula se activarán.
3. Seleccione el elemento **Tipo**. Cada tipo de elemento ofrece varias opciones de formato. Son las siguientes:
 - **Columnas**. Indique aquí cuántas columnas debe ocupar el campo.
 - **Agrupar**. Seleccione esta opción para todos los elementos que desee agrupar. Por ejemplo, una caja de texto de campo y su correspondiente botón de remonte.

A continuación se indican los tipos de elementos que se pueden agregar a una página JSP de mantenimiento.

Tabla 3. Tipos de elementos en las páginas JSP

Elemento	Objetivo y parámetros
Texto	Caja de texto de una sola línea que se emplea para introducir valores de campo. Seleccione el código identificativo del campo e indique mediante las casillas de verificación correspondientes si se trata de un campo habilitado o necesario. Especifique el tamaño de la caja en el campo Tamaño y el número máximo de caracteres en el campo Tamaño máximo. En las cajas en las que desea invocar la traducción y validación de códigos, marque la casilla de verificación Validar y a continuación indique: • Cómo desea traer los datos de validados: – Si desea traer al puesto cliente solamente el valor de la traducción marque el botón Uno a uno. Implica una transacción con el servidor para cada traducción que se haga (función m4translatepick). Suele utilizarse para listas voluminosas. – Si desea traer al puesto cliente todos los valores de la lista marque el botón Lista. Implica una transacción para todas las traducciones que se hagan (función m4translatelist). Es útil para listas de valores poco voluminosas. • Tarea de la página: indique la tarea de la página de lista y de validación. • Página destino: indique la página jsp de lista que se invoca en el caso de que la llamada a la página de validación no devuelva datos porque el valor introducido por el usuario sea incorrecto, no exista en la lista. • Página de validación: indica la página encargada de la traducción de códigos. Esta página contiene en formato xml la indicación del Meta4Object y nodo del que se van a extraer los datos, los campos de entrada con sus tipos, los campos de salida, etcétera. Una misma página de validación puede ser utilizada por diferentes páginas y en cada una de ellas pedir la traducción de un número diferente de elementos. Por tanto cuando cree una página de validación debe seleccionar el mayor número de campos que prevea va a necesitar. • Parámetros: campos por los que desea filtrar y que se van a pasar como parámetros a la página de validación. Normalmente el campo será un ID. • Valores de retorno: lista de campos cuyos valores desea mostrar en la página que invoca la traducción. Normalmente será el Nombre correspondiente al ID por el que ha filtrado. Si la página de validación no devuelve ningún valor se invoca la página de lista. Para las páginas jsp desarrolladas antes del SP3 el robot M4robotjsp2k2 permite hacer estos cambios de forma masiva. Más información, Incluir validaciones de datos en las cajas de texto del capítulo Operaciones automáticas sobre páginas JSP: Robot M4robotjsp2k2.
Etiqueta	Etiqueta de campo de sólo texto. Seleccione el código identificativo del campo e indique mediante la casilla de verificación correspondiente si se trata de un campo necesario.

Elemento	Objetivo y parámetros
Enlace	Etiqueta de campo de hipervínculo. Seleccione el código identificativo del campo e indique mediante la casilla de verificación correspondiente si se trata de un campo necesario. Para identificar el destino del enlace, introduzca el nombre del archivo y la tarea en los campos Tarea de la página y Página destino . Indique también los campos que va a utilizar como filtros para el enlace en el campo Parámetros .
Texto multilinea	Caja de texto de varias líneas que se emplea para introducir valores de campo. Seleccione el código identificativo del campo e indique mediante la casilla de verificación correspondiente si se trata de un campo necesario. Especifique el tamaño de la caja en los campos Columnas (para indicar el número de caracteres de una fila) y Filas (para indicar el número de filas).
Marca	Casilla de verificación. Seleccione el código identificado del campo cuyo valor determina si la casilla de verificación debe estar activada por defecto o no. Utilice los campos Marcada y Desmarcada para introducir los valores que determinan si la casilla de verificación debería aparecer marcada o no por defecto.
Opción	Botón de opción. Seleccione el código identificativo del campo. Si el botón de opción debe aparecer activado por defecto, introduzca un "1" en el campo Marcado . En caso contrario, introduzca un "0".
Lista	Botón de remonte. Seleccione el código identificativo del campo para el que debe encontrar los valores el botón de remonte. Introduzca la Tarea de la página y la Página destino de la lista que contiene los valores del campo. En el campo Parámetros , introduzca los códigos identificativos de los campos que desea encontrar en la base de datos; en el campo Valores de retorno introduzca los códigos identificativos de los campos de la página en los que deben aparecer los valores.
Calendario	Botón de calendario que permite seleccionar una fecha para introducirla en un campo asociado. Seleccione el código identificativo del campo asociado.
Moneda	Botón de moneda que permite seleccionar una moneda y una cantidad para introducirlas en un campo asociado. Seleccione el código identificativo del campo asociado e introduzca la información sobre la moneda y el cambio.
Texto	Este tipo de elemento es la etiqueta de texto que se definió en el paso <i>Definir etiquetas de texto para la página JSP de tipo mantenimiento</i> del asistente. Seleccione el código identificativo de la etiqueta de texto que desea incluir en la celda seleccionada actualmente.

4. Rellene los campos asociados a cada tipo de elemento.

5. Haga clic en el botón **Aplicar**.

Tenga en cuenta lo siguiente a la hora de diseñar la presentación de los campos en el área de edición:

-
- Piense en función de las columnas. Si desea mostrar en la página dos campos en una sola fila, cada fila de la cuadrícula deberá constar de cuatro columnas, una para cada caja de texto y etiqueta.
 - Utilice el parámetro Columnas. En el caso de las filas que contengan menos campos que otras de la misma página, establezca el parámetro de columnas de modo que los campos cubran el mismo número de columnas que las filas más largas.
 - Tenga en cuenta que los elementos agrupados mediante el parámetro Agrupar ocuparán una sola "celda" en la página resultante, a menos que se haya configurado lo contrario con el parámetro Columnas.

Al generar la página JSP de mantenimiento, se da formato automáticamente a los campos según las especificaciones de la cuadrícula de diseño. Una vez finalizado el proceso de generación, active la casilla de verificación **Probar páginas** para abrir la página y revisar su formato.

Si desea realizar algún cambio, reinicie la opción Mantenimientos y Listas y ajuste el diseño de la página. En caso de que sólo desee modificar el diseño de la página de un Meta4Object existente, no es necesario volver a generar el Meta4Object asociado, sólo la página. Por tanto, al reiniciar la opción, desactive la casilla de verificación **Generar Meta4Object** en el paso Introducir datos del Meta4Object del asistente.

Finalización del proceso

Una vez finalizado el proceso de definir el Meta4Object nuevo y las páginas nuevas, se procede a generar estos objetos.

Generar Meta4Objects y páginas

Sin embargo, antes de hacerlo, el sistema verifica si hay posibles conflictos porque los objetos ya existan. Desde la ventana **Informe del proceso** puede indicar qué se debe hacer al encontrar conflictos.

1. Verifique si hay conflictos en la parte del Meta4Object.
 - Seleccione la opción **Actualizar**, si quiere machacar el Meta4Object existente con los cambios nuevos.
 - Seleccione la opción **Crear nuevo**, si quiere borrar el Meta4Object existente y volver a crearlo partiendo de los datos nuevos.
2. Verifique si hay conflictos en la parte de las páginas.
 - Seleccione la opción **Actualizar**, si quiere machacar la página existente con los cambios nuevos.

- Seleccione la opción **Crear nuevo**, si quiere borrar la página existente y volver a crearla partiendo de los datos nuevos.
- 3. Asegúrese de desactivar las casillas de verificación de las páginas que no quiere tocar.
- 4. Haga clic en el botón **Sí** para completar la generación.



El proceso de generación puede ser lento.

Al terminar la generación del Meta4Object y las páginas definidas, el sistema prepara un informe que se muestra a continuación.

Ver el informe del proceso de generación

Esta ventana le indica si el proceso de generación se ha realizado correctamente o no.

En caso afirmativo, podrá probar inmediatamente la página o páginas generadas durante el proceso. Para ello, compruebe que la casilla de verificación **Probar páginas** está activada y haga clic en el botón **Aceptar**.

En caso contrario, si el proceso no se ha realizado correctamente, existen dos niveles de errores:

- Si el error no es importante, la casilla de verificación **Probar páginas** está activada y deshabilitada. Le permite testear las páginas.
- Si el error es importante, la casilla de verificación **Probar páginas** está desactivada y deshabilitada. No le permite testear las páginas.

Haga clic en el botón **Detalles** para ampliar la ventana y conocer los errores concretos. Las pestañas agrupan los detalles de los errores relacionados con el proceso del asistente, la generación del Meta4Object y la generación de la página o páginas.

Para solucionar los errores del asistente o del Meta4Object, salga de esta ventana y reinicie la opción Páginas de lista y mantenimiento.

Para solucionar los errores de creación de páginas, en caso de existir conflictos:

1. Haga clic en un mensaje para ir directamente a la página.
2. Identifique el conflicto.

Si el sistema no sabe qué código utilizar, marca el inicio del código antiguo conflictivo con una fila nueva que incluye <<<< y el fin de este código con === en otra línea. A continuación proporciona el código nuevo hasta su fin, que marca con >>>> en otra fila.

EXAMPLE

```
<<<<<< file.txt
...
código antiguo
...
=====
...
código nuevo
...
>>>>>> 1.1.1.2
```

3. Elimine el trozo de código que no corresponde junto con estas marcas.

Uso de la opción Asistentes

En este apartado del capítulo se detallan todos los pasos del proceso de creación o modificación.

Para empezar, seleccione la siguiente opción de menú:

Acceso

Herramientas de desarrollo | Herramientas de cliente ligero | Asistentes

La opción Asistentes es una herramienta eficaz e intuitiva que permite crear y mantener las páginas JSP de tipo asistente para un proceso funcional. Utilice esta opción para:

- Crear nuevos Meta4Objects y sus correspondientes páginas JSP.
- Heredar Meta4Objects existentes y sus correspondientes páginas JSP.
- Modificar y volver a generar Meta4Objects existentes y sus páginas JSP.



Esta opción *no* se debe utilizar para modificar los Meta4Objects que se crearon originalmente de modo manual. Utilícela sólo para los Meta4Objects y páginas JSP que se crearon mediante esta opción.

Esta opción funciona como un asistente que le guía a través de un proceso en dos fases. La primera de ellas consta de siete pasos en los que:

1. Se indica si se está creando un Meta4Object nuevo o se está trabajando con uno ya existente, sea un Meta4Object heredado o no.
2. Se definen o vuelven a definir las características principales del Meta4Object.
3. Se define los datos del proceso guiado que corresponde al asistente.
4. Se definen los campos del filtro para el asistente.
5. Se define cómo se conecta el filtro del asistente a una lista determinada.
6. Se especifica las etiquetas de las páginas JSP del asistente.
7. Se define el orden de los pasos del asistente y se crean nuevos pasos.

La segunda fase del proceso le guía a través de la definición de los pasos siguientes del asistente con sus páginas que se van a generar. Esta fase consta de tres o cuatro pasos, en función de si se crean para un registro o varios:

1. Se define un nodo del Meta4Object del asistente para el paso en concreto.
2. Se conectan o enlazan los campos de la tabla base del asistente con los campos del paso.
3. Si el paso que se está modificando permite varios registros, se identifican los campos que formarán las columnas de la tabla con separadores.
4. Se definen los campos de edición de la página del paso correspondiente del asistente.

Seleccionar un Meta4Object nuevo o existente

La opción Asistentes le permite crear un Meta4Object nuevo, o modificar la definición de uno existente y volverlo a generar. Al seleccionar esta opción se abre la ventana **Seleccionar un Meta4Object nuevo o existente**.

Si está creando un objeto nuevo, se pide la tabla que se utilizará como la tabla base del asistente.

Si está heredando un objeto ya existente, introduzca el código identificativo del Meta4Object existente en el campo correspondiente. Recuerde que, para que el Meta4Object se procese correctamente, debe haberse creado con esta herramienta. Además, habrá metadatos no modificables en este Meta4Object heredado que vienen del Meta4Object padre.

Si está regenerando un objeto ya existente, introduzca el código identificativo del Meta4Object existente en el campo correspondiente. Recuerde que, para que el Meta4Object se procese correctamente, debe haberse creado con esta herramienta. Si, por el contrario, se ha creado manualmente, no debe modificarse de este modo, sino de forma manual. Consulte el capítulo 2 de este manual para obtener información sobre la modificación manual de estos Meta4Objects y sus páginas JSP.

Además puede ver los elementos por identificador o nombre en las siguientes pantallas.

Introducir datos del Meta4Object

En el primer paso, se definen o actualizan las características principales del Meta4Object del asistente.



Al crear un Meta4Object nuevo, algunos campos de esta ventana se rellenan automáticamente mediante reglas de denominación y valores predeterminados estándar; no obstante, se pueden editar.

Rellene las propiedades básicas de la pestaña Información Meta4Object de la ventana teniendo en cuenta que:

- Al introducir el **ID Meta4Object** único el sistema permite regenerar los nombres por defecto de algunas propiedades avanzadas.
- Al introducir el **Nombre Meta4Object** intuitivo, automáticamente se rellenan los campos del **Título de la página** y la **Cabecera de la página** con el mismo texto por defecto. Estos campos aparecen como títulos y cabeceras de ventana en las páginas de los pasos.

En Propiedades avanzadas, generalmente no se modifica los parámetros.

- El **SQL lógico** corresponde a la tabla base ya seleccionada para el nodo raíz de este Meta4Object. Haga clic en el botón de remonte situado junto a la sentencia SQL lógico para abrir el Editor de sentencias.
- El **ID Tabla base del asistente** e **ID Tabla base filtro tipo lista** se rellenan automáticamente con la tabla base solicitada en el paso anterior.
- El **Proyecto** se utiliza para asociar este Meta4Object a un proyecto determinado. Consulte el Explorador de proyectos en la ayuda en línea.
- El **Indicador de propietario** es un valor numérico que sale por defecto con el que depende de la configuración de la aplicación.
- La casilla de verificación **Generar Meta4Object** indica si, al finalizar el proceso, se debe regenerar el Meta4Object junto con las páginas JSP. Desactive esta casilla de verificación si está utilizando la herramienta para hacer cambios sólo en las páginas JSP y va a dejar intacto el Meta4Object.



Cualquier cambio al Meta4Object requiere automáticamente la regeneración del Meta4Object. Por ejemplo, si modifica el ID o el nombre del Meta4Object, la casilla de verificación Generar Meta4Object se activa pero se deshabilita para asegurarse de que el Meta4Object se regenere automáticamente.

Si hereda el Meta4Object, se podrá agregar a la sentencia SQL lógico, pero no se podrá eliminar ninguna parte de la definición heredada desde el Editor de sentencias.

Además, si hereda un Meta4Object, los parámetros de **Título de la página**, **Cabecera de la página**, **ID Tabla base del asistente** y **ID Tabla base filtro tipo lista** no son editables porque corresponden a los metadatos del Meta4Object padre.

En la pestaña Información herencia, especifique el Meta4Object y otros elementos de los que el nuevo Meta4Object heredará su estructura. El valor por defecto hace referencia a un Meta4Object de asistencia estándar que prácticamente *no es necesario modificar*.

Si hereda un Meta4Object el parámetro **ID EN** a heredar que refiere al ID de la estructura de nodo del Meta4Object padre, no es editable.

Introducir datos del proceso guiado

En este paso, se definen los datos globales del proceso guiado o el asistente que se está creando.

Rellene los campos de la ventana teniendo en cuenta que:

- La **Ruta de las páginas** es la ruta de acceso en el servidor a los directorios de la *página JSP correspondientes a las tareas*. Por defecto, se utiliza la ruta que se utilizó la última vez.
- El **ID tarea** es el nombre del directorio en el que la página o páginas del asistente se deben agrupar en el servidor.
- Para la **Página del asistente**, introduzca el nombre de la página principal del asistente. Introduzca aquí sólo el nombre del archivo, no la ruta de acceso. Al crear páginas nuevas para los distintos pasos del asistente, los nombres de los archivos JSP se escriben automáticamente mediante reglas de denominación, pero siguen siendo editables.
- El **Nivel de menú** debe reflejar el nivel al que desea que aparezca la página o páginas dentro de la estructura de menús.
- Active la casilla de verificación **Volver a evaluar los pasos** si quiere verificar que se han cumplido las condiciones antes o después de proceder al siguiente paso. Para mayor información acerca de esta propiedad, véase el apartado "Proceso reanudable" a continuación.
- El **ID tarea del filtro tipo lista** es el nombre del directorio en el que la página o páginas de lista se deben agrupar en el servidor.
- Para la **Página del filtro tipo lista**, introduzca el nombre de la página JSP de lista a utilizar como filtro. Introduzca aquí sólo el nombre del archivo, no la ruta de acceso.
- Para los JSP de lista, active la casilla de verificación **Crear botón de nuevo filtro tipo lista** si la lista va a permitir a los usuarios crear registros nuevos.
- Active la casilla de verificación **Proceso reanudable** si quiere permitir al usuario final parar temporalmente en algún paso del asistente y completar los datos de los pasos restantes posteriormente.
- El **ID tarea retorno** es el nombre del directorio en el que la página o páginas del asistente se deben agrupar en el servidor.

Para la **Página de retorno**, introduzca el nombre de la página que se devuelve al usuario la próxima vez que reanude el proceso guiado. Le permite continuar desde donde había dejado el proceso y completar los pasos y datos restantes que faltan. Introduzca aquí sólo el nombre del archivo, no la ruta de acceso.

- Active la casilla de verificación **Habilitar botón de guardar** si quiere permitir la grabación de datos durante el proceso guiado.
- Active la casilla de verificación **Cargar los datos temporales** si quiere cargar los datos del proceso incompleto temporalmente guardados.

Si hereda un Meta4Object y sus páginas, no podrá editar los parámetros **ID tarea**, **Página del asistente**, **ID tarea retorno** y **Página de retorno**.

Cuando los valores de la ventana sean los deseados, haga clic en el botón **Siguiente** si quiere definir los campos para el filtro del asistente. Si no, haga clic en **Finalizar**, sobre todo para procesos modificados.

Proceso reanudable

Cuando un proceso es reanudable, significa que el usuario debe decidir si el proceso queda finalizado, se guarda en un paso determinado o se cancela. Esto permite mantener abierto un proceso durante varias sesiones del usuario. Para ello, cualquier acción realizada se actualiza sobre una tabla temporal que contiene los datos temporales del proceso abierto. Por lo tanto, al activar la casilla de verificación **Proceso reanudable**, se activa automáticamente las casillas de verificación **Habilitar botón de guardar** y **Cargar los datos temporales**. En particular, los procesos guiados con un gran número de pasos o de información serán temporales, por ejemplo, el alta o la petición de vacantes.

Cuando un proceso no es reanudable, significa que cualquier acción realizada (acciones del tipo Insertar, Actualizar y Borrar) se efectúa directamente sobre la base de datos. En particular, los datos de las páginas de mantenimiento que sean de varios nodos son siempre procesos no reanudables.

Seleccionar campos para filtrar el asistente

En esta ventana, utilice los botones de flecha para mover los campos entre los cuadros de lista hasta que el cuadro **Campos del filtro** contenga sólo los campos de la tabla base que se requiere para filtrar los pasos del asistente, por debajo. Por ejemplo, la alta de empleado típicamente consiste en varios pasos en los cuales sólo se necesita trabajar con un único empleado. Se entiende por tabla base del asistente la tabla que sea el sujeto de todo el proceso guiado. Por ejemplo, el asistente de la alta de empleado utilizaría la tabla de empleados como tabla base. Por defecto, los campos de clave primaria aparecen en el cuadro **Campos del filtro**. En este cuadro pueden aparecer varios tipos de elementos:

-  Campos estándares
-  Campos heredados en color gris claro

Al heredar un Meta4Object y sus páginas, por lógica, los campos de filtro heredados no se pueden modificar y no se pueden quitar de la lista de campos del filtro. En este caso, sólo se podrá agregar más campos del Meta4Object a la lista de campos del filtro.

Cuando los campos incluidos en el cuadro **Campos del filtro** sean los deseados, haga clic en el botón **Siguiente**.

En esta ventana, utilice los botones de flecha para mover los campos entre los cuadros de lista hasta que el cuadro **Campos de la tabla** contenga sólo los campos con los que desea formar las columnas de la tabla de lista.

Definir el filtro tipo lista

Esta ventana permite enlazar y desenlazar los campos de filtro con los campos de la lista. De este modo, el sistema devolverá los registros cuyos datos en los campos conectados cumplan los criterios del filtro.

Para conectar un campo filtro con un campo lista:

1. Seleccione un campo de filtro.
2. Seleccione un campo de lista.
3. Haga clic en el botón  **Enlazar**.
4. Repita estos pasos hasta que estén conectados todos los campos filtro.
5. Haga clic en el botón **Siguiente**. Este botón sólo se habilita cuando todos los campos filtro estén conectados.

Para desconectar un campo filtro de un campo lista:

1. Seleccione un campo de filtro ya conectado.
2. Haga clic en el botón  **Desenlazar**.

Al heredar un Meta4Object tipo asistente y sus páginas correspondientes, por lógica, se hereda los enlaces entre los campos de filtro y los campos de lista que no se pueden modificar. De hecho, quedan deshabilitados los botones de **Enlazar** y **Desenlazar**. Sin embargo, se puede agregar enlaces adicionales.

Definir etiquetas de texto para la página JSP

Los datos de este paso son exactamente iguales que el paso que se describe en el apartado Definir etiquetas de texto para la página JSP en la sección anterior Uso de la opción Mantenimientos y Listas.

Definir los pasos del asistente

En este paso se definen los pasos guiados del asistente y el orden que seguirán.

El asistente que se está definiendo es, en realidad, un Meta4Object cuyos nodos libres corresponden a los pasos guiados. Hasta ahora se ha definido los parámetros globales del asistente, es decir, los parámetros que afectan a todo el proceso guiado como, por ejemplo, si los pasos son reanudables.

Ahora se pasa a definir los pasos guiados y sus parámetros, donde cada paso guiado corresponde a un nodo libre y los parámetros incluyen algunos datos de la estructura de nodo.

Al heredar el Meta4Object del asistente, los pasos heredados están en color gris claro.

- Haga clic en el botón **Editar** para completar o modificar la definición de los datos del nodo del paso guiado seleccionado. Consulte Definir un paso guiado del asistente.
- Haga clic en el botón **Nuevo** para crear un nuevo paso del asistente. Al crear un nuevo paso, el nuevo ID del paso se genera por defecto de forma numérica. En el caso de un asistente heredado, también se genera un nuevo ID numérico, pero por defecto se agrega un prefijo numérico para indicar el nivel de herencia. Por ejemplo, 1 corresponde al nivel de una herencia primaria. Tras crear un paso nuevo, hace falta completarlo. Para ello, haga clic en el botón **Editar**.
- Haga clic en el botón **Borrar** para eliminar el paso seleccionado. No se puede borrar pasos heredados.
- Haga clic en el botón **Renombrar** para renombrar el paso seleccionado. No se puede renombrar pasos heredados.
- Haga clic en los botones de flecha para ordenar los pasos del asistente. Se pueden ordenar pasos heredados.
- Haga clic en el botón **Finalizar** cuando todos los pasos estén completos. Procederá a

Definir un paso guiado del asistente

En este paso, se definen o actualizan las características principales de este paso guiado, que corresponde a un nodo del Meta4Object subyacente del asistente.



Al crear un nuevo paso guiado, algunos campos de esta ventana se rellenan automáticamente mediante reglas de denominación y valores predeterminados estándar; no obstante, se pueden editar.

Se define la sentencia SQL lógico que se ejecutará para este nodo y los siguientes propiedades e identificadores que definen algunos datos del nodo del paso guiado del asistente. Rellene el resto de los campos de la ventana teniendo en cuenta que:

- El **ID Nodo** lo genera el sistema para el paso guiado.
- La **Página del asistente** no es modificable, ya viene predefinida desde el paso anterior.
- La **Posición** indica el orden del paso guiado en el flujo del asistente, que también viene predefinido desde el paso anterior.
- El **Título de la página** aparece como título del paso que aparece en la caja de navegación en la página.

Si hereda un Meta4Object, el **Título de la página** no es editable. Sin embargo, en todos los pasos guiados nuevos que se agreguen sí es editable.

- El **Archivo de ayuda** corresponde a la página HTML de ayuda asociada a este paso guiado.
- La casilla de verificación **Habilitado** indica si el paso será visible o no. Por ejemplo, cuando se hereda un asistente de cinco pasos, puede que en la parametrización de un cliente sólo se necesite tres de estos pasos. En lugar de borrar estos pasos, simplemente se deshabilita, por si los necesite más adelante. El primer paso guiado del asistente siempre está habilitado.
- La casilla de verificación **Obligatorio** indica que este paso es imprescindible dentro del flujo de la funcionalidad del asistente. Afecta a la navegación entre pasos guiados del asistente:
 - No existen pasos obligatorios: La navegación es libre, se puede realizar de manera secuencial o directa, sin orden obligatorio alguno.
 - Pasos obligatorios ordenados: La navegación no es libre, hay que pasar por los pasos obligatorios primero. Además, entre los pasos obligatorios también se establece un orden que se tiene que seguir para poder continuar.

Lo habitual es que un paso prioritario se encargue de cargar datos en el resto de los pasos guiados del asistente.

- La casilla de verificación **Cargar** indica si los datos de este paso se cargan o no. Si está desactivada, el paso se carga vacío, listo para una nueva entrada de datos. El primer paso guiado del asistente siempre está cargado.
- Seleccione el botón de opción **Uno** o **Varios** si quiere mostrar un registro o varios registros para este paso guiado. Si selecciona **Varios**, el paso mostrará un registro y por debajo la lista de registros desde los cuales puede seleccionar uno para su modificación o consulta.
- Haga clic en el botón de remonte situado junto a la sentencia SQL lógico para abrir el Editor de sentencias, donde podrá modificar la sentencia.

Si hereda el Meta4Object y sus nodos, podrá agregar a la sentencia SQL lógico, pero no podrá eliminar ninguna parte de la definición heredada desde el Editor de sentencias.

Introduzca el **ID Estructura de nodo** para la estructura de nodo a utilizar.

- El **ID EN a heredar** indica la estructura de nodo que se hereda por debajo, que prácticamente *no es necesario modificar*.

Definir el filtro del paso guiado del asistente

Esta ventana permite enlazar y desenlazar los campos de filtro con los campos del paso. De este modo, el sistema devolverá los registros cuyos datos en los campos conectados cumplan los criterios del filtro.

Para conectar un campo del filtro con un campo del paso:

1. Seleccione un campo del filtro.
2. Seleccione un campo del paso.
3. Haga clic en el botón  **Enlazar**.
4. Repita estos pasos hasta que estén conectados todos los campos del filtro.
5. Haga clic en el botón **Siguiente**. Este botón sólo se habilita cuando todos los campos filtro estén conectados.

Para desconectar un campo del filtro de un campo del paso:

1. Seleccione un campo de filtro ya conectado.
2. Haga clic en el botón  **Desenlazar**.

Al heredar un Meta4Object tipo asistente y sus páginas correspondientes, por lógica, se hereda los enlaces entre los campos del filtro y los campos del paso que no se pueden modificar. De hecho, quedan deshabilitados los botones de Enlazar y Desenlazar. Sin embargo, se puede agregar enlaces adicionales.

Seleccionar los campos para la tabla del asistente

Los datos de este paso son exactamente iguales que el paso que se describe en el apartado Seleccionar los campos para la tabla del asistente en la sección anterior Uso de la opción Mantenimientos y Listas.

Definir los campos de edición de la página JSP del paso

Los datos de este paso son exactamente iguales que el paso que se describe en el apartado Definir los campos de edición de la página JSP en la sección anterior Uso de la opción Mantenimientos y Listas.

Finalización del proceso

Una vez finalizado el proceso de definir el Meta4Object nuevo y las páginas nuevas, se procede a generar estos objetos.

Generar Meta4Objects y páginas

Una vez finalizado el proceso de definir el Meta4Object nuevo y las paginas nuevas, se procede a generar estos objetos.

Sin embargo, antes de hacerlo, el sistema verifica si hay posibles conflictos porque los objetos ya existen.

1. Verifique si hay conflictos en la parte de Meta4Object.
 - Seleccione la opción **Actualizar**, si quiere machacar el Meta4Object existente con los cambios nuevos.
 - Seleccione la opción **Crear nuevo**, si quiere borrar el Meta4Object existente y recrearlo partiendo de los datos nuevos.
2. Verifique si hay conflictos en la parte de las páginas.
 - Seleccione la opción **Actualizar**, si quiere machacar la página existente con los cambios nuevos.
 - Seleccione la opción **Crear nuevo**, si quiere borrar la página existente existente y recrearla partiendo de los datos nuevos.
3. Asegúrese de desactivar las casillas de verificación de las páginas que no quiere tocar.



Las páginas que se muestran en gris claro se deben regenerar automáticamente porque son del sistema, y por tanto no se puede desactivar la casilla correspondiente.

4. Haga clic en el botón **Sí** para completar la generación.



El proceso de generación puede ser lento.

Al terminar la generación del Meta4Object y las páginas definidas, el sistema prepara un informe que se muestra a continuación.

Ver el informe del proceso de generación

Los datos de este paso son exactamente iguales que el paso que se describe en el apartado Ver el informe del proceso de generación en la sección anterior Uso de la opción Mantenimientos y Listas.

Hojas de estilos: CSS

4

Acerca de este capítulo

El cliente Web incorpora la posibilidad de cambiar la apariencia de la aplicación. Estos cambios se refieren al formato de las páginas. Así es posible cambiar el color, el tipo de letra, los iconos, etcétera. Todas estas características de formato se gestionan en un hoja de estilos.

En este capítulo se detalla:

- Qué hojas de estilo hay
- Qué es lo que se puede modificar
- Cómo gestionar nuevas hojas de estilo por el desarrollador
- Cómo el usuario final puede indicar la hoja de estilo con la que desea trabajar

Hojas de estilos del producto estándar

Antes de nada, hay que recordar que cualquier cambio que afecte a la apariencia de la aplicación nunca debe ser realizado directamente en el código de las páginas. Todos los cambios se realizan en la hoja de estilos o CSS.

Todas las hojas de estilos se encuentran en el directorio `../style` del servidor Web. En el producto estándar existe varias hojas de estilos pero los dos más importantes son:

- **shco_gen100.css**: para las páginas generales de la aplicación estándar tras logarse. Con esta hoja de estilos acompaña un juego de imágenes acabado en `_100` en el directorio `/images`.
- **tc_login.css**: para las páginas de conexión, es decir las páginas de login, login avanzado y error de login. Al proceder a iniciar una sesión con la aplicación el sistema necesita saber cuál debe usar.

Existe otras hojas de estilos de la aplicación que no se suministran: las imágenes acabadas en 0, 1 y 2 ni las hojas de estilos `shco_gen0.css`, `shco_gen1.css` y `shco_gen2.css`.

Gestión de hojas de estilos

A continuación, se describe cómo se gestionan en la aplicación las hojas de estilo que se creen nuevas. No es objetivo de este apartado detallar el proceso de creación de las mismas que siguen el estándar CSS especificado en el capítulo de Metodología de este mismo manual. Consultar este capítulo también para conocer qué herramientas se pueden utilizar en su generación y validación.

Hoja de estilos general estándar

En el producto estándar se facilita un juego de CSS/iconos terminado en el número 100. El 100 es el que está configurado por defecto en la aplicación. Por ello no se debe modificar los archivos terminados en 100.

Esto hace que la hoja de estilos `style/shco_gen100.css` y el juego de imágenes acabado en `_100` de la carpeta `images` no será modificable por parte de los desarrolladores funcionales ni los usuarios finales.

Dependiendo de la aplicación al que se conecta, el sistema utilizará la hoja de estilos correspondiente, por defecto, es `shco_gen100.css`. No se debe de

modificar pero se puede utilizar como base para una hoja de estilos nueva para las páginas generales de la aplicación.

Respecto al sufijo de la hoja de estilos, el sistema lo gestiona de la siguiente manera:

- La referencia al archivo css se hace desde la página **/shco_g0/shco_gen_css.jsp** para todas las páginas y usuarios de la siguiente manera:

```
<link href="/style/shco_gen<%=zcuser%>.css" type="text/css"
rel="stylesheet" />
```

- Posteriormente, desde cada página en el encabezamiento se hace una llamada a esta página:

```
<%@ include file="../../shco_g0/shco_gen_css.jsp" %>
```

- Para el juego de iconos asociados a una hoja css concreta, se sigue un proceso similar. Así la nomenclatura de los mismos debe ser:
 - Icon_nombre_0.gif
 - Icon_nombre_1.gif ...

Se guardan en la carpeta **/images** del servidor Web.

- La llamada a los iconos se realiza de la siguiente forma:
 - Por cada juego de iconos existente en la carpeta /images susceptibles de cambiarse, se crea una página .jsp en la carpeta /files_gif. Se recomienda que tenga el mismo nombre del icono pero sin terminación numérica.

Así para el juego de iconos: Icon_nombre_0.gif, Icon_nombre_1, existirá una página genérica Icon_nombre.jsp.

En esta página se hace la siguiente llamada:

```
src="/images/icon_nombre<%=zcuser%>.gif" width="36"
height="36"
```

- En las páginas que utilicen estos iconos se hace una llamada a esta página de la siguiente forma:

```
<%@ include file="../../files_gif/ic_cabec.jsp" %>
```

Creación y configuración de una nueva hoja de estilos general

Se utiliza la hoja de estilo general estándar shco_gen100.css como base para las nuevas. Para ello, hay que respetar la nomenclatura definida para que el sistema pueda gestionar la hoja de estilos personalizada correctamente.

Los archivos css de las hojas de estilos parametrizables seguirán la siguiente nomenclatura:

shco_gen100

Donde el número final coincide con el identificador de la hoja de estilo que seleccione el usuario para trabajar.



Recuerde que la hoja de estilo con el sufijo 100, es reservado y no se debe de modificar nunca.

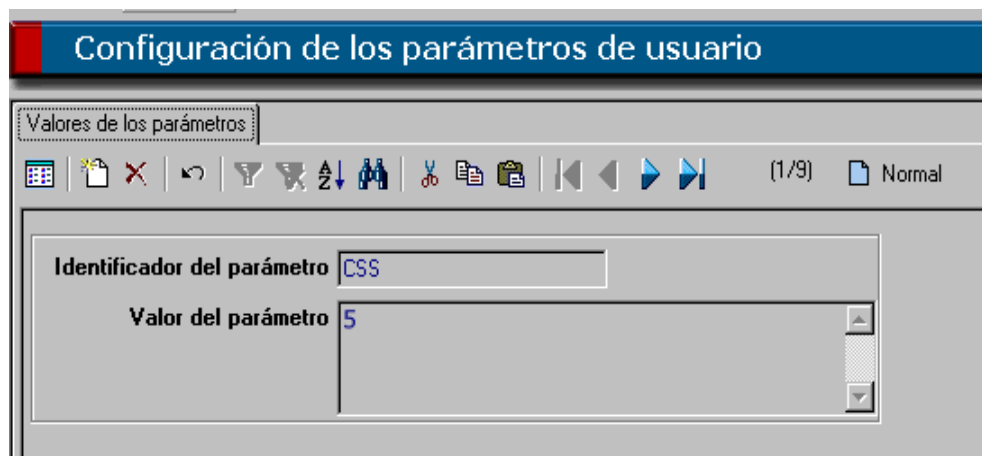
Los pasos a seguir son:

1. Haga una copia de shco_gen100.css y renómbrelo para la parametrización según la nomenclatura establecida.

Ejemplo: shco_gen5.css

2. Modifique las clases existentes en esta hoja de estilos y agregue nuevas clases si hace falta.
3. Agregue la nueva hoja de estilos a la aplicación a través de la opción de menú **Herramientas de desarrollo | Configuración funcional Cliente Ligerio | Configuración de los parámetros del usuario**. El objetivo es que el usuario pueda seleccionarla entre el resto de las css existentes. Indique el número correspondiente a su hoja de estilo en el valor del parámetro.

Figura 36. Pantalla de configuración de hojas de estilos disponibles al usuario final.



Creación y configuración de la hoja de estilos de conexión

Las páginas de conexión que son la página de login, de login avanzado y de error de login utilizan la hoja de estilos que está definida en el configclient.xml, en la sección STYLESHEET. Por defecto, el valor es tc_login.css:

```
<THINCLIENT>
```

```
<UPLOADDIR>c:emp</UPLOADDIR>
<MAXFILESIZE>50000</MAXFILESIZE>
<DEFAULTPAGE>/servlet/CheckSecurity/JSP/tctools/tc_portal.jsp</
DEFAULTPAGE>
<YLESHEET>shco_gen.css</YLESHEET>
<THINCLIENTROOT></THINCLIENTROOT>
```

Si el valor es nulo, es decir, por defecto se lee shco_gen.css (por compatibilidad hacia atrás).

Los pasos a seguir son:

1. Haga una copia de tc_login.css y renómbrelo para la parametrización.
Ejemplo: mi_login.css
2. Modifique las clases existentes en esta hoja de estilos.
3. Modifique la sección YLESHEET en el configclient.xml e indique la nueva hoja de estilos.

Cambio de hoja de estilo del usuario

Una vez que el desarrollador ha definido una apariencia para la aplicación (hoja de estilo e iconos), el usuario final puede seleccionarla en su pantalla de parámetros.

Conectados al servidor Web, en la ruta **Herramientas | Opciones**, elegimos la hoja de estilos que queremos utilizar en el campo **Hoja de estilos en cascada**. La 100 es la configurada por defecto en el producto estándar.

Figura 37. Parámetros de usuario



PARÁMETROS DE USUARIO

Visualización de la fecha: ddMMaa

Formato de la fecha: dd-MM-yyyy

Separador de la fecha: -

Separador decimal: ,

Separador de miles: .

Visualización de los números: 123456

Separador de miles: ☐ Número de decimales: 5

Visualización de las monedas: \$ 234.56000000

Separador de miles: ☒ Número de decimales: 9

Número de decimales fijos: 9

Hoja de estilos en cascada: 100

Desactivar advertencias e informaciones: 0

Confirmar navegación en asistentes: 1

100

2

Al crear nuevas hojas de estilos para ajustar a las necesidades particulares de una parametrización, estas figurarán en esta lista.

Parámetros del webservices

5

Acerca de este capítulo

En este capítulo se explican los parámetros con los que se desarrolla la funcionalidad de las páginas JSP y Meta4Object. Estos parámetros se han creado para facilitar el acceso, y de este modo, simplificar el proceso de desarrollo funcional y la gestión de los datos de la aplicación.

Acceso a parámetros desde páginas jsp

Existen una serie de parámetros en la aplicación que se mantienen en memoria accesibles para todas las páginas. Este conjunto de parámetros se definen en la página shco_gen_bag.jsp y se conocen como bolsa.

Estos parámetros que se obtienen desde la bolsa se encuentran disponibles como variables java (*strings*), ya definidas y con el valor correspondiente, desde todas las páginas, debido al uso del include: shco_gen_bag.jsp.

Los parámetros obtenidos desde la bolsa son:

Parámetro	Descripción
zlanguserld	Idioma del usuario. Valores posibles: ■ es: Español ■ en: Inglés ■ fr: Francés
cssuser	Hoja de estilo del usuario.
zrole	ID del Rol del usuario.
zbrowser	ID del <i>browser</i> en uso. Valores posibles: ■ IE Internet Explorer. ■ NS NetScape.
zdateformat	ID de formato fechas del usuario.
zidcur	ID de la moneda por defecto del usuario.
znmcur	Nombre de la moneda por defecto del usuario.
zextype	Tipo de cambio por defecto del usuario.
zdecnb	Número de decimales de la moneda.
zsoc	Sociedad del usuario.
zapprod	ID de la aplicación.

Acceso a parámetros desde Meta4Objects

Para obtener los parámetros que se requieran desde un M4Object, se ejecuta el método: SHCO_GET_PARAM de la TI: SHCO_GN_TOOLS con uno de los siguientes parámetros:

Parámetro	Descripción
CSS	Hoja de estilo del usuario.
ID_APP_ROLE	ID del rol del usuario
DATE_FORMAT	ID del formato fechas del usuario.
ID_CURRENCY	ID de la moneda por defecto del usuario.
EX_TYPE	Tipo de cambio por defecto del usuario.
ID	ID persona asociada al usuario.
USER	ID usuario.
DEBUG	Nivel de información de depuración del usuario.
SOC	ID sociedad del usuario.
DT_EXCHANGE	Fecha de cambio por defecto del usuario.
DATA_START_DATE	Fecha de inicio por defecto del usuario.
DATA_END_DATE	Fecha de fin por defecto del usuario.

Parámetros del webservice

En este apartado se tratan dos tipos de parámetros: la configuración global del cliente Web y las opciones de usuario.

Configuración del cliente Web

Para establecer estos parámetros, seleccione **Herramientas de desarrollo | Configuración de la aplicación | Parámetros de la aplicación**.

- En la ventana **Parámetros de la aplicación**, se encuentran en la categoría **Parámetros del cliente Web**, en los grupos de Formatos y parámetros del portal.
- En el árbol Parámetros avanzados (Herramientas – Avanzadas), despliegue la rama THINCLIENT y, a continuación, las ramas PORTAL_PARAM y FORMAT.

En la tabla siguiente se describen algunos parámetros del cliente Web seleccionados:

Parámetros	Descripción	Valor por defecto	Editabile
CSS	Hoja de estilo del usuario	0	Si
DATE_FORMAT=DATE	Formato fechas	dd-MM-yyyy	Si
DEBUG	Nivel de información de depuración	0	Si
DEBUG WIZARD	Mensajes de advertencia sobre la posibilidad de perder datos	1	Si
_APP_MN	Identificador de la opción de menú bajo la que cuelgan las opciones de menú de cliente Web para la aplicación.	MN_TC_MENUS	No
STANDAR	Standard aplicado por la sociedad	CO	No

Parámetros	Descripción	Valor por defecto	Editabile
CASH_COMP	Activar / Desactivar los <i>links</i> al módulo Compensación salarial localizados en el alta y en las fichas de <i>job</i> y de rol. El resto del módulo permanece activo. Para desactivarlo, es necesario descolgar las opciones de menú desde windows.	1	No
NAV_RC	Valor 1 indica que estamos en navegación Rich Client, y bajo este tipo de navegación cuando se invoca a páginas web, no se mostrarán los menús ni el disclaimer. Con Valor 0 se mostrarán en las páginas el menú y el disclaimer.	Depende del producto instalado	No



Consulte el capítulo de Metodología para más detalles acerca de la Configuración del producto Web.

Opciones generales

A continuación se indican aquellos parámetros de entre los anteriormente señalados que el usuario puede modificar en la página Herramientas – Opciones del cliente Web.

Parámetros	Descripción
CSS	Hoja de estilos. Elija entre 0, 1, 2, etc. Para modificar los valores de este parámetro o agregar otros nuevos, seleccione Herramientas de desarrollo Configuración funcional del cliente Web Configuración del parámetro del usuario. Consulte el capítulo Hojas de estilos en este manual para más información.

Parámetros	Descripción
DATE	Formato en el que el usuario verá las fechas en el cliente Web. Elija entre: ■ dd-MM-aaaa ■ dd-aaaa-MM ■ MM-dd-aaaa ■ MM-aaaa-dd ■ aaaa-dd-MM ■ aaaa-MM-dd Para modificar los valores de este parámetro o agregar otros nuevos, seleccione Herramientas de desarrollo Configuración funcional del cliente Web Configuración de parámetros del usuario
DEBUG_WIZARD	Este parámetro sólo puede tomar dos valores, por lo que no se incluye en la tabla Configuración de parámetros del usuario .



Consulte el capítulo de Metodología para más detalles acerca de Formato de fechas, números y monedas.

Gestión de menús

6

Acerca de este capítulo

Este capítulo describe:

- el sistema de funcionamiento y configuración de menús del cliente Web
- la gestión de la seguridad de los menús y
- la generación de las páginas de menú de cada módulo.

Funcionamiento de los menús

Los menús actuales se generan en archivos .js en el servidor de aplicaciones y se mueven al directorio temporal del servidor Web que se crea por sesión de usuario. Esta generación se hace cada vez que se conecta un usuario, pues de esta forma se generan en el idioma de conexión y en función de los roles del usuario con el que se conecta.

Se crea un archivo .js por nivel de menú. A nivel de página, una variable java `znivelmenu` define el nivel de menú en el que se encuentra. En el proceso de carga de la página se cargará el archivo .js de menús asociado al nivel definido en la página y vía javascript se dibujan.

Este sistema de generación de menús requiere configurar el servidor de aplicaciones y el servidor Web de forma que se puedan traspasar los archivos generados de los menús del servidor de aplicaciones al servidor Web. Más información, en la Guía de instalación.



Recuerde que si tiene establecido el parámetro de webservices NAV_RC a 1 los menús no serán mostrados en su página. Ver cómo modificar dichos parámetros en el apartado "Parámetros de webservices/Configuración del cliente web" de este mismo documento.

Configuración de las opciones de menú

La configuración de los menús de cliente Web se realiza también desde el Diseñador de menús de Windows en la ruta **Meta4 PeopleNet Ksystem | Herramientas de desarrollo | Menús | Diseñador de menús**.

En Diseñador de menús, observe que existe una opción de menú en la que se cuelgan los menús de cliente Web siguiendo una metodología de niveles: **Meta4 PeopleNet Ksystem | Herramientas de desarrollo | Configuración funcional cliente ligero | Configuración de menús**.

A continuación se detalla:

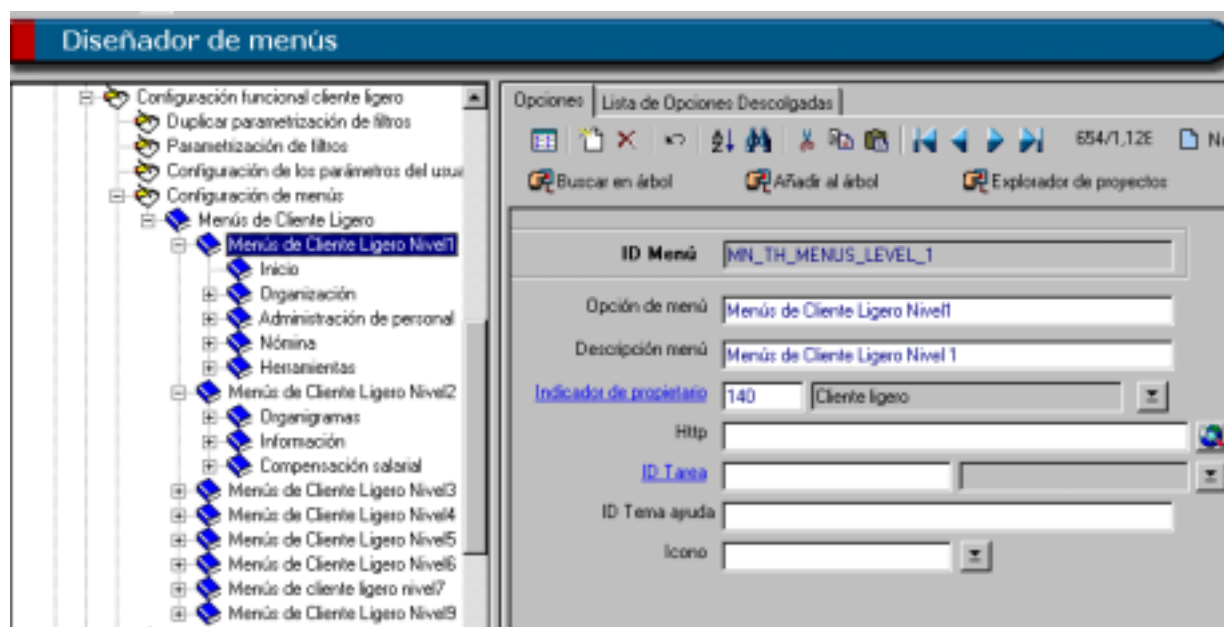
- Clasificación de los menús
- Gestión de los niveles de los menús
- Gestión de los menús

Clasificación de los menús

Los menús se clasifican en primer lugar por aplicación y dentro de cada aplicación por nivel. Estos niveles se corresponden con los niveles de las páginas .jsp indicados en la variable `znivelmenu`.

Esta clasificación se reflejan en el árbol de menús de la siguiente forma:

Figura 38. Árbol de menú



Cada nivel de menú tiene asociado sus menús. A continuación, se explica cómo se gestionan los niveles y cómo se gestionan los menús asociados a cada nivel.

Gestión de niveles

Se genera un archivo de menús .js por cada nivel de menú. Así, si disponemos de 9 niveles, tendremos nueve archivos de menús.

Cada archivo se llama **shco_menu_znivelmenu.js**, donde el valor para la variable znivelmenu se toma del ID MENU del nivel. Concretamente, se toma el número que viene tras el último guión bajo (_).

EJEMPLO

El nivel de menú con ID = MN_TH_MENU_NIVEL_50 genera el archivo shco_menu_50.js y será incluido en aquellas páginas cuya variable znivelmenu sea 50.

Los valores para znivelmenu de los distintos módulos son los siguientes:

Estándar	Rango de niveles menú - znivelmenu
Corporativo	1-39

Estándar	Rango de niveles menú - znivelmenu
Standard Francés	40-49
Standard Español	50_59
Standard UK	90-99
Cliente	200-209



Al menos debe estar disponible el nivel 1 que es el que usa el portal.

Un cliente no debería modificar los menús y sus niveles directamente a no ser que correspondan al rango reservado para su parametrización.

Gestión de menús dependientes de los menús de nivel

Cada opción de menú se identifica con los siguientes datos:

- **ID MENU:** Identificador único en todo el árbol de menús. A continuación se describe una metodología de creación de identificadores para evitar *id's* duplicados. Se trata de una guía y no asegura que no se genere un id que ya este dado de alta en otra zona de la aplicación.

Es obligatorio tener la raíz correcta para identificar los menús por aplicación y standard:

- Menús de aplicación: **MN_TH_NombreAplicacion_Standard**. Por ejemplo: **MN_TH_PN_UK** para los menús de cliente Web de UK, **MN_TH_MENUS** para Menús de cliente Web en la aplicación corporativa.
- Menús de nivel: **MN_TH_MENUS_NombreAplicacion_Standard_LEVEL_i** donde i es el **Nivel de Menú** (valor de la variable `znivelmenu`). Por ejemplo, **MN_TH_MENUS_LEVEL_2**, **MN_TH_MENUS_LEVEL_1**.
- Resto de Menús: **MN_TH_NombreAplicacion_Standard Modulo_j. MN_TH_NombreAplicacion_Standard _Modulo _j_z...** indicando el nivel de profundidad.

Por ejemplo, la opción de menú Administración de personal tiene un ID igual a **MN_TH_G2** y la opción dependiente Información de la persona un **MN_TH_G2_1**.

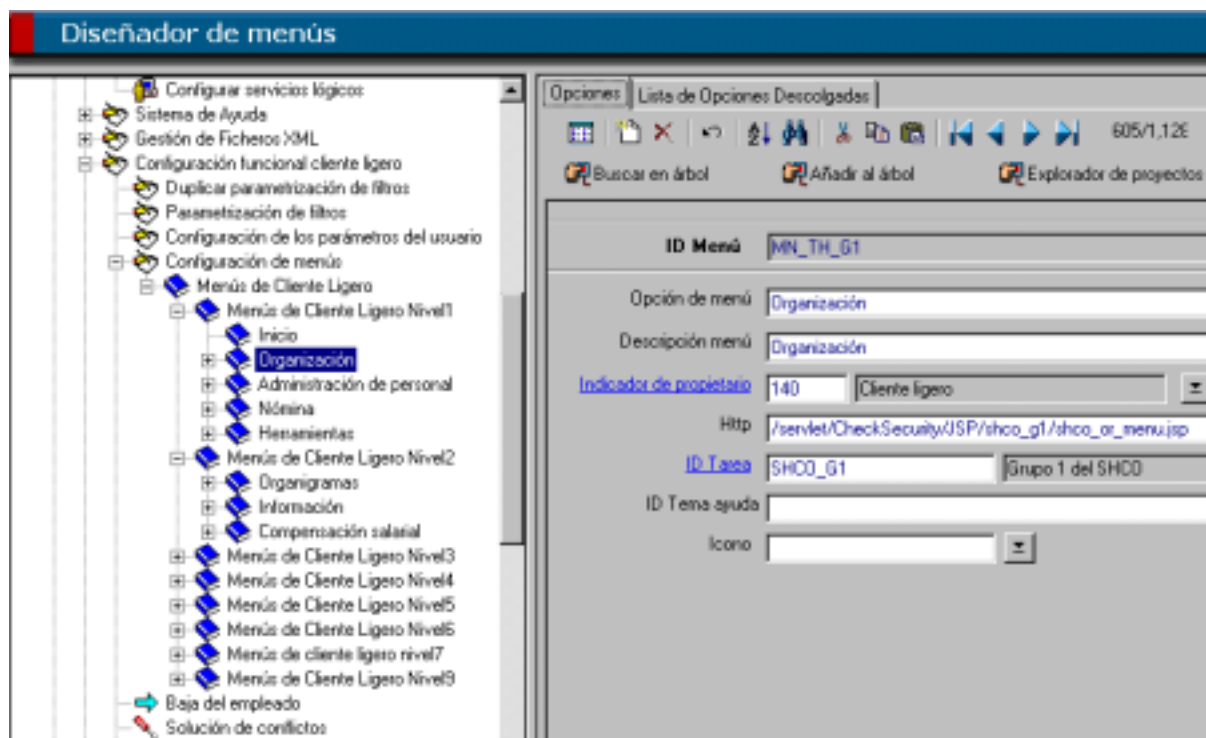
- **Opción de menú:** literal traducido que aparecerá como opción de menú. No tiene que tener formato HTML de esto ya se encarga el proceso de generación de los menús.
- **Descripción de menú:** descripción de la opción de menú. No se necesita que tenga formato HTML porque no se muestra en cliente Web.
- **Indicador de propietario:** Se pueden utilizar los Id de cliente Web por tener agrupadas las opciones de menú.
- **Http:** URL asociada con la opción de menú. Puede ser la invocación a una página (`/servlet/CheckSecurity/JSP/shco_g1/shco_or_menu.jsp`), e incluso usar javascript... es decir, indicar la misma información que se daba hasta ahora como link de una opción de menú.



Este campo es un campo traducido por lo cual 1) hay que modificarlo siempre en el idioma de desarrollo. 2) Hay que tener un *script* de actualización del campo del idioma de desarrollo al resto de idiomas para tener la aplicación configurada en varios idiomas correctamente.

- **ID Tarea:** tarea que nos indica la seguridad de la opción de menú. Es decir, si el usuario logado no tiene permisos para ejecutar esta tarea no aparece como opción de menú salvo en el caso excepcional de que tenga submenús con permisos. En este caso aparece la opción de menú pero no tendrá un link asociado. Normalmente aquí se establece la tarea indicada en la URL (campo Http). Si estamos usando javascript y establecemos una URL u otra en función de algún parámetro, la tarea a establecer será la más restrictiva. (P.e lista + página de retorno)
- **ID Tema Ayuda:** no se usa a nivel de cliente Web
- **Icono:** no se usa a nivel de cliente Web

Figura 39. Organización



Dentro de esta generalidad nos encontramos con las siguientes peculiaridades:

- **Opción de menú inicial**

La primera opción de menú del primer nivel debe ser la opción de menú de inicio (link al portal). En la imagen, de la opción Menús de Cliente Web Nivel 1 cuelga la opción Inicio cuyo Http es un link al portal: `/servlet/CheckSecurity/JSP/shco_g0/shco_gen_portal.js`.

Para el resto de niveles esta opción de inicio se genera automáticamente conteniendo todas las opciones de menú del primer nivel.

- **Disclaimer** que aparece en todas las páginas en la parte inferior se genera con las opciones de menú del primer nivel.

- Opción de menú padre.

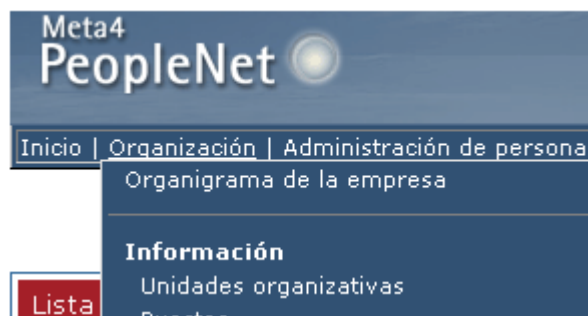
Como ya se ha dicho, las opciones de menú pueden estar distribuidas por aplicación. Es decir, crear un árbol de menús por cada aplicación.

La forma de indicar al proceso de generación de los archivos .js de menús, las opciones de menú que debe procesar es pasándole el identificador de la opción de menú padre de la aplicación (raíz del árbol).

Se parametriza en el archivo shco_gen_login.jsp localizado en la carpeta de idioma correspondiente en \scho_g0 del servidor Web, dando valor al parámetro **_app_mn**. En la aplicación estándar corporativa tiene el valor MN_TC_MENUS.

- Opción que pinta una línea tras la primera opción dependiente de la opción de menú (primer nivel) de cada módulo.

Figura 40. Opciones dependientes de una opción de menú del primer nivel



En el Diseñador de menús se dejará una opción en blanco indicando exclusivamente su ID.

Opciones de menú no modificables

El HTML Development Framework suministra los menús Inicio, Herramientas y Herramientas avanzadas que no se deben modificar nunca. El ID menú base correspondiente a estas opciones reservadas es MN_TC_MENUS.

Las opciones de menú están separadas en dos niveles:

- **Nivel 1:** el nivel asociado al portal (a través del parámetro global znivelmnu = 1) y corresponde al ID menú MN_TH_MENUS_LEVEL_1.

Dentro de este nivel están las opciones: Inicio (MN_TH_G0), Herramientas (MN_TH_TOOLS) y Herramientas avanzadas (MN_TH_ADV_TOOLS). Esta última opción y todas las opciones que de ella cuelgan tienen asociada la tarea SHCO_ADV_TL y sólo serán visible para los usuarios de aplicación que tengan el rol SHCO_ADV_DEVELOPER. Dentro de este nivel los

menús MN_TH_TOOLS y MN_TH_ADV_TOOLS tendrán las posiciones 500 y 501 respectivamente, de forma que por defecto aparecerán a la derecha.

- **Nivel 6:** el nivel asociado al menú Herramientas y corresponde al ID menú MN_TH_MENUS_LEVEL_6.

Dentro de este nivel están las opciones de menú: Lista de tareas, Editar Tareas, Cambio de Contraseña, Opciones, Información de la Conexión y Planificador de tareas.

Ver el menú Herramientas Avanzadas

Para poder ver el menú Herramientas Avanzadas del Nivel 1 (o nivel del portal) se debe asignar el rol fijo, SHCO_ADV_DEVELOPER, a un usuario que necesita tener acceso a esta opción. Para ello, debe utilizar la herramienta de desarrollo Seguridad desde un cliente Windows.

Desde el cliente Windows en la aplicación Meta4, los pasos a seguir son:

1. Seleccione la opción **Herramientas de desarrollo | Seguridad | Usuarios de aplicación**.
2. Seleccione el usuario de aplicación a modificar.
3. Haga clic en el botón  **Asignar roles fijos**.
4. Asigna el rol SHCO_AV_DEVELOPER al usuario de aplicación.
5. Haga clic en el botón **Aplicar** y a continuación en el botón **Aceptar**.
6. Guarde el cambio.



Consulte la ayuda en línea correspondiente para Asignar roles fijos para más información.

Seguridad

La gestión de la seguridad a nivel de opciones de menú es la misma que se realiza en Windows.

Seguridad por *outsourcing*

En el caso de *outsourcing* de las opciones de menú que cuelgan de cliente Web sólo se muestran aquellas que:

- No estén asociadas a proyectos
- Están asociadas a algún proyecto válido para la sociedad seleccionada en el *login*.

Seguridad por tarea

Para todas las opciones de menú que cuelgan de los menús de cliente Web, y que en el caso de *outsourcing* sean válidas para mi sociedad, se comprueba la seguridad de la tarea que tengan asociadas de forma que se muestra aquellas opciones que:

- No tienen tarea asignada
- Tienen tarea asignada sin seguridad
- Tienen tarea asignada, con seguridad y alguno de los roles asociados a la misma coinciden con alguno de los roles asignados al usuario que se loga. Tener en cuenta que si se trabaja con sociedad, de todos los roles asociados a un usuario sólo se tienen en cuenta aquellos que son válidos para la sociedad del login.

Generación dinámica de las páginas de menús de cada módulo

Este apartado describe cómo generar de manera dinámica las páginas de menú de cada módulo.

Figura 41. Página de menú de Gestión del tiempo



Estas páginas se crean utilizando la información que se ha cargado en la línea de menús.

A continuación se detalla el proceso a seguir:

- Definición del código de la página jsp que hay que utilizar
- Edición del archivo shco_portal_es.properties
- Nomenclatura de las imágenes empleadas

Se utiliza el módulo de Gestión del tiempo en todo el proceso como ejemplo.

Definición del código de la página jsp

Toda la generación de la página de menú de un módulo se basa en la variable `znivelmenu` al que pertenece el módulo.

El código de una página de menú de módulo es el siguiente:

```
<%@ include file="../../../shco_g0/shco_gen_taglib.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_arg.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_bag.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_css.jsp" %>
<%@ include file="../../../shco_g0/shco_portal_trans.jsp" %>

//Escribir la dirección de la página:
String zdireccion = "shco_ta_menu.jsp";

//Escribir el nivel de menú:
znivelmenu = "4";

// Declarar la variable de ayuda en línea si hay un ID_TOPIC asociado. El
objetivo es que en la opción Ayuda del disclaimer muestre la ayuda en línea
específica para este tópico y no la página genérica de ayuda.

String zhelp="SHCO_TA_MENU.htm";
zsLocalizeHelp= zhelp;

<%@ include file="../../../shco_g0/shco_gen_menu.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_disclaimer.jsp" %>
</body></html>
```

Edición del archivo `shco_portal_idioma.properties`

En el archivo `shco_portal_idioma.properties` se define el título y la descripción funcional de cada página de menú de módulos. En el nombre del archivo idioma es igual a: es, en, o fr en función del idioma con el que se trabaje:

- `shco_portal_es.properties`
- `shco_portal_en.properties`
- `shco_portal_fr.properties`

Es necesario añadir dos literales. Si el nivel de menú es 4, como en el ejemplo utilizado, se escribirá los literales de la siguiente forma:

[illegible]

El segundo es una breve descripción funcional del módulo que se utiliza tanto en la página del portal como en la propia del menú.

[illegible]

Imágenes

El nombre de la imagen que va en la página principal de cada módulo tiene que seguir la siguiente nomenclatura:

- `ic_menu_4_147_100_0.gif`, donde:
 - 4, se corresponde con `znivelmenu`
 - 0, se corresponde con el `css` funcional. Para las imágenes de tecnología se utiliza el 100. Ver el capítulo de la hoja de estilo de este manual.

Configuración Funcional

7

Introducción

Algunos procesos funcionales del cliente Web cuentan con requisitos específicos de configuración para los datos de la aplicación. En este capítulo se explica cómo agregar un proceso a la **Lista de tareas** del cliente Web.

Agregar un proceso a la Lista de tareas

En la función **Editar tareas personales** del cliente Web, los usuarios pueden definir y mantener una lista de tareas pendientes de realizar. Estas tareas pueden estar asociadas a una aplicación **Proceso a ejecutar**. Éste es un filtro tipo lista que contiene una serie de procesos que el usuario puede seleccionar.

En este apartado se explica el procedimiento para agregar nuevos procesos a la lista de seleccionables mediante la opción **Editar tareas personales**. Para ello, se debe incluir la tarea en la categoría con el ID 150, y asociarla al código de cliente Web. Esto se lleva a cabo mediante el Diseñador de tareas del cliente Windows.

1. Acceda a **Herramientas de desarrollo | Menús | Diseñador de tareas**.
2. Seleccione **Archivo | Nuevo | Tarea**.
3. Seleccione una tarea "genérica" en la ventana **Nueva tarea** y haga clic en **Aceptar**. Se abre el **Asistente para nueva tarea**. Para obtener información sobre cómo utilizar este asistente, consulte la Ayuda en línea.
4. En el paso 4 del asistente, active la casilla de verificación **Código de cliente Web**.

Al finalizar el asistente, habrá creado la tarea, pero no la habrá asociado a la categoría 150.

5. Busque la categoría con el ID **150**. Puede hacerlo de dos maneras:
 - Haga clic en la categoría principal del árbol y revise la lista de tareas que aparece en el área de lista de la ventana. Busque el ID **150** y escriba el nombre de la categoría.
 - Haga clic en una de las categorías que dependen de la principal y elija **Categoría - Buscar**. Ejecute la búsqueda de ID = 150 y se resaltará automáticamente la categoría en el árbol.
6. Una vez identificada la categoría, selecciónela en el árbol de menús.
7. Elija **Categoría | Asociar tarea a categoría**. Utilice los filtros que aparecen para seleccionar la tarea recién creada y haga clic en **Aceptar**.

La tarea se agrega a la categoría. Ahora ya se puede seleccionar en el cliente Web.

Página de moneda

8

Acerca de este capítulo

Este capítulo describe:

- La funcionalidad de la página de moneda
- Cómo se invoca la moneda desde las páginas jsp

Funcionalidad de la página de moneda

En todas las páginas que hagan uso de valores monetarios aparecerá el botón  al lado de los campos de valor y moneda. Al hacer clic en él, se muestra la página de monedas (shco_gen_currency.jsp) de la figura.

Figura 42. Monedas



La interfaz 'MONEDAS' contiene los siguientes elementos:

	Valor	Moneda	Tipo de cambio	Fecha de cambio
*	20.2	Euro	Oficial	15-01-2003

En la parte inferior de la interfaz hay dos botones: uno con una marca de verificación (confirmar) y otro con una 'X' (cancelar).

A continuación se explica cómo se utiliza la moneda desde el código de cualquier página jsp.

Forma de invocación a la página

La invocación a la página de moneda es a través de la función `m4currency` cuyos parámetros son los siguientes:

```
m4currency(sIdForm, sIdQuantity, sIdCurrency, sIdExchangetype, sIdExchangeDate, sDecimalNumber, sIdCurrencyName)
```

Dónde:

- `sIdForm`: identificador del formulario, en el ejemplo `NombreFormulario`.
- `sIdQuantity`: identificador del *Input* cantidad en el ejemplo.
- `sIdCurrency`: identificador del *Input* `idmoneda` en el ejemplo.
- `sIdExchangetype`: identificador del *Input* `idtipocambio` en el ejemplo
- `sIdExchangeDate`: identificador del *Input* `fechacambio` en el ejemplo
- `sDecimalNumber`: número de decimales asociados a la moneda
- `sIdCurrencyName`: identificador del contenedor del nombre de la moneda. Puede ser un *td*, un *input*. En este objeto se escribirá el nombre de la moneda elegida.

Devuelve en los *inputs* `sQuantity`, `sIdCurrency`, `sIdExchangetype`, `sIdExchangeDate` del formulario `sIdForm` los valores seleccionados. Además si los valores de los *input* llegan rellenos los establece en la pantalla.

EJEMPLO

```
<table>
  <form action = "" method = "post" name='NombreFormulario'
  id=NombreFormulario'>
    <tr>
      <td>
        <input type = "text" id="cantidad" name="cantidad"
value="" />
      </td>
      <td id="nombremoneda"> Pesetas
      </td>
      <td>
        <a href="javascript:m4currency('NombreFormulario',
'cantidad', 'idmoneda', 'idtipocambio',
'fechacambio', 'numerodecimales', 'nombremoneda')">
          <img <%@ include file = "../..filles_gif/ic_curr.jsp"%>
alt="Monedas" />
        </a>
```

```





        </td>
    </tr>
</form>
</table>

```

Los valores por defecto de estos campos están accesibles desde todas las páginas (definidos en **shco_gen_bag.jsp**) como variables java y son los siguientes:

- **zidcur**: id de moneda por defecto
- **zncur**: nombre de moneda por defecto
- **zextype**: tipo de cambio por defecto
- **zdecnb**: número de decimales por defecto

Para validar que la información de la moneda se introduce correctamente en cualquier página jsp, se comprueba que todos los campos estén rellenos y además que el valor para la moneda sea el correcto. Para validar el valor de la moneda se utiliza la validación **tipo_currency_oblig** indicando el número de decimales asociados a la moneda elegida.

Obtención del listado de monedas y tipos de cambio

A continuación se detalla a modo informativo cómo se obtiene internamente el listado de monedas y los tipos de cambio. Realmente, en el desarrollo de nuevas páginas no habrá que modificar esto.

La lista de tipos de moneda y tipos de cambio se crean a partir de una variable **zzurcurr** que viene en la bolsa (**shco_gen_bag.jsp**). Más información, en el capítulo Parámetros del webservices.

Esta variable es capaz de obtener todos los tipos de moneda y los tipos de cambio siguiendo el formato:

```
Zzurcurr = ListadoMonedas$$ListadoTiposdeCambio
```

dónde:

ListadoMonedas = Moneda₁||...Moneda_n||
Moneda_i = IdMoneda;;NombreMoneda;;NumeroDecimales
ListadoTiposdeCambio = TipoCambio₁||... TipoCambio_n||
TipoCambio_i = IdTipoCambio;;NombreTipoCambio

EJEMPLO

zzurcurr="ESP;;Peseta;;0||EUR;;Euro;;2||\$\$1;;Cambio1;;2Cambio2||"

Temporalidad: control de fechas

9

Acerca de este capítulo

Es fundamental conocer el comportamiento de la aplicación en cuanto a fechas de trabajo para asegurarse de que los datos manejados son los que realmente se necesitan. En todo momento, es importante saber con qué fechas se trabaja.

La aplicación se invoca desde windows o a través de la conexión directa al cliente Web:

- Si la conexión se realiza directamente al cliente Web, se trabaja con las fechas que se establezcan por defecto en el enlace **Login avanzado**. Concretamente, es la casilla de verificación **Fechas por defecto** la que permite especificar las fechas de trabajo iniciales. Las posibilidades son:
 - Si la casilla está activada indica que se trabaja con las fechas límites del sistema y por tanto estarán disponibles todos los datos.
 - Si la casilla está desactivada, se indican las fechas con las que se desea trabajar. En este caso existe la posibilidad de trabajar con una fecha o con un periodo:
 - Para trabajar con los datos vigentes en una fecha, se indica ésta tanto en la fecha de inicio como en la fecha de fin. Así se trabaja con aquellos datos entre cuyas fechas de inicio y de fin se encuentre la fecha que se introduzca en esta opción.
 - Para trabajar con registros cuyo periodo se encuentre incluido en un rango de fechas, se introducen las fechas límites de dicho intervalo en las fechas de inicio y en las fechas de fin.
- Si la conexión se realiza desde el *SingleSignOn* (conexión unificada) de windows se trabaja con las fechas de sesión de windows.

Independientemente del modo de acceso, nada más producirse la conexión la aplicación trabaja o con una fecha de corte o con un intervalo de corte.



Si en algún momento, se olvidan las fechas especificadas en la conexión, se pueden consultar desde la opción de menú **Herramientas | Información de la conexión**.

Conforme se navega por la aplicación, se utilizan diferentes tipos de páginas y se debe tener claro también con qué fechas se trabaja en cada una de ellas. A continuación, se explica el comportamiento de las fechas en las distintas funcionalidades:

- En las listas y filtros tipo lista
- En los asistentes
- En las consultas

Listas y filtros tipo lista

Básicamente, el objetivo de los filtros tipo lista es facilitar el acceso a una página y el de las listas facilitar la introducción de información en los campos.

Teniendo en cuenta su funcionalidad, los filtros tipo lista mostrarán la información filtrada por las fechas de trabajo establecidas en la conexión. Así, la página siguiente, contará ya con la información filtrada.

En el caso de las listas, la información mostrada se filtra atendiendo a las fechas de trabajo establecidas en la conexión y no a las fechas de la entidad principal que contiene esa lista.

EJEMPLO

En el mantenimiento del rol, en el botón de lista que permite seleccionar el puesto, muestra la relación de los puestos válidos para las fechas de trabajo, no para las fechas de inicio y fin del rol. Es decir, que si no se han establecido las fechas adecuadas en la conexión, puede suceder que no todos los puestos relacionados sean válidos para el periodo de vigencia del rol. Así, es el propio usuario, el que debe comprobar que está seleccionando el correcto. Otra opción, es volver a conectarse con las fechas apropiadas.

Para que este filtro se aplique de manera automática es necesario dar a los nodos del M4Object con el que se trabaje, la propiedad *Filtro automático con valor histórico*.

En los asistentes

En los asistentes se muestran los datos históricos en cada paso. Normalmente se accede a un asistente a través de un filtro tipo lista en el que se ha filtrado ya la información. Entonces, para la información seleccionada en el filtro tipo lista, se muestran todos sus datos históricos.

Existe un caso especial en el tratamiento de fechas en los asistentes. Si un histórico tuviera muchos registros, se podría hacer lo mismo que en las listas, filtrar la información desde del Meta4Object.

Tener siempre presente que los datos más recientes tienen que ser los primeros en aparecer.

En las consultas de información

El objetivo de las consultas es mostrar la información más completa y reciente. Si se consulta la información de una persona, interesa saber todos sus periodos de alta en la empresa y sus roles en las fechas de trabajo. Y para cada periodo y para cada rol interesa conocer los datos vigentes de los históricos. En el ejemplo se distingue un nodo raíz y los históricos:

- El raíz carga la información principal: los periodos. Este nodo muestra todos los periodos de una persona comprendidos en las fechas de trabajo.
- Para mostrar la información de los históricos asociados a un periodo del RH se calcula una fecha de corte a partir de las fechas de inicio y fin del nodo raíz, no a partir de las fechas de inicio y fin de trabajo de la sesión.

Esta fecha se obtiene llamando al método **SHCO_FILTER_DATE** del nodo **SHCO_GN_TOOLS**. Este método recibe dos argumentos, la fecha de inicio y la fecha de fin del periodo seleccionado en el nodo raíz. Si el nodo raíz no tuviera fechas el método toma las fechas de sesión.

Para entender el cálculo de esta fecha se indica el siguiente ejemplo.

EJEMPLO

En una nueva conexión al cliente Web, parametrizando la fecha de inicio con 01-01-2001 y fecha de fin con 02-02-2001, para una persona que tenga los siguientes periodos:

- 01-01-1800 - 31-06-2000
- 07-09-2000 - 10-01-2001
- 08-01-2001 - 01-01-4000
- 10-01-2001 - 30-01-2001
- 04-02-2001-

se muestran los que tengan alguna intersección con el periodo de trabajo (01-01-2001 / 02-02-2001):

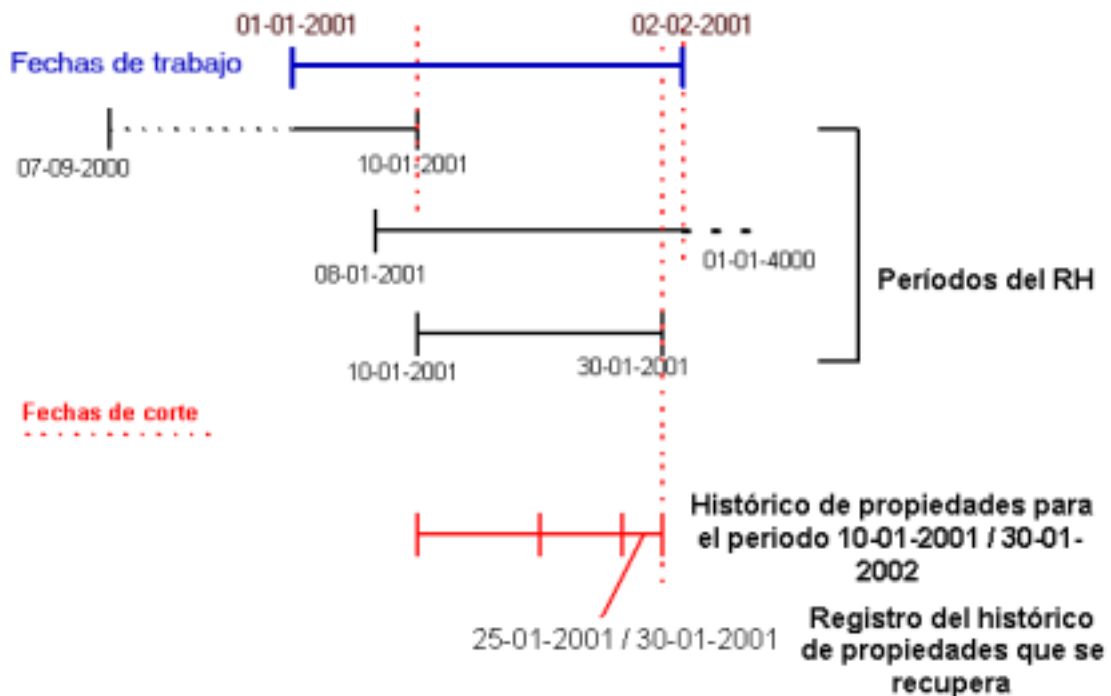
- 07-09-2000 - 10-01-2001
- 08-01-2001 - 01-01-4000
- 10-01-2001 - 30-01-2001

A la hora de obtener registros de históricos correspondientes a algún periodo, se utiliza la fecha de corte, que se calcula a partir de las fechas de inicio y fin del periodo seleccionado y de la fecha de corte teórica (02-02-2001). Así, para cada uno de periodos recuperados, la fecha de corte será:

- 07-09-2000 - 10-01-2001 fecha corte real: 10-01-2001
- 08-01-2001 - 01-01-4000 fecha corte real: 02-02-2001
- 10-01-2001 - 30-01-2001 fecha corte real: 30-01-2001

Así, si se selecciona el periodo 10-01-2001-30-01-2001, de los históricos muestra los registros vigentes a fecha 30-01-2001, como se muestra en la figura.

Figura 43. Cálculo de la fecha de corte



Gestión de caracteres especiales (HTML, JAVASCRIPT)

10

Acerca de este capítulo

Este capítulo describe cómo formatear los valores recuperados de los Meta4Objects para:

- Poder trabajar correctamente con caracteres especiales como la comilla simple, comilla doble...
- Cumplir el estándar XHTML 1.0 en las páginas

Este formato se hace a través de atributos asignados a tags concretos. A continuación se describen estos atributos, el modo de uso de los mismos y los casos especiales.

Atributos de formato

Para el formato de valores a html existe el atributo **htmlsafe** para los tags `m4:item` y `m4:label` y para el formato a tipo javascript tenemos el atributo **jsafe** para los mismos tags.

Ambas funciones de formato están accesibles a través de funciones java cuando se trabaja con *items/labels* directamente con la *m4operations*. Más información, en el apartado *Funciones equivalentes en Java*.

A continuación se describe:

- Uso de atributo `htmlsafe`
- Uso de atributo `jsafe`
- Casos especiales
- Funciones equivalentes en Java
- Ejemplos de uso

Uso de atributo htmlspecialchars

Si asignamos a un *tag* `m4:item/m4:label` el atributo `htmlspecialchars` con valor a `true`, (`htmlspecialchars="true"`), conseguimos que la información nos llegue con notación `html`.

EJEMPLO

Organización lo devuelve como Organizaci´n

Se utiliza siempre que la información devuelta por el *tag* `m4:item/m4:label` se utilice dentro de código `html`, bien sea para mostrarla o para utilizarla en eventos (`href`, `onclick`..). Así se consigue que las páginas pasen las validaciones `HTML` del `W3C`.

No se usa nunca este atributo:

- En código `javascript` puro, es decir, en secciones `<script.. > </script>` pues `javascript` no entiende la codificación de caracteres *&entity*; de `HTML` y los interpreta literalmente.
- Cuando la información devuelta por `m4:item/m4:label` se recoge en una variable `java` con el atributo `m4varname` y esta variable no se va a mostrar en `HTML`.

EJEMPLO

```
<m4:item outputdef="DataAskParam" item="ID_ITEM" m4varname="sIDItem" />
```

Uso de atributo jsafe

Si se asigna a un *tag* `m4:item/m4:label` el atributo `jsafe` con valor `true`, (`jsafe="true"`), conseguimos que la información nos llegue con los caracteres especiales de `javascript` codificados para poder tratarlos correctamente: `apóstrofe`, `comilla doble`, `retorno de carro`.

EJEMPLO

O'neill lo convierte a O\'neill

Se utiliza siempre que la información devuelta por el *tag* m4:item/m4:label se utilice dentro de código javascript. En caso contrario, si el valor del *item/label* tiene caracteres especiales para javascript (ver siguiente tabla) podemos tener errores de funcionamiento en las páginas.

Hay que entender como código javascript, todo el código que va entre los tags `<script..</script>`, así como el código que se ejecuta a través de eventos de html como por ejemplo `onclick = ..` o un link que ejecutará javascript (`href="javascript:..."`).

Tabla 4. Caracteres especiales

Caracteres especiales para javascript	
Apóstrofe	'
Retorno	\r \n
Comillas	"
Backslash	\

No se utiliza nunca en tags que se muestran directamente en HTML, pues HTML interpreta los caracteres de escape de javascript literalmente, con lo cual aparecería la barra de escape.

Casos especiales

Existen variables java cuyo contenido no se ha codificado para poder visualizarlas correctamente en HTML y que si se van a visualizar. Por ejemplo, los input que dibujan sus valores a través de variables java.

En estos casos, antes de mostrar el valor hay que formatearlo a HTML para que funcione correctamente. En caso contrario, puede suceder que si viene como valor una comilla doble se corte el valor. Dado que se trabaja en java hay que usar la función equivalente a `htmlspecialchars="true"`. Más información, apartado Funciones equivalentes en java.

Este caso se da en los MT con listas, de forma que cuando se pasa de una ventana de resultados (1-10) a otra ventana (11-20), dado que se mantienen los valores del formulario éstos se pasan a través de un *request* por variables java. Es en estos casos, hay que formatear el valor a html antes de asignarlo al valor de la caja.

EJEMPLO

```
<input type="text" id="X<%=zIdInc%>" name="X<%=zIdInc%>"
value="<%=zIdP1%>" />
```

Convertirlo a :

```
<input type="text" id="X<%=zIdInc%>" name="X<%=zIdInc%>"
value="<%=M4PresentationUtilTaglib.cookHTML(zIdP1)%>" />
```

Funciones equivalentes en java

Las funciones java equivalentes a los atributos utilizados para el correcto tratamiento de los caracteres en html y javascript son las siguientes:

Tabla 5. Funciones equivalentes en java

Atributo	Función java
Htmlsafe="true"	M4PresentationUtilTaglib.cookHTML(zItem)
Htmlsafe="false"	M4PresentationUtilTaglib.uncookHTML(zItem)
jsafe="true"	M4PresentationUtilTaglib.escape(zItem)
jsafe="false"	M4PresentationUtilTaglib.unescape(zItem)

Ejemplos de uso

A continuación se detallan ejemplos de cada uno de los casos expuestos en este capítulo.

Código javascript puro

```
<script type="text/javascript" language="Javascript1.5">

function comprobar(){
    var sfunciones =
    "m4valinput('_alfanum_oblig','NombreFormulario',sIdPk,1,'<m4:label
m4name="<%=zIdPk1%>" jsafe="true"/>')";

    var verr=m4valform(sfunciones);
    if (verr == 1){m4submit("NombreFormulario");}
}

</script>
```

Código javascript embebido en HTML

```
<m4:loop from="<%=zregistroinicials%>" to="<%=zregistrofinals%>">
<tr>
href="javascript:m4valor('NombreFormulario','X<%=zcampoID1%>',
'<m4:item m4name="<%=zIdPk1%>" jsafe="true" htlsafe="true"/
>','set');
m4valor('NombreFormulario','<%=zcampoNombre1%>', '<m4:item
m4name="<%=zNPk1%>" jsafe="true" htlsafe="true"/>','set');">
file="../../files_gif/ic_bor.jsp" %> /></a></td>

</tr>
</m4:loop>
```

Código HTML sin javascript

```

<table><tr>
<m4:item m4varname = "varzIdPk1" m4name="<%=zIdPk1%>" />

<td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zIdPk1%>"
htmlsafe="true"/>
</td>

<td><a>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zLNPk1%>" htmlsafe="true"/>"
</td>

</tr>

```

Input recogiendo el valor de variable java

```

<input tabindex="<%= (zTab + 1)%>" class="disabled"
readonly="readonly" type="text" id="X<%=zIdInc%>"
name="X<%=zIdInc%>" size="9" maxlength="9" title="<m4:label
m4name="<%=zSHCOLBWRITE%>" htmlsafe="true"/> <m4:label
m4name="<%=zIdIncr%>" htmlsafe="true"/>"
value="<%=M4PresentationUtilTaglib.cookHTML(zIdP1)%>" />

```

Páginas JSP multiidioma

11

Acerca de este capítulo

La mayor parte de las páginas JSP, saca la información traducida a través de elementos de Meta4Objects. Pero hay ciertas páginas que necesitan incluir literales directamente en la página, por ejemplo los archivos .jsp de tipo botón (`files_gif`).

Para estos casos, es decir, para aquellas páginas que no tengan un Meta4Object detrás, del que se puedan extraer los literales traducidos, se pretende dar una solución que permita:

- Tener una única versión de la página aunque tenga literales susceptibles de traducirse, para facilitar el mantenimiento.
- Unificar la localización de los literales a traducir para facilitar el proceso de traducción.

Para la correcta gestión de las páginas multiidioma es necesario:

- Crear los archivos de propiedades con los literales traducidos: se detalla el proceso de generación, su localización en el servidor y cuál debe ser su contenido.
- Crear la página de carga de las traducciones generadas en el primer paso: se detalla el proceso de creación, su contenido, su localización en el servidor, la forma de llamarse y utilizarse en las páginas JSP.

Crear archivos de literales traducidos

Para las traducciones se crea un archivo de propiedades por tarea de cliente Web e idioma de logado: **shco_g0_es.properties**, **shco_g0_en.properties**, **shco_g0_fr.properties**.

El nombre de estos archivos debe seguir esta nomenclatura:

`Idtarea_pais_idioma.properties`

Dónde:

IdTarea: nombre del directorio donde están las páginas con literales a traducir.

País: Identificador del país propietario de los literales: sp, en, fr. De forma que si en la aplicación corporativa existen los `shco_g0_es.properties`, `shco_g0_en.properties`, `shco_g0_fr.properties` y por ejemplo Francia quiere crear sus literales específicos, crearía: `shco_g0_fr_es.properties`, `shco_g0_fr_en.properties`, `shco_g0_fr_fr.properties`.

Idioma es el valor de la variable java **zlanguser** que se tiene en los *includes* genéricos **shco_g0/shco_gen_bag.jsp** que tienen todas las páginas: es, fr, en

Si los archivos fuente sobre los que se debe traducir son los de idioma español (**..es.properties**), del resto de idiomas destino se encarga el proceso de traducción que antes de la traducción copia el contenido del archivo en español al del resto de idiomas.

El contenido de cada uno de estos archivos sigue la siguiente sintaxis:

`IDLiteral=texto traducido`

Donde,

El **IDLiteral** es el código identificativo del literal. Para construir este código se sigue la metodología de definición de clases en las css. Por ejemplo, todos los literales a los que se hagan referencia desde botones ponerle **Button.IdLiteral**.

```
Button.Filter=Filtro
Literal.Org=Organizacio&acute;n
Menu.Reports=Informes
Menu.TODOList=Lista de tareas
Menu.Top=Arriba
Menu.Start=Inicio...
```

Una vez que se han creado estos archivos, la manera de llamarlos desde las páginas que utilizan estos literales es a través de una página intermedia cuya creación se muestra a continuación.

Crear la página de carga de las traducciones

Para cada tarea, se crea un archivo cuya función es cargar las traducciones. El nombre debe seguir esta nomenclatura:

`Idtarea_trans.jsp`

Por ejemplo: **shco_g0_trans.jsp**

Se debe situar en el directorio *root* de la tarea a la que corresponde.

Para crear este archivo en una nueva tarea se siguen estos pasos:

1. Copiar el archivo **shco_g0_trans.jsp** sobre el directorio raíz de la tarea. Este archivo se encuentra en el directorio **shco_g0**.
2. Renombrar el archivo, cambiando **shco_g0** por el nombre de la tarea.
3. Modificar el contenido:
 - Cambiar el nombre de la variable java. Reemplazar **Tran_scho_g0** en los dos sitios donde aparece por **Tran_IdTarea**.
 - Cambiar el nombre del archivo de propiedades. Hay que cambiar **shco_g0** por el nombre de tu tarea.

```
<%  
    java.util.Properties Tran_shco_g0 = new Properties();  
    Tran_shco_g0.load(application.getResourceAsStream("/  
translations/shco_g0_" + zlanguser + ".properties"));  
%>
```

Una vez creado el archivo de carga, todas aquellas páginas que necesiten trabajar con archivos de propiedades susceptibles de traducir tienen que hacer la carga del mismo al principio de la página. La forma de hacer la carga del archivo será a través de un *include* `<%@ include file="archivo de carga.jsp"`. Por ejemplo: `<%@ include file="../../shco_g0/shco_g0_trans.jsp" %>`.

Además, en el cuerpo de la página para hacer referencia a los literales traducidos, se debe utilizar la variable java **Tran_Idtarea** definida en el archivo de carga. En el ejemplo se trabaja con la de **shco_g0**.

```
widowctrl(85)Código Fuente2  
    alt="<%=Tran_shco_g0.getProperty("Button.Filter")%>"  
(85)    Código Fuente2
```

Organización de páginas multiidioma

Métodos de traducción de literales

Recapitulando, la traducción de todo literal que se necesite en una página depende de si hay un Meta4Object detrás o no.

- Si hay un Meta4Object detrás, se sigue utilizando el mecanismo de crear una propiedad y usar la etiqueta **m4:label**. Para más información de esta etiqueta, consulte la "Referencia de la biblioteca de M4Taglib" en el manual Desarrollo de funcionalidad en el cliente HTML
- Si no hay Meta4Object detrás, se utilizan los archivos de *properties*.

No existen páginas dependientes del idioma. Con el objeto *properties* todas las páginas tienen el mismo código.



Por tanto al gestionar el control de versiones, deberían figurar como archivos compartidos.

Organización de páginas multiidioma

Todas las páginas nuevas que se incluyan y que sean multiidioma, bien porque traigan la información traducida desde un Meta4Object o porque usen el objeto de traducciones se gestionan de la siguiente forma:

- Si son páginas que se invocan vía *servlet/checksecurity*. Se desarrolla la página y se cuelga en el directorio raíz de la tarea. Y dado que necesitamos para que sea invocada que esté en el directorio del idioma, se incluye en este último directorio una página que llame a la página del directorio raíz.
- Si son páginas de tipo include directamente se coloca en el directorio raíz de la tarea, fuera del idioma.

Acerca de este capítulo

Se describe la gestión de errores en un desarrollo en cliente Web. Esta gestión depende de si los errores se circunscriben en la parte servidor o en la parte cliente. Es decir, los errores se pueden ejecutar en servidor o en cliente, pero todos se visualizarán en cliente. Así, se divide este capítulo en:

- Gestión de errores en servidor
- Gestión de errores en cliente

Gestión de errores en servidor

El usuario tiene la posibilidad de parametrizar el grado de depuración que desea. Existen 4 niveles de depuración:

- Nivel 0 - Usuario: aparecen todos los mensajes funcionales. Es decir, los pertenecientes a módulos distintos del 0, salvo los incluidos en los submódulos 1 que se reservan para control de flujo de ejecución.

Se hará una excepción con los incluidos en el módulo de multimoneda (120) en los que no se filtrará por submódulo. También aparecen los mensajes de error generados por la BDL.

- Nivel 1 - Usuario Avanzado: se suprimen los mensajes informativos y alertas funcionales para agilizar el manejo de la aplicación.

Aparecen los errores funcionales. Es decir, los pertenecientes a módulos distintos del 0 salvo los incluidos en los submódulos 1 (si se trata de un mensaje de multimoneda no se excluye ningún submódulo) y los provenientes de la BDL.

- Nivel 2 - Desarrollador: aparecen todos los mensajes tanto funcionales como tecnológicos.
- Nivel 3 - Desarrollador Avanzado: se reduce el log eliminando alertas y avisos tecnológicos.

Aparecen todos los mensajes funcionales y de los tecnológicos únicamente pasarán el filtro los errores.

Los mensajes de error de cliente Web incluidos en el estándar corporativo estarán incluidos en el módulo 61 y el submódulo indicará la funcionalidad.

Los submódulos 1 se reservarán para mensajes destinados únicamente a desarrolladores.

A continuación se detalla el proceso para generar los mensajes en servidor.

Proceso

Los pasos para utilizar la depuración de errores son los siguientes:

1. Generar los *SetLogs*:

Utilizar los *Setlogs* dentro del código LN4, teniendo en cuenta que el nivel del error determinará su visualización por parte del usuario.

El submódulo 1 se reserva para mensajes únicamente destinados a desarrolladores. Se hará una excepción con los ya existentes de multimoneda.

2. Leer la pila de errores.

Antes de terminar cualquier proceso, esto es, antes del *Return* (**M4_SUCCESS**) o del *Return* (**M4_ERROR**) que finaliza el proceso en LN4, hay que ejecutar el método: **SHCO_DEBUG_MAKER** del nodo **SHCO_GN_TOOLS**.

Este método se recorre la pila de los errores, recupera aquellos mensajes de error válidos para el nivel de depuración que tenga el usuario en estos momentos, y los deposita en el ítem **SHCO_TEXT** del nodo **SHCO_GN_COMUNICACION**. Igualmente, en el ítem **SHCO_ACTIVE_DEBUG** del mismo nodo, marca (1/0) si se ha recuperado o no algún mensaje de error (este es el valor utilizado para levantar o no la ventana de errores).

3. Conectar con la página JSP. Para que en una página aparezca la ventana de errores, es necesario lo siguiente:

1. Traerse el **OUTPUTDEF** del nodo **SHCO_GN_COMUNICACION**.

2. Tras la transacción, leer el item **SHCO_ACTIVE_DEBUG**.

```
<%String zerror="";  
try {  
    M40perations m = new M40perations(request);  
    zerror =  
    m.getItem(znodocom,zm4object,znodocom,"","SHCO_ACTIVE_DEBUG");  
}catch(Exception e) {}  
%>
```

3. Incluir tras el anterior el *include* de la ventana de error:

```
<%@ include file="shco_gen_error.jsp" %>
```

Gestión de errores en cliente

En este apartado se explica cómo incluir nuevos mensajes de errores funcionales, con el objetivo de que dichos mensajes tenga una organización clara por país y funcionalidad.

Se detalla:

- la política de mantenimiento de los archivos de error
- la organización de los archivos de errores y dónde meter cada mensaje de error funcional, así como la metodología para establecer el identificador para dichos mensajes y su invocación desde las página jsp.

Estos puntos son importantes ya que tener una buena estructuración de archivos nos facilitará los procesos de actualización y mantenimiento y el mantener una metodología para establecer los identificadores de mensajes nos evita que haya duplicación de identificadores.

Política de mantenimiento de los archivos de error

Los archivos de los mensajes de error se clasifican por país e idioma, y la política de mantenimiento de los mismos sigue los siguiente puntos:

- La versión inicial de estos archivos para todos los países e idiomas se creará automáticamente.
- En las sucesivas actualizaciones sólo se pasarán los archivos de errores de corporativo, para no sobrescribir los mensajes de los países.

- Desde corporativo no se modificarán los archivos de mensajes de error de los distintos países. Por tanto en el proceso de globalización, todos los mensajes de error que se añaden deberán ser corporativos.
- Cada país deberá incluir los id de los mensajes en el idioma de desarrollo de forma que el equipo de traducciones tomará la última versión del archivo asociado al país y al idioma de desarrollo y actualizará los archivos de ese país en el resto de idiomas.

Organización de los archivos de error

Los archivos de errores funcionales se encuentran en el directorio *translations*. a partir del *root* de la aplicación, y están clasificados por país e idioma de la siguiente forma:

`m4err_ClavePais_ClaveIdioma.js`

Donde:

Clave País		Clave Idioma	
España	sp	Español	es
Francia	fr	Francés	fr
United Kingdom	uk	Inglés	en

EJEMPLO

Archivo con errores funcionales de España en español:

`m4err_sp_es.js`

Archivo con errores funcionales de España en inglés:

`m4err_sp_en.js`

Existe un archivo más que es `m4err_cl_ClaveIdioma.js` que también se carga y se utilizará para mensajes de error propios del cliente.

Dentro de cada archivo se relacionan el conjunto de mensajes de error. Para ello es necesario conocer:

- Cómo se identifican y redactan estos mensajes
- Cómo se invocan y escriben desde las páginas jsp

Forma de declarar y escribir los mensajes en los archivos

Cada mensaje de error son variables javascript por tanto hay que declararlos como tales, es decir:

1. Precedidos de la palabra reservada var.
2. El valor entre comillas dobles.
3. Terminar la definición con punto y coma.

EJEMPLO

```
var _sl_p_ta_0= "La fecha de inicio debe ser superior a la fecha de fin";
var _empleado_baja = "Seleccione empleados a los que dar de baja.";
//var _cancel_mod_baja = "Seleccione empleados a los que cancelar o modificar la de baja.";
var _delete_recol = "La persona no puede ser borrada";
```

Identificadores

Respecto al identificador del mensaje, hay que tener en cuenta que se clasifican por país y módulo de funcionalidad de la siguiente forma:

después de var irá _sl_ClavePais_ClaveModulo_Contador

Dónde:

ClaveModulo será igual a gn para mensajes genéricos utilizados en varios módulos.

EJEMPLO

//Mensajes de error de España para el módulo de Time &Attendance:

```
var _sl_sp_ta_0 = "...";
var _sl_sp_ta_1="..."
```

//Mensajes de error de España para el módulo de Cash Comp

```
var _sl_co_cc_1 = "El número de bandas debe ser mayor que cero.";
var _sl_co_cc_2 = "Debe especificar un código identificativo para la estructura salarial.";
var _sl_co_cc_3 = "El número de zonas debe ser mayor que cero.";
```

```
//
```

Parámetros

Para añadir parámetros a los mensajes se utiliza la siguiente sintaxis:

`&%Posicion&`

Con Posicion igual a 0 ..n

EJEMPLO

```
var _sl_p_ta_0="La fecha de inicio &%0& debe ser superior a la
fecha de fin &%1&";
var _sl_co_pa_8 = "El Número de Cuenta insertado tiene menos de 8
caracteres, el espacio restante será rellenado. Por ejemplo, si
insertaste &%0&, se guardará como &%1&. Este campo puede almacenar
&%2& caracteres aunque solo los 8 últimos son utilizados. Los
caracteres adicionales son requeridos para tipos de cuenta
especializadas.";
```

Mostrar los mensajes de error

Una vez que están redactados y declarados los mensajes en los archivos de error correspondientes, ya se pueden invocar desde las páginas jsp que se desee.

La función javascript para invocar y mostrar un mensaje de error es la `m4showmessage` que muestra el mensaje identificado por `IdMensajeError` con la sustitución correspondiente de los argumentos.

La función javascript `m4getmessage`, no muestra el mensaje, en su lugar devuelve una cadena con el mensaje identificado por `IdMensajeError` con la sustitución correspondiente de los argumentos.

En ambos casos los parámetros son:

- `IdMensajeError`
- Conjunto de parámetros del mensaje en orden. Es decir, el orden en el que se pasan a la función se corresponderá con la posición establecida en el mensaje de error. El primer parámetro que se pasa, será sustituido por el parámetro definido en el mensaje con posición 0, el segundo con posición 1...

EJEMPLO

```
m4showmessage("_sl_p_ta_0",10-10-2001,08-10-2001);
```

Carga de los mensajes de los diferentes estándares

Si utiliza el atributo `_PROD` para configurar y personalizar la aplicación tendrá que crear una página jsp que cargue los archivos con los mensajes javascript de las diferentes estándares.

Si `_PROD = pn`, deberá crear la siguiente página jsp **shco_go_pn/shco_gen_load_js_msg.jspf**, con el siguiente contenido:

EJEMPLO

```
<%--
@(#)FileVersion: 601.000.003
@(#)FileDescription: include genérico de msg js
@(#)CompanyName: Meta4 Spain, S.A.
@(#)LegalCopyright: (c)1998
@(#)ProductName: Meta4 PeopleNet Ksystem
@(#)ProductVersion: 6.0
@(#)InternalName: shco_gen_load_js_msg.jsp
@(#)Date: 21/02/2002
--%>

<%String zlanguser = (String) request.getAttribute("zlanguser");%>
<script type="text/javascript" src="/translations/
m4err_co_<%=zlanguser%>.js"></script>
<script type="text/javascript" src="/translations/
m4err_uk_<%=zlanguser%>.js"></script>
<script type="text/javascript" src="/translations/
m4err_fr_<%=zlanguser%>.js"></script>
<script type="text/javascript" src="/translations/
m4err_sp_<%=zlanguser%>.js"></script>
<script type="text/javascript" src="/translations/
m4err_cl_<%=zlanguser%>.js"></script>
```

Si el valor para el atributo `_PROD` o la etiqueta `MainProductID` del `configclient.xml` tienen valor nulo o vacío no será necesario incluir ningún archivo pues la tecnología cargará esos archivos para mantener la compatibilidad hacia atrás.

Acerca de este capítulo

En este capítulo se describe cómo se puede integrar el filtro en ejecución o filtro dinámico en procesos del cliente Web, así como los distintos tipos de integración existentes.

Introducción

El filtro dinámico es un proceso que, dado un Meta4Object,

- lista los nodos asociados que tengan filtro dinámico. Un nodo se considera que tiene filtro dinámico si tiene establecida la propiedad avanzada de nodo *Filtro dinámico* a *Sí*.
- permite, para cada uno de los nodos listados, establecer una sentencia de filtro a través del filtro HTML. El filtro HTML permite realizar búsquedas desde el cliente Web seleccionando los criterios que quiere en el interfaz del usuario y permite configurar y crear criterios complejos en el entorno Web basándose en cualquier tabla. A la hora de establecer el filtro, se puede hacer directamente sobre la tabla base del nodo o sobre un conjunto de tablas que vengan agrupadas por un escenario que haya seleccionado previamente. Para un nodo sólo se permite seleccionar aquellos escenarios cuya tabla base coincida con la del nodo.

El filtro HTML también permite crear filtros predefinidos que se pueden volver a utilizar en otras ocasiones y funcionalidades.



Para más información sobre el Filtro HTML consulte el manual *Desarrollo de funcionalidad para el cliente HTML*.

- permite aplicar o devolver el filtro global retornado a la página de invocación o navegando a otra página.



Si el filtro dinámico se ejecuta sobre un Meta4Object que no tenga nodos con filtro dinámico, actuará igual que si se hubiera aplicado el filtro, es decir, retornando o invocando la página que corresponda.

Cómo llamar al filtro dinámico

Para invocar al filtro dinámico desde una página, hay que incluir la página *shco_g0/shco_gen_dynfilter_include.jsp* y después llamar a la función genérica de javascript *m4dynfilter()*.

Función de invocación al filtro: *m4dynfilter()*

Parámetros de entrada

m4dynfilter(ssub,sm4o,sm4oalias,sapplymode,sretmode,sretpage,sretpagewidth,sretpageheight,sreturnform,scallbackfunction,sidresultinput)

Para invocar al filtro *m4dynfilter* se utilizan los siguientes parámetros de entrada:

Parámetro	Descripción	Tipo
sm4o	Meta4Object cuyos filtros dinámicos se quieren editar.	Obligatorio
ssub	Sesión sobre la que debe trabajar el filtro dinámico. Se requiere para aquellos casos en los que haya que aplicar el filtro dinámico sobre un Meta4Object previamente instanciado.	Opcional
sm4oalias	Alias del Meta4Object sobre el que se trabaja. Se requiere para aquellos casos en los que haya que aplicar el filtro dinámico sobre un Meta4Object previamente instanciado en página.	Opcional

Parámetro	Descripción	Tipo
sapplymode	Forma de aplicar el filtro: Si = 1, que es el valor por defecto, devuelve el filtro en formato nodo item valor sin aplicar. Si = 2, devuelve el filtro en forma JIT sin aplicar. Si = 3, aplica el filtro sobre el Meta4Object (sm4o) instanciado en página. Si = 4, aplica el filtro sobre el Meta4Object (sm4o) instanciado vía LN4. Para sapplymode = 1 ó 2, es la página de retorno o la página a la que se navega, que se debe encargar de la aplicación del filtro según el caso.	Opcional
sretmode	Si = 1, que es el valor por defecto, tras la invocación con <i>m4window</i> se retorna vía <i>m4submit</i> a <i>zreturnpage</i> . Si = 2, tras la invocación con <i>m4window</i> se retorna sobre la página de invocación con <i>m4returnvalues</i> . Si = 3, tras invocación con <i>m4windowcallback</i> se retorna sobre la página de invocación con <i>m4returnvalues</i> con la consiguiente invocación a la función que corresponda. Si = 4, tras invocación vía <i>m4submit</i> se retorna vía <i>submit</i> a <i>zreturnpage</i> .	Opcional
sretpage	Página a retornar tras la aplicación del filtro. Se requiere para <i>sretmode</i> = 1 o <i>sretmode</i> = 4.	Obligatorio si <i>sretmode</i> = 1 ó 4
sretpagewidth	Ancho de la página de retorno para el caso de <i>zretmode</i> = 1. Defecto: 800	Opcional
sretpageheight	Alto de la página de retorno para el caso de <i>zretmode</i> = 1. Defecto: 600	Opcional
sreturnform	Identificador del formulario donde retornar los valores. Se requiere para <i>sretmode</i> = 2 o <i>sretmode</i> = 3. El formulario del <i>include</i> tiene como ID: <i>frmcalldynfilter</i> y se puede usar como retorno.	Obligatorio si <i>sretmode</i> = 2 ó 3
scallbackfunction	Función a la que invocar una vez que el filtro dinámico retorna.	Obligatorio si <i>sretmode</i> = 3

Parámetros de salida

Parámetros de salida para *sretmode* = 1 ó 4

Estos parámetros se recogen en la página vía *request*.

EJEMPLO

```
<%
zdynfilter=request.getParameter("zdynfiltersinfo")
%>
```

Los parámetros de salida para este caso son:

Parámetro	Descripción
zdf_sub	Sesión sobre la que se ha aplicado el filtro dinámico.
zdf_m4o	Meta4Object cuyos filtros dinámicos se han establecido.
zdf_m4oalias	Alias del Meta4Object sobre el que se trabaja. Se requiere para aquellos casos en los que haya que aplicar el filtro dinámico sobre un Meta4Object previamente instanciado en página.
zdynfiltersinfo	Filtro en formato cadena para aquellos en los que no se aplica (<i>sapplymode</i> <> 3).

Parámetros de salida para *sretmode*= 2 ó 3

Son retornos vía la función javascript *m4returnvalues*. Se devuelve directamente el filtro en formato cadena sobre el *input* del formulario indicado en el parámetro *sreturnform*.

Si *sapplymode* = 3 este valor vendrá vacío.

Cómo usar el filtro dinámico

Los pasos que debe seguir para usar el filtro dinámico son:

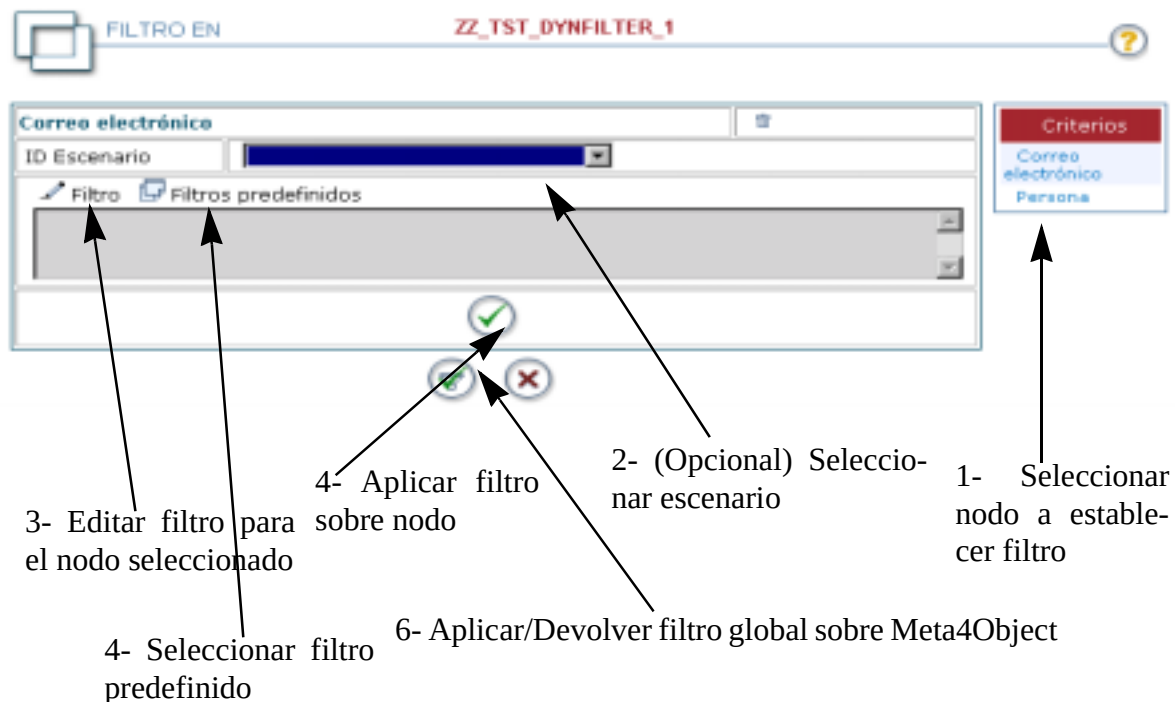
1. Seleccione en la lista uno de los nodos sobre los que se quiere establecer el filtro.
2. Seleccione el escenario sobre el que quiere trabajar a la hora de filtrar. Si no se hace se trabajará sólo con la tabla base del nodo.



Sólo aparecen listados aquellos escenarios cuya tabla base coincida con la tabla base del nodo cuyo filtro estamos editando.

3. Edite el filtro o seleccione un filtro predefinido.
4. Aplique el filtro sobre el nodo.
5. Repita el proceso para cada nodo que quiera filtrar.
6. Aplique el filtro de forma global al Meta4Object.

Figura 44. Filtro dinámico o en ejecución.



Ejemplos de uso

sretmode = 1

En este caso el filtro dinámico se abre en una ventana hija y se navega a una página posterior.

Para ello se usará la función *m4dynfilter* indicando *sretmode* = 1, así como la página a la que navegar después (*sreturnpage*). En el caso de que el filtro no se aplique directamente sobre el Meta4Object (*sapplymode* <> 3) la página *sreturnpage* recibirá la información del filtro.

EJEMPLO

```
<%@ include file="../../shco_g0/shco_gen_dynfilter_include.jsp"%>
<script type="text/javascript">
m4dynfilter("SES_DYN_FILTER", "ZZ_TST_DYNFILTER_1", "", "1", "1", "shco_g0/shco_gen_dynfilter_return.jsp")
</script>
```

sretmode = 2

En este caso se abre el filtro dinámico en una ventana hija y se retorna a la ventana padre.

Para ello se usará la función *m4dynfilter* indicando *sretmode* = 2. Además si el tipo de aplicación del filtro es <> 3, se indicará el identificador del formulario y del *input* donde devolver el filtro.

EJEMPLO

```
<%@ include file="../../shco_g0/shco_gen_dynfilter_include.jsp"%>
<script type="text/javascript">
m4dynfilter("SES_DYN_FILTER", "ZZ_TST_DYNFILTER_1", "", "1", "2", "",
    "", "", "frmcalldynfilter", "", "zdynfiltersinfo");
</script>
```

sretmode = 3

En este caso se abre el filtro dinámico en una ventana hija y se retorna a la ventana padre invocando esta función.

Para ellos se usará la función *m4dynfilter* indicando *sretmode* = 3, así como la función de la ventana padre a invocar (*scallbackfunction*). Además si el tipo de aplicación del filtro es *<>3*, se indicará el identificador del formulario y del *input* donde devolver el filtro.

EJEMPLO

```
<%@ include file="../../../shco_g0/shco_gen_dynfilter_include.jsp"%>
<script type="text/javascript">
m4dynfilter("SES_DYN_FILTER", "ZZ_TST_DYNFILTER_1", "", "1", "3", "",
    "", "", "frmcalldynfilter", "returnDynFilter()", "zdynfiltersinfo");
</script>
```

sretmode = 4

En este caso se abre el filtro dinámico en la misma ventana y se navega a una página posterior.

Para ello se usará la función *m4dynfilter* indicando *sretmode* = 4, así como la página a la que navegar después (*sreturnpage*). En el caso de que el filtro no se aplique directamente sobre el *Meta4Object* (*sapplymode* *<>* 3) la página *sreturnpage* recibirá la información del filtro.

EJEMPLO

```
<%@ include file="../../../shco_g0/shco_gen_dynfilter_include.jsp"%>
<script type="text/javascript">
m4dynfilter("SES_DYN_FILTER", "ZZ_TST_DYNFILTER_1", "", "1", "4", "
shco_g0/shco_gen_dynfilter_return.jsp");
</script>
```

Ejecución y planificación de informes

14

Acerca de este capítulo

En este capítulo se describen los pasos que se tienen que seguir para ejecutar o planificar la ejecución de un informe y establecer ciertos parámetros de configuración desde un entorno de cliente Web.

La aplicación le permite optar entre ejecutar un informe en una fecha determinada, es decir, planificar la ejecución, o hacerlo inmediatamente.

Este capítulo describe las características del sistema:

- cómo ejecutar o planificar la ejecución de un informe y obtener los resultados de esa ejecución, y
- cómo introducir los parámetros en el sistema.

Introducción

El sistema permite ejecutar o planificar un informe y establecer ciertos parámetros de configuración desde un entorno de cliente Web.

La ejecución del informe puede tener los siguientes objetivos:

- Visualizar el informe en el navegador. Los formatos permitidos son HTML, PDF y Texto.
- Guardar el archivo resultado de la ejecución en la máquina cliente. Los formatos permitidos son HTML, PDF, Texto, PCL, PostScript y ESCP.
- Guardar el archivo resultado de la ejecución en el sistema de archivos del servidor. Los formatos permitidos son HTML, PDF, Texto, PCL, PostScript y ESCP.
- Imprimir el informe a través de una impresora de la máquina cliente. Los formatos de impresión permitidos son PCL, PostScript, Texto y ESCP.
- Imprimir el informe a través de una impresora del servidor. Los formatos de impresión permitidos son PCL, PostScript, Texto y ESCP, aunque dependerá de cuales tiene disponible cada impresora.



Estas impresoras están definidas en un Meta4Object del sistema. Su mantenimiento se puede realizar a través de la opción de menú **Herramientas de desarrollo | Informes | Herramientas de prueba de informes | Impresoras del servidor**.

El sistema sólo permite planificar los siguientes tipos de ejecución:

- Guardar el archivo resultado de la ejecución en el servidor. Los formatos permitidos son HTML, PDF, Texto, PCL, PostScript y ESCP.
- Imprimir el informe a través de una impresora del servidor. Los formatos de impresión permitidos son PCL, PostScript, Texto y ESCP, aunque dependerá de cuales tiene disponible cada impresora.



Estas impresoras están definidas en un Meta4Object del sistema. Su mantenimiento se puede realizar a través de la opción de menú **Herramientas de desarrollo | Informes | Herramientas de prueba de informes | Impresoras del servidor**.

Cómo ejecutar o planificar la ejecución de un informe

Para planificar y ejecutar un informe hay que invocar la página */CheckSecurity/JSP/shco_rp/shco_m4throw.jsp* con los parámetros necesarios. El único parámetro obligatorio es el que indica el informe a ejecutar. Existen dos formas de traspasar los parámetros:

- a través de un formulario HTML
- a través de un Meta4Object

Los parámetros en su totalidad están definidos en el apartado *Configuración del sistema* de este mismo capítulo.

Transferencia de parámetros a través de un formulario

En este caso la página origen contiene un formulario cuya propiedad *action* es */CheckSecurity/JSP/shco_rp/shco_m4throw.jsp* y los parámetros de ejecución se definen en campos de entrada tipo texto.

EJEMPLO

```
<html><head><title></title></head>
<body>
<form action="/servlet/CheckSecurity/JSP/shco_rp/shco_m4throw.jsp"
method="post" name="NombreFormulario" id="NombreFormulario" >
<input type="submit" value="ejecutarinforme" />
<input type="hidden" id="ID_REPORT" name="ID_REPORT"
value="ZZ_TEST_PUB_INFOMES_REP1" />
<input type="hidden" id="OUTPUT_TYPE" name=" OUTPUT_TYPE "
value="VISUALIZE"/>
<input type="hidden" id="OUTPUT_FORMAT" name=" OUTPUT_FORMAT "
value="PDF"/>
...
</form>
</body>
</html>
```

Al hacer el *submit*, se ejecuta el informe sobre una nueva ventana. Si el usuario ha utilizado el ASKCONFIGPARAM, se levantará la ventana de configuración.

Transferencia de parámetros a través de un Meta4Object

Es posible suministrar los parámetros a través de un nodo de un Meta4Object. En este caso, previamente a la invocación de la página */CheckSecurity/JSP/shco_rp/shco_m4throw.jsp*, tiene que haber añadido como registros de un nodo de un Meta4Object los parámetros de ejecución de su informe.

Para que el sistema pueda recoger esos parámetros debe indicar cual es el nombre del Meta4Object y del nodo donde están los parámetros, así como la sesión en la que está trabajando.

Los parámetros que se tienen que indicar en el *request* al invocar la página *shco_m4throw.jsp* son:

- **zsubsesion**: identificador de la subsesión donde se han hecho las transacciones para añadir los parámetros de ejecución.
- **zm4o**: ID del Meta4Object que contiene los parámetros de ejecución.
- **zalias**: alias utilizado al definir el Meta4Object. Si no se especifica el sistema usará el que se indique en *zm4o*.
- **znode**: ID del nodo donde se almacenan en forma de registro los parámetros de ejecución del informe. Si no se especifica, el sistema tomará como nombre del nodo **SHCO_GN_PARAM_VALUES**.



Obligatoriamente los ID de elementos donde se almacenan el nombre del parámetro y su valor tienen que ser SHCO_PARAM y SHCO_VALUE respectivamente.

- **zopenmode**: indica sobre que ventana se abrirán el informe y la ventana de configuración. Sus posibles valores son:
 - 0: sobre la misma ventana que llama
 - 1: sobre una nueva ventana (valor por defecto).

Si se llama a la página *shco_m4throw.jsp* haciendo un *window.open*, se debe pasar un *zopenmode = 0* y el ID de la ventana se debe construir dinámicamente para que tenga un valor distinto cada vez que se ejecute. De esta forma, se permiten distintas ejecuciones al mismo tiempo, pero evitando que dos ejecuciones levanten la misma ventana.

EJEMPLO:

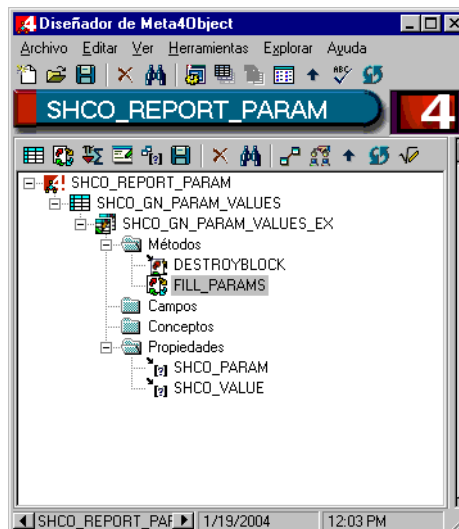
```
<script type="text/javascript">
var hoy = new Date();
    var sNewWindow = "Wnd" + hoy.getDay() + hoy.getHours() +
hoy.getMinutes() + hoy.getSeconds();
//Abro la ventana
window.open("",sNewWindow,"top=20,left=20,toolbar=no,scrollbars=yes,
directories=no,status=yes,menubar=no,resizable=yes,width=850,height=560");
urlLista = '/servlet/CheckSecurity/JSP/shco_rp/
shco_m4throw.jsp?zsubsesion='+ '<%=zsubsesion%>'+'&z4o='+ '<%=z4ob
ject%>'+'&zopenmode=0';
msgWindow = window.open(urlLista,
sNewWindow,"width=800;height=450,resizable,scrollbars,toolbar,menu
bar,status");
window.history.back();
</script>
```

-
- **Obligatorio:** *zalias* y *zm4o* tienen que ser distinto de *zsubsesion*.

Estructura del Meta4Object

La estructura del Meta4Object debe tener un nodo sobre el que se añadan como registros los distintos parámetros:

Figura 45. Estructura del Meta4Object



El nombre del nodo puede ser cualquiera, pero si es **SHCO_GN_PARAM_VALUES** no es necesario indicárselo como parámetro a la página en el momento de ejecutar. Las propiedades que obligatoriamente deben tener como ID son:

- **SHCO_PARAM**, tipo básico cadena fija de longitud 30, donde se almacenan los nombres de los parámetros.
- **SHCO_VALUE**, tipo básico *LONG* de forma que permita almacenar cualquier tipo de valor para los parámetros.

Este M4Object se carga desde una página *jsp* por lo que hay que especificar:

- la subsesión donde se están realizando las transacciones (en el parámetro *zsubsesion*)
- el nombre y el alias del Meta4Object (en los parámetros *zm4o* y *zalias*). Si no se especifica el alias, se establece uno igual al nombre Meta4Object especificado en *zm4o*. Por tanto es obligatorio suministrar, al menos, *zsubsesion* y *zm4o*.

Finalmente, hay que indicar si la página de configuración y, posteriormente el resultado del informe, se quiere abrir sobre la misma ventana o sobre una nueva. Para ello se utiliza el parámetro *zopenmode* y los posibles valores son:

- 1: para que se abra en una nueva ventana y
- 0: para que se abra sobre la actual.



Si no se especifica se abrirá sobre una nueva ventana.

EJEMPLO

Si en una página *jsp* ejecutamos un método para añadir los registros:

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionId="DAVID">
<m4:job>
<m4:datadef m4o="SHCO_REPORT_PARAM" m4name="SHCO_REPORT_PARAM" />
<m4:exec
m4method="SHCO_REPORT_PARAM!SHCO_GN_PARAM_VALUES.FILL_PARAMS">
</m4:exec>
</m4:job>
</m4:page>
```

El código LN4 del método FILL_PARAMS añade los parámetros de ejecución deseados a través del **Editor de fórmulas**:

```
AddRegister ()
SHCO_PARAM = "ID_REPORT"
SHCO_VALUE = "ZZ_TEST_GRUPOS"
AddRegister ()
SHCO_PARAM = "OUTPUT_TYPE"
SHCO_VALUE = "FILE_CLIENT"
AddRegister ()
SHCO_PARAM = "ASKCONFIGPARAM"
SHCO_VALUE = "FALSE"
```

Para llamar a la página *shco_m4throw.jsp* pasándole el Meta4Object donde están definidos los parámetros, se puede hacer otra página *jsp* con el siguiente código:

```
<html><head><title></title></head>
<body>
<form action="/servlet/CheckSecurity/JSP/shco_rp/
shco_m4throw.jsp?zsubsesion=DAVID&zm4o=
SHCO_REPORT_PARAM&zopenmode=1" method="post"
name="NombreFormulario" id="NombreFormulario" >
<input type="submit" value="ejecutarinforme" />
</form>
</body>
</html>
```

Cómo configurar la ejecución de un informe

Ejecución de un informe

Para facilitar la introducción de parámetros, el sistema permite al usuario visualizar una ventana con la mayoría de las opciones de configuración. Esta ventana de configuración está dividida en 3 pasos:

1. Seleccione el tipo de ejecución, **Ejecución directa**.



Los tipos y los formatos de salida quedan restringidos a la opción seleccionada.

2. Especifique el tipo y formato de salida:

Figura 46. Tipo y formato de salida

Pasos:
 Tipo y formato de salida
 Opciones de dispositivo
 Opciones avanzadas

CONFIGURACIÓN DE LA SALIDA DE UN INFORME

Tipo de ejecución
☒ Ejecución directa ☐ Planificación

Tipo de salida
☒ Visualización
☐ Archivo en máquina cliente
☐ Archivo en servidor
 Ruta relativa al directorio temporal de informes:
☐ Impresora en máquina cliente
☐ Impresora en servidor
 *ID impresora de servidor:

Formato de salida
☒ HTML ☐ PDF ☐ Texto ☐ PCL ☐ ESCP ☐ PostScript

Navigation icons: back, forward, cancel, OK, help.

Si el tipo de salida elegido era **Archivo en máquina cliente**, el sistema presentará un caja de diálogo que permitirá guardar el archivo en la ubicación deseada del disco local.

Si el tipo de ejecución es **Impresora en máquina cliente**, el sistema mostrará una lista combinada para que el usuario seleccione la impresora.

Si el tipo de salida elegido es **Impresora en servidor**, el sistema muestra la lista de impresoras disponibles y, una vez seleccionada una, establece el idioma por defecto asociado a la impresora elegida.



El usuario puede visualizar este paso restringiendo la lista de tipos de salida y de formatos disponibles de forma que se puedan evitar las ejecuciones no deseadas.

3. Establezca los parámetros específicos de dispositivos de impresión PCL, PostScript o ESCP.



Estas opciones sólo estarán disponibles si el formato de salida elegido en el paso anterior es uno de los correspondientes: PCL, PostScript o ESCP.

Figura 47. Opciones de dispositivo

Pasos
Tipo y formato de salida
Opciones de dispositivo
Opciones avanzadas

CONFIGURACIÓN DE LA SALIDA DE UN INFORME

Opciones PCL/PostScript

Bandeja: Superior

A doble cara: Desactivado

Copias:

Generar página: Seguir dispositivo

Resolución: Auto (300)

Aplicar color: ☒ Blanco y negro ☒ Color

Opciones ESCP

A doble cara: Desactivado

Avance de línea:

Factor X:

- Indique las opciones de cambio de moneda para los informes que usan esta funcionalidad.

Figura 48. Opciones avanzadas

Pasos
Tipo y formato de salida
Opciones de dispositivo
Opciones avanzadas

CONFIGURACIÓN DE LA SALIDA DE UN INFORME

Opciones de moneda

☐ Establecer moneda

ID Moneda: Peseta

Tipo de cambio: Oficial

Fecha de cambio: 21/01/2004

Planificación de un informe

El sistema permite la planificación de un informe para su posterior ejecución. Dicha ejecución se realiza en el servidor de aplicaciones y los resultados, para su posterior obtención o visualización, quedan almacenados en el gestor de ejecuciones del planificador de tareas.

La tarea de planificación para cada informe está definida en un Meta4Object del sistema. En caso de que no tuviera se usaría la tarea estándar.

El sistema permite realizar la planificación desde:

- una página determinada del usuario
- el planificador de tareas, en concreto desde el gestor de grupos de tareas.



Los tipos y formatos de salida quedan restringidos al tipo de ejecución seleccionado.

Figura 49. Planificación

Resultados de la ejecución del informe

Según el tipo de salida elegido, el sistema realiza diferentes acciones sobre los archivos generados tras la ejecución del informe para que estén accesibles:

- Visualización e impresión en máquina cliente: Los archivos se generan en el directorio temporal de sesión y permanecen ahí hasta que se cierre o expira la sesión.
- Archivo en máquina cliente: Los archivos se almacenan temporalmente en elementos de tipo básico *blob* en un *Meta4Object* del sistema hasta que son guardados. Una vez guardados en la máquina cliente los archivos no estarán ya disponibles en el directorio temporal.
- Archivo en servidor e impresión en servidor: En este caso los archivos se generan en el directorio temporal de informes del servidor de aplicaciones. Será responsabilidad del usuario el acceso a los mismos.



Recuerde que si ha planificado la ejecución de un informe, los archivos resultantes sólo estarán accesibles en el servidor y, por tanto, únicamente puede guardarlos o imprimirlos en el servidor de aplicaciones.

Ejecución de informes con grupos

Cuando el informe ejecutado genera varios archivos, el sistema muestra una ventana en la que se informa de los distintos archivos generados. Según haya sido el tipo de salida elegida, el usuario tendrá que seleccionar el archivo que desea visualizar o guardar en máquina cliente.

Si el tipo de salida es visualización, se mostrará la siguiente ventana:

Figura 50. Tipo de salida: Archivo en máquina cliente, PCL



RESULTADO DE LA EJECUCIÓN DEL INFORME

Informe generado correctamente.

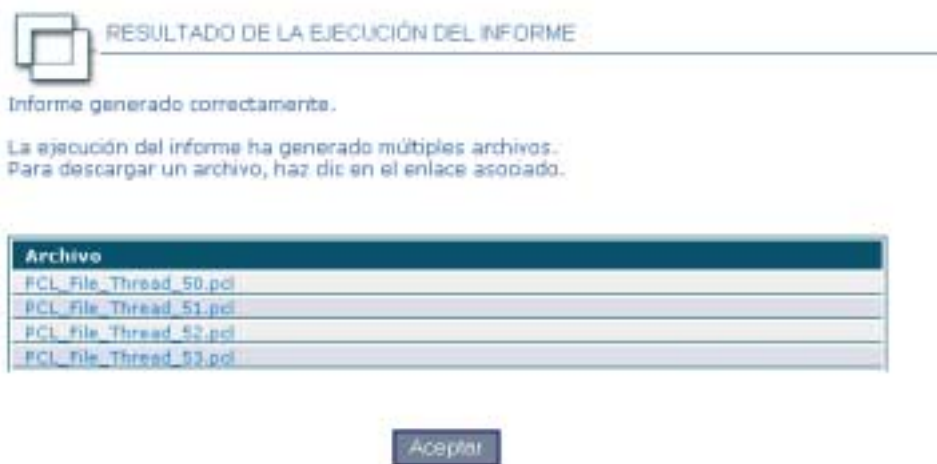
La ejecución del informe ha generado múltiples archivos.
Para descargar un archivo, haz clic en el enlace asociado.

Archivo
TXT_File_Thread_70.txt
TXT_File_Thread_71.txt
TXT_File_Thread_72.txt
TXT_File_Thread_73.txt

Aceptar

Si el tipo de salida es archivo en máquina cliente, la ventana será del tipo:

Figura 51. Tipo de salida: Archivo en máquina cliente, Texto



Configuración del sistema

Parámetros

A continuación se muestra un listado de los posibles parámetros junto con sus posibles valores, los valores por defecto, la obligatoriedad de los mismos y la posibilidad de aplicación.

Parámetro	Descripción
OPCIONES GENERALES	
ID_REPORT	Identificador del <i>Report</i> . ■ Obligatorio: SI
PROCESS_MODE	Modo de procesamiento. ■ Valores permitidos: ONLINE_EXECUTION, ejecución on-line del informe JS_EXECUTION, planificación de la ejecución del informe. ■ Obligatorio: NO ■ Defecto: "ONLINE_EXECUTION"
PROCESS_MODE_VISIBLE	Valores permitidos: "TRUE", "FALSE" ■ Obligatorio: NO ■ Defecto: "TRUE" si PROCESS_MODE contiene el valor "ONLINE_EXECUTION" o "JS_EXECUTION"

Parámetro	Descripción
PROCESS_MODE_NOT_ALLOWED	Valores permitidos: Indica los PROCESS_MODE no permitidos vinculados a los parámetros ONLINE_EXECUTION_AVAILABLE o JS_EXECUTION_AVAILABLE. ■ Obligatorio: No ■ Defecto: Los valores por defecto dependen de los valores vinculados a los parámetros ONLINE_EXECUTION_AVAILABLE y JS_EXECUTION_AVAILABLE, a continuación se describen los distintos casos. – Si ONLINE_EXECUTION_AVAILABLE = "FALSE" y JS_EXECUTION_AVAILABLE = "TRUE" entonces el valor por defecto es "ONLINE_EXECUTION " – Si ONLINE_EXECUTION_AVAILABLE = "TRUE" y JS_EXECUTION_AVAILABLE = "FALSE" entonces el valor por defecto es "JS_EXECUTION " – Si ONLINE_EXECUTION_AVAILABLE = "FALSE" y JS_EXECUTION_AVAILABLE = "FALSE" entonces el valor por defecto es "ONLINE_EXECUTION JS_EXECUTION " – Si ONLINE_EXECUTION_AVAILABLE = "TRUE" y JS_EXECUTION_AVAILABLE = "TRUE" entonces el valor por defecto es "="
ONLINE_EXECUTION_AVAILABLE	Indica si se permite o no la ejecución on-line. Implica mostrar o no el botón de ejecución on-line en la pantalla de configuración. ■ Valores permitidos: "TRUE", "FALSE" ■ Obligatorio: NO ■ Defecto: "TRUE" ■ Parámetro aplicable solo si: ASKCONFIGPARAM = "TRUE"
JS_EXECUTION_AVAILABLE	Indica si se permite o no la planificación de informes. Implica sacar o no la opción de planificación en la pantalla de configuración. Permite confirmar si se deshabilita o se oculta. ■ Valores permitidos: "TRUE", "FALSE" ■ Obligatorio: NO ■ Defecto: "TRUE" ■ Parámetro aplicable solo si: ASKCONFIGPARAM = "TRUE"
ASKCONFIGPARAM	Sacar pantalla de petición de parámetros de configuración de la salida. ■ Obligatorio: NO ■ Defecto: "TRUE"

Parámetro	Descripción
OUTPUT_TYPE	Tipo de salida del informe. ■ Valores permitidos: "VISUALIZE": Visualizar informe "FILE_CLIENT": Archivo en máquina cliente "FILE_SERVER": Archivo en el servidor "PRINTER_CLIENT": Imprimir en máquina cliente "PRINTER_SERVER": Imprimir en servidor ■ Obligatorio: NO ■ Defecto: Si PROCESS_MODE = "JS_EXECUTION" o "EDITION", entonces "FILE_SERVER" El resto será "VISUALIZE" El valor por defecto dependerá del parámetro OUTPUT_TYPE_NOT_ALLOWED. Por ejemplo, si el usuario descarta VISUALIZE, nunca podrá ser el valor por defecto (sólo si ASKCONFIGPARAM = TRUE).
OUTPUT_FORMAT	Formato de la salida. ■ Valores permitidos: según OUTPUT_TYPE ver tabla 4. La lista completa es: HTML, PDF, TXT, ESCP, PSC, PCL. ■ Obligatorio: NO ■ Defecto: Si OUTPUT_TYPE= "VISUALIZE" es HTML. Si OUTPUT_TYPE= "PRINTER_SERVER" es la establecida por defecto para la impresora o una válida para la misma en su defecto (si no hay ID_SERVER_PRINTER será "PCL"). Resto de casos es PCL. Dependerá de OUTPUT_FORMAT_NOT_ALLOWED_VISUALIZE, es decir, si el usuario ha eliminado HTML, el defecto será PDF (sólo si ASKCONFIGPARAM =True).
OUTPUT_TYPE_NOT_ALLOWED	Representan los tipos de salida que el usuario no podrá seleccionar al visualizar la pantalla de configuración. ■ Valores permitidos: Ver parámetro OUTPUT_TYPE. - Si hay más de uno deben especificarse separándose entre comas. Por ejemplo: "FILE_CLIENT, VISUALIZE" ■ Obligatorio: NO ■ Defecto: "", excepto si PROCESS_MODE = EDITION o PROCESS_MODE = REEDITION que será "VISUALIZE, FILE_CLIENT, PRINTER_CLIENT" ■ Parámetro aplicable sólo si: ASKCONFIGPARAM = "TRUE"

Parámetro	Descripción
OUTPUT_FORMAT_NOT_ALLOWED_VIS	Lo mismo que el anterior pero para los formatos de salida. Por tanto, representan los formatos de salida para la salida "VISUALIZE". ■ Valores permitidos: Ver parámetro OUTPUT_FORMAT y tabla 4. Si hay más de uno deben especificarse separándose entre comas. Por ejemplo: "HTML, PDF" ■ Obligatorio: NO ■ Defecto= "" ■ Parámetro aplicable solo si: ASKCONFIGPARAM = "TRUE"
OUTPUT_FORMAT_NOT_ALLOWED_FC	Lo mismo que el anterior pero para los formatos de salida. Por tanto, representan los formatos de salida para la salida "FILE_CLIENT". ■ Valores permitidos: Ver parámetro OUTPUT_FORMAT y tabla 4. Si hay más de uno deben especificarse separándose entre comas. Por ejemplo: "TXT, PCL" ■ Obligatorio: NO ■ Defecto = "" ■ Parámetro aplicable solo si: ASKCONFIGPARAM = "TRUE"
OUTPUT_FORMAT_NOT_ALLOWED_FS	Lo mismo que el anterior pero para los formatos de salida. Por tanto, representan los formatos de salida para la salida "FILE_SERVER". ■ Valores permitidos: Ver parámetro OUTPUT_FORMAT y tabla 4. Si hay más de uno deben especificarse separándose entre comas. Por ejemplo: "TXT, PCL" ■ Obligatorio: NO ■ Defecto = "" ■ Parámetro aplicable solo si: ASKCONFIGPARAM = "TRUE"
OUTPUT_FORMAT_NOT_ALLOWED_PC	Lo mismo que el anterior pero para los formatos de salida. Por tanto, representan los formatos de salida para la salida "PRINTER_CLIENT". ■ Valores permitidos: Ver parámetro OUTPUT_FORMAT y tabla 4. Si hay más de uno deben especificarse separándose entre comas. Por ejemplo: "TXT, PCL" ■ Obligatorio: NO ■ Defecto = "" ■ Parámetro aplicable solo si: ASKCONFIGPARAM = "TRUE"

Parámetro	Descripción
OUTPUT_FORMAT_NOT_ALLOWED_PS	Lo mismo que el anterior pero para los formatos de salida. Por tanto, representan los formatos de salida para la salida "PRINTER_SERVER". ■ Valores permitidos: Ver parámetro OUTPUT_FORMAT y tabla 4. Si hay más de uno deben especificarse separándose entre comas. Por ejemplo: "TXT, PCL" ■ Obligatorio: NO ■ Defecto: "" ■ Parámetro aplicable solo si: ASKCONFIGPARAM = "TRUE"
OTHER_CONFIG_PARAMS	Resto de parámetros de configuración. Sintaxis tipo línea de comando. Ejemplo: "##debug:all:crazy## ##XFACTOR:125##" Estos parámetros tienen prioridad sobre los parámetros que se establezcan de forma individual en las pantallas de configuración. ■ Valores permitidos: Ver sintaxis tipo línea. ■ Obligatorio: NO ■ Defecto: ""
OPCIONES DE ARCHIVO	
FILE	Path relativo al directorio temporal de <i>reports</i>, junto con el nombre del archivo, de normal se usa únicamente con el nombre del archivo. Ejemplo: "temp/payslip" ■ Obligatorio: NO ■ Defecto: "" ■ Parámetro aplicable solo si: OUTP_TYPE="FILE_SERVER"
PRESERVE_AUTO	Para indicar que no se sobrescriban los archivos. ■ Valores permitidos: "TRUE", "FALSE" ■ Obligatorio: NO ■ Defecto: "FALSE" ■ Parámetro aplicable solo si: OUTPUT_TYPE="FILE_SERVER"
CREATE_DIR	Para indicar que se cree el directorio si se establece y no existe. ■ Valores permitidos: "TRUE", "FALSE" ■ Obligatorio: NO ■ Defecto: "TRUE" ■ Parámetro aplicable solo si: OUTPUT_TYPE="FILE_SERVER"

Parámetro	Descripción
DATOS DE MONEDA	
ID_CURRENCY	Identificador de la moneda que se quiere establecer al informe. Si viene el parámetro indica que hay que establecer moneda al informe. ■ Valores permitidos: Monedas dadas de alta en el usuario. ■ Obligatorio: NO ■ Defecto: se carga con los defectos del Meta4Object sesión.
EXCHANGE_DATE	Fecha de cambio de la moneda a establecer en el informe. Si viene el parámetro indica que hay que establecer moneda al informe. ■ Valores permitidos: cualquier tipo de fecha ■ Obligatorio: NO ■ Defecto: se carga con los defectos del Meta4Object sesión.
EXCHANGE_TYPE	Tipo de cambio de la moneda a establecer en el informe. Si viene el parámetro indica que hay que establecer moneda al informe. ■ Valores permitidos: Tipos de alta dados de alta en el usuario ■ Obligatorio: NO ■ Defecto: se carga con los defectos del Meta4Object sesión
CURRENCY_INF_ENABLED	Indica si es posible cambiar la moneda, Habilita/Deshabilita los datos de moneda para que se puedan o no tocar. Útil para cuando metes la moneda desde parámetro y no quieres que la modifiquen o quieres que se establezca la de por defecto y no se toque. ■ Valores permitidos: "TRUE","FALSE" ■ Obligatorio: NO ■ Defecto: "TRUE"
STABLISH_DEFAULT_CURRENCY	Indica si hay que establecer la moneda por defecto del Meta4Object sesión al informe. Si este parámetro está a true se invalidan ID_CURRENCY, EXCHANGE_DATE, EXCHANGE_TYPE. ■ Valores permitidos: "TRUE","FALSE" ■ Obligatorio: NO ■ Defecto: "FALSE"

Parámetro	Descripción
DATOS DE IMPRESORA	
ID_SERVER_PRINTER	Identificador de impresora para impresoras en server. Se toma el dispositivo asociado en la tabla. ■ Valores permitidos: Identificador de alguna de la impresoras de servidor dadas de alta en el usuario. ■ Obligatorio: NO ■ Defecto: "" ■ Parámetro aplicable solo si: OUTPUT_TYPE = "PRINTER_SERVER"
OPCIONES DE DISPOSITIVO	
Para los parámetros de esta opción consulte la Tabla 1 de este apartado	
APPLY_COLOR	Aplicar o no color. ■ Valores permitidos: "TRUE","FALSE" ■ Obligatorio: NO ■ Defecto: si OUTPUT_TYPE = PRINTER_SERVER será el que tenga la impresora cuyo identificador es ID_SERVER_PRINTER. En caso contrario será "FALSE". ■ Parámetro aplicable solo si: OUTPUT_FORMAT ■
RESOLUTION	Resolución a establecer. ■ Valores permitidos: "600","300" ■ Obligatorio: NO ■ Defecto: "600". ■ Parámetro aplicable solo si: OUTPUT_FORMAT
DUPLEX	Opción de doble cara. ■ Valores permitidos: "OFF", "EMULATED", "REAL" ■ Obligatorio: NO ■ Defecto: "OFF". ■ Parámetro aplicable solo si: OUTPUT_FORMAT

Parámetro	Descripción
DUPLEX_ESCP	Opción de doble cara. ■ Valores permitidos: "OFF", "EMULATED" ■ Obligatorio: NO ■ Defecto: "OFF". ■ Parámetro aplicable solo si: OUTPUT_FORMAT
COPIES	Número de copias. ■ Valores permitidos: 0..999 ■ Obligatorio: NO ■ Defecto: "1". ■ Parámetro aplicable solo si: OUTPUT_FORMAT
BIN	Bandeja de impresión. ■ Valores permitidos: "MANUAL", "UPPER", "LOWER" ■ Obligatorio: NO ■ Defecto: "UPPER". ■ Parámetro aplicable solo si: OUTPUT_FORMAT
TUMBLE	Opción de girar páginas. ■ Valores permitidos: "OFF", "FALSE", "TRUE" ■ Obligatorio: NO ■ Defecto: "OFF". ■ Parámetro aplicable solo si: OUTPUT_FORMAT y DUPLEX (o DUPLEX_ESC) = "REAL"
FACTOR_X	Factor X ■ Valores permitidos: 0..999 ■ Obligatorio: NO ■ Defecto: "60". ■ Parámetro aplicable solo si: OUTPUT_FORMAT
FACTOR_Y	Factor de avanzar entre líneas. ■ Valores permitidos: 0..999 ■ Obligatorio: NO ■ Defecto: "180". ■ Parámetro aplicable solo si: OUTPUT_FORMAT

Parámetro	Descripción
PARÁMETROS DEL META4OBJECT ASOCIADO AL INFORME	
M4O_SERIALICE_PARAMS	Cadena con los datos a instanciar sobre el M4Object del informe. Estos datos harán referencia a items propiedad de nodo. Su sintaxis es Nodo 1 Elemento1 Valor1; Nodo2 Elemento2 Valor2;...; NodoX ElementoX Valor X;...". donde NodoX, ElementoX y Valor X representan el valor que se quiere establecer sobre el Elemento cuyo ID es ElementoX en el nodo cuyo ID es NodoX. Si ElementoX es de tipo básico cadena (o tipos similar) entonces ValorX debe especificarse entre dobles comillas: "Valor" (si se establece desde HTML deben escaparse las dobles comillas, si se establece desde código LN4 no es necesario). Si ElementoX es de tipo básico fecha o fecha y hora debe especificarse entre llaves: {Valor}. Para tipos numéricos se escribirá sin ningún carácter de apertura o cierre. Si los tipos básicos son: <pre> 'M4 Types M4_CHAR = 1 M4_VARCHAR = 2 M4_LONG = 3 M4_DATE = 4 M4_TIMESTAMP = 5 M4_NUMBER = 6 M4_VARIANT = 7 M4_CURRENCY = 8 M4_MINI_VARIANT = 9 M4_BLOB = 10 M4_BYN_STRING = 11 M4_HOUR_TP = 12 M4_NULL = 13 </pre> Se pueden considerar cadena (M4_CHAR, M4_VARCHAR, M4_LONG, M4_VARIANT, M4_BLOB, M4_BYN_STRING), fecha (M4_DATE, M4_TIMESTAMP, M4_HOUR_TP (concatenar por delante "1970-09-25")) y numérico (M4_NUMBER, M4_CURRENCY, M4_MINI_VARIANT). ■ Valores permitidos: Tuplas (Nodo item valor) ■ Obligatorio: NO ■ Defecto: ""

Parámetro	Descripción
M4O_AUTOLOAD	Indica si establecer un modo de carga automático para el Meta4Object asociado al informe o por el contrario invocar la carga completa. Si no se establece valor el motor de informes utiliza la opción "OFF" que implica la carga explícita de todo el M4Object asociado al informe. ■ Valores permitidos: "OFF", "BLOCK", "PRG", "NODESAYS" ■ Obligatorio: NO ■ Defecto: "OFF"
GESTIÓN OUTSOURCING	
ID_M4OBJECT	Indica el M4Object a usar para la ejecución del informe. Se utiliza para los casos de informes que trabajan sobre meta4Objects sobrescritos. ■ Valores permitidos: M4Objects dados de alta en la aplicación. ■ Obligatorio: NO ■ Defecto: El asociado al ID_REPORT en la tabla SRP_EXECUTION_TPS considerando ID_EXECUTION_TYPE = 1

Reglas de validación de parámetros

Tabla 1: Parámetros aplicables en función del formato de la salida

PARAM	OUTPUT_FORMAT					
	TXT	HTML	PDF	ESCP	PCL	PSC
BIN	NO	NO	NO	NO	SI	SI
DUPLEX	NO	NO	NO	SI (sólo "EMULATED")	SI	SI
TUMBLE	NO	NO	NO	NO	SI	SI
COPIES	NO	NO	NO	NO	SI	SI
RESOLUTION	NO	NO	NO	NO	SI	SI
APPLY_COLOR	NO	NO	NO	NO	SI	NO
X_FACTOR	NO	NO	NO	SI	NO	NO
Y_FACTOR	NO	NO	NO	SI	NO	NO

Tabla 2: Tipos de ejecución aplicables en función del tipo de salida

OUTPUT_TYPE	PROCESS_MODE	
	ONLINE_EXECUTION	JS_EXECUTION (Planificación)
VISUALIZE	SI	NO
PRINTER_CLIENT	SI	NO
PRINTER_SERVER	SI	SI
FILE_CLIENT	SI	NO
FILE_SERVER	SI	SI (Con formato <> HTML)

Tabla 3: Parámetros de archivo permitidas en función del tipo de salida

OYUTPUT_TYPE	OPCIONES DE ARCHIVO			
	PATH	FILE	PRESERVE_AUTO	CREATE_DIR
VISUALIZE	NO	NO	NO	NO
PRINTER_CLIENT	NO	NO	NO	NO
PRINTER_SERVER	NO	NO	NO	NO
FILE_CLIENT	NO	NO	NO	NO
FILE_SERVER	NO	SI	SI	SI

Tabla 4: Formatos de salida válidos en función del tipo de salida

OUTPUT_TYPE	OUTPUT_FORMAT					
	TXT	HTML	PDF	ESCP	PCL	PSC
VISUALIZE	SI	SI	SI	NO	NO	NO
PRINTER_CLIENT PRINTER_SERVER	SI	NO	NO	SI	SI	SI
FILE_CLIENT	SI	NO	SI	SI	SI	SI
FILE_SERVER	SI	SI	SI	SI	SI	SI

Tabla 5: Parámetros obligatorios en función del tipo de procesamiento

PROCESS_MODE	PARÁMETROS OBLIGATORIOS
ONLINE_EXECUTION	ID_REPORT

Restricciones

Los parámetros no pueden tener en el nombre o en el valor ninguno de los siguientes caracteres: # y ~ .

Compilación e instanciación de páginas JSP

15

Acerca de este capítulo

Este capítulo tiene por objetivo detallar el funcionamiento de las herramientas de compilación de páginas *.jsp* y de instanciación de clases y comprobación de tamaño del código de las clases.

Con el objetivo de agilizar la tarea de compilación de páginas *.jsp*, Meta4 pone a disposición una herramienta que compila automática y masivamente páginas JSP; entendiendo por masiva la posibilidad de compilar, en una única petición, las páginas ubicadas en un directorio de forma recursiva o todo un servidor.

Asimismo, se pone a disposición una herramienta para instanciar clases también de manera automática y masiva, así como para comprobar el tamaño del código JAVA compilado. Esta utilidad para detectar el tamaño viene originada por la limitación del propio JAVA que no soporta un código compilado superior a 64 KB.

Requisitos

Esta funcionalidad se basa en JSPC de JRun, tanto de la versión JRun 3.1 como 4.0. Funciona y está certificado con este *contenedor Web*; si el entorno está con cualquier otro (iPlanet, etcétera), no funciona.

Proceso general de compilación e instanciación

Una de las tareas más rutinarias era compilar páginas *.jsp*, ya que hasta la fecha la compilación se realizaba individualmente página a página. Actualmente, y por medio de esta nueva herramienta, se da la posibilidad de compilar automáticamente y de manera masiva páginas *.jsp* en una única petición.

El proceso de compilación e instanciación requiere dos fases, cada una con una serie de pasos que se describen a continuación:

■ **Compilar las páginas *.jsp*:**

1. Definir el archivo *.xml* con la lista de las páginas *.jsp* que se quieren compilar
2. Indicar la lista de páginas *.jsp* que el archivo *.xml* debería incluir
3. Lanzar la ejecución de compilación utilizando el archivo *.xml*.

El resultado de la compilación se localizará en un directorio preseleccionado donde se ubicarán las clases generadas

■ **Instanciar las clases:**

A continuación se pasa directamente al proceso de la instanciación de las clases, sin ello no se podrá trabajar con estas páginas.

1. Definir el archivo *.xml* con la lista de las clases que se quieren instanciar
2. Indicar la lista de clases que el archivo *.xml* debería incluir
3. Lanzar la ejecución de la instanciación utilizando el archivo *.xml*.

El resultado de la instanciación indica si se pueden cargar las páginas *.jsp* correctamente en memoria o no. Además se muestra el tamaño del código JAVA para asegurar de que no sobrepase el límite fijado.

Ya que el límite del código puede estar muy ajustado en las páginas traspasadas a países y clientes, una de las funcionalidades adicionales es la posibilidad de marcar un límite por debajo de los 64 KB para tener controladas aquellas páginas que pueden ser susceptibles de originar problemas una vez se introduzca la parametrización adicional en cliente.



En caso de sobrepasar el límite, véase el apartado "Reducción del tamaño del código JAVA" del capítulo *Gestión del límite de código compilado JAVA* donde encontrará información detallada para depurar y reducir código en las páginas JSP.

Compilación e instanciación en JRun 3.1

El proceso de compilación e instanciación en un entorno que utiliza JRun 3.1 se compone de dos fases que tienen una serie de pasos concretos:

- Definir y compilar las páginas JSP
- Instanciar las clases de las páginas JSP

Definir y compilar las páginas JSP

Para compilar páginas *.jsp* y obtener las clases correspondientes, siga estos pasos:

1. Acceda la página Compilación JSP desde la opción de menú **Herramientas Avanzadas | Compilación páginas JSP** en el cliente Web.

También puede acceder la página Compilación JSP a través del explorador a la URL:

```
//[servidor]/m4jspsc/[idioma]/m4jspc_def.jsp
```

Dónde

[*servidor*] es el nombre del servidor adonde está ubicado Jrun

[*idioma*] es el nombre de directorio del idioma del interfaz que quiere utilizar.

EJEMPLO

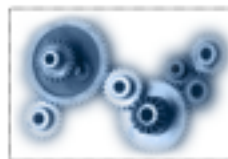
```
http://miwebserver.meta4.com:10528/m4jspc/english/m4jspc_def.jsp
```

A continuación en la página se muestra:

- El directorio raíz de Jrun, este valor viene cumplimentado por defecto y no es editable. Es el directorio adonde está localizado Jrun
- El nombre del servidor de Meta4Mind, este valor viene cumplimentado por defecto y no es editable. Generalmente, toma el valor *default*
- La aplicación Web, este valor viene cumplimentado por defecto y no es editable.
- El archivo XML que contenga la lista de archivos JSP a compilar.
- El directorio destino, donde se desea ubicar todas las páginas compiladas, es decir, las clases.

Figura 52. Compilador páginas .jsp

Parámetros de la compilación	
Obligatorios	
Directorio raíz de JRun	C:\Program Files\Allaire\JRun
Servidor	default
Aplicación web	m4tags-app
Ficheros a compilar	C:\Program Files\Meta4\M4WS\default\m4jsp\m4jsp.xml
Directorio destino	C:\Program Files\Meta4\M4WS\default\WEB-INF\jsp
Opcionales	
☐	Muestra el fichero segun lo va compilando
☐	Muestra los mensajes de depuración
☐	Muestra los mensajes de ayuda
☐	Compila sólo las JSPs que sean más actuales que su correspondiente fichero .class



- Si el archivo .xml no existe, hay que crearlo. Para ello, haga clic en el botón asociado al campo **Ficheros a compilar**, .

Figura 53. Generar archivo .xml para la lista de páginas JSP a compilar

Generador de fichero XML para compilación de páginas JSP	
Parámetros	
Directorio fuente	C:\Program Files\Meta4\M4WS\default
Fichero destino	C:\Program Files\Meta4\M4WS\default\m4jsp\m4jsp.xml



- Introduzca los parámetros de la definición del archivo XML:
 - Directorio Fuente: desde donde se desea leer todas las páginas .jsp que se utiliza para crear la lista en el archivo XML de la definición del compilado. Esta lista es recursiva, es decir, que incluye este directorio y sus hijos.
 - Archivo Destino: donde se quiere guardar el archivo XML de la definición del compilado.
- Haga clic en el botón  en ésta página para generar el archivo XML con la lista de páginas JSP especificadas.

Automáticamente, el sistema procede a generar la lista de páginas JSP a compilar en el archivo XML definido y genera un aviso le indicará si se terminó de generar el archivo XML con éxito.

5. Haga clic en el botón de retorno  para volver a la página de Compilación JSP.
6. (Opcional) Revise el contenido del archivo XML con un editor. A veces se requiere eliminar algunas páginas JSP de un directorio determinado, en este caso proceda a eliminar.



En XML, la lista de archivos JSP se define entre dos tags `<m4jspcfiles>` y `</m4jspcfiles>`.

EJEMPLO DE UNA LISTA DE PÁGINAS JSP EN UN ARCHIVO XML EN UN ENTORNO UNIX

```
...
<m4jspcfiles>
...
<file>/home/m4webservices/default/shco_g0/english/
shco_gen_portal.jsp</file>
<file>/home/m4webservices/default/shco_g0/espanol/
shco_gen_portal.jsp</file>
<file>/home/m4webservices/default/shco_g0/francais/
shco_gen_portal.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_inf.jsp</file>
<file>/home/m4webservices/default/shco_g0/browser.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_logout.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_p_wz_end.jsp</
file>
<file>/home/m4webservices/default/shco_g0/shco_gen_p_wz_exec.jsp</
file>
<file>/home/m4webservices/default/shco_g0/shco_gen_tr.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_wz_end.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_show_error.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_login.jsp</file>
...
</m4jspcfiles>
...
```

Figura 54. Compilador páginas .jsp

Parámetros de la compilación	
Obligatorios	
Directorio raíz de JRun	C:\Program Files\Allaire\JRun
Servidor	default
Aplicación web	m4tags-app
Ficheros a compilar	C:\Program Files\Meta4\M4WS\default\m4jsp\m4jsp.xml
Directorio destino	C:\Program Files\Meta4\M4WS\default\WEB-INF\jsp
Opcionales	
☐	Muestra el fichero según lo va compilando
☐	Muestra los mensajes de depuración
☐	Muestra los mensajes de ayuda
☐	Compila sólo las JSPs que sean más actuales que su correspondiente fichero .class



7. Introduzca los parámetros de compilación, tras generar el archivo xml:
 - Ficheros a compilar: la ruta y el nombre de archivo *.xml* que contendrá la lista de todas las páginas *.jsp* a compilar.
 - Directorio Destino: la ruta donde se desea ubicar todas las páginas compiladas, es decir, las clases.
8. Seleccione los parámetros opcionales de compilación activando las siguientes casillas:



La elección de estos parámetros se verá reflejada en el archivo *.log*

- muestra el archivo según se va compilando
 - muestra los mensajes de depuración
 - muestra los mensajes de ayuda
 - compila sólo las páginas que sean más actuales; esta opción sirve para optimizar la compilación, ya que sólo trata aquellas páginas que han sufrido algún tipo de modificación con respecto al último archivo compilado (concretamente a su archivo *.class*)
9. Haga clic en el botón  en la página de Compilación JSP, si está conforme con la lista definida en el archivo XML, para iniciar la compilación de las páginas JSP indicadas.

La ejecución se realiza página a página, detallándose el resultado de la compilación (número de páginas compiladas correctamente y aquellas con errores).

Figura 55. Resultado de la compilación

Páginas compiladas	Errores	Resultado
jschco.ta.sselespanofamit.java.jsp		Éxito
jschco.ta.sselespanofamit.java.man.jsp		Éxito
jschco.ta.sselespanofamit.test.jsp	.../log2.log	Fallo
jschco.ta.sselespanofamit.dar.calendar.tr.jsp		Éxito
jschco.ta.sselespanofamit.ta.ait.calendar.jsp		Éxito

Resultados de la compilación	
Páginas compiladas con éxito	4
Páginas que no pueden ser compiladas	1



El sistema muestra las páginas que va tratando a medida que se van compilando. Aquellas páginas que tienen algún tipo de fallo en su compilación se muestran en color rojo; su traza queda disponible para su depuración en un archivo `.log` a través de un enlace.

Una vez finalizada la compilación de las páginas, las clases quedan localizadas en el directorio de destino preseleccionado en los parámetros de compilación.

Instanciar las clases de las páginas JSP

Deberá instanciar las clases de estas páginas JSP para cargar en memoria las páginas y poder ejecutarlas correctamente. Antes de instanciar clases, las páginas `.jsp` se han debido compilar previamente y correctamente.

Para instanciar clases y comprobar su tamaño, siga estos pasos:

1. Acceda la página Compilación JSP desde la opción de menú **Herramientas Avanzadas | Instanciación de clases** en el cliente Web.

También puede acceder la página Compilación JSP a través del explorador a la URL:

```
//[servidor]/m4jspsi/[idioma]/m4jspsi_def.jsp
```

Dónde

[servidor] es el nombre del servidor adonde está ubicado Jrun

[idioma] es el nombre de directorio del idioma del interfaz que quiere utilizar.

EJEMPLO

http://miwebserver.meta4.com:10528/m4jspssi/english/m4jspssi_def.jsp

Figura 56. Instanciar clases

- Si el archivo .xml no existe, hay que crearlo. Para ello, haga clic en el botón asociado al campo **Clases a instanciar**, .

Figura 57. Generar archivo .xml para la lista de clases a instanciar

- Introduzca los parámetros de la definición del archivo XML tras generar el archivo xml:
 - Directorio Fuente: desde donde se desea leer todas las clases que se utiliza para crear la lista en el archivo XML de la definición de la instanciación. Esta lista es recursiva, es decir, que incluye este directorio y sus hijos.
 - Archivo Destino: donde se quiere guardar el archivo XML de la definición de la instanciación.
- Haga clic en el botón  en ésta página para generar el archivo XML con la lista de clases especificadas.

Automáticamente, el sistema procede a generar la lista de clases a instanciar en el archivo XML definido y genera un aviso le indicará si se terminó de generar el archivo XML con éxito.

-
5. Haga clic en el botón de retorno  para volver a la página de Instanciación.
 6. (Opcional) Revise el contenido del archivo XML con un editor. A veces se requiere eliminar algunas clases de un directorio determinado, en este caso proceda a eliminar.



En XML, la lista de clases se define entre dos tags `<m4jspifiles>` y `</m4jspifiles>`.

EJEMPLO DE UNA LISTA DE CLASES PARA LAS PÁGINAS JSP DEFINIDAS ANTERIORMENTE EN UN ARCHIVO XML EN UN ENTORNO UNIX

```
<m4jspifiles>
...
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__espanol__shco_gen_portal2ejsp24.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_act2ejsp19.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_calendar2ejsp1e.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_currency2ejsp1e.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_inf2ejsp19.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$FileInfo.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$HttpMultiPartParser.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$FileComp.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$Writer2Stream.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__m4jpspi__english__m4jpspi_def2ejsp20.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__m4jpspi__english__m4jpspi_xml2ejsp20.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__m4jpspi__english__m4jpspi_xml_rt2ejsp23.class</class>
...
</m4jspifiles>
```

Figura 58. Instanciar clases

7. Introduzca los parámetros de instanciación:

- Directorio clases: la ruta del directorio adonde están las clases generadas
- Clases a instanciar: la ruta y el nombre de archivo *.xml* que contendrá la lista de todas las clases a instanciar.
- Límite al máximo permitido: (Máximo 65535) con formato bytes. Uno de los principales problemas a la hora de instanciar clases es exceder el tamaño del código compilado JAVA. Se recomienda que el tamaño máximo permitido no debe sobrepasar los 64 KB, de lo contrario la clase no se carga en memoria y no puede ejecutarse la página *.jsp*.

8. Haga clic en el botón en la página de Generar archivo XML, si está conforme con la lista definida en el archivo XML, para iniciar la compilación de las páginas JSP indicadas.

La ejecución se realiza clase a clase, detallándose el resultado de la instanciación (número de clases instanciadas correctamente y aquellas con errores y su tamaño).

Figura 59. Resultado de la instanciación

Clases instanciables	Resultado	Errores	Tamaño
run shco ta sse espanol lint java2jsp024.class	Éxito		60085
run shco ta sse espanol lint java_max2jsp020.class	Fallo	...log1.log	70459
run shco ta sse espanol lint test2jsp023.class	Fallo	...log2.log	0
run shco ta sse espanol shco ta lint calendar2jsp02f.class	Éxito		24099
Resultados de la instanciación			
Clases instanciadas con éxito			2
Clases no instanciadas			2



El sistema muestra si se pueden cargar en memoria para poder ser ejecutadas posteriormente. Aquéllas clases que al instanciar tienen algún error se muestran en color rojo; su traza queda disponible en un archivo *.log* a través de un enlace. Las clases que sobrepasen el límite fijado se muestran también en color rojo.



Para depurar código siga los consejos detallados en el capítulo *Gestión del límite de código compilado JAVA*, en concreto véase el apartado "Reducción del tamaño del código JAVA".

Compilación e instanciación en JRun 4.0

El proceso de compilación e instanciación en un entorno que utiliza JRun 4.0 se compone de dos fases que tienen una serie de pasos concretos:

- Definir y compilar las páginas JSP
- Instanciar las clases de las páginas JSP

Definir y compilar las páginas JSP

Para compilar páginas *.jsp* y obtener las clases correspondientes, siga estos pasos:

1. Acceda la página Compilación JSP a través del explorador a la URL:

`//[servidor]/m4jspsc/[idioma]/m4jspc_def.jsp`

Dónde

[*servidor*] es el nombre del servidor adonde está ubicado Jrun

[*idioma*] es el nombre de directorio del idioma del interfaz que quiere utilizar.

EJEMPLO

`http://miwebserver.meta4.com:10528/m4jspc/english/m4jspc_def.jsp`

Figura 60. Compilador páginas *.jsp*



COMPILACIÓN DE PÁGINAS JSP

Parámetros obligatorios	
Archivos a compilar	C:\Program Files\meta4\M4WS\default\m4jspc\xml\m4jspc.xml
Directorio destino	C:\Program Files\meta4\M4WS\default\WEB-INF\jsp



2. Si el archivo .xml no existe, hay que crearlo. Para ello, haga clic en el botón asociado al campo **Ficheros a compilar**, .

Figura 61. Generar archivo .xml para la lista de páginas JSP a compilar


GENERADOR DE ARCHIVO XML PARA COMPILACIÓN DE PÁGINAS JSP

Parámetros	
Directorio fuente	C:\Program Files\meta4\M4WS\default\
Archivo destino	C:\Program Files\meta4\M4WS\default\m4jspc\xml\m4jspc.xml



3. Introduzca los parámetros de la definición del archivo XML:

- Directorio Fuente: desde donde se desea leer todas las páginas .jsp que se utiliza para crear la lista en el archivo XML de la definición del compilado. Esta lista es recursiva, es decir, que incluye este directorio y sus hijos.
- Archivo Destino: donde se quiere guardar el archivo XML de la definición del compilado.

Automáticamente, el sistema procede a generar la lista de páginas JSP a compilar en el archivo XML definido y genera un aviso le indicará si se terminó de generar el archivo XML con éxito.

4. Haga clic en el botón de retorno  para volver a la página de Compilación JSP.
5. (Opcional) Revise el contenido del archivo XML con un editor. A veces se requiere eliminar algunas páginas JSP de un directorio determinado, en este caso proceda a eliminar.



En XML, la lista de archivos JSP se define entre dos tags <m4jspcfiles> y </m4jspcfiles>.

EJEMPLUDE UNA LISTA DE PÁGINAS JSP EN UN ARCHIVO XML EN UN ENTORNO UNIX

```
...
<m4jspcfiles>
...
<file>/home/m4webservices/default/shco_g0/english/
shco_gen_portal.jsp</file>
```

```

<file>/home/m4webservices/default/shco_g0/espanol/
shco_gen_portal.jsp</file>
<file>/home/m4webservices/default/shco_g0/francais/
shco_gen_portal.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_inf.jsp</file>
<file>/home/m4webservices/default/shco_g0/browser.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_logout.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_p_wz_end.jsp</
file>
<file>/home/m4webservices/default/shco_g0/shco_gen_p_wz_exec.jsp</
file>
<file>/home/m4webservices/default/shco_g0/shco_gen_tr.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_wz_end.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_show_error.jsp</file>
<file>/home/m4webservices/default/shco_g0/shco_gen_login.jsp</file>
...
</m4jspcfiles>
...

```

Figura 62. Compilador páginas .jsp



COMPILACIÓN DE PÁGINAS JSP

Parámetros obligatorios	
Archivos a compilar	C:\Program Files\meta4\M4WS\default\m4jspc\xml\m4jspc.xml
Directorio destino	C:\Program Files\meta4\M4WS\default\WEB-INF\jsp



6. Introduzca los parámetros de compilación, tras generar el archivo xml:
 - Ficheros a compilar: la ruta y el nombre de archivo *.xml* que contendrá la lista de todas las páginas *.jsp* a compilar.
 - Directorio Destino: la ruta donde se desea ubicar todas las páginas compiladas, es decir, las clases.
7. Haga clic en el botón  en la página de Compilación JSP, si está conforme con la lista definida en el archivo XML, para iniciar la compilación de las páginas JSP indicadas.

La ejecución se realiza página a página, detallándose el resultado de la compilación (número de páginas compiladas correctamente y aquellas con errores).

Figura 63. Resultado de la compilación



RESULTADOS DE LA COMPILACIÓN

Páginas compiladas	Errores	Resultado
config/add_app_param.jsp		Éxito
config/add_app_server.jsp		Éxito
config/delete_app_param.jsp		Éxito
config/delete_app_server.jsp		Éxito
config/english/add_app_param.jsp		Éxito
config/english/add_app_server.jsp		Éxito
config/english/ayuda.jsp		Éxito
config/english/config.jsp		Éxito
config/english/delete_app_param.jsp		Éxito
config/english/delete_app_server.jsp		Éxito
config/english/thendient.jsp		Éxito
config/expand/add_app_param.jsp		Éxito
config/expand/add_app_server.jsp		Éxito
config/expand/ayuda.jsp		Éxito
config/expand/config.jsp		Éxito
config/expand/delete_app_param.jsp		Éxito
config/expand/delete_app_server.jsp		Éxito
config/expand/thendient.jsp		Éxito
images/comision/gen_act.jsp		Éxito
images/config/add_app_param.jsp		Éxito
images/config/add_app_server.jsp		Éxito
images/config/delete_app_param.jsp		Éxito
images/config/delete_app_server.jsp		Éxito
images/config/english/add_app_param.jsp		Éxito
images/config/english/add_app_server.jsp		Éxito
images/config/english/ayuda.jsp		Éxito
images/config/english/config.jsp	...log/20.log	Fallo
images/config/english/delete_app_param.jsp		Éxito
images/config/english/delete_app_server.jsp		Éxito

El sistema muestra las páginas que va tratando a medida que se van compilando. Aquellas páginas que tienen algún tipo de fallo en su compilación se muestran en color rojo; su traza queda disponible para su depuración en un archivo `.log` a través de un enlace.

Una vez finalizada la compilación de las páginas, las clases quedan localizadas en el directorio de destino preseleccionado en los parámetros de compilación.

Instanciar las clases de las páginas JSP

Deberá instanciar las clases de estas páginas JSP para cargar en memoria las páginas y poder ejecutarlas correctamente. Antes de instanciar clases, las páginas `.jsp` se han debido compilar previamente y correctamente.

Para instanciar clases y comprobar su tamaño, siga estos pasos:

1. Acceda la página Compilación JSP a través del explorador a la URL:

```
//[servidor]/m4jspsi/[idioma]/m4jspsi_def.jsp
```

Dónde

[*servidor*] es el nombre del servidor adonde está ubicado Jrun

[*idioma*] es el nombre de directorio del idioma del interfaz que quiere utilizar.

EJEMPLO

http://sisifo.meta4.com:10528/m4jspsi/english/m4jspsi_def.jsp

Figura 64. Instanciar clases



INSTANCIACIÓN DE CLASES

Parámetros para instanciación	
Directorio clases	C:\Program Files\meta4\M4WS\default\WEB-INF\jsp
Clases a instanciar	C:\Program Files\meta4\M4WS\default\m4jspsi\xml\m4jspsi.xml
Límite de tamaño del código (bytes). Máximo 65535.	40000



2. Si el archivo .xml no existe, hay que crearlo. Para ello, haga clic en el botón asociado al campo **Clases a instanciar**,

Figura 65. Generar archivo .xml para la lista de clases a instanciar



GENERADOR DE ARCHIVO XML PARA INSTANCIACIÓN DE CLASES

Parámetros	
Directorio fuente	C:\Program Files\meta4\M4WS\default\WEB-INF\jsp
Archivo destino	C:\Program Files\meta4\M4WS\default\m4jspsi\xml\m4jspsi.xml



3. Introduzca los parámetros de la definición del archivo XML tras generar el archivo xml:
 - Directorio Fuente: desde donde se desea leer todas las clases que se utiliza para crear la lista en el archivo XML de la definición de la instanciación. Esta lista es recursiva, es decir, que incluye este directorio y sus hijos.
 - Archivo Destino: donde se quiere guardar el archivo XML de la definición de la instanciación.

Automáticamente, el sistema procede a generar la lista de clases a instanciar en el archivo XML definido y genera un aviso le indicará si se terminó de generar el archivo XML con éxito.

4. Haga clic en el botón de retorno  para volver a la página de Instanciación.
5. (Opcional) Revise el contenido del archivo XML con un editor. A veces se requiere eliminar algunas clases de un directorio determinado, en este caso proceda a eliminar.



En XML, la lista de clases se define entre dos tags <m4jspifiles> y </m4jspifiles>.

EJEMPLO DE UNA LISTA DE CLASES PARA LAS PÁGINAS JSP DEFINIDAS ANTERIORMENTE EN UN ARCHIVO XML EN UN ENTORNO UNIX

```
...
<m4jspifiles>
...
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__espanol__shco_gen_portal2ejsp24.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_act2ejsp19.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_calendar2ejsp1e.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_currency2ejsp1e.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__shco_gen_inf2ejsp19.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$FileInfo.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$HttpMultiPartParser.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$FileComp.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__shco_g0__browser2ejsp14$Writer2Stream.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__m4jpspi__english__m4jpspi_def2ejsp20.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__m4jpspi__english__m4jpspi_xml2ejsp20.class</class>
<class>/home/m4webservices/default/WEB-INF/jsp/
jrun__m4jpspi__english__m4jpspi_xml_rt2ejsp23.class</class>
```

</m4jspifiles>

...

Figura 66. Instanciar clases



INSTANCIACIÓN DE CLASES

Parámetros para instanciación	
Directorio clases	C:\Program Files\meta4\M4WS\default\WEB-INF\jsp
Clases a instanciar	C:\Program Files\meta4\M4WS\default\m4jsp\xml\m4jsp.xml
Límite de tamaño del código (bytes). Máximo 65535.	40000



6. Introduzca los parámetros de instanciación:

- Directorio clases: la ruta del directorio adonde están las clases generadas
- Clases a instanciar: la ruta y el nombre de archivo *.xml* que contendrá la lista de todas las clases a instanciar.
- Límite al máximo permitido: (Máximo 65535) con formato bytes. Uno de los principales problemas a la hora de instanciar clases es exceder el tamaño del código compilado JAVA. Se recomienda que el tamaño máximo permitido no debe sobrepasar los 64 KB, de lo contrario la clase no se carga en memoria y no puede ejecutarse la página *.jsp*.

7. Haga clic en el botón en la página de Generar archivo XML, si está conforme con la lista definida en el archivo XML, para iniciar la compilación de las páginas JSP indicadas.

La ejecución se realiza clase a clase, detallándose el resultado de la instanciación (número de clases instanciadas correctamente y aquellas con errores y su tamaño).

Figura 67. Resultado de la instanciación

Classes for Instantiation	Results of Instantiation	Errors on Instantiation	Size of the Longest Method
run.shcn.q0.espanol.shcn.gen.art2jsp21.class	Success		8830
run.shcn.q0.espanol.shcn.gen.let.scenarios2jsp22.class	Success		20507
run.shcn.q0.espanol.shcn.gen.portaf2jsp24.class	Success		25914*
run.shcn.q0.shcn.gen.art2jsp19.class	Success		4326
run.shcn.q0.shcn.gen.calendar2jsp16.class	Success		2873
run.shcn.q0.shcn.gen.citizens2jsp18.class	Success		2823
run.shcn.q0.shcn.gen.doctiter2jsp17.class	Success		22482
run.shcn.q0.shcn.gen.htmliter2jsp20.class	Success		37891*
run.shcn.q0.shcn.gen.hr2jsp19.class	Success		6849
run.shcn.q0.browser2jsp14.class	Success		7486
run.shcn.q0.browser2jsp14\$FileInfo.class	Failure	...log/10.log	48
run.shcn.q0.browser2jsp14\$HttpMultiPartParser.class	Failure	...log/11.log	964
run.shcn.q0.browser2jsp14\$FileComp.class	Failure	...log/12.log	382
run.shcn.q0.browser2jsp14\$Writer2Stream.class	Failure	...log/13.log	31
run.milipri.english.milipri.def2jsp20.class	Success		826

El sistema muestra si se pueden cargar en memoria para poder ser ejecutadas posteriormente. Aquéllas clases que al instanciar tienen algún error se muestran en color rojo; su traza queda disponible en un archivo `.log` a través de un enlace. Las clases que sobrepasen el límite fijado se muestran también en color rojo.



Para depurar código siga los consejos detallados en el capítulo *Gestión del límite de código compilado JAVA*, en concreto véase el apartado "Reducción del tamaño del código JAVA".

PARTE II

DESARROLLO AVANZADO EN CLIENTE WEB

Los capítulos de esta parte no son imprescindibles para el desarrollo en cliente Web, pero sí aportan información adicional importante para casuísticas concretas.

Se recomienda revisar las introducciones y apartados generales de estos capítulos para tener presente en qué momento del desarrollo pueden ayudar.

Gestión del límite de código compilado JAVA

16

Acerca de este capítulo

Este capítulo tiene por objetivo definir distintos modos de reducir el tamaño del código compilado Java generado tras la compilación de las páginas JSP desarrolladas con metodología interna de Meta4.

Por metodología interna de Meta4 se entiende tanto aspectos técnicos, tales como la utilización de TAGS propios, como aspectos funcionales, tales como la existencia de *Tooltips* que incrementen la usabilidad de las páginas por parte de los usuarios.

Destacar que existe una limitación del propio JAVA que no soporta un código compilado superior a 64 KB. Así, esta limitación se reproduce en los distintos *servlet engines* (Jrun, iPlanet, etcétera) certificados por Meta4.

Por todo ello y considerando que el desarrollo liberado por Meta4 puede ser parametrizado por los clientes, toda liberación de funcionalidad estándar debe contemplar un margen que permita dicha parametrización.

A continuación se detalla:

- El proceso para conocer el tamaño del código compilado
- Las formas de reducir el tamaño del código compilado

Comprobar el tamaño del código

Para saber el tamaño del código compilado JAVA, hasta que se encuentre una manera más eficaz, se deben realizar los pasos indicados a continuación:

1. Localizar el directorio del servidor Web donde se almacenan los archivos generados por el *servlet engine*. Por ejemplo:
`\\ m4ws in mywebserver \default\WEB-INF\jsp`
2. Localizar el archivo compilado con extensión *class* referente a la página funcional en cuestión. Por ejemplo:
`jr_un__shco_g0__english__shco_gen_calendar2ejsp26.class`
3. Editar dicho archivo y borrar todo el contenido desde el inicio hasta el código del *tag* `</html>`, dejando únicamente el código compilado JAVA generado.
4. Verificar el tamaño del archivo.



Tiene la posibilidad de comprobar de manera automática el tamaño del código JAVA en el momento de instanciar una clase, véase el apartado "Instanciar las clases de las páginas JSP" del capítulo *Compilación de páginas JSP*.

Reducción del tamaño del código JAVA

Las distintas formas de reducir el tamaño del código compilado JAVA de las páginas se agrupan en cinco niveles de acción, enumerados a continuación del más efectivo al menos, tomando como criterio la capacidad de reducción y al mismo tiempo el impacto en las páginas ya desarrolladas:

- **Jsp:include:** consiste en la sustitución del código de una página por inclusiones a páginas a través de esta directiva.
- **Tags Outputdef y Move:** consiste en cambiar la forma de definir los distintos *tags* *Outputdef* en las páginas así como la sustitución del *tag* *Move*.
- **Etiquetas genéricas:** consiste en sustituir los *tags* *Label* utilizados para leer las etiquetas genéricas por variables JAVA con el contenido traducido de dichas etiquetas.
- **Etiquetas y Valores de items propios de la página:** optimización del número de *Tags* utilizados en las páginas funcionales, reutilizando el valor devuelto por un *Tag* asignándolo a una variable JAVA y reutilizándolo en los demás sitios de la página.

-
- Reducción de *Tags*: sustitución de *Tags* en las páginas por llamadas directas a la *m4operations*. Esta opción no es recomendada dado que se aleja de la metodología de desarrollo y debe ser utilizada únicamente en casos extremos en los que tras realizar los 3 niveles de cambio anteriormente descritos no se haya alcanzado el resultado esperado.

A continuación se detallan estos 5 niveles de acción.

Uso de <jsp:include>

Se trata de la sustitución de parte del código de una página por inclusiones a páginas a través de esta directiva.

La página que se incluye ha de ser una página que funcione correctamente ya que es ejecutada de manera independiente. El ahorro que se produce en el código compilado JAVA corresponde casi por completo con el tamaño de la página incluida.

Es importante tener en cuenta que la página incluida se compila y ejecuta de manera independiente, creándose sus propios archivos .java y .class. Las variables java por lo tanto no se comparten, pero se pueden pasar parámetros entre ambas páginas.

Una descripción detallada de esta directiva la puede encontrar en:

<http://java.sun.com/products/jsp/syntax/1.2/syntaxref1214.html#8828>.

EJEMPLO

A continuación se detalla la manera en la que se ha sustituido el *disclaimer* de las páginas:

Código anterior:

```
...
<table width="100%" class="disclaimer">
<tr>
<td class="disclaimer">
<script type="text/
javascript">generate_disclaimer(mnames_disc,mlinks_disc);</script>
<%if (existURI("/help/"+zLangFolder+"/output/
"+zsLocalizeHelp,pageContext) == true){%>
<a title="<%=Tran_shco_g0.getProperty("Menu.Help")%>"
href="javascript:m4help('<%=zLangFolder%>', '<%=zsLocalizeHelp%>');
">[<%=Tran_shco_g0.getProperty("Menu.Help")%>]</a>
<%}else{%>
<a title="<%=Tran_shco_g0.getProperty("Menu.Help")%>"
href="javascript:m4help('<%=zLangFolder%>');">[<%=Tran_shco_g0.get
Property("Menu.Help")%>]</a>
```

```

<%}%>
<a title="<%=Tran_shco_g0.getProperty("Menu.Top")%>"
href="#Top">[<%=Tran_shco_g0.getProperty("Menu.Top")%>]</a>
</td>
</tr>
<tr>
<td class="disclaimer" align="center">&#169; 2003<a title="Meta4"
href="http://www.meta4.com/">Meta4 Spain, S.A.</a><br /
><%=Tran_shco_g0.getProperty("Msg.Disclaimer")%>
</td>
</tr>
</table>
...

```

Nuevo código:

```

...
<% request.setAttribute("zsLocalizeHelp",zsLocalizeHelp); %>
<jsp:include page="/shco_g0/shco_gen_include_disclaimer.jsp"
flush="true" />
...

```

donde shco_g0/shco_gen_include_disclaimer.jsp es básicamente lo mismo que había antes, pero con nuevas instrucciones y definiciones de variables, para que la página funcione correctamente de manera independiente.

```

<% response.setDateHeader("Expires", -1); %>

<%@ taglib uri="M4Tags" prefix="m4" %>
<%@ page import="java.io.*" %>
<%@ page import="java.util.*" %>
<%@ page import="com.meta4.session.*" %>
<%@ page import="com.meta4.configuration.*" %>

<%@ include file="../shco_g0/shco_gen_functions.jsp" %>

<%
    M4SessionManager zsessionmanager =
M4Context.getSession(request);
    int iLang = zsessionmanager.getLanguageID(); // 2, 3, 4.... 8
    String zLangFolder= CheckConfig.checkFolderLanguage(new
Long(iLang).intValue(), CheckConfig.THCL);

```

```

        String zsLocalizeHelp =
        (String)request.getAttribute("zsLocalizeHelp");
    %>

    <jsp:useBean id="Tran_shco_g0" scope="session"
    class="java.util.Properties" />

    <br/>
    <table width="100%" class="disclaimer">
    <tr>
    <td class="disclaimer">
    <script type="text/javascript">
    generate_disclaimer(mnames_disc,mlinks_disc);</script>
    <%if (existURI("/help/"+zLangFolder+"/output/"
    "+zsLocalizeHelp,pageContext) == true){%>
    <a title="<%=Tran_shco_g0.getProperty("Menu.Help")%>"
    href="javascript:m4help('<%=zLangFolder%>', '<%=zsLocalizeHelp%>');
    ">[<%=Tran_shco_g0.getProperty("Menu.Help")%>]</a>
    <%}else{%>
    <a title="<%=Tran_shco_g0.getProperty("Menu.Help")%>"
    href="javascript:m4help('<%=zLangFolder%>');">[<%=Tran_shco_g0.get
    Property("Menu.Help")%>]</a>
    <%}%>
    <a title="<%=Tran_shco_g0.getProperty("Menu.Top")%>"
    href="#Top">[<%=Tran_shco_g0.getProperty("Menu.Top")%>]</a>
    </td>
    </tr>
    <tr>
    <td class="disclaimer" align="center">&#169; 2003<a title="Meta4"
    href="http://www.meta4.com/">Meta4 Spain, S.A.</a><br /
    ><%=Tran_shco_g0.getProperty("Msg.Disclaimer")%>
    </td>
    </tr>
    </table>

```

EJEMPLO

Unas páginas donde puede interesar el uso del `jsp:include` para reducir el tamaño son las páginas de información. Gracias a que se organizan en diferentes bloques de información, podemos sacar alguno, varios o todos los bloques a otras páginas.

Código anterior:

```
...
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block1.jsp" %>
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block2.jsp" %>
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block3.jsp" %>
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block4.jsp" %>
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block5.jsp" %>
...
```

Nuevo código:

Por ejemplo se pueden agrupar los 3 primeros includes en un sólo `jsp:include`:

```
...
<%request.setAttribute("zm4object",zm4object);
request.setAttribute("znodo1",znodo1);
request.setAttribute("zcount1",zcount1);
request.setAttribute("znodo2",znodo2);
request.setAttribute("zcount2",zcount2);
request.setAttribute("znodo3",znodo3);
request.setAttribute("zcount3",zcount3);%>

<!-- jsp:include de los 3 primeros bloques --!>
<jsp:include page="../../../shco_or_ext_organization_f_blocks1_2_3.jsp"
flush="false" />

<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block4.jsp" %>
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block5.jsp" %>
...
```

Donde la nueva página `shco_or_ext_organization_f_blocks1_2_3.jsp` sería:

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<%@ page import="java.util.*" %>
<%@ page import="com.meta4.session.*" %>
```

```

<%@ page import="com.meta4.m4operations.*" %>
<%
String zm4object = (String)request.getAttribute("zm4object");
int zcount1 =
Integer.parseInt((String)request.getAttribute("zcount1"));
int zcount2 =
Integer.parseInt((String)request.getAttribute("zcount2"));
int zcount3 =
Integer.parseInt((String)request.getAttribute("zcount3"));
String znodo1 = (String)request.getAttribute("znodo1");
String znodo2 = (String)request.getAttribute("znodo2");
String znodo3 = (String)request.getAttribute("znodo3");

String zraiz1 = znodo1 + ":" + zm4object + "!" + znodo1 + ".";
String zraiz2 = znodo2 + ":" + zm4object + "!" + znodo2 + ".";
String zraiz3 = znodo3 + ":" + zm4object + "!" + znodo3 + ".";
String zcomun2 = znodo2 + ":" + zm4object + "!" + znodo2 +
"&VAR.m4lix]" + ".";

//Definición de los elementos necesarios, por ejemplo:
String zlSCODTSTART1 = zraiz1 + "SCO_DT_START";
String zlSCODTSTART2 = zraiz2 + "DT_START";
String zlSCODTSTART3 = zraiz3 + "STD_DT_START";
%>

<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block1.jsp" %>
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block2.jsp" %>
<%@ include file="../../../shco_g1_sse/
shco_or_ext_organization_f_block3.jsp" %>

```

Tags Outputdef y Move

Se trata de un cambio en la forma de definir los distintos *Tags* outputdef en las páginas así como la sustitución del *Tag* Move, cuya utilización se encuentra muy bien localizada en las páginas.

El ahorro que se produce en el código compilado JAVA con dicho cambio es de:

- 477 Bytes por cada Tag m4:outputdef
- 411 Bytes por cada Tag m4:move

Este cambio deberá ser adoptado como obligatorio en nuevos desarrollos y será progresivamente implementado en las funcionalidades ya desarrolladas.

Debido al ahorro que dicho cambio produce se trata de un cambio especialmente interesante para páginas con muchos nodos de información, como pueden ser las páginas de consulta de información o bien asistentes con muchos pasos y opciones.

EJEMPLO

Tag m4:outputdef

Cabe destacar que el cambio del Tag Outputdef indicado a continuación sólo se puede hacer para los casos en los que no se definen los items a traer, que es lo habitual en las páginas.

Código Anterior:

```
<m4:outputdef m4alias="<%=znodo%>" >
    <m4:param name="m4name0" value="<%=zoutputdef%>"/>
</m4:outputdef>
donde String zoutputdef = zsubsesion + "!" + znodo + "[" + "]" ;
```

Nuevo Código:

```
<m4:outputdef m4alias="<%=znodo%>"
    m4object="<%=zsubsesion%>" node="<%=znodo%>" records="*" />
```

Tag m4:move

Este cambio supone la sustitución del Tag por código JAVA aunque no siga las recomendaciones de la metodología pero supone un riesgo menor de legibilidad de la página.

Código Anterior:

```
<m4:move>
    <m4:param name="<%=zsubsesion%>" value="<%=zmove%>"/>
</m4:move>
<m4:move>
    <m4:param name="<%=zsubsesion%>" value="<%=zmove2%>"/>
</m4:move>
```

```
donde String zmove = znodo + ":" + znodo + "[FIRST]";
String zmove2 = znodo2 + ":" + znodo2 + "[FIRST]";
```

Código nuevo:

```
<%try {
    M4Operations m4f = new M4Operations(request);
    m4f.moveData (znodo,zsubsesion,znodo,"0");
    m4f.moveData (znodo2,zsubsesion,znodo2,"0");}
catch(Exception e) {}
%>
```

Etiquetas genéricas

Se trata de la sustitución de los Tags Label utilizados para leer las etiquetas genéricas por variables JAVA con el contenido traducido de dichas etiquetas.

Su definición se incluirá en la página shco_gen_labels, por lo que estará disponible en las plantillas de Asistente, Mantenimiento y Consultas.

El ahorro que se produce en el código compilado JAVA con dicho cambio es de:

- 102 Bytes por cada Tag m4:label

Debido al ahorro que dicho cambio produce se trata de un cambio especialmente interesante para páginas de entrada de datos con muchos campos, como por ejemplo el alta de empleados.

Es importante resaltar que con el cambio propuesto deben ser mantenidos todos los *tooltips* de todos los campos, botones, etcétera, en las páginas, siguiendo la metodología actualmente establecida de desarrollo.

EJEMPLO

Cabe destacar que este cambio es compatible hacia atrás, es decir, se va a mantener ambas alternativas y evitar así la necesidad de ejecutar el proceso automático de actualización en todas las paginas ya desarrolladas.

Nuevas variables JAVA disponibles tras incluir el archivo **shco_gen_labels.jsp**:

```
String zSHCOLBCLEAN_val
String zSHCOLBDEL_val
String zSHCOLBEDIT_val
String zSHCOLBINSERT_val
String zSHCOLBLIST_val
String zSHCOLBNEXT_val
String zSHCOLBORD_val
```

```
String zSHCOLBPREV_val  
String zSHCOLBREFRESH_val  
String zSHCOLBSEND_val  
String zSHCOLBWRITE_val  
String zSHCOLBNOHELP_val  
String zSHCOLBHELP_val  
String zSHCOLBCAB_val  
String zSHCOLBTITERROR_val  
String zSHCOLBBACK_val
```

Código anterior:

```
<input tabindex="<%= (zTab + 1)%>" class="form" type="text"  
      id="<%=zId2%>" name="<%=zId2%>" size="3" maxlength="3"  
      title="<m4:label m4name="<%=zSHCOLBWRITE%>"  
htmlsafe="true"/>  
      <m4:label m4name="<%=zId2r%>" htmlsafe="true"/>"  
      value=""/>&nbsp;
```

Código nuevo:

```
<input tabindex="<%= (zTab + 1)%>" class="form" type="text"  
      id="<%=zId2%>" name="<%=zId2%>" size="3" maxlength="3"  
      title="<%=zSHCOLBWRITE_val%>"  
      <m4:label m4name="<%=zId2r%>" htmlsafe="true"/>"  
      value=""/>&nbsp;
```

Etiquetas y valores de items propios de la página

Se trata de optimizar el número de *Tags* utilizados en las páginas funcionales. Se reutiliza el valor devuelto por un *Tag* asignándolo a una variable JAVA y reutilizándolo en el resto de la página.

Dicha optimización, debe realizarse a criterio del desarrollador teniendo en cuenta la estructura de su página y las posibilidades de reutilizar valores recuperados a través de *Tags* en su propia página.

El ahorro que se produce en el código compilado JAVA con dicho cambio es:

- 102 Bytes por cada *Tag* m4:label o m4:item

Es importante resaltar que con el cambio propuesto deben ser mantenidos todos los *tooltips* de todos los campos, botones, etcétera en las páginas, siguiendo la metodología actualmente establecida de desarrollo.

EJEMPLO

A continuación, se detalla la manera de volcar a una variable JAVA el valor de un campo. La variable JAVA se define directamente por el Tag `m4:item` y en el caso de que ya haya sido definida se produce un error.

```
<m4:item m4name="<%=zIdHRcm%>" htmsafe="true"
m4varname="mivariable"/>
```

A continuación se detalla la manera de volver a una variable JAVA el valor traducido de una etiqueta. En el caso de un etiqueta, la variable JAVA debe haber sido definida previamente.

```
<% String mivariable = null;%>
<m4:label m4name="<%=zSHCOLBDEL%>" htmsafe="true"
var="mivariable"/>
```

El ejemplo presentado a continuación refleja una casuística normal del desarrollo en el cual dentro de un bucle se construye el código javascript del botón editar, el código del botón borrar y se utiliza el *Tag* para mostrar el valor en la página.

Código anterior:

```
<m4:loop from="<%=zregistroinicials%>" to="<%=zregistrofinals%>">
<%@include file="../../../shco_g0/shco_gen_loop.jsp"%>
<tr>
<td class="boton<%=zpos%>">
<a title="<m4:label m4name="<%=zSHCOLBEDIT%>" htmsafe="true"/
>"href=""
onclick="
m4valor('NombreFormulario','X<%=zIni%>',
'<m4:item m4name="<%=zInicm%>" jsafe="true"
htmsafe="true"/>',
'set');
m4valor('NombreFormulario','X<%=zIdHR%>',
'<m4:item m4name="<%=zIdHRcm%>" jsafe="true"
htmsafe="true"/>',
'set');
m4valor('NombreFormulario','X<%=zOrRol%>',
'<m4:item m4name="<%=zOrRolcm%>" jsafe="true"
htmsafe="true"/>',
'set');
return false;">
<img alt="<m4:label m4name="<%=zSHCOLBEDIT%>"
htmsafe="true"/>"
<%@ include file="../../../files_gif/ic_mod.jsp" %> />
</a>
</td>
<td class="valor<%=zpos%>" >&nbsp;
```

```
        <m4:item m4name="<%=zIdHRcm%>" htmlsafe="true"/>
    </td>
    <td class="valor<%=zpos%>">
        <m4:item m4name="<%=zOrRolcm%>" htmlsafe="true"/>
    </td>
    <td class="valor<%=zpos%>">&nbsp;
        <m4:item m4name="<%=zInicm%>" htmlsafe="true"/>
    </td>
    <td class="boton<%=zpos%>" colspan="2">
    <a title="<m4:label m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/>"
        href="javascript:m4wzeliminar('X<%=zIni%>',
            '<m4:item m4name="<%=zInicm%>" jsafe="true"
htmlsafe="true"/>',
            'X<%=zOrRol%>',
            '<m4:item m4name="<%=zOrRolcm%>" jsafe="true"
htmlsafe="true"/>',
            'X<%=zIdHR%>',
            '<m4:item m4name="<%=zIdHRcm%>" jsafe="true"
htmlsafe="true"/>');">
        <img alt="<m4:label m4name="<%=zSHCOLBDEL%>"
htmlsafe="true"/>"
        <%=@ include file="../../files_gif/ic_bor.jsp" %> />
    </a>
    </td>
</tr>
</m4:loop>
```

Reducción de *Tags*

Se trata de la sustitución de *Tags* en las páginas por llamadas directas a la `m4operations`.

Esta opción no se recomienda dado que se aleja de la metodología de desarrollo y debe ser utilizada únicamente en casos extremos en los que tras realizar los tres niveles de cambio anteriormente descritos no se haya alcanzado el resultado esperado.

Dicha optimización se realiza a criterio del desarrollador teniendo en cuenta la estructura de su página y las posibilidades de reutilizar valores recuperados a través de *Tags* en su propia página.

El ahorro que se produce en el código compilado JAVA dependerá de los *Tags*

que sean sustituidos. A continuación, se pueden encontrar los valores orientativos del ahorro que se produce por *Tag*:

- 102 Bytes por cada Tag *m4:label* o *m4:item*
- 477 Bytes por cada Tag *m4:outputdef*
- 411 Bytes por cada Tag *m4:move*

El ejemplo presentado a continuación refleja una casuística en la que se ha tenido que prescindir de la utilización de *Tags* por utilización directa de la *m4operations*.

EJEMPLO

Código anterior

```
<%if(zcount6>0){%>
<table class="datos" width="100%" cellpadding="0" border="1">
<thead>
<tr class="titulo"><th colspan="2">&nbsp;<m4:label
m4name="<%=znamenodo6%>"/></label></th></tr>
</thead>
<tbody>
<m4:loop from="0" to="<%=new Integer(new
Integer(zcountv6).intValue()-1).toString()%">
<tr class="valor"><td>&nbsp;<m4:item
m4name="<%=zSTDNLOCATIONTYPE6%>"/></td><td>&nbsp;<m4:item
m4name="<%=zSTDEMAIL6%>"/></td></tr>
</m4:loop>
</tbody></table>
<br />
<%}%>
```

Código nuevo

```
<%if(zcount6>0){String zlcoNode6 = "";
    String zlcoNode6_1 = "";
    String zlcoNode6_2 = "";
    try {
        M4Operations mco6 = new M4Operations(request);
        zlcoNode6 = mco6.getLabel(znodo6,zsubsesion,znodo6,"");
        zlcoNode6_1 =
mco6.getLabel(znodo6,zsubsesion,znodo6,"STD_EMAIL");
        zlcoNode6_2 =
mco6.getLabel(znodo6,zsubsesion,znodo6,"STD_N_LOCATION_TYPE");
    }
    catch(Exception e){}
```

```

        zlcoNode6 =
com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zlcoNode6)
;

        zlcoNode6_1 =
com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zlcoNode6_
1);

        zlcoNode6_2 =
com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zlcoNode6_
2);%>
<table class="datos" width="100%" cellpadding="0" border="1">
<thead><tr class="titulo"><th colspan="2"><a name="block6"></
a>&nbsp;<%=zlcoNode6%></th></tr><tr class="titulo2"><th
>&nbsp;<%=zlcoNode6_1%></th><th >&nbsp;<%=zlcoNode6_2%></th></
tr></thead>
<tbody>
<%try {
        M4Operations m6cof = new M4Operations(request);
        int ico6 = 0;
        for (ico6 =0; ico6 < zcount6; ico6++){
            String idco6 = String.valueOf(ico6);
            m6cof.moveData (znodo6,zm4object,znodo6,idco6);
            zlcoNode6_1 =
m6cof.getItem(znodo6,zsubsesion,znodo6,"","STD_EMAIL");
            zlcoNode6_2 =
m6cof.getItem(znodo6,zsubsesion,znodo6,"","STD_N_LOCATION_TYPE");

            zlcoNode6_1 =
com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zlcoNode6_
1);

            zlcoNode6_2 =
com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zlcoNode6_
2);
            %>
<tr class="valor"><td>&nbsp;<%=zlcoNode6_1%></
td><td>&nbsp;<%=zlcoNode6_2%></td></tr>
            <%}%>

        <%}
        catch(Exception e){}
        %>
</tbody></table><br />
<%}%>

```


Depuración de páginas JSP

17

Acerca de este capítulo

Tras desarrollar una nueva funcionalidad en cliente Web y comprobar que no funciona correctamente, es necesario averiguar cuál es el origen del problema. Este capítulo describe qué pasos se pueden seguir para encontrar la causa del funcionamiento incorrecto.

El proceso general que se debe seguir es el siguiente:

- Ejecutar la página y comprobar que es correcta.
- Revisar los diferentes archivos de trazas.
- En el cliente windows probar el Meta4Object que usará la página jsp para entrada, proceso, salida de información. Asegurarse de que la ejecución de toda la secuencia que realizará la página (instanciación, ejecución de métodos, etcétera) es la correcta.

El capítulo se divide en tres apartados:

- Depuración de las páginas
- Configuración del sistema de trazas de los servicios Web
- Errores más frecuentes



Existen herramientas externas a Meta4 que permiten también depurar páginas JSP paso a paso y con puntos de ruptura. La configuración y manejo de estas herramientas depende de su proveedor y no está dentro del ámbito de Meta4 por lo que no se trata en este Manual.

Depuración de una página JSP

Después de desarrollar una página JSP, puede que esta funcione de modo incorrecto o inesperado. En tal caso, se tendrán que depurar las páginas JSP.

En general las principales fuentes de ayuda van a ser los archivos de trazas que genera el motor de servlets, los propios servicios Web e incluso el propio servidor de aplicaciones.

A continuación se describen los pasos a dar para realizar dicha depuración:

- Comprobar los errores de sintaxis
- Revisar los archivos de traza del servidor Web
- Añadir más información de depuración

Comprobar errores de sintaxis

Para comprobar que la página no tiene errores de sintaxis, de compilación, se invoca directamente sin la llamada al *servlet* *CheckSecurity*.

EJEMPLO

Si se está invocando

```
//servidor/servlet/CheckSecurity/JSP/shco_tl/shco_tl_change_password.jsp
```

para comprobar la sintaxis bastaría hacer

```
//servidor/shco_tl/espanol/shco_tl_change_password.jsp
```

Si la página no compila, debe examinar las trazas en su motor de servlets para determinar en qué líneas de la página están los errores de sintaxis.

En el caso de utilizar JRun, el proceso se hace más fácil, porque en el navegador puede aparecer una pila que contiene el motivo por el cual la página no ha podido ser compilada. Si tiene Internet Explorer 5 o superior debe deshabilitar los mensajes descriptivos para ver la pila de errores directamente. Para ello, en el navegador en **Tools | Internet Options | Advanced**, desactive *Show friendly HTTP error messages* (Mostrar mensajes descriptivos de los errores http).

EJEMPLO

La página index.jsp tiene una directiva escrita de manera incorrecta.

500 Internal Server Error

/index.jsp:

```
javax.servlet.ServletException: Parse Error in JSP page: The page directive on line  
17 has an invalid attribute
```

Revisar archivos de traza del servidor Web

Una vez que la página compila, si no funciona como se espera, se pasa a examinar los distintos archivos de traza del servidor Web que contienen información complementaria:

- Archivos de traza del motor de *servlets*
- Archivo de trazas de los servicios Web
- Archivo de trazas XML

Archivo de trazas XML (*connect.log*)

En este archivo se muestra detalle de las transacciones entre el servidor Web y el servidor de aplicaciones. Estas trazas pueden ser especialmente útiles cuando se observen comportamientos distintos del esperado en el acceso a datos a través de los tags de cliente Web.



Este archivo ocupa mucho espacio en disco. Por ello, es recomendable borrarlo de vez en cuando.

Para activar el fichero *connect.log* es necesario modificar la sección de Variables Globales del *m4logsystem.cfg*, asignando a **M4_TRACE_LEVEL** el valor *debug*. Más información, en el apartado Configuración del sistema de trazas de este capítulo.

Puede ser que debido a un problema de caché de datos se ejecute un estado del *Meta4Object* anterior al actual. Es decir, que no refleje los cambios que se hechos en el *Meta4Object*. Dependiendo del tipo de cambio que se haya realizado el proceso varía:

- Si los cambios afectan exclusivamente al Meta4Object es suficiente con desconectarse y volver a conectarse.
- Si los cambios afectan a tablas, opciones de menú, tareas, etcétera entonces para eliminar las cachés del servidor, hay que ir al directorio de instalación del servidor de aplicaciones y borrar todos los archivos *.mrc del directorio app-configuration\temp\m4cache\server\MCR.

La ubicación de estos archivos de trazas varía según el entorno:

- Windows: <m4log-dir>\connect.log
- Solaris: <m4log-dir>/connect.log

Donde:

- <m4log-dir> es el parámetro M4_LOG_DIR del archivo m4logsystem.cfg.

Más información, en el apartado Configuración del sistema de trazas de este capítulo.

Para visualizar el contenido de este archivo, existe el archivo m4xml.xml que permite visualizar el contenido de la connect.log en formato árbol de xml, lo cual facilita su interpretación.

Este archivo, cuya localización tras la instalación es WEB-INF\classes\properties, hay que copiarlo en el mismo directorio donde se encuentra el archivo de log: <m4log-dir>/. Se hace doble clic sobre el mismo para visualizar en el explorador el contenido del archivo connect.log.

El contenido de este archivo se interpreta a continuación:

- Este archivo contiene una zona inicial referente a la conexión del usuario (se carga el Meta4Object de tareas, etcétera), aunque no es decisiva para el desarrollo. Y a continuación vienen las ejecuciones propias de cada página.
- Una vez que hayamos ejecutado la página que deseamos depurar, hay que abrir el archivo y buscar, desde el final del archivo hacia arriba, el meta4object (m4oid="mi_meta4object") que estamos ejecutando desde la página.
- Podrá encontrar dos tipos de bloques: los referentes a transacciones de envío de datos desde el servidor Web(action="sending") y los referentes a transacciones de datos devueltos por el appserver (action="receiving").
- A continuación, se muestra la connect.log correspondiente a una página que ejecuta la carga de un filtro de Meta4Object y solicita registros de un nodo.

EJEMPLO

Primero aparece una transacción de lo que envía la página al servidor de aplicaciones:

```
<transaction number="16" action="Sending"
ipaddress="172.17.14.146">
  <m4xml version="1.0">
```

Definición del contexto: el ID de la subsesión, y la página utilizada.

```
    <context subsessionid="SHCO_PA_MT_PERSON"
subsessionmode="current"
    info="/shco_g2_sse/espanol/shco_pa_list_person.jsp"
    debuginfo="/shco_g2_sse/espanol/shco_pa_list_person.jsp">
```

Características del Meta4Object.

```
    <m4oalias alias="SHCO_PA_MT_PERSON"
m4oid="SHCO_PA_MT_PERSON" preserve="yes"
    find="no" mode="both" />
    <commandset id="result">
```

Método que se va a ejecutar, y su argumento, haciendo esta llamada desde el cliente windows, pasando este mismo argumento tendremos el mismo comportamiento que el que queremos en esta ejecución.

```
    <method
item="SHCO_PA_MT_PERSON!SHCO_GN_ROOT[].SHCO_LOAD_FILTER">
    <arg name="LOAD_TYPE_ARG"
    value="ASCO_GB_NAME*A1*Piara{BSTD_ID_PERSON*A1*{" />
  </method>
```

Filtro de ordenación que está pidiendo la página

```
    <filter type="sort"
item="SHCO_PA_MT_PERSON!SHCO_PA_MT_PERSON.STD_ID_PERSON"mode="add">
    <filterarg name="STD_ID_PERSON" mode="asc" />
  </filter>
```

Conjunto de datos pedidos.

```
    <datadef id="SHCO_PA_MT_PERSON">
    <nodedef id="SHCO_PA_MT_PERSON!SHCO_PA_MT_PERSON">
    <registerdef index="0-39" />
    </nodedef>
  </datadef>
  </commandset>
  </context>
</m4xml>
</transaction>
```

Después aparece la misma transacción con la parte que recibe la página del servidor de aplicaciones:

```
<transaction number="16" action="Receiving"
ipaddress="172.17.14.146">
<m4xml version="1.0">
```

Definición del contexto: el ID de la subsesión, y la página utilizada.

```
<context subsessionid="SHCO_PA_MT_PERSON"
subsessionmode="current"
info="/shco_g2_sse/espanol/shco_pa_list_person.jsp"
debuginfo="/shco_g2_sse/espanol/
shco_pa_list_person.jsp">
```

Conjunto de datos devueltos

```
<dataset id="result">
```

Resultado de la ejecución del método

```
<method id="" result="0" value="0.000000000" cpptype="1">
<data id="SHCO_PA_MT_PERSON">
```

Datos y metadatos para este nodo

```
<node id="SHCO_PA_MT_PERSON!SHCO_PA_MT_PERSON"
name="SHCO_PA_MT_PERSON"
count="1" current="0">
```

Primero los metadatos

```
<iteminfo id="SCO_GB_NAME" name="Nombre" scope="3"
type="3" m4type="2" />
<iteminfo id="SHCO_CONTROL" name="Control del borrado"
scope="1" type="2"
m4type="2" />
<iteminfo id="SHCO_SENTENCE" name="Aplica una sentencia"
scope="2" type="2"
m4type="3" />
<iteminfo id="STD_ID_PERSON" name="Id. Persona"
scope="3" type="3"
m4type="2" />
<iteminfo id="STD_N_FIRST_NAME" name="Nombre"
scope="3" type="3"
m4type="2"/>
<iteminfo id="STD_N_FAMILY_NAME_1" name="Apellidos"
scope="3" type="3"
m4type="2" />
```

Luego los datos, los primeros elementos son de bloque....

```
<item id="SHCO_CONTROL" value="" />
```

```

        <item id="SHCO_SENTENCE" value="FROM &STD_PERSON A
Where( A.SCO_GB_NAME
        Like 'Piara%')" />
... y los siguientes de registro.
        <register index="0">
        <item id="SCO_GB_NAME" value="Piara Hernández" />
        <item id="STD_ID_PERSON" value="M10007" />
        <item id="STD_N_FIRST_NAME" value="María" />
        <item id="STD_N_FAMILY_NAME_1" value="Piara Hernández" >
        </register>
        </node>
    </data>
Trazas del servidor
<log>
Esto es un mensaje de depuración.
<logmess code="1023475719" severity="_debugInfo_" value="El nodo
        SHCO_PA_MT_PERSON se ha cargado." />
</log>
</dataset>
</context>
</m4xml>
</transaction>
<timer number="16"total="290"
start="1038574853422"stop="1038574853712"
ipaddress="172.17.14.146" />

```

Archivo de trazas de los servicios Web (webservices.log)

En estos archivos se muestran:

- Excepciones y errores en el uso del código Java del cliente Web.
- Trazas del código java que se está ejecutando.
- Trazas que provienen del servidor de aplicaciones.

La ubicación de estos archivos de trazas varía según el entorno:

- Windows: <m4log-dir>\webservices.log
- Solaris: <m4log-dir>/webservices.log

Donde:

- `<m4log-dir>` es el parámetro `M4_LOG_DIR` del archivo `m4logsystem.cfg`.

Más información, en el apartado *Configuración del sistema de trazas*.

EJEMPLO

En el uso del tag `m4:saveconfig` se ha encontrado una excepción al escribir en disco. Esta información se obtuvo de `webservices.log`:

Jun 27 15:31:22 [- jcp-6] FATAL: [EXCEPTION] to save new file:

```
java.io.FileNotFoundException: c:\program
files\meta4\m4webservices\servlets\properties\configclient.xml (Access is denied)
at java.io.FileOutputStream.open(Native Method)
at java.io.FileOutputStream.<init>(FileOutputStream.java:102)
at java.io.FileOutputStream.<init>(FileOutputStream.java:62)
at com.meta4.configuration.M4ConfigClient.createConfigClient(Unknown Source)
at com.meta4.taglib.M4TagSaveConfig.doEndTag(M4TagSaveConfig.java:41)
at
jrun__config__espanol__config2ejsp1a._jspService(jrun__config__espanol__confi
g2ejsp1a.java:56)
[...]
```

Archivos de traza del motor de servlets

En estos archivos se vuelcan excepciones y errores del API de *servlets* y JSP.

La ubicación de estos archivos de trazas varía según el motor de *servlets*:

- JRun:
 - Windows: `<jrun-root>\logs\<jrun-server>-<level>.log`
 - Solaris: `<jrun-root>/<jrun-server>/default-<level>.log`

Donde:

- `<jrun-root>` es el directorio de instalación de JRun, por defecto `c:\program files\allaire\jrun` en Windows.
- `<jrun-server>` es el identificador de la instancia de JRun, por defecto, "default".
- `<level>`:
 - *out* para las trazas volcadas al estándar *output*
 - *err* para las trazas volcadas al estándar error y
 - *event* para registrar excepciones.

- IWS

- Solaris: <iws-root>/https-<iws-server>/config/<level>.

Donde:

- <iws-root>: es el directorio de instalación de IWS.
- <iws-server>: es el identificador de la instancia de IWS, por defecto el nombre del *host*.
- <level>:
 - *access* registra los accesos al servidor Web y
 - *errors* contiene todas las trazas de compilación y ejecución del motor de *servlets*.

Revisar archivos de trazas del servidor de aplicaciones

En el cliente ligero además existe la posibilidad de parametrizar el grado de depuración, de tal forma que se puedan visualizar las trazas del logfile.html al ejecutar una página. Esto se hace a través del editor de parámetros de aplicación, modificando el parámetro DEBUG de la sección PORTAL_PARAM de la aplicación THINCLIENT.



Para más información sobre los archivos de trazas, consulte el apartado "Sistemas de trazas" del capítulo *Solución de problemas* del documento *Administración_Servidor_Aplicaciones.pdf*.

Existen 4 valores del parámetro correspondientes a 4 niveles de depuración:

- *Valor 0 - Nivel Usuario*

Muestra todos los mensajes funcionales, es decir, pertenecientes a módulos distintos del 0, salvo los incluidos en los submódulos 1 que se reservan para control de flujo de ejecución. Se hace una excepción con los incluidos en el módulo de multimonedas (120) en los que no se filtrará por submódulo.

Recoge también los mensajes de error generados por la BDL.

- *Valor 1 - Nivel Usuario Avanzado*

Se suprimen los mensajes informativos y alertas funcionales para agilizar el manejo de la aplicación. Aparecen los errores funcionales, es decir, pertenecientes a módulos distintos del 0, salvo los incluidos en el submódulos 1 y los provenientes de la BDL. Si se trata de mensajes de multimonedas no se excluye ningún submódulo.

- *Valor 2 - Nivel Desarrollador*

Aparecen todos los mensajes tanto funcionales como tecnológicos.

■ Valor 3 - Nivel Desarrollador Avanzado

Se reduce el log eliminando alertas y avisos tecnológicos. Aparecen todos los mensajes funcionales. De los tecnológicos únicamente pasarán el filtro los errores.

Revisar el Meta4Object en el cliente Windows

Para ello, partiendo del connect.log, se obtiene el ID del Meta4Object y el método que ejecuta la página.

A partir de estos datos se puede levantar el Meta4Object, ejecutar el método y utilizar las diferentes herramientas de depuración para comprobar que la ejecución hace lo que realmente se desea.

EJEMPLO

Dado el connect.log del ejemplo anterior, en la parte *sending* donde se hace la petición al servidor:

```
<m4oalias alias="SHCO_PA_MT_PERSON"m4oid="SHCO_PA_MT_PERSON"
preserve="yes"find="no" mode="both" />
<commandset id="result">
<method item="SHCO_PA_MT_PERSON!SHCO_GN_ROOT[ ].SHCO_LOAD_FILTER">
    arg name="LOAD_TYPE_ARG"
    value="ASCO_GB_NAME*A1*Piara{BSTD_ID_PERSON*A1*{" />
</method>
```

Tenemos que se está levantando el Meta4Object SHCO_PA_MT_PERSON, y se está ejecutando el método SHCO_LOAD_FILTER, del nodo SHCO_GN_ROOT, con el argumento LOAD_TYPE_ARG, que tiene como valor ASCO_GB_NAME*A1*Piara{BSTD_ID_PERSON*A1*{.

Con estos datos podemos levantar el depurador LN4, añadir un punto de ruptura el dicho método, levantar el Meta4Object y ejecutar el método pasando los argumentos necesarios, con lo que podremos depurar la ejecución del método. Una vez hecho esto podemos levantar el inspector y ver el resultado de la ejecución en el Meta4Object.

En particular podemos hacer lo anterior creando un Meta4Object de prueba que sea mononodo, con un Alias de Meta4Object que apunte al SHCO_PA_MT_PERSON y un método con el siguiente código:

```
SHCO_PA_MT_PERSON! SHCO_GN_ROOT.
SHCO_LOAD_FILTER("ASCO_GB_NAME*A1*Piara{BSTD_ID_PERSON*A1*{.")
```

Añadir más información de depuración desde las páginas JSP

Se puede conseguir que nuestras páginas jsp vuelquen trazas a un archivo, utilizando el mismo sistema de *log* que utilizan los servicios Web. Para ello se utilizará Java.

Para la parte de javascript se usará la función *alert*. Al ejecutarse esta función se muestra un mensaje con la información que se indique.

EJEMPLO

```
<script type="text/javascript" language="Javascript1.5">
//El valor de una variable javascript
alert (vJSVariable);
//El valor de una variable java
alert ( '<%=zJavaVariable%>' );
</script>
```

El archivo de trazas de páginas jsp es el jsp.log. Contiene la información volcada desde páginas jsp.

Su ubicación depende del entorno:

- Windows: <m4log-dir>\jsp.log
- Solaris: <m4log-dir>/jsp.log

Donde

- <m4log-dir> es el parámetro M4_LOG_DIR del archivo m4logsystem.cfg.

Más información, en el apartado *Configuración del sistema de trazas* de este capítulo.

Para conseguir volcar datos al archivo *log* desde la página jsp hay que seguir estos pasos:

1. Importar la clase:

```
<%@ page import="com.meta4.common.utils.logsystem.M4i18nCategory" %>
```

2. Crear con java el objeto *Log*:

```
<%
// Creates a log object for these jsp pages.
M4i18nCategory m_log = (M4i18nCategory)
M4i18nCategory.getInstance("com.meta4.jsp");
%>
```

3. Utilizar los distintos métodos de volcado según el nivel de debug del error. Tener en cuenta que al archivo irán las trazas hasta el nivel indicado en el parámetro TRACE_LEVEL del archivo m4logsystem.cfg.

Más información, en el apartado *Configuración del sistema de trazas* de este capítulo.

```
m_log.fatal("my message");
m_log.fatal("my message", e); // Dump message and exception stack
m_log.error("my message");
m_log.warn("my message");
m_log.info("my message");
m_log.debug("my message");
```

Configuración del sistema de trazas de los servicios Web

Las trazas de servicios Web se configuran en el archivo m4logsystem.cfg localizado bajo el directorio de instalación de los servicios Web en la carpeta: \web_inf\classes\properties.

Los parámetros que se configuran en este archivo son los siguientes:

Tabla 6. Configurar sistemas de trazas

PARÁMETROS	DESCRIPCIÓN
M4_LOG_DIR	Indica la ubicación de las trazas. Para el nombre del directorio usar el separador "/". No usar el separador "\"
M4_TRACE_LEVEL	Indica el nivel de trazas. De mayor a menor, los posibles niveles son: <i>debug</i> <i>info</i> <i>warn</i> <i>error</i> <i>fatal</i>

Errores más frecuentes

A continuación se relacionan los problemas habituales y una guía de cuál puede ser la solución.

PROBLEMAS	POSIBLES SOLUCIONES
Un campo no admite más dígitos a pesar de estar definido en la BD con un tamaño superior	Controlar que la longitud de los campos coincide con las propiedades <code>size=""</code> y <code>maxlength=""</code> definidas en las páginas
Cuando navego en la página porque tiene más registros de los que muestra la primera vez pierdo los datos del formulario	Controlar que la página tiene el formulario oculto y que en este formulario se han modificado también los campos de nuestra página
No se ven los nombres traducidos	No los esta trayendo correctamente el <code>Meta4Object</code> o no se están trayendo como el resto de campos del <code>Meta4Object</code> a la página
Al dar al botón de limpiar pierdo los datos que tenía porque iba a modificar el registro	Controlar que este botón tiene el control de nuevo/modificado
Existe el botón de nuevo pero no lleva a ninguna página	Esta información no tiene pantalla para mantener los datos y hemos olvidado quitar este botón (variable <code>java znew</code>)
Al irme a consultar el botón de lista de la cabecera, luego no vuelvo a la página	Controlar que la página tiene definida la variable <code>zcarril</code> y esta tiene modificado el valor con el nombre correcto de la página
Existen literales en la página no traducidos	Se ha incluido literales en las páginas directamente y no en el archivo que se traduce. Sacar todos los literales a este archivo
Estoy incluyendo una fecha y puedo poner cualquier cosa o mi campo es numérico y puedo introducir letras	Controlar que cada campo que incluimos en el formulario tiene un formato especial y que hemos utilizado la función correspondiente de la librería para controlarlo
Estoy trabajando con fechas de inicio y fin pero puedo incluir cualquier fecha o no obtengo valores correctos de duración	Controlar que se utilizan los controles especialmente definidos para fechas de inicio y fechas de fin antes trabajar con funciones de cálculo de días

PROBLEMAS	POSIBLES SOLUCIONES
Cuando cambio de pantalla el formulario aparece más desplazado hacia la izquierda	Controlar que delante de cada campo del formulario hay un espacio
Los campos de la PK no me aparecen marcados en azul	No se ha hecho el control que lo identifica como campo de la PK
Puedo modificar el ordinal en la PK que se genera automáticamente	Controlar que el campo en la página esta deshabilitado

Operaciones automáticas sobre páginas JSP: Robot M4robotjsp2k2

18

Acerca de este capítulo

El robot M4robotjsp2k2 se utiliza para modificar el código de un conjunto de páginas jsp de forma automática. En este manual se describen los procesos implementados en el robot.

A continuación se detalla:

- Procesos implementados en el robot
- Uso del robot
- Interpretación de los resultado de ejecutar el robot sobre páginas JSP

Procesos implementados en el robot

Los procesos que puede ejecutarse con el robot sobre las páginas JSP son los siguientes:

- Inyección de dobles comillas en los atributos de los tags de meta4
- Gestión de caracteres especiales para html y javascript
- Actualización de tópicos de ayuda
- Modificar imágenes para la gestión multiproducto
- Establecer estándares en el tag `<m4:input>` y en expresiones JAVA
- Incluir validaciones de datos en las cajas de texto

Inyección de dobles comillas en los atributos de los tags de meta4

Para funcionar correctamente con JRun 3.1 se requiere que el valor de los atributos de un tag vayan entre dobles comillas. En versiones anteriores de JRun no era necesario. Por este motivo se recomienda su uso cuando sea necesario pasar un producto a una versión de JRun 3.1

El proceso busca tags de Meta4 en las páginas jsp y todos los atributos cuyo valor no vaya entre comillas dobles se las inyecta. Concretamente, trata los tags definidos en el fichero M4Tags.tld. Así, en cada versión se tratan solo los tags que se usan.

Gestión de caracteres especiales para html y javascript

Teniendo en cuenta el capítulo Gestión de caracteres especiales (html y javascript) del Manual HTML Development Framework, el robot para este proceso se encarga de hacer los siguientes tratamientos y en el orden indicado:

- Tratamiento para las invocaciones a la función javascript `m4checkradio`
- Tratamiento `HtmlJsafe`
- Tratamiento de los valores asignados al atributo `value` de los objetos `Input`.

Tratamiento para las invocaciones javascript a `m4checkradio`

En aquellos casos en los que a la función `m4checkradio` se le invoca pasándole como argumentos los valores de los tags `m4:item` o `m4:label` hay que

asegurarse que dichos valores van entre comillas simples para evitar que falle el tratamiento HtmlJSafe.

EJEMPLO

Antes de pasar el robot: `m4checkradio (.., <m4:item />..)`

Después de pasar el robot: `m4checkradio (.., '<m4:item />'.)`

Tratamiento HtmlJSafe

Dada una página JSP se tratan los tags `m4:item` y `m4:label` que se usan directamente en ella. Se considera que estos tags se usan directamente en la página si no se está utilizando el atributo `m4varname` para recoger el valor a través de una variable java.

Para las correctas notaciones será necesario añadir los atributos `htmlsafe` y `jsafe` dependiendo del uso que se esté haciendo de los mismos:

- A todos los `m4:item`, `m4:label`, que se usan dentro de los eventos `onclick= "..."` y `href= "..."`, se les añaden los atributos `jsafe="true"` y `htmlsafe="true"`.
- A todos los `m4:item` y `m4:label` que se usan en secciones javascript, aquellas que van entre las etiquetas `<script> </script>`, se les añade el atributo `jsafe="true"`.
- A todos los `m4:item` y `m4:label` que se usan en las secciones que no sean de javascript, código fuera de las etiquetas `<script> </script>`, se les añade el atributo `htmlsafe="true"`.

Si uno de los atributos que se va a añadir ya existe en el tag no se modifica.

Tratamiento de los valores asignados al atributo value de los objetos Input

Para los objetos de tipo Input que no sean hidden y que tengan asignado en el atributo `value` una expresión java se les añade la función java equivalente a utilizar el atributo `htmlsafe` en un `m4:item` / `m4:label`.

El motivo es que normalmente a nivel corporativo los input indicados se utilizan para pasar valores entre páginas en los procesos de paginación. Así, se visualizarán correctamente según la notación HTML.

EJEMPLO

```
<input type="text" value="<%=zval1%>" /> -->
<input type="text"
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML("zval1")%>"
/>
```

Actualización de tópicos de ayuda

El proceso general de ayuda consiste en mostrar una página html de ayuda para cada página jsp de la aplicación. Si no existiera una página específica de ayuda se muestra la página general del browser de ayuda. Para ello:

- Todas las páginas JSP que sean susceptibles de tener el botón de ayuda deben tener definida la variable java zHelp <% String zhelp = "";%>.
- La norma general es que la página de ayuda asociada a una página JSP tiene el mismo nombre pero con extensión .htm. Es decir, shco_ta_inc_data.jsp tendría como tópico de ayuda SHCO_TA_INC_DATA.htm.

Este proceso del robot consiste en buscar por las páginas jsp todas las variables zhelp (en minúsculas puesto que es la variable que luego se usa para invocar la ayuda) y aquellas páginas que la tengan, actualiza el valor de dicha variable con el nombre de la página de ayuda correspondiente todas las veces que lo encuentre.

El hecho de que una página JSP no tenga desarrollada la página de ayuda correspondiente no quiere decir que la variable zhelp no tenga asignado el nombre del página. El proceso que se encarga de invocar a la ayuda llamará a la página principal si la página concreta no existe.

El proceso es verdaderamente útil cuando se integra por primera vez la ayuda en un producto, facilitando la actualización de todas las páginas. Una vez que ya se tiene integrada la ayuda es útil pasarlo sobre las páginas JSP nuevas.

Existen páginas jsp cuya funcionalidad cambia, por ejemplo en función de un parámetro. En estos casos la ayuda a mostrar va a depender precisamente de la funcionalidad que esté dando la página en cada momento, y por tanto el valor asignado a la variable zhelp habrá que gestionarlo manualmente teniendo en cuenta los factores que influyen en la funcionalidad que da la página en cada momento. Un ejemplo son las tablas de valor.

Sobre estas páginas, si se pasara el robot se invocaría siempre a la misma página de la ayuda independientemente de la funcionalidad de la página en

cada momento. Este comportamiento no es el deseado. Para más información, sobre el tratamiento de estos casos, vea capítulo Ayuda en línea del Manual HTML Development Framework.

Modificar imágenes para la gestión multiproducto

Esta función le facilita la tarea de modificar los ficheros jsp del directorio files_gif para que en un producto las imágenes puedan ser diferentes a las desarrolladas bajo las directrices del Development Framework. Si no pasa el robot no se pueden personalizar las imágenes a no ser que hagan los cambios manualmente.

Concretamente lo que hace es modificar los ficheros de files_gif para que las imágenes puedan ser sobrescritas. A efectos prácticos, la información de la imagen se desdobra en dos ficheros de la siguiente forma:

- En cada página jsp de files_gif inserta una llamada a un fichero con el mismo nombre terminado en code. Por ejemplo, en la página ic_help.jsp sustituye el contenido por un include a ic_help_code.jsp:

```
<jsp:include page='/files_gif/ic_help_code.jsp' flush="false" />
```

- Para cada página jsp de imagen se genera un fichero tipo _code. Cada fichero code debe incluir dos ficheros genéricos (shco_gen_images_arg y shco_gen_images) y el nombre y localización de la imagen. Todo el código debe estar en formato java:

```
<%Include src="shco_g0/shco_gen_images_arg.jsp"%>
<%zFileName = "ic_help.jsp" ;
zStdFileContent = "src=\"/images/ic_help_25_31_" + zcssuser
+".gif\" width=\"25\" height=\"31\""; %>
<%Include src="shco_g0/shco_gen_images"%>
```

El fichero shco_gen_images.jsp se encarga de comprobar si existe en el directorio files_gif del producto actual (files_gif_appprod, donde appprod es el identificador del producto) un fichero con el nombre establecido en la variable zFilename. Si es así, se incluye dicho fichero (jsp:include) y sino se deja el contenido inicial que vendrá establecido en la variable java zStdFileContent.

Establecer estándares en el tag <m4:input> y en expresiones JAVA

Con el objetivo de hacer las páginas JSP más estándares se realizan cambios concretos de sintaxis:

- En las expresiones `<%= Tran.getProperty("a"); %>` se quita el punto y coma quedando: `<%=Tran.getProperty("a") %>`
- En el tag `m4:input` se cambia el atributo `class` por `styleclass`

Incluir validaciones de datos en las cajas de texto

Esta funcionalidad del robot permite insertar las validaciones de datos en páginas JSP desarrolladas con anterioridad al SP3 tecnológico. A partir de este Service Pack la funcionalidad se ha implementado directamente en las Herramientas de desarrollo | Herramientas de cliente ligero. Más información, Introducir la información de la página JSP del capítulo Generación automática de páginas del cliente web.

A grandes rasgos, esta funcionalidad consiste en validar los datos que introduce un usuario en un campo determinado contra una lista y recuperar su literal correspondiente. Si no existe, se llama a la lista para que el usuario pueda seleccionar uno de los valores de ésta. Antes de ejecutar esta funcionalidad, se recomienda la lectura del apartado Validaciones de código JavaScript del capítulo Metodología en el que se detalla como insertar las validaciones manualmente, sin ayuda del robot.

El proceso consta de dos pasos:

- Extracción de la información. Los objetivos de esta fase son:
 - Extraer de las páginas jsp la información necesaria sobre las listas que existen, campos de retorno, etcétera. Se genera un archivo con esta información puesto que será utilizada en el segundo paso.
 - Crear las páginas de validación correspondientes con la información obtenida.
- Inclusión de las validaciones. El objetivo es insertar las llamadas a las páginas de validación, creadas en el paso anterior, en las páginas jsp correspondientes.

Extracción de la información

El objetivo final de este paso es crear las páginas de validación. Para ello se buscan las páginas de lista existentes con el objetivo de extraer información útil necesaria para la creación de las páginas de validación. Por ejemplo:

- El `Meta4Object` y nodo utilizados para listar valores serán los mismos que se utilicen para la página de validación. Lo mismo ocurre con el nodo raíz que contiene los métodos de filtrado y carga del `Meta4Object`.

- Campos de entrada que serán los campos de filtrado en la validación.
- Campos de salida que serán los campos de retorno de la página de validación y que se pintarán en la página que invoca dicha validación.

El proceso interno del robot para localizar las listas es el siguiente:

1. Buscar las llamadas a las listas con la función `m4filtro()`. Se obtiene así información de la página de lista utilizada y de sus campos de retorno. El robot considera llamada válida para ser procesada si tiene al menos dos campos de retorno.



No detecta las listas que estén bajo la función `m4filtrocallback`

2. Buscar las llamadas a los zoom de cada lista. Por cada `m4filtro()` se busca la llamada `m4zoom()` correspondiente para localizar los campos de entrada y los campos de salida. Siempre se considerará como prioritaria la información encontrada en la función `m4filtro()` y en caso de no encontrar el zoom se tomará como campo de entrada el primero de la llamada al `m4filtro()`.
3. Buscar la página de lista que se llama desde la función `m4filtro()` para buscar el resto de la información. Esta página se busca en los directorios de los idiomas en el siguiente orden: *espanol, francais, english, y portugues*. Para cada página de lista localizada, se toma la siguiente información:
 - El `Meta4Object` de validación. Se toma el `Meta4Object` que consta en el valor de la variable java `zm4object` o `zsubsesion`. Como nodo de datos se toma el valor de la variable java `znodo` y como nodo raíz se toma el valor fijo `SHCO_GN_ROOT`. Si no encuentra el `Meta4Object` de validación porque la asignación de estas variables se hace de otra forma, se recomienda crear en la página de lista un comentario que incluya esta información, evitando así que la asignación influya en su funcionalidad.
 - Campos de retorno. Se toman tantos como valores se rellenen en el array javascript `aval` y los campos de los nombres se toman de las variables java `zField` o `zcampo`. Los campos se consideran siempre de tipo cadena.

Toda la información de la lista que va recabando se guarda en el fichero que indique en **Archivo de listas**. Se utilizará para generar las páginas de validación y para insertar las validaciones en los diferentes campos. Este fichero se genera en su directorio personal de trabajo pero no lo puede leer puesto que se genera en binario.

4. Generar y grabar la página de validación con toda la información recopilada. Un ejemplo de página de validación sería esta:

```
<%@   page import="com.meta4.validate.XMLValidate" %><%
```

```
// Response Headers...
//response.setDateHeader("Expires", -1);
response.setHeader("Cache-Control", "max-age=3600,public");
XMLValidate valobject = new XMLValidate();
    valobject.setM4object("SHCO_TK_MT_COUNTRY");
valobject.setNodeData("SHCO_TK_MT_COUNTRY");
valobject.setNodeRoot("SHCO_GN_ROOT");
valobject.setInputFields(new String[]{"STD_ID_COUNTRY"});
valobject.setInputTypes(new String[]{"4"}); // "4" cadenas,
"10" fechas y "11" números
valobject.setOutputFields(new String[]{"STD_N_COUNTRY"});
valobject.process(request, response);    %>
```

Inclusión de las validaciones

Para incluir las llamadas a las páginas de validación en las páginas JSP que utilizan listas el robot internamente realiza los siguientes pasos:

1. Busca las llamadas a las listas. Se vuelven a buscar todas las llamadas a la función `m4filtro()` con el objetivo de localizar los campos relacionados con esta llamada que necesitan incluir una validación.
2. Para cada `m4filtro()` localizado, busca la información, generada por el paso Extracción de la información, para la página de lista correspondiente.
3. Busca los campos a validar. Se localizan todos los campos input con id igual a los campos de entrada de la página de validación.

Para cada uno de ellos se incluye el evento onblur que realiza la invocación a la validación. La función de validación que se utilice dependerá de los parámetros indicados en la ejecución del robot. Por defecto, siempre inserta la función `m4translatepick`. Si desea insertar la `m4translatelist` para algunos campos validados, genere un fichero con un listado con los ID de los Meta4Objects implicados. Éstos deben estar separados por retornos de carro. Para generar la lista puede utilizar la tabla T3S y seleccionar los cacheables.

Indique al robot el nombre del fichero y ruta en la que se encuentra en **Archivos de lista cacheables**. Se incluirá `m4translatelist` sólo para las validaciones cuyo Meta4Object asociado figure en dicho fichero. Para el resto se incluirá `m4translatepick`.

Recuerde que `m4translatelist` trae a la máquina cliente la lista con todos los valores y la función `m4translatepick` sólo trae el valor que se quiere traducir. La primera es indicada para listas cortas y la segunda para listas voluminosas. Más información, Implementación de traducciones de código del capítulo Metodología.



Si en una página ya existe el evento onblur, el robot no hace nada.

La llamada a la página de validación tendría la estructura del ejemplo.

EJEMPLO

```
<input id="<%=zFieldX_Std_id_country%>" onblur="m4translatelist(
'shco_py/shco_tk_xml_country.jsp', 'shco_py/
shco_tk_list_country.jsp', new
Array('<%=zFieldX_Std_id_country%>'), new
Array('<%=zFieldX_Std_n_country%>'));"value="<%=com.meta4.taglib.u
til.M4PresentationUtilTaglib.cookHTML(zParam_Std_id_country)%>"/>
```

4. Si en la página se ha añadido al menos una validación es necesario incluir, justo después del tag <body>, la llamada a m4valdata.js que contiene el código javascript de las validaciones:

```
<script type="text/javascript" src="/library/m4valdata.js"></script>
```

Cómo instalar el robot

Para instalar el robot es necesario:

- Tener instalado J2RE (Java 2 Runtime Environment) versión 1.3 o superior de Sun Microsystems.

El robot se publicará internamente en m4toolkit\m4jsprobot2k2.

Se distribuye en el CD de tecnología.

El robot consta de los siguientes ficheros:

- m4jsprobot2k2.bat
- m4jsprobot2k2.jar
- oro.jar
- xercesimpl.jar
- xmlparserapis.jar

Todos los ficheros tienen que estar en el mismo directorio para que funcione correctamente la herramienta.

- Para ejecutar el robot, haga clic en el m4jsprobot2k2.bat.

Uso del robot

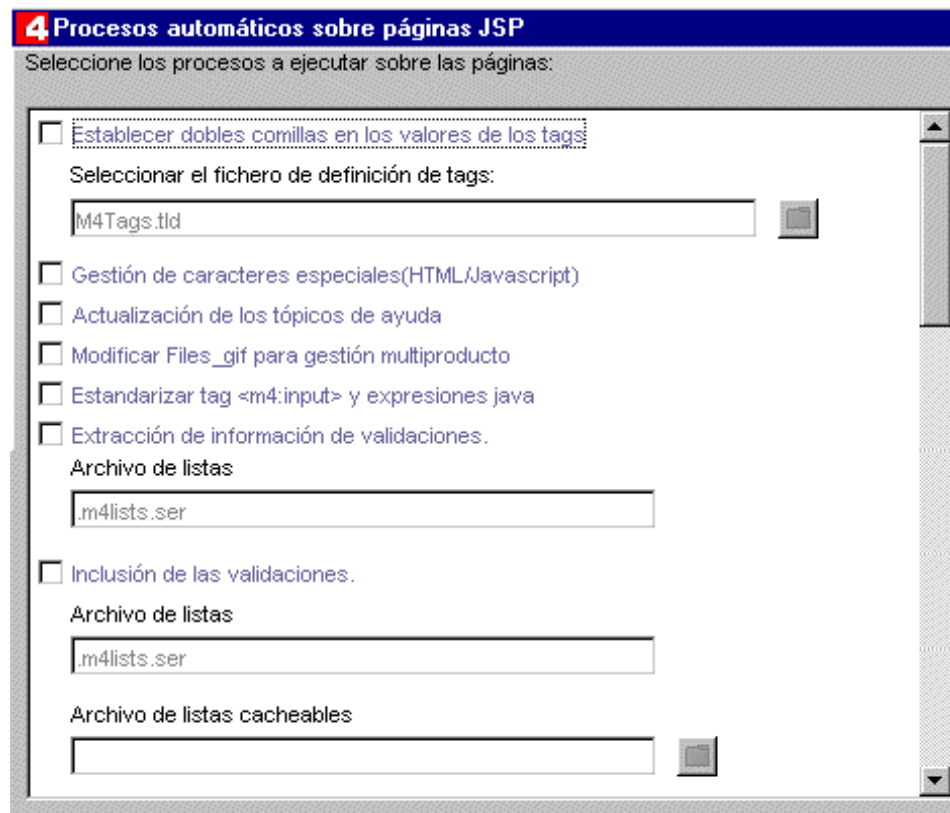
Antes de utilizar el robot conviene hacer las siguientes recomendaciones:

- Hacer una copia previa de las páginas o si están en un repositorio de control de versiones tipo Sourcesafe se recomienda etiquetar. Así, siempre se puede comparar lo que había con lo que ha hecho el robot por si hay que revisar posibles errores.
- El robot debe ser pasado sobre JSP debidamente desarrolladas, es decir, que tengan un funcionamiento correcto y no estén en proceso de desarrollo.

El proceso de ejecución del robot es el siguiente:

1. Hacer doble clic sobre el fichero m4jsprobot2k2.bat tras haber instalado el robot.

Figura 68. Seleccionar proceso



2. Seleccionar los procesos que quiere ejecutar sobre las páginas. Puede pasar todos los procesos a la vez si lo desea.

En el caso de seleccionar el primer proceso debe indicar además el fichero de extensión tld en el que se definen los tags que se tratarán.

Más información, Inyección de dobles comillas en los atributos de los tags de meta4 .

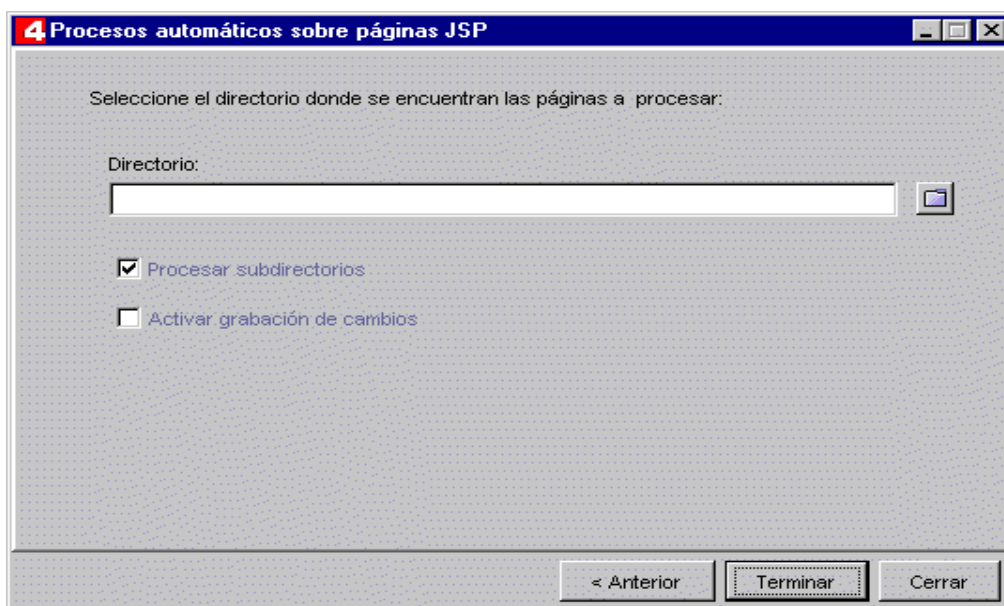
En el caso de seleccionar el proceso **Extracción de información de validaciones** debe indicar el fichero en el que se incluirá la información de las listas o dejar el que se indica por defecto.

En el caso de seleccionar el proceso **Inclusión de las validaciones**, debe indicar el fichero de listas resultante del proceso de extracción o dejar el que se indica por defecto. Si la extracción y la inclusión se realiza a la vez este fichero debe ser el mismo. Si además necesita indicar en qué casos se va a utilizar la función m4translatelist debe indicar la ruta y el fichero que contiene la relación de Meta4Objects sobre los que se aplicará.

Más información, Incluir validaciones de datos en las cajas de texto .

3. Hacer clic en Next.

Figura 69. Selección de carpeta que contiene las páginas

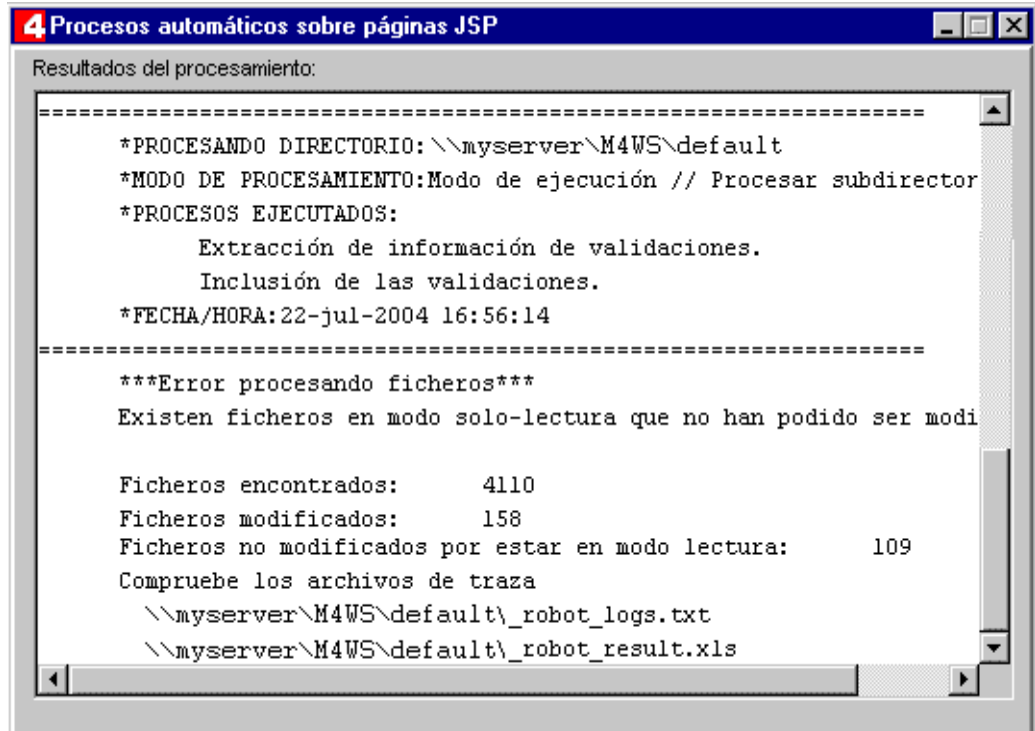


4. Seleccionar el directorio donde se encuentran las páginas a procesar. indicando además si:

- se procesan las páginas de los subdirectorios activando la casilla de verificación correspondiente
- el robot se pasa en modo de chequeo desactivando la casilla de verificación **Activar grabación de cambios** o guardando los cambios que se realicen activando la casilla de verificación.

Una vez que se pone en marcha, el robot muestra el directorio cuyas páginas está procesando y al finalizar indica el resultado del procesamiento.

Figura 70. Resultado del proceso en modo testeo



```
4 Procesos automáticos sobre páginas JSP
Resultados del procesamiento:
*****
*PROCESANDO DIRECTORIO: \\myserver\M4WS\default
*MODO DE PROCESAMIENTO: Modo de ejecución // Procesar subdirector
*PROCESOS EJECUTADOS:
    Extracción de información de validaciones.
    Inclusión de las validaciones.
*FECHA/HORA: 22-jul-2004 16:56:14
*****
***Error procesando ficheros***
Existen ficheros en modo solo-lectura que no han podido ser modi

Ficheros encontrados:      4110
Ficheros modificados:      158
Ficheros no modificados por estar en modo lectura:      109
Compruebe los archivos de traza
    \\myserver\M4WS\default\_robot_logs.txt
    \\myserver\M4WS\default\_robot_result.xls
```

Como se aprecia en la figura, tras el proceso se generan dos ficheros en el mismo directorio de proceso:

- el fichero `_robot_logs.txt` que contiene la información general de proceso y que se muestra en la parte superior de la figura.
- el fichero `_robot_result.xls` en el que detalla el estado final de cada fichero para cada proceso. Si ya existiera este fichero y estuviera en uso, la aplicación genera un nuevo fichero `_robot_results1.xls` y así sucesivamente hasta que encuentre un fichero xls disponible sobre el que pueda escribir datos. Más información, Interpretación de los resultados al ejecutar el robot sobre páginas JSP .

Interpretación de los resultados al ejecutar el robot sobre páginas JSP

Cómo consecuencia de pasar el robot sobre un conjunto de páginas, se obtendrá información relativa el estado final de cada fichero para cada uno de los procesos a los que se ha sometido. Esta información se recoge en el fichero `_robot_result.xls` que se crea en el mismo directorio que contiene las páginas procesadas.

El robot se puede pasar:

- En modo testeo sin guardar cambios para tener una previsión de las alteraciones que se efectuarán
- Guardando cambios

A continuación se explica cómo interpretar la información que se facilita en `_robot_result.xls` sobre el estado de un fichero en estos dos modos.

Paso de procesos en modo testeo

Cuando el robot se pasa en modo testeo, por cada fichero tratado se genera una fila de información. Esta fila contiene el nombre del fichero (columna `FILE`), una columna por cada proceso que se lanza, y una columna con el resultado global (columna `PROCESS_RESULT`).

La columna que recoge el resultado para cada proceso puede tener los siguientes valores:

Valores posibles	Descripción
OK	Fichero que no necesita ser modificado por el proceso.
SHOULD_BE_MODIFIED	Fichero que precisa ser modificado por alguno de los procesos
ERROR	Fichero que necesita ser modificado.

La columna que recoge el resultado global para cada fichero (`PROCESS_RESULT`) puede tener los siguientes valores:

Valores posibles	Descripción
OK	Fichero no modificado por ninguno de los procesos porque está correcto.
SHOULD_BE_MODIFIED	Fichero que precisa ser modificado por alguno de los procesos

ERROR	Error en alguno de los procesos. Ver el resultado correspondiente a cada proceso.
-------	---

Paso de procesos guardando cambios

Cuando el robot se pasa grabando los cambios, por cada fichero tratado sale una fila de información. Esta fila contiene el nombre del fichero (columna FILE), una columna por cada proceso que se lanza, y una columna con el resultado global (columna PROCESS_RESULT).

La columna que recoge el resultado para cada proceso puede tener los siguientes valores:

Valores posibles	Descripción
OK	Fichero que no necesita ser modificado por el proceso.
Modified	Fichero modificado por el proceso.
ERROR (Read-Only file)	Fichero que necesita ser modificado pero está en sólo lectura.
ERROR	Error en el tratamiento del fichero.

La columna que recoge el resultado global para cada fichero (PROCESS_RESULT) puede tener los siguientes valores:

Valores posibles	Descripción
OK	Fichero no modificado por ninguno de los procesos porque está correcto.
Modified	Fichero modificado debido a alguno de los procesos que se le han pasado. Ver el resultado correspondiente a cada proceso.
ERROR	Error en alguno de los procesos. Ver el resultado correspondiente a cada proceso.

Ayuda en línea

19

Acerca de este capítulo

El objetivo de este capítulo es explicar los pasos que se tienen que seguir a la hora de integrar el sistema de ayuda en el cliente Web. No es objetivo de este capítulo describir el proceso de generación de la ayuda en línea que se explica en el manual *Generación de sistemas de ayuda en línea 6.0*.

Descripción funcional

Acceso a las ayudas en línea

La ayuda en línea en el cliente Web se invoca de estas formas:

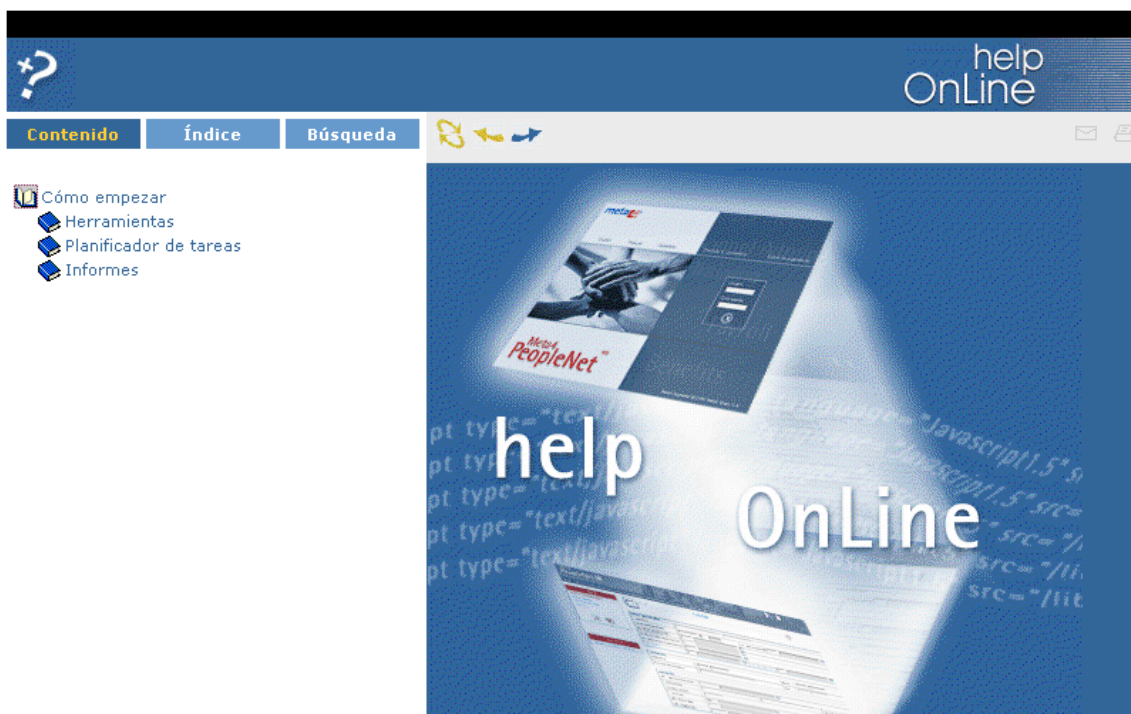
- A través de la inclusión en las páginas del botón 
- A través del enlace de Ayuda en el *disclaimer*

[Inicio](#) [\[Organización\]](#) [\[Administración de personal\]](#) [\[Nómina\]](#) [\[Herramientas\]](#) [\[Ayuda\]](#) [\[Arriba\]](#)
 © 2003 Meta4 Spain, S.A.
 Todos los derechos reservados.

- A través del enlace de **Ayuda** en el Portal, en la página de conexión.

El proceso general de ayuda consiste en mostrar una página HTML de ayuda para cada página *jsp* de la aplicación. Si no existiera una página específica de ayuda se muestra la página general del *browser* de ayuda.

Figura 71. Browser de ayuda que se muestra por defecto



Parametrización de las ayudas en línea

El HTML Development Framework proporciona, por separado, las ayudas en línea que corresponden a la tecnología que subyace y apoya las funcionalidades de una aplicación de Meta4 PeopleNet Ksystem.

Cuando se conecta con la aplicación que utiliza la tecnología del HTML Development Framework, como por ejemplo Meta4 PeopleNet Ksystem, tendrá acceso a dos sistemas de ayudas en línea: tecnológica y funcional.

Para cambiar de un sistema de ayudas a otro, haga clic en el botón  en la ventana del sistema de las ayudas en línea que sea.

Sistema de ayudas en línea tecnológica: no parametrizable

Este incluye la información acerca de las herramientas y utilidades avanzadas que apoyan la funcionalidad. Esta información es reservada y no parametrizable, así como las páginas y las opciones de menú tecnológicas correspondientes de la aplicación.

Sistema de ayudas en línea funcional: parametrizable

Este incluye información acerca de los módulos funcionales de la aplicación. Esta información es editable en función de la parametrización de los módulos funcionales.

Integración de un sistema de ayudas en línea funcional

Una vez generada, el contenido, estructura y *browser* de la ayuda en línea para integrarla en la aplicación es necesario:

- Preparar las páginas de la aplicación para que se pueda invocar la ayuda desde ellas. Existen tres formas de definir la ayuda dependiendo de la página desde la que se llama:
 - Páginas funcionales JSP
 - Páginas de menú de módulos
 - Páginas de tablas de valor

Además, en el *disclaimer*, siempre existirá un enlace a la ayuda en línea. Este enlace mostrará la ayuda específica de la página en la que se encuentra o la página genérica de la ayuda si no existiera una página htm concreta de ayuda.

- Colocar las páginas de contenido en el directorio adecuado en el servidor.

Ayudas para nuevas páginas funcionales jsp

Para que una nueva página tenga ayuda en línea los pasos a seguir son:

1. Definir en la página la variable java **zhhelp** de tipo *string*, e indicar el nombre de la página htm de ayuda correspondiente en mayúsculas. Normalmente, a todas las páginas jsp les corresponde una página de ayuda htm cuyo nombre coincide. Así para la página shco_ta_mt_incidence.jsp la página de ayuda es SHCO_TA_MT_INCIDENCE.htm
2. Incluir el archivo de cabecera correspondiente, normalmente el **shco_gen_cab.jsp**. Existen otros archivos cabecera relativos a diferentes funcionalidades: shco_gen_cab_period.jsp, shco_gen_cab_person.jsp, shco_gen_cab_role.jsp, etcétera.

Este archivo genera la cabecera y a su vez incluye el archivo **shco_gen_help.jsp** que es el que se encarga de:

- Dibujar el icono de ayuda en el caso de que la variable **zhhelp** tenga un valor distinto de cero.
- Llamar a la función m4help.js encargada de llamar al sistema de ayuda y mostrar la página htm indicada **zhhelp**.

La inclusión de este archivo se debe realizar después de definir la variable **zhhelp**.

Figura 72. Ejemplo de página jsp con ayuda

The screenshot shows a web application interface for defining incidents. At the top, there is a header bar with the title "DEFINIR INCIDENCIAS" and a help icon (a question mark inside a circle) on the right. Below the header, there is a form titled "Nueva registro" (New record). The form contains several fields and checkboxes:

- * ID Incidencia: A text input field.
- * Incidencia: A text input field.
- * Inicio: A date picker field.
- Fin: A date picker field.
- Tipo de incidencia: Two radio buttons, "Ausencia" (selected) and "Presencia".
- * ID Grupo de incidencias: A text input field.
- * Alcance incidencia: Two checkboxes, "Periodo de RH" (checked) and "Rol".
- Datos pago: Three checkboxes, "Incidencia de larga duración", "Incidencia incluida en nómina" (checked), and "Incidencia remunerada" (checked).
- Periodo de pago: Two radio buttons, "Actual" (selected) and "Siguiendo".

Para incluir ayuda en esta página shco_ta_mt_incidence.jsp el código debe contener:

EJEMPLO

```
String zhhelp="SHCO_TA_MT_INCIDENCE.htm";  
%><%@ include file="../../shco_g0/shco_gen_cab.jsp" %>
```

El valor de **zhhelp** siempre debe ir en mayúsculas.

Ayudas en las páginas de menú genéricas

La forma de añadir la ayuda en una página de menú de un módulo es:

1. Declarar la variable **zhelp** asignándole el nombre del archivo htm de ayuda en mayúsculas. Normalmente coincidirá con el nombre de la página jsp en mayúsculas con la extensión htm.
2. Localizar la ayuda asignando el valor de **zhelp** a la variable **zsLocalizeHelp**. El objetivo es que el enlace Ayuda del *disclaimer* muestre la página htm de ayuda específica y no la genérica. Es necesario hacerlo así porque en estas páginas no se incluye el archivo de cabecera shco_gen_cab.jsp.

Figura 73. Menú principal de Administración de personal



El código para incluir la ayuda en esta página sería:

EJEMPLO

Página **shco_ta_menu.jsp**

```
<%@ include file="../../../shco_g0/shco_gen_taglib.jsp" %><html><%@
include file="../../../shco_g0/shco_gen_arg.jsp" %><%@ include
file="../../../shco_g0/shco_gen_bag.jsp" %><%@ include file="../../../
shco_g0/shco_gen_css.jsp" %><%@ include file="../../../shco_g0/
shco_portal_trans.jsp" %>

<head><%// Declaración del M4Object y de la página
//Escribes la dirección de la pagina
String zdireccion = "shco_ta_menu.jsp";
//Escribes el nivel de menu
znivelmenu = "4";
String zhelp = "SHCO_TA_MENU.htm";
zsLocalizeHelp= zhelp;
%>

<%@ include file="../../../shco_g0/shco_gen_menu.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_disclaimer.jsp" %>

</body></html>
```

Tablas de valor

En la aplicación estándar, las páginas relacionadas con la introducción de valores en la aplicación tienen el siguiente comportamiento respecto a la ayuda:

- En la página de configuración de parámetros correspondiente a cada tabla de valor, por ejemplo `shco_class_pyroleparams.jsp`, se incluye el siguiente atributo:

```
//página de ayuda
request.setAttribute("Help.Page", "shco_pyvalues_1.htm");
%>
```

`Help.Page` indica la página `htm` (`shco_pyvalues_1.htm`) de ayuda correspondiente a la página de valores.



Más información sobre las páginas de configuración de las tablas de valor en el capítulo de plantillas.

Todas las tablas de valores tendrán siempre asignados alguna de las páginas siguientes:

- Para todos los valores fijos tanto del rol como del periodo: `shco_pyvalues_1.htm`
- Para todos los valores incidencias tanto del rol como del periodo (excepto valores masivos): `shco_pyvalues_2.htm`
- Para todos los valores incidencias masivas tanto del rol como del periodo: `shco_pyvalues_2_1.htm`
- Para todas las tablas de valores generales: `shco_pyvalues_3.htm`

Así, las nuevas páginas que se generen tendrán alguno de estos valores dependiendo del grupo al que pertenezcan.

Además, existen una serie de páginas auxiliares relacionadas con la gestión de valores que llevan el `Help.Page` y además un `include` a la página `shco_pyinchelp.jsp` que inserta el botón de ayuda:

```
<% request.setAttribute("Help.Page", "shco_pyvalues_1_2.htm"); %>
<%@ include file="shco_pyinchelp.jsp" %>
```

Estas páginas son:

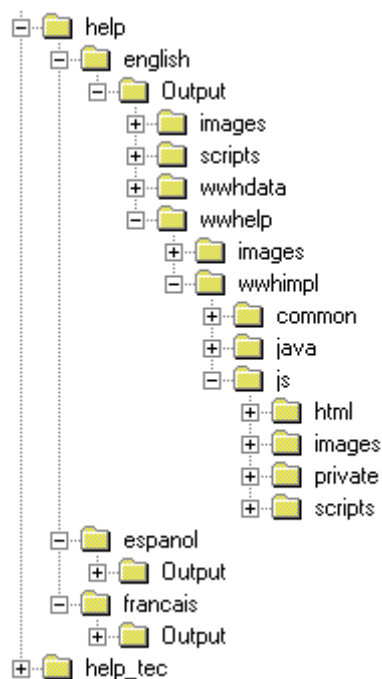
- En los históricos de imputación y de pago (`shco_historyvalues.jsp`): `SHCO_PYVALUES_1_2.HTM`
- Selección de componentes de cálculo (`shco_list_concepts.jsp`): `SHCO_PYVALUES_1_1.HTM`

Páginas de ayuda en el servidor

Normalmente, el proceso de publicación del sistema de ayuda escapa a la etapa de desarrollo. Es el proceso de documentación el encargado de esta tarea detallada en el manual *Generación de sistemas de ayuda en línea 6.0*. No obstante, conviene tener una idea clara de la estructura de la ayuda.

La ayuda funcional se localiza en la carpeta **help** del servidor y la ayuda tecnológica en la carpeta **help_tec**. Estas contienen para cada uno de los idiomas (english, espanol, francais) la carpeta **Output**.

Figura 74. Estructura del servidor Web para la ayuda en línea



En el directorio **Output** están las páginas de ayuda relacionadas con el contenido.

La página de invocación a la ayuda frames.htm está bajo el directorio:
\\help\\espanol\\Output\\wwwhelp\\wwhimpl\\js\\html\\frames.htm.

El resto de carpetas contiene archivos auxiliares que permiten la gestión del sistema de ayuda.

Configuración de la conexión unificada

20

Acerca de este capítulo

Con esta nueva funcionalidad, se da la opción de poder cambiar de la aplicación Windows a la aplicación Web sin necesidad de identificarse de nuevo, es decir, usando el usuario y contraseña con el que se ha entrado.

Así, podemos definir opciones de menú en la aplicación Windows que directamente dirijan a una página del producto Web, o que dependiendo de la parametrización unas veces se dirijan a Windows y otras a Web.

En este capítulo se describen los cambios que son necesarios hacer en la aplicación para que esto sea posible.

Proceso para utilizar la conexión unificada

El proceso se divide en dos partes principales:

- Primero hay que trabajar con los parámetros de la aplicación
- En segundo lugar, tratar adecuadamente las opciones de menú para:
 - Obtener acceso directo a Web
 - Obtener acceso combinado Web/Windows

Parámetros de aplicación

El primer paso es dar el valor adecuado a los parámetros implicados en la opción de menú **Herramientas de desarrollo | Configuración de la aplicación | Parámetros de aplicación**:

- Activar (valor 1) el parámetro de aplicación **Mostrar cliente Ligero** que se encuentra en **Configuración funcional | Portal**.
- Introducir el nombre del servidor Web en el parámetro **WEBSERVER** que se encuentra en **Forms | Singlesignon**. Además, indicar el **PORT** y el **PROTOCOL**.

Opciones de menú con acceso directo a Web

Para aquellas opciones de menú nuevas que llaman directamente a la página HTML hay que crear una tarea asociada a cada opción de menú con el siguiente código:

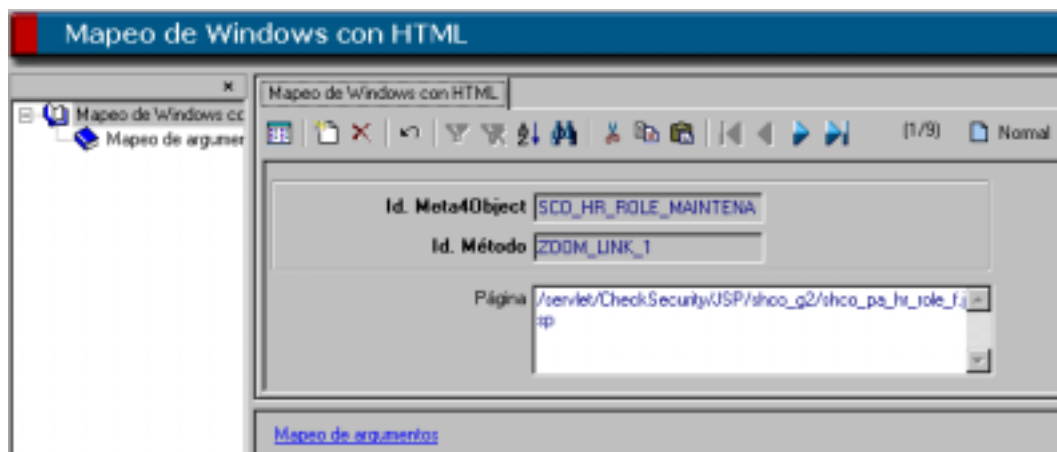
```
Stform.ptcmfLink
SetConstStr URL, "meta4ws://servlet/CheckSecurity/JSP/shco_g0/
...jsp" (Nombre página portal)
SetConstStr Sourceargs, "_REDIRECT"
SetConstStr Sourceobjs, "**F* "/servlet/CheckSecurity/JSP/...jsp "
(Dirección de la página a la que me quiero dirigir)
```

Opciones de menú con acceso combinado Windows/Web

En este apartado se tratan opciones de menú mantenidas de la parte Windows y que en función de los datos parametrizados en el Meta4Object **SCO_SSO_MAP**, se dirigen a la presentación Windows o a la página Web. Para conseguir esto se realizan los siguientes cambios:

1. Rellenar los datos de las tablas **M4SCO_SSO_MAP** y **M4SCO_SSO_MAP_ARG** a través de la opción de menú **Configuración funcional cliente ligero | Mapeo de Windows con HTML**, con la correspondencia entre el Meta4Object, el método al que se llama desde la aplicación y la página Web, y los argumentos que se pasan a la página.

Figura 75. Mapeo de Windows con HTML



2. Añadir la propiedad HTML_STRING a la estructura de nodo del Meta4Object que carga la opción de menú. Por ejemplo, si la opción de menú es mantenimiento del rol, se abre el Meta4Object **SCO_HR_ROLE_MAINTENANCE** y en la estructura de nodo **SCO_MT_HR_ROLE** se incluye la propiedad.

Id: HTML_STRING

Scope: Nodo

Type: Long

Status: N

C/S: ambas partes

3. Modificar el método al que se llama desde la presentación para que en el inicio compruebe el parámetro de aplicación para ver si tiene que mostrar Windows o HTML. Si es HTML, genera la cadena **HTML_STRING**. En caso de fallar esta generación o que la opción sea Windows, el método se ejecuta como antes de ser modificada la regla.

```

DefineInstance("SCO_SSO_MAP", "SCO_SSO_MAP", 0, 2)
Code ="Return(SCO_SSO_MAP!SCO_SSO_MAP.SHOW_HTML())"
ShowHTML = ClcExecuteLN4JIT(Code)
If ShowHTML="1" Then
    'Tenemos que levantar HTML
    ID_METHOD = "..."
    'Generacion de la página
    Code ="Return(SCO_SSO_MAP!SCO_SSO_MAP.LOAD_T3_DATA(ID_T3,
ID_METHOD))"
    iReturn =
    ClcExecuteLN4JIT(Code, "ID_T3", GetChannelId(), "ID_METHOD", ID_METHO)
    If (iReturn=M4_SUCCESS) Then
        'Generación de los argumentos
        For i=1 To GetArgumentNumber()
            sIdArg = "" : iTypeArg = -1 : iPrec = -1 : iRef = -1
            GetNodeItemArgument(sIdArg, iTypeArg, iPrec, iRef, i-1,
GetThisNodeId(), ID_METHOD)
            Code ="SCO_SSO_MAP!SCO_SSO_MAP.GENERATE_PARAMS_STRING("
+ Chr(DOUBLE_QUOTE) + sIdArg + Chr(DOUBLE_QUOTE) + "," + sIdArg +
"," + Format(iTypeArg,0) + "," + Format(iPrec,0) + ")"
            ClcExecuteLN4JIT(Code, sIdArg, GetArgument(i))
        Next
        Code = "Return(SCO_SSO_MAP!SCO_SSO_MAP.PARAMS_STRING)"
        HTML_STRING = ClcExecuteLN4JIT(Code)
        Return (M4_SUCCESS)
    EndIf
EndIf

```

4. Modificar la presentación que llama la opción de menú para ver si tiene que levantar una página Web o seguir con el *Obj* de Windows. Se añade el siguiente código antes del **BEGIN Form**:

```

BEGIN Action_script Action_html_or_windows
    Code = "DefineInstance("SCO_SSO_MAP", "SCO_SSO_MAP", 0,
2):Code ="Return(SCO_SSO_MAP!SCO_SSO_MAP.SHOW_HTML())":ShowHTML =
    ClcExecuteLN4JIT(Code):Return(ShowHTML)"
    Onfalse = "FormMain"
END
BEGIN Action_script Action_html_defined

```

```

        Code = "If (ARG_STRING<>'') AND (ARG_STRING<>NullValue())
Then Return('1') Else Return('0')"
        Ontrue = "Action_set_BP_PARAM_PAGE"
        Sourcenames = "ARG_STRING"
        Sourceobjects = "HTML_STRING"
    END

```

Y el siguiente código antes del último **END**:

```

BEGIN Action_goto GotoEnd
Nextblock = "FIN"
END
BEGIN Action_set Action_set_BP_PARAM_PAGE
    Set = "*C*Sourceobjs:=*F*" [HTML_STRING] ""
END
BEGIN Action_bp Action_OPEN_HTML
    Idbp = "BP_SSO_MAP"
    Localcontext = False
END
BEGIN Action_goto FIN
END

```

Diseño de una página JSP con enlaces entre sitios

Edite la página JSP del servidor origen desde la que desea realizar un enlace al servidor destino.

Para ello, utilice la etiqueta personalizada `<m4:crosslink/>`. Si desea obtener la página `/servlet/CheckSecurity/JSP/some_task/some_page.jsp`, utilice la siguiente sintaxis:

```

<m4:crosslink uri="/servlet/CheckSecurity/JSP/some_task/
some_page.jsp" provider="0"/>

```

Este tag agregará en su página una cadena que por defecto será la siguiente:

```

/servlet/CrossPoster?_URI=/servlet/CheckSecurity/JSP/some_task/
some_page.jsp

```

Así que si desea realizar un enlace con el tag A de HTML puede emplear la siguiente sintaxis:

```

<a href=<m4:crosslink
    uri="/servlet/CheckSecurity/JSP/task/page.jsp"
    provider="0"/>>click on this link that will take you away</a>

```

Si suprime el atributo `provider` se tomará por defecto el servidor remoto indicado en el parámetro `URL_0` del fichero de configuración.

```
<a href=<m4:crosslink
uri="/servlet/CheckSecurity/JSP/task/page.jsp"/>>click on this
link that will take you away</a>
```

Si indica otro identificador de proveedor, se proporcionará un enlace con el servidor que se haya incluido en el parámetro URL_N del archivo configclient.xml.

```
<a href=<m4:crosslink
uri="/servlet/CheckSecurity/JSP/task/page.jsp"
idprovider="1"/>>click on this link that will take you away</a>
```

En ese caso, el enlace que se generará, será como el siguiente:

```
/servlet/CrossPoster?_URI=/servlet/CheckSecurity/JSP/some_task/
some_page.jsp&_PROVIDER=1
```

Por supuesto, para poder ejecutar una página que contenga un tag `<m4:crosslink>` es necesario tener una sesión web activa. Por lo cual si intenta ejecutar una página JSP con dicho tag sin haberse conectado o después de un periodo de inactividad que haya hecho que su sesión haya expirado, el tag no pintará la cadena de texto deseada en la página HTML, sino que no pintará nada.

Desarrollo de componentes que difieren del diseño estándar del development framework

21

Acerca de este capítulo

En este capítulo se detallan los elementos que pueden cambiarse y cómo modificarlos sin seguir las especificaciones genéricas del Development Framework. Esta nueva funcionalidad permite desarrollos de componentes cambiando parte de la funcionalidad y apariencia estándares.

Para ver aplicaciones concretas vea el apartado Uso más frecuente.

Desarrollo de componentes modificables

Cuando quiera diseñar un componente que no siga con precisión el Development Framework, el proceso es el siguiente:

- Primero hay que identificar el producto de forma obligatoria
- En segundo lugar, modificar los elementos deseados. Sólo puede modificar los elementos que se señalan a continuación:
 - Elementos relacionados con el aspecto de la aplicación: css, imágenes, cabecera.
 - Disclaimer
 - Ayuda en línea
 - Asistentes

La filosofía de esta funcionalidad es que la aplicación compruebe si ciertos ficheros están sobrescritos:

- En caso negativo se actúa de forma estándar.
- En caso afirmativo se muestran los ficheros específicos del producto.

Identificación del producto

Si un producto utiliza elementos desarrollados fuera de las líneas de desarrollo del Development Framework es imprescindible identificar dicho producto. Existen dos formas:

- Se recomienda identificar el producto desde la página de conexión. Para ello, en la página de conexión del producto correspondiente se elimina la antigua identificación del producto a través del parámetro `_app_prod` y se añade el atributo `_PROD`.

EJEMPLO

Por ejemplo para las páginas de PeopleNet en la página `/shco_g0_pn/shco_gen_login.jsp` se elimina el parámetro de producto:

```
String zurl = "/servlet/CheckSecurity/JSP/shco_g0_pn/shco_gen_portal.jsp?_app_prod=pn";
```

Quedando así:

```
String zurl = "/servlet/CheckSecurity/JSP/shco_g0_pn/shco_gen_portal.jsp
```

Se añade a continuación el parámetro `_PROD` de la siguiente forma:

```
String zurl = "/servlet/CheckSecurity/JSP/shco_g0_pn/  
shco_gen_portal.jsp...  
<% request.setAttribute("LANG", zlang);  
// Identificador de producto  
request.setAttribute("_PROD", "PN");  
request.setAttribute("URL", zurl);...  
<jsp:include page="/shco_g0/shco_gen_login_box.jsp" flush="false" />
```

Es importante que la identificación se haga antes del `jsp:include`
<jsp:include page="/shco_g0/shco_gen_login_box.jsp" flush="false" />
puesto que esta página necesita el valor del atributo `_PROD`.

- A través de la etiqueta `MainProductID` del `configclient.xml`. Recuerde que este fichero se modifica a través de las páginas de configuración del producto. Si la identificación no consta en la página de conexión, se toma por defecto el definido en este fichero. Pero como ya se ha comentado se recomienda identificar el producto en la conexión.

Elementos relacionados con el aspecto

Hojas de estilo

Aquellos productos que desean una hoja de estilo diferente a la estándar deben crear en el directorio `shco_g0_appprod` (`approd`=identificación del producto) la página `shco_gen_css.jsp` añadiendo en esta página la hoja de estilo propia del producto.

En esta página está accesible la variable del usuario de css, a través de los atributos del `request`, por si la necesita:

```
String zL_cssuser = (String)request.getAttribute("zcssuser");
```

El resto de datos que tenga que utilizar deberán incluirse de forma explícita.

Iconos

Para poder parametrizar iconos estándares para productos específicos es necesario:

- Tener preparados los ficheros estándar de `files_gif` para que puedan ser parametrizados. Para comprobar si están preparadas compruebe si existe para su imagen en el directorio `files_gif` una página con el mismo nombre terminada en `_code`. En caso de no estar preparados, las modificaciones se pueden realizar automáticamente utilizando el robot `M4robotjsp2k2`. Más información, [Modificar imágenes para la gestión multiproducto](#).

- Parametrizar las imágenes que se deseen. Para ello siga los pasos siguientes:
 - Generar un fichero, con el mismo nombre del fichero de la imagen que figura en el directorio files_gif, en el directorio específico del producto files_gif_approd (approd=identificación del producto). Si queremos localizar para nuestro producto el icono de ayuda, en el directorio files_gif_approd creamos el fichero ic_help.jsp.
 - El contenido de este fichero debe seguir la misma estructura que el fichero original de files_gif:
 - Las imágenes deben tener obligatoriamente los atributos src, width y height que son los atributos del fichero original. Y además siguiendo la misma sintaxis definida en la variable java zStdFileContent del fichero correspondiente _code pero con notación htm, no java.
 - Opcionalmente se pueden añadir eventos onmouseout y onmouseover.
 - Si se cambia la imagen se recomienda generarla en el directorio images del producto.
 - En esta página se tiene accesible el usuario css a través de los atributos del request:

```
String zcssuser = (String) request.getAttribute("zcssuser");
```

EJEMPLO

En el producto ESS se genera una nueva imagen para el icono que invoca la ayuda en línea. La nueva imagen varía de tamaño y se le incluye los eventos onmouseout y onmouseover. El proceso es el siguiente:

- Partimos de la base de que ic_help.jsp ya está preparada para ser parametrizada. Comprueba para ello que en el directorio files_gif existe la página ic_help_code.jsp. Si no está preparada, recuerde que este proceso se puede realizar automáticamente con el robot M4robotjsp2k2.
- En el directorio images_ess del ESS se ha creado la imagen ic_help_28_28_css.gif
- En el directorio files_gif_ess se añade la página ic_help.jsp con el siguiente contenido:

```
<% String zl_cssuser = (String)
request.getAttribute("zcssuser");%>
src="/images_ess/ic_help_28_28_<%=zl_cssuser%>.gif" width="28"
height="28" onmouseout="" onmouseover=""
```

Cabeceras

Las modificaciones que puedes hacer en la cabecera afectan a:

- La generación de los menús
- Inclusión de más información en la cabecera
- Modificación de los botones

La generación de los menús

Si desea parametrizar la generación de los menús para un producto, Meta4 permite modificar el fichero `shco_gen_menusup.jsp` de tal forma que si se encuentra este fichero parametrizado en el producto no se ejecuta el método de generación de menús estándar ni la generación de la cabecera.

Figura 76. Ejemplo de cabecera, con sus menús y disclaimer



[Inicio] [Herramientas] [Herramientas avanzadas] [Ayuda] [Arriba]
© 2003 Meta4 Spain, S.A.
Todos los derechos reservados.

El proceso es el siguiente:

- Generar obligatoriamente el fichero `shco_gen_menusup.jsp` en el directorio `shco_g0` asociado al producto `shco_g0_approd`. Esta página es la encargada de imprimir la cabecera y generar los menús dinámicamente. Aquí debe desarrollar su nueva generación de menús. Desde la página tiene accesibles a través de los atributos del request los siguientes datos:

```
//Menu Level
String zl_MenuLevel = (String)request.getAttribute("znivelmenu");
//Track info
String zl_Track = (String)request.getAttribute("TRACK");
//Application Product
String zl_AppProd = (String)request.getAttribute("_APP_PROD");
//Login Url
```

```
String zl_LoginUrl = (String)request.getAttribute("LOGIN_URL");
```

- Opcionalmente puede crear la página shco_gen_menus_load.jsp en el directorio shco_g0 asociado al producto shco_g0_approd, y definir aquí la generación de los menús. Esta página se invoca en la carga del portal y no en la carga de cada página como el shco_gen_menusup.jsp. Así, se optimiza la generación de los menús que se hace una única vez.

Debe tener en cuenta que si parametriza la generación de los menús, en el disclaimer estándar sólo se generan los enlaces Ayuda y Arriba. No genera el resto de enlaces al nivel 1 de menús. Si desea que en el disclaimer aparezcan los enlaces a los menús que ha parametrizado vea el apartado Disclaimer.

Figura 77. Disclaimer estándar con menús localizados por producto

[Ayuda] [Arriba]
© 2003Meta4 Spain, S.A.
Todos los derechos reservados.

Inclusión de más información en la cabecera

A parte de la información que figura en la cabecera estándar puede necesitar incluir otro tipo de datos adicionales. La posición en la que se ha previsto añadir esta nueva información es a continuación de la cabecera estándar. Y la inclusión se hará normalmente en forma de tabla.

La página de generación de cabecera es shco_g0/shco_gen_menusup.jsp. Esta página permite incluir por producto un fichero shco_gen_menusup2.jsp en el que se puede añadir información adicional.

El proceso es el siguiente:

- En el directorio del producto shco_g0_approd genere el fichero shco_gen_menusup2.jsp. En este fichero codifique la información que desea que aparezca. Tenga en cuenta que dispone de los siguientes datos accesibles a través de los atributos del request:

```
<%String zperson = (String) request.getAttribute("zperson");  
String zappprod = (String) request.getAttribute("zappprod");  
String zlanguser = (String) request.getAttribute("zlanguser");%>
```

Es importante que la nueva información no se solape con las siguientes divisiones de la página. Si la nueva tabla o división que dibuja ocupa un espacio tal que la siguiente división de la página requiere un top superior a 85 pxls debe establecerlo en el parámetro CAPA_CUERPO_DIV_TOP del request. Se trata en definitiva de indicar con exactitud dónde comienza a pintarse la nueva información. Si no se indica nada en este parámetro se toma por defecto el valor 85.

Para la nueva tabla que se añade se recomienda utilizar la clase `menusup_prod` definida en la hoja de estilo.

EJEMPLO

El producto ESS quiere mostrar la hora y el día debajo de la cabecera estándar. El proceso es el siguiente:

- Genere el archivo `shco_gen_menusup2.jsp` en el directorio `shco_g0_ess` con el siguiente contenido:

```
<%String zperson = (String) request.getAttribute("zperson");
String zappprod = (String) request.getAttribute("zappprod");
String zlanguser = (String) request.getAttribute("zlanguser");%>

<%@ include file="/shco_g0/shco_gen_js.jsp" %>
<div id="capa_menu" style="position:absolute; left:1%; top:90px;
width:99%; z-index:2">
<table width="100%" class="menusup_prod" >
<tr><td class="menusup_prod"><script type="text/
javascript">document.write(m4today())</script></td>
</tr>
</table>
</div>

<%request.setAttribute("CAPA_CUERPO_DIV_TOP", "140");%>
```

Como se observa en el fichero se utiliza la clase `menusup_prod` y además, se indica que la nueva información comience a pintarse en el pixel 140 en lugar del 85 definido por defecto.

Modificación de los botones de la cabecera

El Development Framework permite modificar el fichero `scho_g0_approd/shco_gen_topmenu.jsp` para modificar parte de la cabecera por producto. En este fichero puede modificar los enlaces que aparecen en la parte derecha de la cabecera. Esta posibilidad junto con la modificación de imágenes detallada en el apartado **Iconos**, le permite cambiar el tamaño de los botones.

Disclaimer

En el disclaimer estándar cuando se ha localizado la generación de menús sólo se generan los enlaces **Arriba** y **Ayuda**. Nunca se generan los enlaces de opciones de menú puesto que al estar localizados no se tiene información de dónde se encuentran. Más información, [La generación de los menús](#).

Para localizar el disclaimer para un producto genere el fichero

shco_gen_disclaimer.jsp en el directorio shco_g0 asociado al producto shco_g0_appprod. En esta página están accesibles a través de los atributos del request los siguientes datos:

```
//Localize help
String zl_LocalizeHelp =
(String)request.getAttribute("zsLocalizeHelp");
//Help folder
String zl_HelpFolder = (String)request.getAttribute("zsHelpFolder");
```

Escriba el código para su disclaimer. Puede tomar como ejemplo el shco_g0/shco_gen_disclaimer.jsp.

Ayuda en línea

Si usted desea utilizar un sistema de ayuda diferente en un producto, puede hacerlo. Los elementos que tendría que modificar son los siguientes:

- Para el link de Ayuda del login tendría que modificar el fichero shco_gen_login.jsp en el directorio shco_g0 asociado al producto shco_g0_appprod.
- Para localizar la ayuda en cada una de las páginas de un producto, tendría que generar el fichero shco_gen_help.jsp en el directorio shco_g0 asociado al producto shco_g0_appprod. Esta página de forma estándar se encarga de generar el botón de ayuda. Introduzca su nuevo código. Tenga en cuenta que en esta página se tendrán accesibles a través de atributos del request los siguientes datos por si los necesita en su desarrollo:

```
//Localize help
String zl_LocalizeHelp =
(String)request.getAttribute("zsLocalizeHelp");
//Help folder
String zl_HelpFolder =
(String)request.getAttribute("zsHelpFolder");
//zhhelp
String zl_Help = (String)request.getAttribute("zhhelp");
//Lang Folder
String zl_LangFolder =
(String)request.getAttribute("zLangFolder");
```

Asistentes: estándar y de procesos

En un asistente puede:

- Añadir un icono específico para un producto en los pasos de los asistentes, en la posición anterior a los enlaces de los pasos.

Figura 78. Icono en pasos



- Modificar los iconos estándar de las acciones de los asistentes. Para este proceso vea el apartado Iconos.

Para parametrizar estos ficheros el proceso es el siguiente:

- Copie los ficheros files_gif/ic_wz_menu.jsp para los asistentes normales y el fichero files_gif/ic_p_wz_menu.jsp para los asistentes de procesos en el directorio files_gif_approd (approd=identificación del producto).

El contenido de estos ficheros debe incluir todo el código html de una imagen con la etiqueta img:

Además puede parametrizar la hoja de estilo para cambiar la clase wzimage. Por ejemplo, puede querer cambiar la alineación de la imagen en la tabla. Para este proceso consulte Hojas de estilo:

```
Shco_gen_css_100.jsp
TABLE.wizard TABLE.subwizard TD.wzimage
{ TEXT-ALIGN: center}
```

En esta página tiene accesibles a través de los atributos del request los siguientes datos:

```
User's css
String zcssuser = (String) request.getAttribute("zcssuser");
```

EJEMPLO

En el producto ESS se añade el icono noname_recibo_72_125 delante de los pasos de los asistentes. Para ello se genera en Files_gif_ess el fichero ic_p_wz_menu.jsp con el siguiente contenido:

```
<% String zcssuser = (String) request.getAttribute("zcssuser");%>

```

Uso más frecuente

Esta funcionalidad se utiliza cuando es necesario compartir componentes entre varios productos. La idea es poder tener una serie de componentes que se utilizan en varios productos y que dependiendo del producto que llame a estos componentes su apariencia y funcionalidad más básica, como puede ser la ayuda en línea y la gestión de los menús, sea diferente.

Un ejemplo claro es el módulo Training compartido para los productos MSS y PeopleNet. Si el módulo se llama desde PeopleNet el módulo sigue todas las especificaciones estándar del Development Framework. Pero si se llama desde el MSS, el aspecto, la generación de los menús y la ayuda en línea pueden parametrizarse para que se adapte al entorno del MSS.

Los requisitos que se deben cumplir para poder utilizar esta funcionalidad son los siguientes:

- El módulo compartido tiene que estar desarrollado con las especificaciones del Development Framework. En el ejemplo Training sigue los estándares.
- Los archivos del módulo Training que varíen entre los productos tienen que estar preparados para que se puedan parametrizar fuera de las directrices generales del Development Framework tal y como se indica en Desarrollo de componentes modificables. El objetivo es que puedan ser utilizados por los productos que no siguen el Development Framework.
- Los productos principales que comparten recursos tienen que tener identificado en el login el producto del que se trata o en el fichero configclient.xml. Más información, Identificación del producto. En el ejemplo, MSS y PeopleNet.
- Los recursos compartidos y el producto desde el que se accede a éstos, se ejecutan en la misma estructura de webservices, es decir, en el mismo JRUN, en el mismo servidor de aplicaciones, en el mismo repositorio.
- La invocación a los recursos compartidos desde un producto que no sigue el Development Framework, se hace a través del portal utilizando el atributo REDIRECT que toma el valor de la página a la que se quiere acceder. El objetivo es que la aplicación no estándar, al acceder por el portal, cuente con toda información que va a necesitar en el nuevo entorno. Este requisito implica que se debe diseñar una página de portal para inicializar todo el entorno aunque nunca se utilice para visualizarse. El portal debe seguir los requisitos especificados en Configuración de un portal para un producto Web. La llamada a la página shco_tr_list_courses.jsp del módulo Training desde el MSS sería de este tipo:

```
http://servidor:puerto/servlet/CheckSecurity/JSP/shco_g0_pn/  
shco_gen_portal.jsp?_REDIRECT=/servlet/CheckSecurity/JSP/shco_tr/  
shco_tr_list_courses.jsp
```

PARTE III

APÉNDICES

Los capítulos de esta parte no son imprescindibles para el desarrollo en cliente Web, pero sí aportan información adicional importante para casuísticas concretas.

Apéndice: Librerías API Javascript



Acerca de este capítulo

Este capítulo describe las funciones javascript desarrolladas en Meta4.

El objetivo de estas funciones es centralizar la funcionalidad javascript que es utilizada desde la mayor parte de las páginas. Así, se facilita el mantenimiento de las mismas.

Todo desarrollador de páginas jsp que tenga que escribir código javascript tendrá que utilizar estas funciones. En caso de necesitar una función que no esté implementada deberá consultar con el responsable de estas librerías antes de proceder a escribir su propia función. En estos casos, se estudiará si se trata de una funcionalidad genérica o específica de la página para incluirla o no dentro de las librerías de Meta4. Además, siempre se documentará debidamente.

Para facilitar este desarrollo de nuevas funciones se detallan también algunas funciones de uso interno.

Estas funciones se agrupan en las siguientes librerías:

Librería	Descripción del contenido
m4duallist.js	Funciones para trabajar con listas duales.
m4gen.js	Funciones genéricas
m4gen_excep.js	Funciones gestión de errores y excepciones.
m4gen_list.js	Funciones a usar en las páginas plantilla de lista.
m4gen_list_ck.js	Funciones a usar en páginas plantillas de lista con <i>checkbox</i> .
m4gen_mt.js	Funciones a usar en las páginas plantilla de mantenimiento.
m4gen_wz.js	Funciones a usar en las páginas plantilla de wizard (asistente).
m4gen_param_wz.js	Funciones usadas en los wizard de proceso
m4val.js	Funciones de validación.

A continuación se detallan todas estas funciones. Para cada una se detalla:

- Sintaxis: En cada una de ellas, se utilizan [..] para recoger aquellos argumentos opcionales y los (..) los obligatorios.
- Descripción
- Parámetros

Manipulación de objetos Javascript

A continuación se detallan aquellas funciones relacionadas con objetos JS. Todas ellas recogidas en la librería m4gen.js:

- Funciones genéricas
- Objeto <INPUT Type=RADIO>
- Objeto <SELECT>

Funciones genéricas

m4clearform

m4clearform(sldform [,sldInput₁.., sldInput_n])

Descripción

Esta función limpia el contenido de los controles (input type=text, input type=checkbox, select, textarea) del formulario identificado por *sldform*. Exceptuando aquellos cuyo identificador se pase por parámetro *sldInputi*.

Parámetros

Parámetro	Tipo	Descripción
sldForm	Cadena	Identificador de formulario
sldInput _i	Cadena	Identificador de los <i>inputs</i> que no se quiere limpiar

m4elemento

m4elemento(sldElemento)

Descripción

Esta función devuelve el objeto javascript contenido en *document* cuya propiedad id sea *sldElemento*.

Parámetros

Parámetro	Tipo	Descripción
sldElemento	Cadena	Identificador del elemento a devolver

Se utiliza para el desarrollo de nuevas funciones.

m4focus

m4focus(sIdForm,sIdInput)

Descripción

Esta función posiciona el foco de la ventana en el input con identificador *sIdInput* del formulario con identificador *sIdForm*.

Parámetros

Parámetro	Tipo	Descripción
sIdForm	Cadena	Identificador de formulario.
sIdInput	Cadena	Identificador del <i>input</i> .

m4help

m4help(sLangFolder[,sHelpFile])

Descripción

Función para invocar a la ayuda en un determinado idioma. En el caso de que venga el parámetro *sHelpFile*, la ayuda se muestra posicionada en la página que se indica, sino sale la página inicial.

Parámetros

Parámetro	Tipo	Descripción
sLangFolder	Cadena	Identifica el idioma en el que levantar la ayuda, dando el nombre del directorio asociado a cada idioma: english, espanol, francais.
sHelpFolder	Cadena	Identifica el nombre de la carpeta donde se encuentra la estructura de la ayuda. Por defecto <i>help</i>
sHelpFile	Cadena	Página de la ayuda en la que posicionarse.

EJEMPLO

```
<a title="<%=Tran_shco_g0.getProperty("Menu.Help")%"
href="javascript:m4help('<%=zLangFolder%>', '<%=zsLocalizeHelp%>');">[
<%=Tran_shco_g0.getProperty("Menu.Help")%>]</a>
```

m4lock

m4lock(sIdForm,sIdObject,sLockMode, sLockUnlockMode [, sClassName])

Descripción

Función que bloquea/desbloquea el objeto identificado por *sIdObject* dentro del formulario *sIdForm*.

Si viene el parámetro *sClassName*, lo utiliza para cambiar la clase de estilo del objeto que se bloquea/desbloquea.

Casos Especiales

a. Anchors <A>. Para poder bloquear un anchor:

1. Debe tener id y name
2. El identificador debe ser único en todo el documento, dado que estos elementos están almacenados a nivel de documento y no a nivel de form. Es decir, este id no puede estar repetido incluso cuando pertenezca a distintos forms.
3. Necesita modificar su propiedad href o onclick. El m4lock únicamente le cambia su aspecto pues el *disabled* no impide el hacer clic y por consiguiente no lo bloquea. Para conseguir el efecto usar en el href un código del estilo:

```
<a id = "anchor1" name = "anchor1" href="javascript:if(bmodelock
== true) {m4nothing();}else{m4calendar(...);}" >...</a>
```

```
<a id = "anchor1" name = "anchor1" onclick
="javascript:if(bmodelock == true) {return false;} " >...</a>
```

La variable de indicador de bloqueo (*bmodelock* en el ejemplo) será gestionada individualmente en las páginas.

b. INPUT file: No se permite la accesibilidad al botón. Por tanto no hay forma de cambiar el color.

Parámetros

Parámetro	Tipo	Descripción
sldForm	Cadena	Identificador del formulario.
sldObject	Cadena	Identificador del objeto a bloquear.
SLockMode	Cadena	Forma de bloqueo ("disabled","readOnly"). Tener en cuenta que el modo readOnly sólo es válido para INPUT TEXT y TEXTAREA. El resto de objetos al tener que bloquearlos utilizando la propiedad <i>disabled</i> no se pasan en el <i>submit form</i> , por tanto, si se quiere que se pase el valor de estos objetos hay que hacer una gestión aparte.
SlockUnlockMode	Cadena	Indicador de bloque o desbloqueo ("lock","unlock")
SClassName	Cadena	Identificador de la clase de estilo a establecer sobre el elemento bloqueado/desbloqueado. Debe aparecer en el archivo CSS correspondiente.

EJEMPLO

```
<input type="hidden" id="cssinput" value="disabled" class="disabled">
m4lock('NombreFormulario','btn1','disabled','lock','cssinput ');
m4lock('NombreFormulario','btn1','disabled','unlock');
<a id = "anchor1" name = "anchor1" href="javascript:if(bmodelock ==
true) {m4nothing();}else{m4calendar(...);}" >...</a>
```

m4markobject

m4markobject(sldObject₁[,...,sldObject_n])

Descripción

Función que marca los objetos que se pasan por argumento. Establece el background del objeto a amarillo. El proceso de validaciones usa este sistema para indicar aquellos input que no pasan las validaciones correspondientes. El color no se saca como argumento para mantener la homogeneidad en la aplicación.

Parámetros

Parámetro	Tipo	Descripción
sIdObject _i	Objeto	Objeto javascript a marcar

EJEMPLO

```
<td><input type="button" id="btn11" name="btn11"
value="m4markobject" onclick="m4markobject(m4objeto('NombreFor-
mulario','txttime2'));" />
```

m4pageparams

m4pageparams(zIdXML,swidth,sheight)

Descripción

Función que permite, dado un identificador de configuración de página de procesos, mostrar una página con la petición de parámetros. Se utiliza por ejemplo para ejecutar un informe. Opcionalmente se puede indicar las dimensiones de esta página.

Parámetros

Parámetro	Tipo	Descripción
zIdXML		Código identificativo de la petición de parámetros
swidth		Anchura de la página en pixels
sheight _i		Altura de la página en pixels

EJEMPLO

```
<input id="" name="" type="button" value ="ZZ_TEST_PERSONAS_XML"
onclick="m4pageparams('ZZ_TEST_PERSONAS_XML')"/>
```

m4nothing

m4nothing()

Descripción

Función que no hace “nada”. Es interesante en algunos casos, como por ejemplo, en las anclas que envuelven a las imágenes, pues en estos casos no queremos navegar sino que queremos ejecutar código del onclick.

EJEMPLO

```
<td class="boton<%=zpos%>"><a title="<m4:label
m4name="<%=zSHCOLBEDIT%>" htmlsafe="true"/
>"href="javascript:m4nothing();"onClick="m4valor('NombreFormulario','
X<%=zcampoID1%>','<m4:item m4name="<%=zIdPk1%>" jsafe="true"
htmlsafe="true"/>','set');m4edit();return false"><img alt="<m4:label
m4name="<%=zSHCOLBEDIT%>" htmlsafe="true"/>" <%@ include file="../../../
files_gif/ic_mod.jsp" %> /></a></td>
```

m4objeto

m4objeto(sidForm,sidElemento)

Descripción

Esta función devuelve el objeto javascript con identificador *sidelemento* del formulario con identificador *sidform*.

Parámetros

Parámetro	Tipo	Descripción
sidForm	Cadena	Identificador del formulario
sidElemento	Cadena	Identificador del elemento

EJEMPLO

```
...
href="javascript:m4calendar(m4objeto('NombreFormulario','STD_DT_BIRTH
'),'calculate_age()')" ..
```

m4prop

m4prop(sIdForm,sIdObject,sIdAttribute,vValue,sMode [,bstyleattribute])

Descripción

Función que establece (sMode ="set") o devuelve (sMode ="get") el valor de la propiedad *sIdAttribute*, del objeto identificado por *sIdObject* dentro del formulario *sIdForm*.

Para el caso de las propiedades de tipo estilo hay que pasar como *bstyleattribute* el valor true. Este parámetro es opcional para el resto de casos.

En el modo 'set' si hay varios controles con el mismo id (sidobject), caso p.e de los radio *button* establece la propiedad para todos ellos. En el modo 'get' para los radio *button* se devuelve el valor del radio *button checked*.



Para las propiedades tipo *readOnly*, *disabled*, *checked* que en HTML se activan si aparecen, para ponerlas a *false* hay que pasar el valor "" (cualquier otro valor no desactiva la propiedad).

Parámetros

Parámetro	Tipo	Descripción
SIdForm	Cadena	Identificador del formulario
SIdObject	Cadena	Identificador del objeto
SIdAttribute	Cadena	Identificador de la propiedad
VValue		Valor a establecer sobre la propiedad. Sólo para el caso de sMode='set' sino poner a vacío (")
SMode	Cadena	'set': Para establecer valor a la propiedad 'get':para recoger el valor de la propiedad
Bstyleattribute	booleano	Indica si la propiedad es de estilo o no. Son de estilo todo lo referente a color.. (Defecto:false)

Parámetros

Parámetro	Tipo	Descripción
sIdForm	Cadena	Identificador de formulario a enviar
sTarget	Cadena	Valor del target a asignar.

m4rewritecell

m4rewritecell(sIdCell,sText)

Descripción

Función que dado el identificador de una celda (TD con identificador único), escribe en la celda el valor pasado por argumento.

Parámetros

Parámetro	Tipo	Descripción
sIdCell	Cadena	Identificador de la celda (TD)
sText	Cadena	Texto a escribir en la celda.

EJEMPLO

```
m4rewritecell('tdfin','*');
```

m4settarget

m4settarget(sIdForm,sTarget)

Descripción

Esta función asigna al atributo *target* del formulario *sIdForm* el valor *sTarget*. Esto es útil cuando se quiere que el *submit* de un formulario sobre una ventana concreta cuyo identificador se pasará como valor del *target*.

Parámetros

Parámetro	Tipo	Descripción
sIdForm	Cadena	Identificador de formulario a enviar
sTarget	Cadena	Valor del target a asignar.

m4textodentrotd

m4textodentrotd(sIdTD,bSetValue,sTextValue)

Descripción

Esta función:

-
- Si `bSetValue = true`, genera un nodo de texto (siguiendo nomenclatura DOM, con `texto=valor`) dentro del objeto "td" con identificador `sIdTD` y valor `sTextValue`.
 - En caso contrario, `bSetValue=false`, devuelve el contenido del nodo de texto del objeto TD siempre que éste exista.

m4submit

`m4submit(sIdForm)`

Descripción

Esta función realiza un submit del formulario identificado por *sIdForm*.

Parámetros

Parámetro	Tipo	Descripción
<code>sIdForm</code>	Cadena	Identificador de formulario a submitir.

EJEMPLO

```
m4submit('NombreFormulario');
```

m4tabfocus

`m4tabfocus(sIdForm,ntabindex)`

Descripción

Función que sitúa el foco en el elemento cuyo valor del atributo `tabindex` es igual al dado en el parámetro *ntabindex* dentro del formulario *sIdForm*.

Parámetros

Parámetro	Tipo	Descripción
<code>sIdForm</code>	Cadena	Identificador del formulario.
<code>ntabindex</code>	entero	Índice de tabulador donde establecer el foco.

m4titulo()

m4titulo()

Descripción

Función que devuelve el contenido de la etiqueta title de la ventana actual.

m4url

m4url()

Descripción

Función que devuelve la URL de la página actual.

m4valor

m4valor(sIdForm,sIdInput,vValue, sMode)

Descripción

Esta función pone (sMode ="set") o recoge (sMode ="get") un valor de un input identificado por *sIdForm* y *sIdInput*.

Si el sMode ="set" el parámetro *vValue* contiene el valor que se desea introducir en la propiedad value del input.

Si el objeto input es de un objeto javascript de tipo radio button, y hay varios objetos con el mismo identificador, se devuelve/establece el valor del/al chequeado.



Para recoger/establecer valores a un objetos select usar preferiblemente las funciona m4select, m4searchoption. Dado que una select no es un objeto simple sino un array de objetos que son los options que lo componen.

Parámetros

Parámetro	Tipo	Descripción
SidForm	Cadena	Identificador del formulario.
SidInput	Cadena	Identificador del input.
Vvalue		Valor a introducir a establecer en el objeto en caso de modo 'set', sino hay que pasarlo a vacío. (")
Smode	Cadena	'Set': Para establecer valor al input 'get':Para recoger el valor del input

Objeto <INPUT Type=RADIO> (m4gen.js)

m4checkradio

m4checkradio(sIdForm,sIdInput,vValue)

Descripción

Se encarga de poner a checked el radio button *sIdInput* con valor *vValue* del formulario *sIdForm*.

Parámetros

Parámetro	Tipo	Descripción
sIdForm	Cadena	Identificador del formulario
sIdInput	Cadena	Identificador del radio button
vValue	Cadena	Valor del radio button a chequear

Objeto <SELECT> (m4gen.js)

m4genoption

m4genoption(objSelect,sId,sValue,sText)

Descripción

Función que crea un objeto "option" dentro del objeto select *objSelect* con valores para las propiedades Id = *sId* , Value = *sValue* y Text = *sText*.

Parámetros

Parámetro	Tipo	Descripción
ObjSelect	Objeto select	Objeto select. Puede ser recogido usando la función m4objeto.
sId	Cadena	Valor para la propiedad id.
sValue	Cadena	Valor para la propiedad Value.
sText	Cadena	Valor para la propiedad Text.

EJEMPLO

```
<td><input type="button" VALUE="genoption"
onclick="m4genoption(m4objeto('NombreFormulario','sel1'),'4','4','4')
;"></td>
```

m4searchoption

m4searchoption(objSelect,sIdOption)

Descripción

Función que busca y posiciona el objeto select *objSelect* en la opción con identificador *sIdOption*.

Parámetros

Parámetro	Tipo	Descripción
ObjSelect	Objeto select	Objeto select. Puede ser recogido usando la función m4objeto.

Parámetro	Tipo	Descripción
SidOption	Cadena	Identificador de la opción donde hay que posicionar la select.

EJEMPLO

```
m4searchoption(m4objeto('NombreFormulario','idmoneda'),'<%=zsselidcurr%>');
```

m4select

- m4select(sIdForm,sIdSelect,sMode)
- m4select(objSelect,sMode)

Descripción

Esta función devuelve el valor de la propiedad *id*, *text* o *value* del elemento seleccionado dentro de un objeto select.

En el primer modo de invocación a la función, el objeto select se obtiene vía su identificador *sIdSelect* y el del formulario al que pertenece (*sIdForm*). En el segundo modo de invocación el objeto select se pasa directamente como parámetro (*objSelect*). Si no existe un elemento seleccionado dentro del objeto select se devuelve un nulo.

Parámetros

Parámetro	Tipo	Descripción
SidForm	Cadena	Identificador del formulario. (Sólo en modo de invocación 1)
SidSelect	Cadena	Identificador del objeto select. (Sólo en modo de invocación 1)
ObjSelect	Objeto select	Objeto select. Puede ser recogido usando la función m4objeto. (Sólo en modo de invocación 2))
Smode	Cadena	Indica la propiedad del elemento seleccionado a devolver. Los valores que puede tomar son: "id", "text", "value"

EJEMPLO

```
m4valor("oculto","zIdB",m4select(m4objeto("NombreFormulario","SHCO_END_REASON2"),"value"),"set");
```

```
m4valor("oculto","zIdB",m4select("NombreFormulario","SHCO_END_REASON2","value"),"set");
```

Manipulación de datos

Cadenas (m4val.js)

m4ltrim

m4ltrim(sText)

Descripción

Función que dada una cadena *sText*, devuelve la misma cadena quitándoles los blancos que aparezcan al principio de la misma.

Parámetros

Parámetro	Tipo	Descripción
sText	Cadena	Cadena a formatear

m4rtrim

m4rtrim(sText)

Descripción

Función que dada una cadena *sText*, devuelve la misma cadena quitándoles los blancos que aparezcan al final de la misma.

Parámetros

Parámetro	Tipo	Descripción
sText	Cadena	Cadena a formatear

m4trim

m4trim(sText)

Descripción

Función que dada una cadena *sText*, devuelve la misma cadena quitándoles los blancos que aparezcan al principio y al final de la misma.

Parámetros

Parámetro	Tipo	Descripción
sText	Cadena	Cadena a formatear

Números (m4gen.js)

m4parseInt

m4parseInt(sValue[,iBase,bValueWithUserFormat,bCurrencyType])

Descripción

Función que convierte el *string* a un entero en la base que se le indique. Por defecto, la conversión es a valor decimal, es decir, en base 10. No conlleva posibles formateos.

Si al valor que devuelve se le quieren aplicar los formatos de números y monedas del usuario hay que utilizar la función m4formatNum.

Por defecto, parte de que el valor que le llega es un valor javascript, con lo cual nunca tiene separadores de dígitos y el separador decimal es el punto.

En el caso de que el valor venga tenga aplicados los formatos del usuario hay que indicarlo mediante *bValueWithUserFormat* = *true* y si, además, el valor hace referencia a un moneda indicarlo con *bCurrencyType* = *true*.



Es importante no usar directamente el parseInt y hay que asegurarse que la base sea 10, porque si el valor viene con los formatos del usuario para número/moneda la función parseInt no va a funcionar correctamente.

Parámetros

Parámetro	Tipo	Descripción
SValue	Cadena	Cadena a convertir a entero.
IBase	Entero	Base en la que convertir el entero (10: decimal, 8:Octal, 16:Hexadecimal) Defecto: 10.
bValueWithUserFormat	Booleano	Indica si es un valor con los formatos de moneda/número del usuario aplicados. Por defecto: <i>false</i> .

Parámetro	Tipo	Descripción
bCurrencyType	Booleano	Indica que es un valor de moneda. Es para detectar en ese caso si lleva o no separador digital. Por defecto: <i>false</i> .

EJEMPLO

```
var ozIndexWizard = m4parseInt('<%=zIndexWizard%>');
var iVal =
m4parseInt(m4valor('NombreFormulario','txt1','','get'),'true');
```

m4parseFloat

m4parseFloat(sValue [,bValueWithUserFormat,bCurrencyType])

Descripción

Función que formatea un número decimal usando la función *parseFloat* de JavaScript. El valor devuelto es un valor decimal en JavaScript. Si al valor que se devuelve se le quieren aplicar los formatos de números/monedas del usuario utilizar la función *m4formatNum*.

Por defecto, parte de que el valor que le llega es un valor JavaScript con lo que nunca tiene separadores de dígitos y el separador decimal es el punto.

En el caso de que el valor tenga aplicados los formatos del usuario hay que indicarlo mediante *bValueWithUserFormat = true* y si además el valor hace referencia a una moneda indicarlo con *bCurrencyType = true*.



Es importante no usar directamente el *parseFloat* porque si el valor viene con los formatos del usuario para número/moneda aplicados la función *parseFloat* no va a funcionar correctamente.

Parámetros

Parámetro	Tipo	Descripción
SValue	Cadena	Cadena a convertir a decimal.
bValueWithUserFormat	Booleano	Indica si tiene los formatos de número/moneda del usuario aplicados. Por defecto: <i>false</i> .

Parámetro	Tipo	Descripción
bCurrencyType	Booleano	Indica que es un valor moneda. Es para detectar en ese caso si lleva o no separador digital. Por defecto: <i>false</i> .

EJEMPLO

```
var sValue2 = m4parseFloat('<%=zValue%>');
var sValue2 =
m4parseFloat(m4valor('NombreFormulario','txt1','','get'),true);
```

m4formatNum

m4formatNum(sValue[,bValueWithUserFormat,bCurrencyType,iNumDecAllowed])

Descripción

Función que formatea un número (entero o decimal) de la siguiente forma:

- Quita los ceros no significativos tanto de la parte entera como de la decimal.

EJEMPLO

```
01      -----> 1
000     -----> 0
0.10    -----> 0.1
0.0     -----> 0
-0      -----> 0
```

- Establece los separadores de dígitos en los miles y elimina los que sobren o no estén correctamente situados.



En los ejemplos se ha supuesto el punto(.) como separador decimal y la coma(,) como para dígitos.

- Para los números de tipo moneda (`bcurrencyType = true`) cuando hay establecidos para el usuario un número de decimales fijos para moneda en conjunción con el parámetro `iNumDecAllowed`, que indicará el número máximo de decimales para la moneda que se está tratando, se rellena con ceros.

Por defecto, parte de que el valor que le llega es un valor javascript, con lo cual nunca tiene separadores de dígitos y el separador decimal es el punto. En el caso de que el valor venga tenga aplicados los formatos del usuario hay que indicarlo mediante `bValueWithUserFormat = true` y si además el valor hace referencia a un moneda indicarlo con `bCurrencyType = true`.

Esta función se usa en las validaciones de tipo `_num`, `_int`, `_decimal`, `_int_decimal`, `_currency`.

Parámetros

Parámetro	Tipo	Descripción
SValur	Cadena	Cadena a convertir a entero
bValueWithUserFormat	Booleano	Indica si es un valor con los formatos de moneda/número del usuario aplicados. Por defecto: <i>false</i> .
bCurrencyType	Booleano	Indica que es un valor de moneda, es para detectar en ese caso si lleva o no separador digital. Por defecto: <i>false</i> .
iNumDecAlloweb	Entero	Número máximo de decimales permitidos. Por defecto: número decimales moneda.

EJEMPLO

```
var sformatnum = m4formatNum(svalue);  
var sformatnum =  
m4formatNum(m4valor('NombreFormulario','txt1','','get'),true);
```

m4changeDecimalSepToPoint

m4changeDecimalSepToPoint (sValue ,[bCurrencyType])

Descripción

Función para cambiar el separador decimal de un valor, el cual depende de la parametrización del usuario y si es o no de moneda (`bCurrencyType`), por el separador de punto.



Esto es necesario cuando se va a trabajar con operadores numéricos de javascript.

Parámetros

Parámetro	Tipo	Descripción
SValue	Cadena	Cadena a convertir.
BCurrencyType	Booleano	<i>True</i> : Indica que es un valor de moneda, es para detectar en ese caso si lleva o no separador digital.

m4changePointToDecimalSep

m4changePointToDecimalSep (sValue ,[bCurrencyType])

Descripción

Función para cambiar el punto de un valor decimal por el separador decimal, el cual depende de la parametrización del usuario y si es o no de moneda (*bCurrencyType*) .

Esto es necesario cuando se va a mostrar un valor numérico que viene como resultado de trabajar con operadores de JavaScript.

Parámetros

Parámetro	Tipo	Descripción
SValue	Cadena	Cadena a convertir.
BCurrencyType	Booleano	<i>True</i> : Indica que es un valor de moneda, es para detectar en ese caso si lleva o no separador digital.

m4takeoutsepdig

m4takeoutsepdig (sValue ,[bCurrencyType])

Descripción

Función para quitar el/los separador/es de dígito/s de un valor. Dicho separador depende de la parametrización del usuario y si es o no de moneda (*bCurrencyType*).

Esto es necesario cuando se va a trabajar con operaciones JavaScript sobre un número, dado que los separadores de dígitos no los considera JavaScript.

Para volver a establecerlos usar el `m4formatNum`.

Parámetros

Parámetros	Tipo	Descripción
SValue	Cadena	Cadena a convertir
BCurrencyType	Booleano	<i>True</i> : indica que es un valor de moneda, es para detectar en ese caso si lleva o no separador digital

Fechas/Horas (m4gen.js)

m4builtdate

`m4builtdate(iDay,iMonth,iYear)`

Descripción

Función que devuelve una fecha a partir del día, mes y año, en el formato del usuario.

Parámetros

Parámetro	Tipo	Descripción
Iday	Entero	Día
Imonth	Entero	Mes
Iyear	Entero	Año

EJEMPLO

```
var dhoy = new Date();
...
m4valor('NombreFormulario','fechadecambio',m4builtdate(dhoy.getDate(),
,dhoy.getMonth() + 1,dhoy.getFullYear()),'set');
```

m4buildtime

m4buildtime(sIdform,sIdInput[,bwithseconds])

Descripción

Función que recoge la hora introducida en el input *sIdInput* del formulario *sIdForm*, la formatea y la vuelve a establecer sobre el input.

El proceso de formateo es el siguiente:

1. Separa las horas, minutos y segundos considerando únicamente como válidos los separadores `:`, `.`. Si hay errores devuelve todos los valores a cero (00:00 / 00:00:00).
2. Usa la función `m4formatime` para crear la cadena considerando o desechando los segundos en función del parámetro *bwithseconds*.



La función no comprueba que la hora introducida sea válida. Hay que pasar antes por `m4valinput(_time / m4valinput(_time_sec..))`.

Parámetros

Parámetro	Tipo	Descripción
sIdform	Cadena	Identificador del formulario.
sIdInput	Cadena	Identificador del input que contiene la hora.
bwithseconds	Boolean	Indica si hay que añadir o no los segundos en la cadena que se devuelve.

m4daydiff

m4daydiff(sInitDate,sLastDate)

Descripción

Función que dadas dos fechas en formato de usuario te devuelve la diferencia en días.

Parámetros

Parámetro	Tipo	Descripción
sInitDate	Cadena	Fecha inicio en formato de usuario.
sLastDate	Cadena	Fecha fin en formato de usuario.

m4formattime

m4formattime(sHours,sMinutes[,sSeconds])

Descripción

Función que devuelve una cadena en formato hh:mm:ss, o hh:mm a partir de las horas (*sHours*) , minutos (*sMinutes*), y si vienen los segundos (*sSeconds*).

Las horas van de 0-23.

Parámetros

Parámetro	Tipo	Descripción
sHours	Entero	Horas
sMinutes	Entero	Minutos.
sSeconds	Entero	Segundos.

m4getyear

m4getyear(sUserFormatDate)

Descripción

Función que devuelve un entero con el año que aparece en la fecha *sUserFormatDate* . Esta fecha debe venir en formato del usuario.

Parámetros

Parámetro	Tipo	Descripción
SUserFormatDate	Cadena	Fecha en formato de usuario.

m4ISOTimeStamp

m4ISOTimeStamp(objDate)

Descripción

Función que dado un objeto tipo fecha de javascript devuelve un string con la fecha y hora en formato ISO.

Parámetros

Parámetro	Tipo	Descripción
ObjDate	Objeto fecha	Objeto fecha de javascript.

m4now

m4now

Descripción

Función que devuelve una cadena con la hora actual en formato hh:mm:ss. (Uso de la función m4formattime).

EJEMPLO

```
m4valor('NombreFormulario','hour',m4now(),'set');
```

m4splitdate

m4splitdate(sUserFormatDate,adateinfo)

Descripción

Función que dada una fecha en formato usuario devuelve en el array adateinfo el día (adateinfo [0]) mes (adateinfo [1]) y año (adateinfo [2]) con tipo entero.

Adateinfo: parámetro por referencia, debe estar creado desde fuera de la forma: *adateinfo = new Array(3);*

Parámetros

Parámetro	Tipo	Descripción
sUserFormatDate	Cadena	Cadena con la fecha en formato usuario.
adateinfo	Array	Array con los datos del día mes y año. Todos los datos son de tipo entero.

m4AddDays

m4AddDays (sDate,sDays)

Descripción

Dada una fecha en formato usuario y un número de días se incrementa la fecha en los días indicados.

Parámetros

Parámetro	Tipo	Descripción
sDate	Cadena	Cadena con la fecha en formato usuario
sDays	Cadena	Cadena con los días de incremento

m4AddMonths

m4AddMonths(sDate,sMonths,sMode):

Descripción

Dada una fecha en formato usuario y un número de meses se incrementa la fecha en los meses indicados.

Parámetros

Parámetro	Tipo	Descripción
sDate	Cadena	Cadena con la fecha en formato usuario
sMonths	Cadena	Cadena con los meses de incremento
sMode	Cadena	Cadena con el comportamiento en diferentes lenguajes. Los posibles valores son: Oracle, LN4, JSP.

Funcionamiento LN4: si al sumar los meses, en la nueva fecha el día no existe

se queda en el último día del mes

```
31/10/2002 + 1 mes = 30/11/2002
30/11/2002 + 1 mes = 30/12/2002
31/12/2002 + 1 mes = 31/01/2003
31/01/2003 + 1 mes = 28/02/2003 (no es bisiesto)
31/01/2003 + 1 mes = 29/02/2003 (bisiesto)
28/02/2003 + 1 mes = 28/03/2003 (no es bisiesto)
28/02/2000 + 1 mes = 28/03/2003 (bisiesto)
29/02/2000 + 1 mes = 29/03/2003 (bisiesto)
```

Funcionamiento ORACLE: mantiene los últimos días de mes

```
31/10/2002 + 1 mes = 30/11/2002
30/11/2002 + 1 mes = 31/12/2002
31/12/2002 + 1 mes = 31/01/2003
31/01/2003 + 1 mes = 28/02/2003 (no es bisiesto)
31/01/2003 + 1 mes = 29/02/2003 (bisiesto)
28/02/2003 + 1 mes = 31/03/2003 (no es bisiesto)
28/02/2000 + 1 mes = 28/03/2003 (bisiesto)
29/02/2000 + 1 mes = 31/03/2003 (bisiesto)
```

Funcionamiento JAVASCRIPT: Si al sumar los meses, el día no existe, pasa al mes siguiente y suma los días

```
31/10/2002 + 1 mes = 01/12/2002
30/11/2002 + 1 mes = 30/12/2002
31/12/2002 + 1 mes = 31/01/2003
31/01/2003 + 1 mes = 03/03/2003 (no es bisiesto)
31/01/2003 + 1 mes = 02/03/2003 (bisiesto)
28/02/2003 + 1 mes = 28/03/2003 (no es bisiesto)
28/02/2000 + 1 mes = 28/03/2003 (bisiesto)
29/02/2000 + 1 mes = 29/03/2003 (bisiesto)
```

m4DaysOfMonth

m4DaysOfMonth (sDate)

Descripción

Dada una fecha en formato usuario devuelve los días correspondientes al mes de la fecha dada. (sDate: Cadena)

m4DaysOfMonthFromDate

m4DaysOfMonthFromDate(dDate):

Descripción

Dada una fecha devuelve los días correspondientes al mes de la fecha dada.
(dDate: Objeto Date de Javascript)

EJEMPLO

// Función para dada una fecha en formato usuario, un número de días y un número de meses, devuelve en formato usuario la fecha a la que se le han añadido los meses y días indicados.

```
function getnewdate(sDate,sNumDays,sNumMonths){  
    sDate = m4AddMonthsOracle(sDate,sNumMonths,"ORACLE");  
    sDate = m4AddDays(sDate,sNumDays);  
    return sDate;}
```

Invocación:

```
<td>  
<input type="text" id="txtNumDays" name="txtNumDays" value="0"/>  
    <input type="text" id="txtNumMonths" name="txtNumMonths"  
value="1"/>  
    <input type="button" value="getnewdate" onclick="var dDate =  
getnewdate(m4valor('NombreFormulario','txtDate','','get'),m4val  
or('NombreFormulario','txtNumDays','','get'),m4valor('NombreFor  
mulario','txtNumMonths','','get'));alert(dDate);"/>  
</td>
```

m4formatdatetoISO

m4formatdatetoIso (sUserFormatDate)

Descripción

Función que dada un fecha en formato usuario la devuelve en formato ISO, es decir, con formato yyyy-dd-MM.

Parámetros

Parámetro	Tipo	Descripción
sUserFormatDate	Cadena	Cadena con la fecha en formato usuario.

Validaciones

En este apartado se especifican las funciones a usar para validar los datos introducidos por el usuario.

Funciones de validación (m4val.js)

m4valinput (Validación de tipos)

m4valinput(sValType,sIdForm,sIdInput,buffer1,buffer2,Param₁...Param_n, [g_M4_NO_FORMAT])

Descripción

Esta función devuelve 1 si el valor contenido en el objeto input *sIdInput* del formulario *sIdForm* es del tipo *sValType*, y 0 en caso contrario.

No se tienen en cuenta los blancos que vienen al principio y al final en el valor. Además en el caso de que la validación sea correcta se modifica el valor del campo suprimiendo estos valores, excepto en la validación de tipo *_comentario*.

Si la validación es correcta para los tipos: *_int*, *int_oblig*, *_decimal*, *_decimal_oblig*, *_int_decimal*, *_int_decimal_oblig*, *_num*, *_num_oblig*, *_currency*, *_currency_oblig* cuando hay separador de dígitos o número de decimales fijos y no se indica explícitamente que no hay formateo (parámetro *g_M4_NO_FORMAT*) se procede al formateo del número y aplicación del mismo en el objeto que se valida. Ver función *m4formatNum*.

Los tipos de validación que llevan el sufijo *_oblig* (ver Tabla de Validaciones), aparte de validar el tipo de valor se valida como campo obligatorio, es decir, para que pase la validación el valor no puede ser vacío o contener sólo blancos. Para los números negativos (*_int*, *_int_decimal*) no es válido el -0.

Los separadores de miles y decimales no contabilizan en el número total de dígitos ni de la parte entera ni de la decimal. El separador de miles:

- es opcional y se permite en cualquier posición del número excepto al inicio.

- Ejemplo válidos con separador "," y que significan 1000 (1,000 1,,000 1,0,0,0 1000,,,))
- Ejemplos no válidos: ,1000

Parámetros

Parámetro	Tipo	Descripción
sValType	Cadena	Tipo de validación. Ver Tabla de Validaciones.
sIdForm	Cadena	Identificador del formulario donde se encuentra el input cuyo valor se quiere validar.
sIdInput	Cadena	Identificador del input cuyo valor se valida.
buffer1	Entero	Parámetro con distinto significado en función del tipo de validación. Ver Tabla de Validaciones.
buffer2		Parámetro con distinto significado en función del tipo de validación. Ver Tabla de Validaciones.
Param _i	Cadena	Datos para construir el mensaje de error en el caso de que el valor no pase la validación. El significado de cada parámetro depende el tipo de validación. Ver Ejemplos de uso. Normalmente el primer parámetro corresponde con el nombre del campo que se valida.
g_M4_NO_FORMAT	Variable JavaScript	Indica que para los casos <i>standard</i> de formateo no se formatee el valor y se deje tal cual.

Tabla de Validaciones

Tipo de validación	Descripción		
_com	Validación comparativa. Ver apartado de validaciones comparativas.		
_comentario	Valida una cadena de caracteres alfanuméricos de longitud máxima (buffer1)		
	Buffer1	Número	Longitud máxima de la cadena
	Buffer2	Cadena	Nombre del campo que se valida.

Tipo de validación	Descripción		
_cp (*) _cp_oblig	Valida un código postal.		
_currency _currency_oblig	Valida una cantidad tipo moneda con un máximo de (buffer1) dígitos en la parte decimal (la parte entera tendrá un máximo de 13 dígitos, el separador es el ".").		
	Buffer1	Número	Número máximo de dígitos en la parte entera.
	Buffer2	Cadena	Nombre del campo que se valida.
_date _date_oblig	Validación de una fecha en formato de usuario.		
	Buffer1	Número	No aplicable. Cualquier número.
	Buffer2	Cadena	Nombre del campo que se valida.
	Arg1		Variable javascript que contiene el formato de fechas de usuario : sformatofechas.
_decimal _decimal_oblig	Valida un número decimal con (buffer1) dígitos como máximo en la parte entera (al menos 1) y un máximo de (buffer2) dígitos en la parte decimal (pudiendo no tener decimales, en cuyo caso no debe tener tampoco separador decimal). El separador es ".".		
	Buffer1	Número	Número máximo de dígitos en la parte entera.
	Buffer2	Número	Número máximo de dígitos en la parte decimal
	Arg1	Cadena	Nombre del campo que se valida.
	Arg2	Número	Número máximo de dígitos en la parte entera, para sacar en el mensaje.
	Arg3	Número	Número máximo de dígitos en la parte decimal, para sacar en el mensaje
_int_decimal _int_decimal_oblig	Igual que _decimal pero permite números negativos		
_email _email_oblig	Valida una dirección e-mail		
_id _id_oblig	Valida un identificador. Cadena con letras y números. Las letras deben estar en mayúsculas.		

Tipo de validación	Descripción		
_interval	Valida que un valor esté dentro de un intervalo dado. Los valores del intervalo, para fechas va entre comilla simple y para numero tal cual sin comillas.		
	Buffer1	Cadena	Tipo de intervalo (_num_oblig, _decimal_oblig, date_oblig)
	Buffer2	Intervalo	Comienzo del intervalo. Si viene vacío (") equivale a -infinito.
	Param1	Intervalo	Fin del intervalo. Si viene vacío (") equivale a +infinito.
_telef (*) _telef_oblig	Valida un número telefónico.		
	Buffer1	Número	Longitud mínima de la cadena
	Buffer2	Cadena	Nombre del campo que se valida.
_nif (*) _nif_oblig	Valida un NIF		
	Buffer1	Número	Longitud mínima de la cadena
	Buffer2	Cadena	Nombre del campo que se valida.
_num _num_oblig	Valida una cadena de caracteres numéricos de longitud mínima (buffer1) NOTA: Usar buffer 1=0 para permitir cualquier longitud.		
	Buffer1	Número	Longitud mínima de la cadena
	Buffer2	Cadena	Nombre del campo que se valida
_alphanumeric _alphanumeric_oblig	Valida una cadena de caracteres alfanuméricos de longitud mínima (buffer1)		
	NOTA: Usar buffer1=0 para permitir cualquier longitud.		
	Buffer1	Número	Longitud mínima de la cadena
	Buffer2	Cadena	Nombre del campo que se valida.
_int _int_oblig	Igual que _num pero permite números negativos		
_oblig	Valida que el campo esté relleno.		
	Buffer1	Número	No aplicable.
	Buffer2	Cadena	Nombre del campo que se valida.

Tipo de validación	Descripción		
_str _str_oblig	Valida una cadena de caracteres de longitud mínima (buffer1)		
	NOTA: Usar buffer1=0 para permitir cualquier longitud.		
	Buffer1	Número	Longitud mínima de la cadena
	Buffer2	Cadena	Nombre del campo que se valida.
_time _time_oblig	Valida un formato de hora. Este formato no es parametrizable. (hh:mm). .soporta tres posibles separadores (: , .) .Permite : horas / horas y minutos / horas , minutos y segundos .Las horas van de 0..23 .los minutos van de 0..59 (ver m4builtime para obtener un formato de hora estándar)		
	Buffer1	Número	No aplicable.
	Buffer2	Cadena	Nombre del campo que se valida.
_time_sec _time_sec_oblig	Igual que _time pero exigiendo segundos (hh:mm:ss) .Los segundos van de 0..59 (ver m4builtimesec para obtener un formato de hora estándar)		
	Buffer1	Número	No aplicable.
	Buffer2	Cadena	Nombre del campo que se valida.



(*)Estas validaciones dependerán en un futuro del estándar elegido. Sin afectar a su uso.

EJEMPLO

```
err = m4valinput('_date', 'NombreFormulario', '<%=zArgIni%>', 1,
'<m4:label m4name=<%=zArgInir%>/>', sformatofechas);
```

m4valinput (Validaciones comparativas)

m4valinput(sValType,sIdForm1,sIdInput1,sIdForm2, sIdInput2, sOpRel, sInputType,buffer₁..buffer_m, Param₁..Param_n)

Descripción

Este modo de invocación de la m4valinput se utiliza para comparar los valores de dos input según el operador relacional *sOpRel*.

Tener en cuenta que antes de realizar la comparación, la función valida los tipos de los elementos y no sigue con la comparación si éstos no son del tipo adecuado. Por tanto la validación de los tipos se debería hacer antes para asegurarnos de que los tipos son correctos, si no queremos que salgan los mensaje de validación del tipo cuando el que esperamos es el de la comparación.

Parámetros

Parámetro	Tipo	Descripción
sValType	Cadena	Hay que poner _com para indicar que es validación comparativa.
sIdForm1	Cadena	Identificador del formulario donde se encuentra el primer input cuyo valor se quiere comparar.
sIdInput1	Cadena	Identificador del primer input cuyo valor se quiere comparar.
sIdForm2	Cadena	Identificador del formulario donde se encuentra el segundo input cuyo valor se quiere comparar.
sIdInput2	Cadena	Identificador del segundo input cuyo valor se quiere comparar.
sOpRel	Cadena	Operador relacional a usar en la comparación (<,. <, >=, ...). Todos los válidos en javascript.
sInputType	Cadena	Tipo de los campos a comparar : _date, _str.... (Ver Tabla de validaciones.)
buffer _i		Parámetros en función del tipo de validación. Ver Tabla de Validaciones .
Param _i	Cadena	Datos para construir el mensaje de error en el caso de que el valor no pase la validación. El significado de cada parámetro depende el tipo de validación. Ver Tabla de Validaciones .

EJEMPLO

```
err =  
m4valinput('_com', 'NombreFormulario', '<%=zArgIni%>', 'NombreFormulario'  
, '<%=zArgFin%>', '<=', '_date', 0, '<m4:label m4name=<%=zArgInir%>/>');
```

m4valform

m4valform (sValidationString)

Descripción

Esta función se encarga de realizar las validaciones que le vienen en la cadena *sValidationString* y para todos los input con valores no válidos marca el background en amarillo además de sacar un único mensaje de error con todos los valores que no han pasado la validación correspondiente.

El formato de la cadena *sValidationString* debe ser la concatenación de invocaciones a la m4valinput de la forma:

m4valinput(...)₁ * * m4valinput()_n

Parámetros

Parámetro	Tipo	Descripción
sValidationString	Cadena	Cadena de validación. El formato de esta cadena debe ser la invocación a llamadas m4valinput deseadas y todas las llamadas concatenadas con el *.

EJEMPLO

```
function val(){  
var sfunciones ="";  
sfunciones = "m4valinput('_date', 'NombreFormulario', '<%=zArgIni%>',  
1, '<m4:label m4name=<%=zArgInir%>/>', sformatofechas)";  
sfunciones =  
sfunciones+"*"+ "m4valinput('_date', 'NombreFormulario', '<%=zArgFin%>',  
1, '<m4:label m4name=<%=zArgFinr%>/>', sformatofechas)";  
var verr=m4valform(sfunciones);  
if (verr == 1){  
m4submit('NombreFormulario');  
}  
}
```

Personalizar el mensaje de error (m4_err_lang.js)

Descripción

A priori para cada tipo de validación existe un mensaje de error estándar que indica el nombre del campo que se valida, más otra serie de parámetros en función del tipo de validación.

Si se quiere sacar un mensaje de error personalizado en vez del estándar, hay que poner como primer parámetro de los parámetros propios del mensaje de error, (Param1) el identificador de nuestro mensaje precedido por la constante javascript: SMSG_ERROR_ID. Esta constante está definida a nivel de m4gen_excep.js.

El mensaje debe estar previamente incluido en el archivo de errores correspondiente (m4err_xx.js).

EJEMPLO

```
var sfunciones
="m4valinput('_com', 'Form1', 'initdate', 'Form1', 'lastdate', '<=', '_date', 0, SMSG_ERROR_ID+ '_ta_year')";
m4valform(sfunciones);
```

Ejemplos de utilización

Validar un campo fecha obligatorio de acuerdo con el formato de fechas del usuario:

```
var sfunciones =
"m4valinput('_date_oblig', 'formu', 'inputp', '0', 'nombre_del_campo')";
m4valform(sfunciones);
```

Validar y comparar dos campos que deben ser caracteres no numéricos:

```
var funciones =
"m4valinput('_com', 'formu', 'inputp', 'formu', 'inputp2', '<=', '_str', 0, 'nombre del campo')";
m4valform(sfunciones);
```

Validar dos campos de acuerdo con sus tipos.

```
var funciones = "m4valinput('_str', 'formu', 'inputp', 0, 'nombre del campo1')*m4valinput('_email', 'formu', 'inputp2', 0, 'nombre del campo2')";
m4valform(sfunciones);
```

Validar un campo dentro de un intervalo numérico (3,+infinito)

```
var funciones = "m4valinput('__interval','formu','inputp',  
'num_oblig', '3', '', 'nombre del campo1')";  
m4valform(sfunciones);
```

Validar un campo dentro de un intervalo de fechas (-infinito,10-11-2001)

```
var funciones = "m4valinput('__interval','formu','inputp',  
'date_oblig', '', '10-11-2001','', 'nombre del campo1')";  
m4valform(sfunciones);
```

Trabajando con monedas (m4gen.js)

m4currency

m4currency(sIdForm,sIdQuantity,sIdCurrency,sIdExchangetype,sIdExchangeDate,sDecimalNumber,sIdCurrencyName)

Descripción

Esta función invoca la página de selección de moneda (shco_gen_currency.jsp), y devuelve en los inputs *sQuantity,sIdCurrency,sIdExchangetype,sIdExchangeDate* del formulario *sIdForm* los valores seleccionados. Además si los valores de los input llegan rellenos los establece en la pantalla.



Para ampliar información ver el capítulo de [Página de moneda](#).

Parámetros

Parámetro	Tipo	Descripción
SIdForm	Cadena	Identificador del formulario que contiene los input.
SIdQuantity	Cadena	Identificador del input donde se establece la cantidad.
SIdCurrency	Cadena	Identificador del input donde se establece el identificador de la moneda.
SIdExchangetype	Cadena	Identificador del input donde se establece el tipo de cambio.
SIdExchangeDate	Cadena	Identificador del input donde se establece la fecha de cambio.
SDecimalNumber	Cadena	Número de decimales asociados a la moneda.
SIdCurrencyName	Cadena	Identificador del objeto donde escribir el nombre de la moneda. Puede ser un TD, un input...

```
<a tabindex="<%=zTab + 1)%%" title="<m4:label  
m4name="<%=zSHCOLBLIST%>" htmlsafe="true"/> <m4:label  
m4name="<%=zSCO_MIN_SALARY%>" htmlsafe="true"/  
>"href="javascript:m4currency('NombreFormulario','SCO_MIN_SALARY','ID  
CURRENCY','EX_TYPE','CUR_DATE','DEC_NB','NM_CURRENCY');"><img  
alt="<m4:label m4name="<%=zSHCOLBLIST%>" htmlsafe="true"/> <m4:label  
m4name="<%=zSCO_MIN_SALARY%>" htmlsafe="true"/>" <%@ include  
file="../../../files_gif/ic_pay.jsp"%> /></a>
```

```
m4currencysdefaultvalues(sidform,sinputcant,sinputidcur,sinputncur,sinputde  
c,sinputexchtype,sinputexchdate,sidcur, snmcur,sdecnb,s sextype)
```

Esta función establece en los *inputs*, *sinputcant*, *sinputidcur*, *sinputncur*, *sinputdec*, *sinputexchtype*, *sinputexchdate* del formulario *sidform* los valores que vienen en los parámetros *sidcur*, *snmcur*, *sdecnb*, *s sextype* más la cantidad que se pone a vacío y la fecha de cambio que se establece al día de hoy. Para que los valores por defecto coincidan con la configuración establecida por el usuario para la moneda hay que pasar como valores los establecidos para las variables globales de java: *zidcur*, *zncur*, *zndec*, *zexchtype*.

Parámetro	Tipo	Descripción
Sidform	Cadena	Identificador del formulario que contiene los input.
Sinputcant	Cadena	Identificador del input donde se establece la cantidad. Se establecerá el valor vacío.
Sinputidcur	Cadenas	Identificador del objeto donde escribir el nombre de la moneda. Puede ser un TD o un input.
Sinputncur	Cadena	Identificador del input/td donde se establece el tipo de cambio.
Sinputdec	Cadena	Identificador del input donde se establece el número de decimales.

Parámetro	Tipo	Descripción
Sinputexctype	Cadena	Identificador del input donde se establece el tipo de cambio.
Sinputexchdate	Cadena	Identificador del objeto donde se establece la fecha de cambio (fecha de hoy)
Sidcur	Cadena	Identificador de la moneda a establecer. El defecto para el usuario está en la variable java: <i>zidcur</i>
Snmcur	Cadena	Nombre de moneda a establecer. El defecto para el usuario está en la variable java: <i>znmcur</i>
Sdecbn	Cadena	Número de decimales a establecer. El defecto para el usuario está en la variable java: <i>zdecbn</i>
Sextype	Cadena	Tipo de cambio a establecer. El defecto para el usuario está en la variable java: <i>zextype</i>

EJEMPLO

```

<input type="text" id="idmoneda" name="idmoneda" value="" />
<input type="text" id="nmoneda" name="nmoneda" value="" />
<input type="text" id="idtipocambio" name="idtipocambio" value="" />
<input type="text" id="nombretipocambio" name="nombretipocambio" value="" />
<input type="text" id="fechacambio" name="fechacambio" value="" />
<input type="text" id="numdec" name="numdec" value="" />
<input type="button" value="setdefcurr"
onclick="m4currencysetdefaultvalues('NombreFormulario','cantidad'
, 'idmoneda', 'nombremoneda', 'numdec', 'idtipocambio',
'fechacambio', '<%=zidcur%>', '<%=znmcur%>',
'<%=zdecbn%>', '<%=zextype%>')" >

```

Gestión de calendarios (m4gen.js)

m4calendar

m4calendar(objDate[,sCallBackFunctionName])

Descripción

Esta función levanta una ventana no modal con un calendario para seleccionar una fecha. Esta fecha se devuelve formateada según la configuración del usuario.

Además, si se le pasa una función a la que invocar tras seleccionar la fecha y cerrar el calendario, *sCallBackFunctionName* se hará una invocación pertinente.



La función se debe pasar entre comillas y con los paréntesis correspondientes aunque no tenga parámetros. Para pasar los parámetros entre comillas usar ";

Parámetros

Parámetro	Tipo	Descripción
ObjDate	Objeto	Objeto input donde devolver la fecha elegida
ScallBackFunctionName	Cadena	Función a invocar tras la selección de la fecha. Debe llevar los paréntesis de invocación e incluso puede llevar argumentos. Esto nos permite simular un comportamiento modal de la ventana.

EJEMPLO

Con invocación a una función sin argumentos

```
<script type="text/javascript">
    function aftercalendarselection{
    ...}
...
</script>

<input type="text" id="txtDate" name="txtDate" value="">

<a href="javascript:
m4calendar(m4objeto('NombreFormulario','txtDate'),'aftercalendarselec
tion()');">
<img <%@ include file="../../files_gif/ic_cal.jsp" %> />
</a>
```

Con invocación a una función con argumentos

```
<script type="text/javascript">
    function aftercalendarselection(svalue){
    ... }
...
</script>

<input type="text" id="txtDate" name="txtDate" value="">

<a href="javascript:
m4calendar(m4objeto('NombreFormulario','txtDate'),
'aftercalendarselection(&quot;valor1&quot;);');" >
<img <%@ include file="../../files_gif/ic_cal.jsp" %> />
</a>
```

Gestión de ventanas (m4gen.js)

En este apartado se especifican las funciones a usar cuando se quiere abrir desde una página otra en una nueva ventana, con posibilidad además de recoger ciertos valores de la página destino en la página origen.

Para todo esto se implementa la clase `m4class_dialogwin`.

Este es el sistema que internamente utilizan por ejemplo las plantillas de mantenimiento.

Para implementar este sistema:

- La página origen debe utilizar la función `m4window()` para invocar a la segunda página. Y si quiere recoger ciertos valores de la página destino, debe pasar los input donde recoger dichos valores así como el formulario al que pertenecen.
- La página destino para devolver valores a la página origen debe utilizar la función `m4returnvalues()`.
- Ambas páginas se sincronizan para establecer el orden en el que se devuelven los valores.

m4window

`m4window(sIdWindow, sPageURL, arResultInputs, sIdform [,iWidth, iHeight, bResizable, bScrollbars, bUseWindowId])`

Descripción

Esta función abre una ventana no modal con la página `sPageURL`.

La ventana que se abre lleva un identificador unívoco `sIdWindow`, de forma que se impide que se levante más de una. Es decir, otro `m4window` con el mismo identificador reutiliza la ventana ya que no puede haber dos ventanas abiertas con el mismo identificador.

Las características físicas de la ventana que se pueden modificar son las dadas por los parámetros: `,iWidth, iHeight, bResizable, bScrollbars`.

Si a esta función se le pasan identificadores de los inputs para que se devuelvan en ellos los resultados, `arResultInputs`, la función se encarga de obtener los objetos asociados a esos identificadores y de guardarlos en propiedades de la ventana origen. En este caso, en la ventana destino hay que invocar a la función `m4returnvalues` con los valores resultados.

Parámetros

Parámetro	Tipo	Descripción
SIdWindow	Cadena	Identificar de la ventana.
SPageURL	Cadena	URL de la página a abrir.
ArResultInputs	Array	Array con los identificadores de los inputs a rellenar con los resultados de la página destino.
SIdform	Cadena	Identificador del formulario a los que pertenecen los input que recogen los resultados.
IWidth	Entero	Ancho de la ventana a abrir. Defecto:800
IHeight	Entero	Alto de la ventana a abrir. Defecto:600
BResizable	Cadena	Indica si la ventana que se abre se quiere que sea redimensionable. ("yes" / "no"). Defecto : "yes"
BScrollbars	Cadena	Indica si la ventana que se abre se quiere que tenga barras de desplazamiento. ("yes" / "no"). Defecto : "yes"
bUseWindowId	Booleano	Para que se use como identificador de ventana el parámetro <i>SidWindow</i> , sino por defecto se usa un número que se genera automáticamente (segundos actuales). Esto nos interesa cuando queremos que se reutilice una ventana concreta. Por ejemplo cuando se quieren pasar parámetros a una ventana que se abre con <i>m4window</i> via <i>post</i> (a través de un formulario) porque son muchos. En estos casos se hace <i>m4window</i> y luego se modifica el <i>target</i> del formulario antes de hacer el <i>submit</i> para que se haga sobre la venta recién abierta. El ID no puede tener / ni ?, por tanto nunca se podría usar en la <i>m4filtro</i> pues el ID es la URL.
bStatus	Booleano	Indica si el estado de la ventana se debe mostrar. ("yes" / "no").

EJEMPLO

```
var scur = "/servlet/CheckSecurity/JSP/shco_g0/  
shco_gen_currency.jsp";  
scur = scur + "?IDCUR="+ m4valor(sidform,sidcurrency, '', 'get')  
+"&CANT="+ m4valor(sidform,squantity, '', 'get')
```

```

        + "&EXTYPE=" +
m4valor(sidform, sidexchangetype, '', 'get') + "&EXDATE=" + m4valor(sidfo
rm, sexchagedate, '', 'get');

var oarcur = new Array();
for(var nt=0; nt <= 4; nt++){
    oarcur[nt] = m4currency.arguments[nt+1];
}
m4window("pagecurrency", scur, oarcur, sidform, 450, 230, "no", "no");

```

m4returnvalues

m4returnvalues(arReturnValues)

Descripción

Esta función se invoca desde una ventana hija para devolver valores a la ventana padre.

La función se encarga de asignar los valores *arReturnValues* a los objetos input de la ventana padre y de cerrar a continuación la ventana.

Parámetros

Parámetro	Tipo	Descripción
arReturnValues	Array	Array con los valores a devolver a la ventana padre.

EJEMPLO

```

m4returnvalues(new
Array(m4valor('NombreFormulario','cantidad','', 'get'), m4select('Nombr
eFormulario','idmoneda','id'), m4select('NombreFormulario','idtipocamb
io','id'), m4valor('NombreFormulario','fechadecambio','', 'get'), m4sele
ct('NombreFormulario','idmoneda','value'), m4select('NombreFormulario'
,'idmoneda','text')));

```

m4window_extension

m4window_extension(sldElement₁[...,sldElement_n])

Descripción

Función para el paso de objetos entre la ventana padre e hija. Se invoca en la ventana padre, antes de función `m4window()` y de esta forma tendremos disponible en la ventana hija, todos los objetos de la padre cuyos identificadores le hayamos pasado. Los identificadores deben ser únicos en toda la página.

Parámetros

Parámetro	Tipo	Descripción
<code>sIdElement</code>	Cadena	Identificador de elemento a pasar a la ventana hija.

EJEMPLO

```
m4window_extension(sidtd);  
m4window(...)
```

m4window_extension_return

`m4window_extension_return(sIdElementInOpenerWindow)`

Descripción

Función complementaria de `m4window_extension`. Se ejecuta en la ventana hija y con ella podemos recuperar una referencia al objeto de la ventana madre que previamente lo habíamos pasado con la función `m4window_extension()`. Si no existe devuelve null.

Parámetros

Parámetro	Tipo	Descripción
<code>SidElementInOpenerWindow</code>	Cadena	Identificador de elemento a pasar a la ventana hija.

EJEMPLO

```
//recoger el objeto de la ventana madre a manipular  
oobject = m4window_extension_return(sidtd);  
oobject.value = m4select('NombreFormulario','idmoneda','text');
```

m4windowcallback

```
m4windowcallback(sIdWindow, sPageURL, arResultInputs, sIdform,
scallBackFunctionName [,iWidth, iHeight, bResizable,
bScrollbars,bUseWindowId])){
```

Descripción

Función igual que m4window, a la que se le añade un argumento más *scallBackFunctionName* que se corresponde con la función de la ventana padre que se invoca con el *m4returnvalues*. Es una forma de que la ventana padre se entere de cuando se están retornando los valores por si quiere hacer algo en ese momento.

Parámetros

(Ver resto de parametros en la m4window)

Parámetro	Tipo	Descripción
scallBackFunctionName	Cadena	Función a invocar en la m4returnvalues. Debe llevar los paréntesis de invocación e incluso puede llevar argumentos.

EJEMPLO

Ventana padre

```
<script type="text/javascript">
function aftercurrencyselect(){
    ...
}
m4windowcallback("pagecurrency", scur,oarcur,sidform,'aftercurrency
select()',450,230,"no","no");
```

m4navegar

```
m4navegar(sURL,arrIdParameters,arrParametersValues)
```

Descripción

Esta función genera una URL y navega a la misma. La URL la genera a partir *sURL*, a la que concatena los pares parámetro=valor según los datos contenidos en los array *arrIdParameters* y *arrParametersValues*.

Los arrays de parámetros y valores se manipulan por posición, es decir, parámetro de la posición *i* se le asigna el valor de la posición *i*.

Parámetros

Parámetro	Tipo	Descripción
sURL	Cadena	Dirección a la que navegar
arrIdParameters	Array	Array con los parámetros a pasar.
arrParametersValues	Array	Array con los valores de los parámetros a pasar.

EJEMPLO

```
href="javascript:var parametros=['zpag','znw'];  
var valores=['shco_ta/shco_ta_wz_week_model_p1.jsp','0'];  
m4navegar('shco_ta_sse/  
shco_ta_list_week_model.jsp',parametros,valores);"
```

Trabajando con plantillas de mantenimiento (m4gen_mt.js)

En la librería (m4gen_mt.js) se encuentran las funciones específicas de la plantilla de tablas de mantenimiento. Hay funciones que se utilizan directamente en las páginas de plantillas de mantenimiento, y otras que se usan desde una página (origen) al levantar otra página para modificar valores (página destino).

La mayor parte de las funciones, cuando se pasa un identificador de input como argumento, excepto en aquellos casos en los que exista un parámetro específico para identificar el formulario de la página origen, se presupone que ese input está definido bajo un formulario identificado por: 'NombreFormulario'.



Las funciones que reciben como parámetro una URL (sIdPage) a cargar, dicho parámetro no puede superar los 256 caracteres debido a los servidores webs que funcionan en UNIX. Para pasar parámetros en la URL usar los formatos que se especifican vía inputs.

Figura 79. Ejemplo de una página de mantenimiento

Inicio | Tablas de la persona | Tablas médicas | Tablas del CV | Tablas del periodo | Tablas rol |

MANTENIMIENTO DE MOTIVOS DE CAMBIO

Nuevo registro

* Id. Motivo cambio * Motivo cambio

Id. Motivo cambio	Motivo cambio	
1	TRASLADO	
2	PROMOCIÓN	
3	REORGANIZACIÓN	
4	CONTRATADO	
5	DEFUNCIÓN	
6	NUEVA ENTRADA	
7	CANCELACIÓN	
8	Despido	
1 - 8		

m4eliminar

m4eliminar(sIdInput₁,sValue₁,...sIdInput_{2n},sValue_{2n})

Descripción

Función para borrar un registro de una tabla de mantenimiento desde una página. Espera como parámetros pares Identificador de campo - Valor del campo a borrar. Esos identificadores de campos deben corresponder con la PK de la tabla.

Parámetros

Parámetro	Tipo	Descripción
SIdInput _i	Cadena	Identificador del objeto input que se asocia al campo a borrar.
sValue _i	cadena	Valor a borrar del campo.

EJEMPLO

Eliminar un tipo de localización:

```
<td class="boton<%=zpos%>"><a title="<m4:label  
m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/>"  
href="javascript:m4eliminar('X<%=zcampoID1%>', '<m4:item  
m4name="<%=zIdPk1%>" jsafe="true" htmlsafe="true"/>');"><img  
alt="<m4:label m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/>" <%@  
include file="../../files_gif/ic_bor.jsp" %> /></a></td>
```

m4filtro

m4filtro(sidPage[sidResultInput₁, ... sidResultInput_n])

Descripción

Función específica para sacar una ventana de listado dónde poder seleccionar los valores pertinentes.

Parámetros

Parámetro	Tipo	Descripción
SidPage	cadena	Nombre de la página asociada a la lista a mostrar

Parámetro	Tipo	Descripción
SIdResultInput _i	cadena	Identificador de los objeto input donde devolver los valores seleccionados.

EJEMPLO

Llamada a la lista de tipos de localización:

```
<a tabindex="<%= (zTab + 1) %>" title="<%= zSHCOLBLIST_val %>" href="
javascript:m4filtro('shco_g2_sse/
shco_pa_list_locat_type.jsp', 'SHCO_PA_WZ_ADDRESS+.+STD_ID_LOCATION_TY
PE', 'STD_N_LOCATION_TYPE', 'STD_N_LOCATION_TYPE' );"> img alt=
"<%= zSHCOLBLIST_val %> <%= zLbLnmLocationType %>" <%@ include file="../
../files_gif/ic_list.jsp"%> /></a>
```

m4filtrocallback

m4filtrocallback(sidPage, scallBackFunctionName[, sidResultInput1, ..., sidResultInputn])

Descripción

Función igual que m4filtro, a la que se le añade un argumento más *scallBackFunctionName* que se corresponde con la función de la ventana padre que se invoca con el *m4returnvalues*. Es una forma de que la ventana padre se entere de cuando se están retornando los valores por si quiere hacer algo en ese momento.

Parametros

(Ver resto de parametros en la m4filtro)

Parámetro	Tipo	Descripción
scallBackFunctionname	Cadena	Función a invocar en la <i>m4returnvalues</i> . Debe llevar los paréntesis de invocación e incluso puede llevar argumentos.

EJEMPLO

Llamada a la lista de tipos de localización:

```
<script type="text/javascript">
function afterlocselection(){
...
}
```



```

    }
</script>

...
<a tabindex="<%= (zTab + 1) %>" title="<%= zSHCOLBLIST_val %>" href="
javascript:m4filtrocallback('shco_g2_sse/
shco_pa_list_locat_type.jsp',
'afterlocselection()', 'SHCO_PA_WZ_ADDRESS+.+STD_ID_LOCATION_TYPE',
'STD_N_LOCATION_TYPE', 'STD_N_LOCATION_TYPE' );"> img alt=
"<%= zSHCOLBLIST_val %> <%= zLbNmLocationType %>" <%@ include
file="../../files_gif/ic_list.jsp"%> /></a>

```

m4filtro_wargs

m4filtro_wargs(sidPage[,sidCountryInput, sidGeoDivInput, sidSubGeoDiv, sidResultInput₁, ... sidResultInput_n])

Descripción

Función específica para sacar una ventana de listado de divisiones/ subdivisiones geográficas dónde poder seleccionar los valores pertinentes.

Este listado respecto al resto tiene la peculiaridad que al listar permite que haya relación entre los valores que se listan del siguiente modo:

- Las divisiones geográficas que se listan, son las válidas para el país seleccionado si está relleno, sino se sacan todas.
- Las subdivisiones geográficas que se listan son válida para el país y división geográfica si están rellenos y sino se sacan todas.

Parámetros

Parámetro	Tipo	Descripción
SidPage	cadena	Nombre de la página asociada a la lista a mostrar
SidCountryInput	cadena	Identificador del objeto input asociado al campo país (SCO_ID_COUNTRY)
SidGeoDivInput	cadena	Identificador del objeto input asociado al campo división geográfica (SCO_ID_GEO_DIV). Sólo aplicable para la lista de subdivisiones geográficas. Para el resto de casos poner vacío ("").
SidResultInput _i	cadena	Identificador de los objeto input donde devolver los valores seleccionados.

EJEMPLO

Llamada a la lista de división geográfica (SCO_ID_GEO_DIV):

```
<a tabindex="<%= (zTab + 1)%>" title="<%=zSHCOLBLIST_val%>"
href="javascript:m4filtro_wargs('shco_g1_sse/
shco_or_list_geo_div.jsp','STD_ID_COUNTRY','','STD_ID_GEO_DIV','STD_N
_GEO_DIV','STD_ID_COUNTRY','STD_N_COUNTRY');"><img alt="m4:label
m4name="<%=zSTD_ID_GEO_DIVcm%>" htmsafe="true"/>" <%@ include
file="../../files_gif/ic_list.jsp"%> /></a>
```

Llamada a la lista de subdivisión geográfica (SCO_ID_SUB_GEO_DIV):

```
<a tabindex="<%= (zTab + 1)%>" title="<%=zSHCOLBLIST_val%>"
href="javascript:m4filtro_wargs('shco_g1_sse/
shco_or_list_sub_geo_div.jsp','STD_ID_COUNTRY','STD_ID_GEO_DIV','STD_
ID_SUB_GEO_DIV','STD_N_SUB_GEO_DIV','STD_ID_GEO_DIV','STD_N_GEO_DIV',
'STD_ID_COUNTRY','STD_N_COUNTRY');"><img alt="m4:label
m4name="<%=zSTD_ID_SUB_GEO_DIVcm%>" htmsafe="true"/>"
<%@ include file="../../files_gif/ic_list.jsp"%> /></a>
```

m4remonte

Dos posibles usos:

- m4remonte(sidPage[sidInput1...,sidInputn])
- m4remonte(sidPage[sldFormulario,sidInput1...,sidInput2n,íM4NEWMODEí])

Descripción

Función para remontarse a una página de mantenimiento. Esta página se abre en una nueva ventana y al ser de mantenimiento te permite editar los valores. Se le puede indicar valores concretos para que se posicione en el mismo.

Dependiendo del modo de funcionamiento:

- El primer modo de funcionamiento:
 - Exige que los input de la página origen y destino se llamen igual.
 - El número de input puede ser 0.
- En segundo modo de funcionamiento:
 - Se identifica cuando el último parámetro de la función tiene el valor 'M4NEWMODE'.

- Modo más flexible pues te permite pasar parámetros desde la página origen a la destino con inputs con distinto nombre. En este caso, la secuencia de identificadores de inputs es una secuencia de parejas de inputs, y por supuesto ha de ser un número par $N = 2 \cdot K$ con K tendiendo a 0 nuevamente. El primer identificador de la pareja es el identificador del input en la página madre y el segundo el identificador del input en la hija, que pueden ser distintos, y si son iguales, de todas formas hay que ponerlos.
- El nombre del formulario bajo el que se definen los inputs de la página también se parametriza.

Parámetros

Parámetro	Tipo	Descripción
SidPage	Cadena	Nombre de la página asociada a la lista a mostrar.
SidFormulario	Cadena	Identificador del formulario de la página origen a la que pertenecen los input.
SidInput _i	Cadena	Identificador del <i>input</i> cuyo valor nos da el posicionamiento en la página.
'M4NEWMODE'	Cadena	Indica el modo de funcionamiento. Sólo para el segundo modo de funcionamiento.

EJEMPLO

Remonte con el primer modo de funcionamiento

```
<td class="campo"><%@ include file="../shco_g0/
shco_gen_wz_asterisc.jsp" %><a tabindex="<%=zTab +
1)%>"title="<m4:label m4name="<%=zIdzGen%>" htmlsafe="true"/>"
href="javascript:m4remonte('shco_g2/
shco_pa_mt_locat_type.jsp', '<%=zField%><%=zIdGen%>');"><m4:label
m4name="<%=zIdzGen%>" htmlsafe="true"/></a></td>
```

Remonte con el segundo modo de funcionamiento

```
<td class="campo" >&nbsp;&nbsp;&nbsp;<a title="<m4:label
m4name="<%=zId3r%>" htmlsafe="true"/>"
href="javascript:m4remonte('shco_g2/
shco_pa_mt_lang_level.jsp', 'NombreFormulario', '<%=zId3%>', 'STD_ID_LAN
G_LEVEL', 'M4NEWMODE');"><m4:label m4name="<%=zId3r%>"
htmlsafe="true"/>&nbsp;&nbsp;&nbsp;</a></td>
```

m4ordenar

m4ordenar(sIdCampo)

Descripción

Función dentro de una página de listado para ordenar por uno de los campos.

Parámetros

Parámetro	Tipo	Descripción
SidCampo	Cadena	Identificador del campo por el que ordenar

EJEMPLO

```
onclick="m4ordenar( '<%=zcampoID1%>' );"
```

m4setPKA

m4setPKA()

Descripción

Función que recoge los valores de las PK del formulario 'NombreFormulario' con su valor y los asigna al input zPKA del formulario 'oculto'.

Los valores recogidos se van concatenando en forma clave=valor{, formato específico para las transacciones con el servidor.

m4setm4tit

m4setm4it()

Descripción

Función para gestionar los estados de nuevo y modificado de una MT. Se gestiona el título y el color de fondo de los campos pk.

m4lim

m4lim()

Descripción

Función que se usa para inicializar el formulario de una MT, cambiando el estado a nuevo y limpiando los valores de los campos.

m4com2

m4com2()

Descripción

Función que se utiliza para comprobar si se han tocado alguno de los valores PK de la MT en el formulario y sacar un mensaje preguntando si se quiere grabar las modificaciones antes de continuar.

m4edit

m4edit()

Descripción

Función para cambiar el estado del formulario de la MT a modo edición. Cambio de título y fondo de los campos PK.

m4cambiofondo

m4cambiofondo(sidMode)

Descripción

Función que recorre los input que son PK del formulario 'NombreFormulario', y si no están en ReadOnly o Disabled les cambia el fondo. El fondo que se les aplica depende del modo en el que estemos (Parámetro sidMode).

Parámetros

Parámetro	Tipo	Descripción
Sidmode	Cadena	"0": Modo inserción "1": Modo normal del formulario

Trabajando con plantillas de listado (m4gen_list.js)

En este apartado se van a describir la funciones genéricas que se utilizan en páginas que son plantillas de listado tal y como vemos en la imagen.

Figura 80. Ejemplo de una página que muestra un listado

ID Persona	Nombre
M10007	Piara, María
M10008	Pérez Ortega, Raúl
M10009	Del Pino PEREZ, José
M10010	Pérez, Luis Carlos
M10011	Ríos, Piero
M10013	Pareja Muñoz, Pedro

m4buscar

m4buscar(sIdFieldSelect,sIdOperSelect,sOperSelectedValue)

Descripción

Función que en la parte de filtro del listado, se encarga de rellenar la select de operadores válidos en función del tipo del campo que se seleccione para el filtro (Cadena, fecha...).

Se invoca al hacer cambiar la selección en la select de campos de filtro.

Si se cambia entre dos campos del mismo tipo el operador se mantiene.

Parámetros

Parámetro	Tipo	Descripción
sldFieldSelect	Cadena	Identificador del objeto Select que contiene los campos por los que se puede filtrar.
sldOperSelect	Cadena	Identificador del objeto Select que contiene los operadores de filtrado.
sOperSelectedValue	Cadena	Valor del operador seleccionado para que se mantenga en caso de que los tipos de los campos sean iguales

EJEMPLO

```
<select tabindex="1" id="list1" class="select20" name="list1"
title="<m4:label m4name="<%=zSHCOLBLIST%>" htmlsafe="true"/>"
onchange="m4buscar('list1','list3')">
```

m4filtrar()

m4filtrar()

Descripción

Función invocada al pulsar el botón de filtrado (lupa) para listar los valores correspondientes al filtro.

EJEMPLO

```
<a title="<m4:label m4name="<%=zSHCOLBFILT%>"
htmlsafe="true"/>" href="javascript:m4filtrar();" tabindex="7"><img
alt="<m4:label m4name="<%=zSHCOLBFILT%>" htmlsafe="true"/>"
<%@include file="../files_gif/ic_env.jsp" %> /></a>
```

m4filtro_wargs

Ver definición en el apartado Trabajando con plantillas de mantenimiento (m4gen_mt.js) de este capítulo.

m4ordenar

m4ordenar (sIdField)

Descripción

Función que se invoca para ordenar una determinada columna del listado. Se usa al hacer clic en las flechas que vienen en cada columna.

Parámetros

Parámetro	Tipo	Descripción
sIdField	Cadena	Identificador del campo por el que se quiere ordenar el listado.

EJEMPLO

```
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zIdPk1%>" htmlsafe="true"/>"
onclick="javascript:m4ordenar('<%=zcampoID1%>');" <%@ include
file="../../files_gif/ic_ord_1.jsp" %> />
```

Valores

valores()

Descripción

Función que se utiliza en un listado en el proceso de paginación para poder mantener los valores del filtro. Uso de un formulario 'oculto'.

EJEMPLO

```
<a
href="javascript:m4valor('oculto','zinicio',<%=ziniciointervalo%>,'se
t');valores();"><%=ziniciointervalo%>&nbsp;  -
&nbsp;  <%=zfinintervalo%></a></td>
```

Trabajando con Plantillas de listado con CheckBox (m4gen_list_ck.js)

Esta librería contiene funciones a usar en caso de listados con multiselección.

Se exige que las *checkbox* se encuentre en un formulario con identificador 'oculto2' y las checks deben tener como id : 'CK'+i donde i es la posición de la check en la lista.

Figura 81. Ejemplo de una página de listado con casillas de verificación

EMPRESAS

Filtro avanzado

Id. Empresa Empieza por

Id. Empresa Empieza por

Buscar

Marcar todos Desmarcar todos Aceptar

☐	Id. Empresa	Empresa	Inicio	Fin	1-6 de 6
☐	rsu	wrtuy	27-06-2002		
☐	testee	testee	10-11-1999		
☐	tets	test	28-10-1999		
☐	yyy	yyy	10-11-1999	27-06-2002	
☐	0001	Technology for Medicine S.A.	01-02-1990		
☐	0002	prueba	01-01-1800		

1 - 6

m4aceptall

m4aceptall(iCheckNumber)

Descripción

Función para marcar los *iCheckNumber* elementos de la lista.

Se exige que las *checkbox* se encuentre en un formulario con identificador 'oculto2' y las checks deben tener como id : 'CK'+i donde i es la posición de la check en la lista.

Parámetros

Parámetro	Tipo	Descripción
iCheckNumber	Entero	Indica el número de elementos de la lista

EJEMPLO

```
<td class="boton"><input id ="AcpAll" name="AcpAll"
tabindex="8"type="button" class="button"
onclick="javascript:m4acceptall(6);"value="Marcar todos" /></td>
```

m4cancelall

m4cancelall(iCheckNumber)

Descripción

Función para desmarcar los *iCheckNumber* elementos de la lista.

Se exige que las checkbox se encuentre en un formulario con identificador 'oculto2' y las checks deben tener como id: 'CK'+i donde i es la posición de la check en la lista.

Parámetros

Parámetro	Tipo	Descripción
iCheckNumber	Entero	Indica el número de elementos de la lista.

EJEMPLO

```
<td class="boton"><input id ="DAcpAll" name="DAcpAll"
tabindex="8"type="button"
class="button"onclick="javascript:m4cancelall(6);"value="Desmarcar
todos" /></td>
```

m4retorno

m4retorno(iCheckNumber,sIdAssociatedField)

Descripción

Se utiliza para devolver en una cadena todos los valores asociados a las checkbox que están chequeadas.

Se exige que las *checkbox* se encuentre en un formulario con identificador 'oculto2' y las *checks* deben tener como id: 'CK'+i donde i es la posición de la *check* en la lista.

Además los campos asociados a cada *checkbox*, deben estar también contenidos en el formulario 'oculto2' y deben identificarse todos con el mismo identificador (sIdAssociatedField) más el indicador de la posición en la lista, como en las *check box* (sIdAssociatedField + i). Es decir, el campo asociado a la *checkbox* 'CK1', debe llamarse sIdAssociatedField + '1'.

Parámetros

Parámetro	Tipo	Descripción
ICheckNumber	Entero	Indica el número de elementos de la lista.
SIdAssociatedField	Cadena	Identificador del campo que guarda el valor asociado a la check.

Trabajando con Plantillas de asistente (m4gen_wz.js)

m4cambioestado

m4cambioestado (sState)

Descripción

Función de cambio de estado en un *wizard*. Los estados que contempla son los de **Nuevo** (nuevo registro) y **Actualización** (actualizando registro).

Esta función se encarga de escribir el estado en la pantalla, y hacer el cambio correspondiente de colores en los campos PK (m4cambiofondo).

Parámetros

Parámetro	Tipo	Descripción
SState	cadena	Cadena que indica el estado al que hay que cambiar. '1': Estado nuevo Resto: Estado actualización

m4comwz

m4comwz()

Descripción

Función que se invoca desde un *wizard* cuando vamos a grabar un registro para controlar si se está modificando la PK. En el caso de que se esté modificando se saca un mensaje solicitando la confirmación de la operación, dado que la modificación de un PK puede tener efectos laterales.

m4editwz

m4editwz()

Descripción

Función que se invoca para editar un registro seleccionado dentro de una lista en una pantalla de *wizard*.

Se encarga de establecer los valores correspondientes en el formulario de edición y de cambiar el estado a actualización.

EJEMPLO

```
<td class="boton<%=zpos%>"><a title="<m4:label
m4name="<%=zSHCOLBEDIT%>" htmlsafe="true"/>"href=""
onclick="m4valor('NombreFormulario','X<%=zIni%>','<m4:item
m4name="<%=zInicm%>" jsafe="true" htmlsafe="true"/>','set');
m4valor('NombreFormulario','X<%=zIdHR%>','<m4:item
m4name="<%=zIdHRcm%>"
jsafe="true" htmlsafe="true"/>','set');
m4valor('NombreFormulario','X<%=zOrRol%>','<m4:item
m4name="<%=zOrRolcm%>"
jsafe="true" htmlsafe="true"/>','set');
m4valor('NombreFormulario','<%=zFin%>','<m4:item m4name="<%=zFincm%>"
jsafe="true" htmlsafe="true"/>','set');
m4valor('NombreFormulario','<%=zNmHR%>','<m4:item
m4name="<%=zNmHRcm%>"
jsafe="true" htmlsafe="true"/>','set');
m4editwz ();return false;">
<img alt="<m4:label m4name="<%=zSHCOLBEDIT%>" htmlsafe="true"/>" <%@
include file="../../files_gif/ic_mod.jsp" %> /></a></td>
```

m4limwz

m4limwz()

Descripción

Función que se invoca para limpiar los campos de un formulario de un *wizard*.

EJEMPLO

```
<th><a title="<m4:label m4name="<%=zSHCOLBCLEAN%>" htmlsafe="true"/>"
href="javascript:m4clearform('NombreFormulario');m4limwz ();"><img
alt="<m4:label m4name="<%=zSHCOLBCLEAN%>" htmlsafe="true"/>" <%@
include file="../../files_gif/ic_des.jsp" %> /></a></th>
```

m4navwz

m4navwz(iNavMode)

Descripción

Función que se invoca al pulsar alguno de los botones de navegación de un *wizard* para que cargue el paso correspondiente y avise previamente de la posible pérdida de datos.

Parámetros

Parámetro	Tipo	Descripción
iNavMode	Entero	Forma de navegación. 0: Hacia atrás Resto: Hacia adelante

EJEMPLO

```
<a tabindex="<%= (zTab+1)%>" title="<m4:label  
m4name="<%=zSHCOLBBACKNAV%">" htmlsafe="true"/>"  
href="javascript:m4navwz(0);"><img alt="<m4:label  
m4name="<%=zSHCOLBBACKNAV%">" htmlsafe="true"/>" title="<m4:label  
m4name="<%=zSHCOLBBACKNAV%">" htmlsafe="true"/>" <%@ include file="../../../  
files_gif/ic_ret.jsp" %> /></a>&nbsp;</pre>
```

m4setm4titwz

m4setm4titwz()

Descripción

Función que se utiliza en la carga de cada paso del wizard para inicializar el estado.

EJEMPLO

```
<body onload="javascript:m4setm4titwz();">
```

m4wzeliminar

m4wzeliminar(sIdField₁,sFieldValue₁,...[, sIdField_n,sFieldValue_n])

Descripción

Función que se invoca para eliminar un registro del listado asociado a un paso de un *wizard*. Se le pasan los identificadores y valores asociados a los campos PK.

Parámetros

Parámetro	Tipo	Descripción
sIdField _i	Cadena	Identificador del campo PK i
SFieldValue _i	Cadena	Valor del campo PKi

EJEMPLO

```
<td class="boton<%=zpos%>" colspan="2"><a
title="<m4:label m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/>"
href="javascript:m4wzeliminar('X<%=zIni%>', '<m4:item
m4name="<%=zInicm%>" jsafe="true" htmlsafe="true"/
>', 'X<%=zOrRol%>', '<m4:item m4name="<%=zOrRolcm%>" jsafe="true"
htmlsafe="true"/>', 'X<%=zIdHR%>', '<m4:item m4name="<%=zIdHRcm%>"
jsafe="true" htmlsafe="true"/>');">
<img alt="<m4:label m4name="<%=zSHCOLBDEL%>"
htmlsafe="true"/>"
<%=@ include file="../../files_gif/ic_bor.jsp" %> /></a>
</td>
```

m4wzins

m4wzins(insertMode)

Descripción

Función que se invoca al pulsar desde algún paso del *wizard* los botones de salvar, anterior + salvar o posterior + salvar.

Parámetros

Parámetro	Tipo	Descripción
iSaveMode	Entero	Modo de salvar. 0: Salvar + goPrevious 1: Salvar 2: Salvar + goNext

EJEMPLO

```
<a tabindex="<%= (zTab+1)%>" title="<m4:label  
m4name="<%=zSHCOLBNEXTINS%>" htmlspecialchars="true"/>"  
href="javascript:m4wzins(2);"><img alt="<m4:label  
m4name="<%=zSHCOLBNEXTINS%>" htmlspecialchars="true"/>" title="<m4:label  
m4name="<%=zSHCOLBNEXTINS%>" htmlspecialchars="true"/>" <%@ include file="../../  
files_gif/ic_ava_ins.jsp" %> /></a>&nbsp;
```

Trabajando con Plantillas de asistente de procesos (m4gen_param_wz.js)

m4wzins

m4wzins(i)

Descripción

Función que se invoca al hacer clic en alguno de los botones de navegación de un *wizard* de proceso para que aplique el paso en el que se encuentra y cargue el paso correspondiente que se le pasa como parámetro.

Parámetros

Parámetro	Tipo	Descripción
i	Entero	Paso del wizard que carga

EJEMPLO

```
<table width="100%">
<tr><td align="center" colspan="<%=zCol%>">
<%if (zIndexWizard == 0 ){%>
<a title="<m4:label m4name="<%=zSHCOLBBACKAPP%>" htmlsafe="true"/>" >
<img alt="<m4:label m4name="<%=zSHCOLBBACKAPP%>" htmlsafe="true"/>"
title="<m4:label m4name="<%=zSHCOLBBACKAPP%>" htmlsafe="true"/>" <%@
include file="../../../files_gif/ic_ret_ins_dis_tmp.jsp" %> /></a>&nbsp;
<%}else{%>
<a tabindex="<%= (zTab+1)%>" title="<m4:label
m4name="<%=zSHCOLBBACKINS%>" htmlsafe="true"/>"
href="javascript:m4wzins(0);"><img alt="<m4:label
m4name="<%=zSHCOLBPREV%>" htmlsafe="true"/>" title="<m4:label
m4name="<%=zSHCOLBBACKINS%>" htmlsafe="true"/>" <%@ include file="../../../
files_gif/ic_ret_ins_tmp.jsp" %> /></a>&nbsp;
<%}%>
<a tabindex="<%= (zTab+1)%>" title="<m4:label m4name="<%=zSHCOLBAPP%>"
htmlsafe="true"/>" href="javascript:m4wzins(1);"><img <%@ include
file="../../../files_gif/ic_ins_tmp.jsp" %> alt="<m4:label
m4name="<%=zSHCOLBAPP%>" htmlsafe="true"/>" ></img></a>&nbsp;
<%if (zIndexWizard == (countStep -1)) { %>
<a title="<m4:label m4name="<%=zSHCOLBNEXTAPP%>" htmlsafe="true"/>" >
<img alt="<m4:label m4name="<%=zSHCOLBNEXTAPP%>" htmlsafe="true"/>"
```

```

title="<m4:label m4name="<%=zSHCOLBNEXTAPP%>" htmlspecialchars="true"/>" <%@
include file="../../../files_gif/ic_ava_ins_dis_tmp.jsp" %> /></a>&nbsp;
<%}else{%>
<a tabindex="<%= (zTab+1)%>" title="<m4:label
m4name="<%=zSHCOLBNEXTAPP%>" htmlspecialchars="true"/>"
href="javascript:m4wzins(2);" ><img alt="<m4:label
m4name="<%=zSHCOLBNEXTAPP%>" htmlspecialchars="true"/>" title="<m4:label
m4name="<%=zSHCOLBNEXTAPP%>" htmlspecialchars="true"/>" <%@ include file="../../../
files_gif/ic_ava_ins_tmp.jsp" %> /></a>&nbsp;
<%}%>
</td></tr>
</table>

```

m4wzsave

m4wzsave(i)

Descripción

Función que se invoca al hacer clic en el botón guardar de un *wizard* de proceso para que aplique el paso en el que se encuentra e invoque el método de grabar parametrizado en el Meta4Object del wizard.

Parámetros

Parámetro	Tipo	Descripción
i	Entero	Paso del wizard en el que se encuentra

EJEMPLO

```

<% // Save Button if (add_save_button == 1){
    if (save == enabled){%>
<a href="javascript:m4wzsave(1);" ><img alt="<m4:label
m4name="<%=zSHCOLBINSERT%>" htmlspecialchars="true"/>" title="<m4:label
m4name="<%=zSHCOLBINSERT%>" htmlspecialchars="true"/>" <%@ include file="../../../
files_gif/ic_save.jsp" %> /></a>&nbsp;
<%}else{%>
<a href="javascript:nullvalue();" ><img alt="<m4:label
m4name="<%=zSHCOLBINSERT%>" htmlspecialchars="true"/>" title="<m4:label
m4name="<%=zSHCOLBINSERT%>" htmlspecialchars="true"/>" <%@ include file="../../../
files_gif/ic_save_dis.jsp" %> /></a>&nbsp;<}}%>

```

m4wzins_term

m4wzins_term(i)

Descripción

Función que se invoca al hacer clic en el botón ejecutar de un *wizard* de proceso para que aplique el paso en el que se encuentra e invoque a la página que se encarga de la ejecución shco_gen_p_wz_exec. jsp.

Si en el paso en el que se encuentra se ha indicado la opción Filtro dinámico se utiliza la función m4wzins_dyn en lugar de la m4wzins_term.

Parámetros

Parámetro	Tipo	Descripción
i	Entero	Paso del wizard en el que se encuentra

EJEMPLO

```
<%if (aceptar==enabled) { %>
<%if (info_dyn.equals("0")) { %>
<a href="javascript:m4wzins_term(1);" ><img alt="m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" title="m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" <%@ include file="../
files_gif/ic_exec.jsp" %> /></a>&nbsp;
<%}else{%>
<a href="javascript:m4wzins_dyn(1);" ><img alt="m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" title="m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" <%@ include file="../
files_gif/ic_exec.jsp" %> /></a>&nbsp;
<%}%>
```

m4wzins_dyn

m4wzins_dyn(i)

Descripción

Función que se invoca al hacer clic en el botón ejecutar de un *wizard* de proceso para llamar al filtro dinámico cuando se haya parametrizado su llamada. A continuación de esta función se ejecutará m4after_filter.

Parámetros

Parámetro	Tipo	Descripción
i	Entero	Paso del wizard en el que se encuentra

EJEMPLO

```
<%if (aceptar==enabled) { %>
<%if (info_dyn.equals("0")) { %>
<a href="javascript:m4wzins_term(1);" ><img alt="<m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" title="<m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" <%@ include file=" ../
files_gif/ic_exec.jsp" %> /></a>&nbsp;
<%}else{%>
<a href="javascript:m4wzins_dyn(1);" ><img alt="<m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" title="<m4:label
m4name="<%=zSHCOLBEXEC%>" htmlsafe="true"/>" <%@ include file=" ../
files_gif/ic_exec.jsp" %> /></a>&nbsp;
<%}%>
```

m4after_filter

m4after_filter()

Descripción

Función que se invoca si se ha marcado la opción de filtro dinámico. Su objetivo es guardar el filtro dinámico que se establezca y a continuación invocar la página que se encarga de la ejecución shco_gen_p_wz_exec. jsp.

EJEMPLO

```
<script type="text/javascript" language="Javascript1.5">
m4dynfilter('<%=zsubsesion%>','<%=zMO%>','<%=zMOApply%>','3','','','frmcall
dynfilter','m4after_filter()','zdynfiltersinfo');
</script>
<%}%>
```

m4generate_merge_word

m4generate_merge_word(vData,vTemplate,vurlData,vurlTemplate)

Descripción

Función que se invoca para ejecutar la combinación de correspondencia con Microsoft Word.

Parámetros

Parámetro	Tipo	Descripción
vData	Cadena	Nombre del fichero que contiene los datos
vTemplate	Cadena	Nombre de la plantilla
vurlData	Cadena	Ruta del fichero de datos
vurlTemplate	Cadena	Ruta del fichero de la plantilla

EJEMPLO

```
<script type="text/javascript">
var
vsol=m4generate_merge_word('<%=zData%>','<%=zTemplate%>','<%=sURLxlsTp
ltData%>','<%=sURLxlsTpltTemplate%>');
</script>
```

Trabajando con dual-list (m4duallist.js)

A la hora de trabajar con *dual-list*, considerando como *duallist* a dos select que se pasan datos de una a otra de forma gráfica, tal y como vemos en el dibujo, se tiene una librería que implementa las funciones más útiles a usar en este tipo de casos.

Esta librería, no se incluye de manera estándar en las cabeceras genéricas y habrá que incluirla en aquellas páginas que la vayan a utilizar de la siguiente forma:

```
<script type="text/javascript" src="/library/m4duallist.js"></
script>
```

Figura 82. Ejemplo de una lista dual



m4funoptionselected

m4funoptionselected(oSelect,sOptionProp,sFunction)

Descripción

Función que permite ejecutar sobre la propiedad *sOptionProp* de los elementos seleccionados de la lista *oSelect* la función *sFunction*.

La implementación de la función deberá estar en la página.

Parámetros

Parámetro	Tipo	Descripción
oSelect	Objeto Select	Objeto select sobre el que se actúa. Concretamente sobre los elementos seleccionados
sOptionProp	Cadena	Propiedad sobre la que se actúa (id, text, value).
sFunction	Cadena	Función a aplicar sobre los elementos seleccionados. Esta función tomará como primer argumento la propiedad del elemento sobre la que actúa y en caso de requerir más argumentos estos deberán pasarse como argumentos opcionales de la función en el orden apropiado.

EJEMPLO

```
function anadirOp(sTextOrigen,sOp){...}  
<input type="button" " name="btnAddOp" id="btnAddOp" value="+"  
onclick="m4funoptionselected(m4objeto('Form1','list2'),'text','anadir  
Op','+')>  
<br />
```

m4moveoption

m4moveoption(oSelectFrom,oSelectTo,bMoveAll)

Descripción

Función que mueve los elementos seleccionados de la lista origen (*oSelectFrom*) a la lista destino (*oSelectTo*). Si *bMoveAll* = true, mueve todos

los elementos de la origen a la destino.

Parámetros

Parámetro	Tipo	Descripción
oSelectFrom	Objeto Select	Objeto de tipo select, desde donde se van a mover los elementos.
oSelectTo	Objeto Select	Objeto de tipo select destino del movimiento.
bMoveAll	Booleano	Para indicar si se quieren mover todos los elementos o solo los seleccionados.

EJEMPLO

<td>

```
<input type="button" name="btnRightMoveAll" id="btnRightMoveAll"
value=" >> "
onclick="m4moveoption(m4objeto('Form1','list1'),m4objeto('Form1','list2'),true);">
<br/>
```

```
<input type="button" name="btnRightMoveSel" id="btnRightMoveSel"
value=" > "
onclick="m4moveoption(m4objeto('Form1','list1'),m4objeto('Form1','list2'),false);">
<br/>
```

```
<input type="button" name="btnLeftMoveSel" id="btnLeftMoveSel"
value=" < "
onclick="m4moveoption(m4objeto('Form1','list2'),m4objeto('Form1','list1'),false);">
<br/>
```

```
<input type="button" name="btnLeftMoveAll" id="btnLeftMoveAll"
value=" << "
onclick="m4moveoption(m4objeto('Form1','list2'),m4objeto('Form1','list1'),true);">
<br/>
```

</td>

m4updownoption

m4updownoption(oSelect,sMode)

Descripción

Función que se encarga de cambiar la posición (subir/bajar) del elemento seleccionado de la lista (*oSelect*).

Parámetros

Parámetro	Tipo	Descripción
Oselect	Objeto Select	Objeto select sobre el que se actúa. Concretamente sobre el elemento seleccionado.
Smode	Cadena	'up' / 'down' : Modo de mover el objeto seleccionado (subir/bajar).

EJEMPLO

```
<td>
<input type="button" name="btnUp" id=" btnUp" value="Subir"
onclick="m4updownoption(m4objeto ('Form1','list2'),'up');" ><br />
<input type="button" name="btnDown" id=" btnDown "> value="Bajar"
onclick="m4updownoption(m4objeto ('Form1','list2'),'down');" </td>
```

Gestión de errores y excepciones

Excepciones (m4gen_excep.js)

En esta librería, se tienen definidas una serie de clases de excepciones que luego se utilizan a la hora de especificar errores en las invocaciones a funciones javascript.

Se utilizarán en el desarrollado interno de funciones javascript.

La generación de nuevas excepciones se realizará heredando de alguna de las anteriores. Esto solo se podrá realizar de manera controlada.

m4class_excepcion

m4class_excepcion(sFunctionName)

Descripción

Clase padre, de la cual heredan todas las demás excepciones de desarrollador.

Parámetros

Parámetro	Tipo	Descripción
sFunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.

m4class_atributonodefinido

m4class_atributonodefinido(sFunctionName,sIdObject,sAttribute)

Descripción

Excepción que ha de lanzarse cuando se intente acceder a una propiedad inexistente de un objeto.

Parámetros

Parámetro	Tipo	Descripción
SFunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.
sIdObject	Cadena	Objeto donde se busca el atributo.

Parámetro	Tipo	Descripción
sAttribute	Cadena	Identificador del atributo que se está buscando

m4class_noexisteelemento

m4class_noexisteelemento(sFunctionName,sIdForm, sIdElement)

Descripción

Excepción que ha de lanzarse cuando se intente acceder a un elemento de un formulario que no existe.

Parámetros

Parámetro	Tipo	Descripción
SFunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.
sIdForm	Cadena	Identificador del formulario donde se busca el objeto
sIdElement	Cadena	Elemento que se busca en el formulario.

m4class_noexisteobjeto

m4class_noexisteobjeto (sFunctionName,sIdElement)

Descripción

Excepción que ha de lanzarse cuando se intente acceder a un elemento/objeto del objeto *Document* de javascript que no existe. No usar para objetos dentro de un formulario, en ese caso usar (m4class_noexisteelemento).

Parámetros

Parámetro	Tipo	Descripción
SfunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.
SIdElement	Cadena	Elemento que se busca.

m4class_indicefuera rango

m4class_indicefuera rango (sFunctionName)

Descripción

Excepción que ha de lanzarse cuando se intente acceder a un índice (de una matriz, de un *for*, de una *select*,...etc) que no existe.

Parámetros

Parámetro	Tipo	Descripción
SfunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción

m4class_tipoerroneo

m4class_tipoerroneo(sFunctionName,sVar,sType,sWrongType)

Descripción

Excepción que ha de lanzarse cuando una variable o un parámetro de una función no es del tipo requerido por la función

Parámetros

Parámetro	Tipo	Descripción
sFunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.
sVar	Cadena	Nombre de la variable/parámetro.
sType	Cadena	Tipo esperado.
sWrongType	Cadena	Tipo recibido que es incorrecto.

m4class_modonodefinido

m4class_modonodefinido(sFunctionName,sWrongMode)

Descripción

Excepción que ha de lanzarse cuando un parámetro de una función que representa un modo de funcionamiento de la misma, no es correcto.

Parámetros

Parámetro	Tipo	Descripción
sFunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.
sWrongMode	Cadena	Modo incorrecto que se recibe.

m4class_numparametrosincorrecto

m4class_numparametrosincorrecto(sFunctionName,nParamNum,nWrongParamNum)

Descripción

Excepción que ha de lanzarse cuando el número de parámetros con los que se invoca a una función son insuficientes.

Parámetros

Parámetro	Tipo	Descripción
sFunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.
nParamNum	Entero	Número de parámetros que espera la función.
nWrongParamNum	Entero	Número de parámetros con los que se invoca a la función.

m4class_constanteindefinida

m4class_constanteindefinida (sFunctionName,sConstName)

Descripción

Excepción que ha de lanzarse cuando faltan las definiciones de constantes requeridas en la página. Se debe lanzar una excepción en la función que vaya a utilizar la constante.

Parámetros

Parámetro	Tipo	Descripción
sFunctionName	Cadena	Nombre de la función dentro de la que se ha producido la excepción.

Parámetro	Tipo	Descripción
sConstName	Cadena	Nombre de la constante no definida.

Excepciones (Ejemplo de uso)

Función que lanza la excepción

```
function m4funcion(sparam1,sparam2){
    .....
    if (m4funcion.arguments.length < 2) throw (new
    m4class_numparametrosincorrecto("m4funcion",m4funcion.arguments.length,2);
    .....
}
```

Invocación a una función que lanza excepciones. Requiere que se ejecute dentro de una sentencia *try/catch* que recoja la excepción, de la forma:

```
try{
    .....
    m4funcion("parametro1");
    .....
}
catch{
    m4err_gen(excepcion);
}
```



Cuando la excepción es particular del usuario, en el *catch*{} debe llevar el bloque de código o la llamada a la función que la trate y no la llamada a la función de *m4err_gen* (que es la de tratamiento genérico).

Funciones genéricas

Estas funciones realizan el tratamiento de mensajes de error definidos de la siguiente forma:

El mensaje de error deberá estar definido a través de una variable javascript en los archivo de error correspondientes(*m4err_xxx_en.js*, *m4err_xxx_fr.js*, *m4err_xxx_es.js*).

Estos mensajes podrán tener parámetros, en cuyo caso hay que indicarlos de la forma *&%i&*, donde *i* es la posición del parámetro dentro de la cadena que

representa el mensaje de error.

Al invocar a cualquiera de la funciones de tratamiento de errores, hay que pasar los argumentos definidos en el mensaje de forma que, el primer argumento será sustituido en el mensaje de error sobre `&%0&`, el segundo sobre `&%1&` ... y así sucesivamente.

EJEMPLO

Mensaje de error identificado por *smen* con tres parámetros.

Definición	<code>var smen = "El campo &%0& no es un &%1& y además es &%2&";</code>
Invocación	<code>m4getmessage("smen","nombre","string","obligatorio");</code>
Mensaje de error resultado	"El campo <i>nombre</i> no es un <i>string</i> y además es <i>obligatorio</i> "

m4getmessage

`m4getmessage (sIdErrorMessage[,MessageArg1.. MessageArgn])`

`m4getmessage (sIdErrorMessage [,arrMesageArg])`

Descripción

Devuelve una cadena con el mensaje identificado por `sidmessage` con la sustitución correspondiente de los argumentos. No muestra el mensaje.

Los argumentos pueden venir separados por comas o directamente en un objeto *array*.

Parámetros

Parámetro	Tipo	Descripción
<code>SidErrorMessage</code>	Cadena	Identificador del mensaje de error a mostrar y que debe estar definido en el archivo de errores correspondiente. Corresponde con el nombre de la variable javascript sobre la que se asigna el mensaje de error.
<code>MessageArg_i</code>	Cadena	Argumento del mensaje de error de la posición <i>i</i> , a sustituir en la cadena del error.

Parámetro	Tipo	Descripción
ArrMesageArg	Array	Array con los argumentos en el orden de sustitución.

m4showmessage

m4showmessage(sIdErrorMessage [,MessageArg₁.. MessageArg_n])

m4showmessage(sIdErrorMessage [,arrMesageArg])

Descripción

Muestra el mensaje identificado por *sIdErrorMessage* con la sustitución correspondiente de los argumentos (uso del *m4getmessage*). Los argumentos pueden venir separados por comas o directamente en un objeto *array*.

Parámetros

Parámetro	Tipo	Descripción
SIdErrorMessage	Cadena	Identificador del mensaje de error a mostrar y que debe estar definido en el archivo de errores correspondiente. Corresponde con el nombre de la variable javascript sobre la que se asigna el mensaje de error.
MessageArg _i	Cadena	Argumento del mensaje de error de la posición i, a sustituir en la cadena del error.
ArrMesageArg	Array	Array con los argumentos en el orden de sustitución.

m4setlog (Uso interno)

m4setlog (sIdErrorMessage[,MessageArg₁.. MessageArg_n])

m4setlog (sIdErrorMessage [,arrMesageArg])

Descripción

Función que se encarga de devolver o mostrar el mensaje de error identificado por *sIdErrorMessage* haciendo previamente la sustitución de los parámetros que se pasan.

Este mensaje vendrá precedido por la cadena genérica: "Se han producido los siguientes mensajes de error:"

Se utiliza en las validaciones de forma que se van concatenando todos los

posibles errores.

Funcionamientos:

- Actúa como m4getmessage si los parámetros se pasan vía *array*
- Actúa como m4showmessage si:
 - los parámetros vienen directamente como cadenas separadas por comas, es decir, no vienen vía *array*.
 - No se le pasan argumentos para el mensaje.

Parámetros

Parámetro	Tipo	Descripción
SidErrorMessage	Cadena	Identificador del mensaje de error a mostrar y que debe estar definido en el archivo de errores correspondiente. Corresponde con el nombre de la variable javascript sobre la que se asigna el mensaje de error.
MessageArg _i	Cadena	Argumento del mensaje de error de la posición i, a sustituir en la cadena del error.

Apéndice: Archivos no modificables



Acerca de este capítulo

Este capítulo describe las páginas desarrolladas que no pueden ser modificadas.

Para montar un producto Web se suministran una serie de páginas tecnológicas no modificables. A este nivel distinguimos entre archivos (páginas JSP, HTML, javascript, gif, etc.) y objetos del repositorio.

Archivos

Existen carpetas cuyo contenido íntegro no se puede modificar. Estas carpetas son:

- config
- help_tec
- image\english
- image\espanol
- images\tcreports
- images\tcadmin
- m4jspc
- m4jssi
- reports
- shco_g0_tec
- shco_js
- shco_rp
- shco_so
- shco_td
- shco_tl
- sitetest
- tc_query
- tcadmin
- tcpubdocs
- tctools

Además de los ficheros contenidos en los directorios anteriores, existe un conjunto de ficheros que tampoco se pueden modificar. A continuación se muestra una relación de dichos ficheros agrupados según el directorio en el que se encuentran. Dentro de cada directorio, se agrupan por idiomas y dentro del epígrafe **Comunes** se relacionan los archivos comunes a todos los idiomas y que se encuentran en el directorio raíz.

files_gif

ic_ace.jsp
ic_ace_code.jsp
ic_apply_filter.jsp
ic_apply_filter_code.jsp
ic_arrow_back.jsp
ic_arrow_back_code.jsp
ic_arrowdown2.jsp
ic_arrowdown2_code.jsp
ic_arrowup2.jsp
ic_arrowup2_code.jsp
ic_ava.jsp
ic_ava_code.jsp
ic_ava_dis.jsp
ic_ava_dis_code.jsp
ic_ava_dis_tmp.jsp
ic_ava_dis_tmp_code.jsp
ic_ava_end.jsp
ic_ava_end_code.jsp
ic_ava_ins.jsp
ic_ava_ins_code.jsp
ic_ava_ins_dis.jsp
ic_ava_ins_dis_code.jsp
ic_ava_ins_dis_tmp.jsp
ic_ava_ins_dis_tmp_code.jsp
ic_ava_ins_tmp.jsp
ic_ava_ins_tmp_code.jsp
ic_ava_tmp.jsp
ic_ava_tmp_code.jsp
ic_bor.jsp
ic_bor_code.jsp
ic_bor_dis.jsp
ic_bor_dis_code.jsp
ic_bus_all.jsp
ic_bus_all_code.jsp
ic_bus_all_des.jsp
ic_bus_all_des_code.jsp
ic_cabec.jsp
ic_cabec_code.jsp

ic_cal.jsp
ic_cal_code.jsp
ic_can.jsp
ic_can_code.jsp
ic_cer.jsp
ic_cer_code.jsp
ic_delrow.jsp
ic_delrow_code.jsp
ic_delttable.jsp
ic_delttable_code.jsp
ic_des.jsp
ic_des_code.jsp
ic_des2.jsp
ic_des2_code.jsp
ic_details.jsp
ic_details_code.jsp
ic_dup.jsp
ic_dup_code.jsp
ic_dup_dis.jsp
ic_dup_dis_code.jsp
ic_env.jsp
ic_env_code.jsp
ic_err.jsp
ic_err_code.jsp
ic_exec.jsp
ic_exec_code.jsp
ic_exec_off.jsp
ic_exec_off_code.jsp
ic_executed_items.jsp
ic_executed_items_code.jsp
ic_executed_task.jsp
ic_executed_task_code.jsp
ic_filter.jsp
ic_filter_code.jsp
ic_help.jsp
ic_help_code.jsp
ic_hide_details.jsp
ic_hide_details_code.jsp
ic_info.jsp
ic_info_code.jsp
ic_ins.jsp

ic_ins_code.jsp
ic_ins_tmp.jsp
ic_ins_tmp_code.jsp
ic_ins_tmp_dis.jsp
ic_ins_tmp_dis_code.jsp
ic_ins_up.jsp
ic_ins_up_code.jsp
ic_lis.jsp
ic_lis_code.jsp
ic_lis_des.jsp
ic_lis_des_code.jsp
ic_list.jsp
ic_list_code.jsp
ic_memu_mod.jsp
ic_memu_mod_code.jsp
ic_meta4.jsp
ic_mod.jsp
ic_mod_code.jsp
ic_mod_dis.jsp
ic_mod_dis_code.jsp
ic_new.jsp
ic_new_code.jsp
ic_new16.jsp
ic_new16_code.jsp
ic_nohelp.jsp
ic_nohelp_code.jsp
ic_ord.jsp
ic_ord_1.jsp
ic_ord_1_code.jsp
ic_ord_2.jsp
ic_ord_2_code.jsp
ic_ord_code.jsp
ic_p_wz_menu.jsp
ic_p_wz_menu_code.jsp
ic_pay.jsp
ic_pay_code.jsp
ic_reload.jsp
ic_reload_code.jsp
ic_report.jsp
ic_report_code.jsp
ic_ret.jsp

ic_ret_beg.jsp
ic_ret_beg_code.jsp
ic_ret_code.jsp
ic_ret_dis.jsp
ic_ret_dis_code.jsp
ic_ret_ins.jsp
ic_ret_ins_code.jsp
ic_ret_ins_dis.jsp
ic_ret_ins_dis_code.jsp
ic_ret_ins_dis_tmp.jsp
ic_ret_ins_dis_tmp_code.jsp
ic_ret_ins_tmp.jsp
ic_ret_ins_tmp_code.jsp
ic_save.jsp
ic_save_code.jsp
ic_save_dis.jsp
ic_save_dis_code.jsp
ic_search.jsp
ic_search_code.jsp
ic_warnig.jsp
ic_warnig_code.jsp
ic_wz_menu.jsp
ic_wz_menu_code.jsp

Images

Todas las carpetas dependientes de images:

english
espanol
tcadmin
tcreports

Comunes

arrow.gif
bola1.gif
bola2.gif
error.gif
flecha.gif
fondo.gif
fondo2.jpg
ic_ace_36_36_100.gif
ic_apply_filter_36_36_100.gif
ic_arrow_back_36_36_100.gif
ic_arrowdown2_36_36_100.gif
ic_arrowup2_36_36_100.gif
ic_ava_36_36_100.gif
ic_ava_dis_36_36_100.gif
ic_ava_end_36_36_100.gif
ic_ava_ins_36_36_100.gif
ic_ava_ins_dis_36_36_100.gif
ic_ava_ins_dis_tmp_36_36_100
ic_ava_ins_tmp_36_36_100.gif
ic_ava_tmp_36_36_100.gif
ic_bor_16_16_100.gif
ic_bor_36_36_100.gif
ic_bor_dis_16_16_100.gif
ic_bus_all_36_36_100.gif
ic_cabec_56_51_100.gif
ic_cal_14_18_100.gif
ic_can_36_36_100.gif
ic_compcpt_16_16_100.gif
ic_compfpfi_16_16_100.gif

ic_compprop_16_16_100.gif
ic_comptotal_16_16_100.gif
ic_compvar_16_16_100.gif
ic_del_36_36_100.gif
ic_delrow_36_36_100.gif
ic_deltable_36_36_100.gif
ic_des_12_12_100.gif
ic_des2_12_12_100.gif
ic_details_16_16_100.gif
ic_dup_16_16_100.gif
ic_dup_dis_16_16_100.gif
ic_env_36_36_100.gif
ic_err_28_28_100.gif
ic_exec_36_36_100.gif
ic_exec_off_36_36_100.gif
ic_executed_items_36_36_100.
ic_executed_task_16_16_100.g
ic_filter_36_36_100.gif
ic_headmiddle_100.gif
ic_help_25_31_100.gif
ic_hide_details_36_36_100.gi
ic_info_28_28_100.gif
ic_ins_36_36_100.gif
ic_ins_tmp_36_36_100.gif
ic_ins_tmp_dis_36_36_100.gif
ic_ins_up_36_36_100.gif
ic_lis_36_36_100.gif
ic_lis_des_36_36_100.gif
ic_list_16_16_100.gif
ic_log_in_36_36.gif
ic_menu_6_147_100_100.gif
ic_meta4_136_38.gif
ic_mod_16_16_100.gif
ic_mod_dis_16_16_100.gif
ic_mods_25_16_100.gif
ic_new_16_16_100.gif
ic_new_36_36_100.gif
ic_nohelp_25_31_100.gif
ic_ord_15_15_100.gif
ic_ord_down_15_15_100.gif
ic_ord_up_15_15_100.gif

ic_pagtable_100.gif
ic_pay_16_16_100.gif
ic_reload_36_36_100.gif
ic_report_graph_36_36_100.gi
ic_ret_36_36_100.gif
ic_ret_beg_36_36_100.gif
ic_ret_dis_36_36_100.gif
ic_ret_ins_36_36_100.gif
ic_ret_ins_dis_36_36_100.gif
ic_ret_ins_dis_tmp_36_36_100
ic_ret_ins_tmp_36_36_100.gif
ic_sav_36_36_100.gif
ic_sav_dis_36_36_100.gif
ic_search_36_36_100.gif
ic_warnig_28_28_100.gif
icono_aceptar_ess_36_36.gif
icono_anterior_36_36.gif
icono_enviar_ess_36_36.gif
icono_siguiete_ess_36_36.gi
logotipo.gif
topo1.gif
ver2.gif
webserviceslogo.gif
wl_calendar.gif

library

English, espanol

calendar.jsp
class_calendar.js
dom_1.js
functions_1.js
functions_tctools.js
functions_wklist.js
tctools_errors.js

Comunes

claseformu.js
date.js
m4calendar.js
m4class_calendar.js
m4class_crosstab.js
m4class_timetable.js
m4duallist.js
m4gen.js
m4gen_adv_functions.js
m4gen_cr.js
m4gen_excep.js
m4gen_list.js
m4gen_list_ck.js
m4gen_mt.js
m4gen_param_wz.js
m4gen_wz.js
m4help.js
m4ie_light.js
m4listconcepts.js
m4listfiltertypes.js
m4menu.js
m4tree.js
m4treeview.js
m4val.js
m4valdata.js
openwin.js

shco_g0

english, espanol, francais, portugues

shco_gen_act.jsp
shco_gen_act_wz.jsp
shco_gen_calendar.jsp
shco_gen_currency.jsp
shco_gen_dynfilter.jsp
shco_gen_htmlfilter.jsp
shco_gen_htmlfilter_name.jsp
shco_gen_inf.jsp
shco_gen_list_scenarios.js
shco_gen_list_table_filter_pred.jsp
shco_gen_p_wz_end.jsp
shco_gen_p_wz_exec.jsp
shco_gen_param_page_maker.jsp
shco_gen_param_process_execute.jsp
shco_gen_path.jsp
shco_gen_tr.jsp
shco_gen_wz_def.jsp
shco_gen_wz_end.jsp
shco_gen_xml_table_filter_pred.jsp
shco_show_error.jsp

Comunes

shco_g0_trans.jsp
shco_gen_act.jsp
shco_gen_act_body.jsp
shco_gen_arg.jsp
shco_gen_bag.jsp
shco_gen_cab.jsp
shco_gen_cab_link.jsp
shco_gen_cal_param.jsp
shco_gen_calendar.jsp
shco_gen_css.jsp
shco_gen_currency.jsp
shco_gen_datadef.jsp

shco_gen_delete_cache.jsp
shco_gen_disclaimer.jsp
shco_gen_dynfilter.jsp
shco_gen_dynfilter_act.jsp
shco_gen_dynfilter_include.jsp
shco_gen_dynfilter_list.jsp
shco_gen_dynfilter_m4def.jsp
shco_gen_dynfilter_simplified.jsp
shco_gen_error.jsp
shco_gen_exec.jsp
shco_gen_formats.jsp
shco_gen_functions.jsp
shco_gen_help.jsp
shco_gen_help_link.jsp
shco_gen_htmlfilter.jsp
shco_gen_htmlfilter_act.jsp
shco_gen_htmlfilter_m4def.js
shco_gen_images.jsp
shco_gen_images_arg.jsp
shco_gen_include_disclaimer.
shco_gen_inf.jsp
shco_gen_js.jsp
shco_gen_label.jsp
shco_gen_label_val.jsp
shco_gen_lang.jsp
shco_gen_list_arg.jsp
shco_gen_list_ck_barbot.jsp
shco_gen_list_ck_js.jsp
shco_gen_list_count.jsp
shco_gen_list_filt.jsp
shco_gen_list_filt_SchTask.jsp
shco_gen_list_filt_SchTaskChecks.jsp
shco_gen_list_filter.jsp
shco_gen_list_filter_buttons.jspf
shco_gen_list_js.jsp
shco_gen_list_nodata.jsp
shco_gen_list_outputdef.jsp
shco_gen_list_preload.jsp
shco_gen_load_js_msg.jsp
shco_gen_login_box.jsp
shco_gen_logout.jsp

shco_gen_loop.jsp
shco_gen_menu.jsp
shco_gen_menus.jsp
shco_gen_menus_load.jsp
shco_gen_menusup.jsp
shco_gen_mt_arg.jsp
shco_gen_mt_bus.jsp
shco_gen_mt_count.jsp
shco_gen_mt_filter.jsp
shco_gen_mt_js.jsp
shco_gen_mt_outputdef.jsp
shco_gen_mt_preload.jsp
shco_gen_normal_arg.jsp
shco_gen_normal_js.jsp
shco_gen_p_wz_act.jsp
shco_gen_p_wz_body.jsp
shco_gen_p_wz_btt.jsp
shco_gen_p_wz_btt_vis.jsp
shco_gen_p_wz_datadef.jsp
shco_gen_p_wz_def.jsp
shco_gen_p_wz_end.jsp
shco_gen_p_wz_error.jsp
shco_gen_p_wz_exec.jsp
shco_gen_p_wz_form.jsp
shco_gen_p_wz_js.jsp
shco_gen_p_wz_menu.jsp
shco_gen_p_wz_nav.jsp
shco_gen_param_page_maker.js
shco_gen_param_page_maker_act.jsp
shco_gen_param_process_execute.jsp
shco_gen_path.jsp
shco_gen_pest.jsp
shco_gen_portal_arg.jsp
shco_gen_portal_bag.jsp
shco_gen_portal_bag.jspf
shco_gen_portal_code_begin.jspf
shco_gen_portal_code_end.jspf
shco_gen_portal_load.jsp
shco_gen_schedule_include.jsp
shco_gen_ss_act.jsp
shco_gen_taglib.jsp

shco_gen_taglib_tree.jsp
shco_gen_tec_include.jspf
shco_gen_title.jsp
shco_gen_today.jsp
shco_gen_topmenu.jsp
shco_gen_tr.jsp
shco_gen_tree_buttons.jsp
shco_gen_tree_filter.jsp
shco_gen_tree_js.jsp
shco_gen_tree_search.jsp
shco_gen_treeview_0.jsp
shco_gen_treeview_1.jsp
shco_gen_treeview_2.jsp
shco_gen_vent_post.jsp
shco_gen_vent_toomuch.jsp
shco_gen_wz_act.jsp
shco_gen_wz_asterisc.jsp
shco_gen_wz_btt.jsp
shco_gen_wz_btt_inc.jsp
shco_gen_wz_def.jsp
shco_gen_wz_end.jsp
shco_gen_wz_error.jsp
shco_gen_wz_input.jsp
shco_gen_wz_js.jsp
shco_gen_wz_menu.jsp
shco_gen_wz_menu_inc.jsp
shco_gen_wz_nav.jsp
shco_login_box_trans.jsp
shco_menu.jsp
shco_show_error.jsp

shco_py

english, espanol, francais, portugues

shco_defaultupdate.jsp
shco_defaultvalues.jsp
shco_historyvalues.jsp
shco_list_concepts.jsp
shco_list_iframe_fpi.jsp

shco_list_iframe_totals.jsp
shco_list_iframe_variables.jsp
shco_proyectpk.jsp
shco_py_gen_datadef.jsp
shco_py_gen_list_filt.jsp
shco_py_grp_dmd_role.jsp
shco_py_incident_values.js
shco_py_list_items.jsp
shco_py_list_payroll_concepts.jsp
shco_pyvalues.jsp
shco_pyvalues_slices.jsp

comunes

m4valuesmain.js
shco_defaultupdatejob.jsp
shco_defaultupdatepage.jsp
shco_defaultvaluesjob.jsp
shco_defaultvaluespage.jsp
shco_historyvaluesjob.jsp
shco_historyvaluespage.jsp
shco_list_concepts.jsp
shco_list_concepts_load.jsp
shco_list_filteritem.jsp
shco_list_iframe_fpi.jsp
shco_list_iframe_totals.jsp
shco_list_iframe_variables.j
shco_proyectpk00.jsp
shco_proyectpkcell.jsp
shco_proyectpkcol.jsp
shco_proyectpkjob.jsp
shco_proyectpkpage.jsp
shco_proyectpkrow.jsp
shco_py_def_cssuser.jspf
shco_py_incident_valuesjob.j
shco_py_incident_valuespage.
shco_pyincerrors.jsp
shco_pyinchelp.jsp
shco_pyvalues_slicesjob.jsp
shco_pyvalues_slicespage.jsp

shco_pyvaluescell00.jsp
shco_pyvaluescellbody.jsp
shco_pyvaluescellhcol.jsp
shco_pyvaluescellhrow.jsp
shco_pyvaluesjob.jsp
shco_pyvaluespage.jsp

shco_py_sse

english, espanol, francais, portugues

shco_py_list_ht_pays.jsp
shco_py_list_pay_freq.jsp

style

jsstyle.css
shco_gen100.css
tc_login.css
tcadmin.css
tcreports_0.css
tech_0.css

translation

m4err_en.js
m4err_es.js
m4err_fr.js
m4err_pt.js
m4jspc_en.properties
m4jspc_es.properties
m4jspc_fr.properties
m4jspc_pt.properties
m4jspsi_en.properties
m4jspsi_es.properties
m4jspsi_fr.properties
m4jspsi_pt.properties
shco_g0_en.properties
shco_g0_es.properties
shco_g0_fr.properties
shco_g0_pt.properties
shco_js_err_en.js
shco_js_err_es.js
shco_js_err_fr.js
shco_js_err_pt.js
shco_login_box_en.properties.jsp
shco_login_box_es.properties.jsp
shco_login_box_fr.properties.jsp
shco_login_box_pt.properties.jsp
shco_portal_code_en.properties.jsp
shco_portal_code_es.properties.jsp
shco_portal_code_fr.properties.jsp
shco_portal_code_pt.properties.jsp
shco_pyvaluespage_en.properties.jsp
shco_pyvaluespage_es.properties.jsp
shco_pyvaluespage_fr.properties.jsp
shco_pyvaluespage_pt.properties.jsp
shco_rp_en.properties
shco_rp_es.properties
shco_rp_fr.properties
shco_rp_pt.properties
shco_show_error_en.properties.jsp
shco_show_error_es.properties.jsp

shco_show_error_fr.properties.jsp
shco_show_error_pt.properties.jsp
shco_so_en.properties
shco_so_es.properties
shco_so_fr.properties
shco_so_pt.properties
shco_tl_en.properties
shco_tl_es.properties
shco_tl_fr.properties
shco_tl_pt.properties
shco_tl_menu_en.properties
shco_tl_menu_es.properties
shco_tl_menu_fr.properties
shco_tl_menu_pt.properties
tc_login_en.properties
tc_login_es.properties
tc_login_fr.properties
tc_login_pt.properties
tc_query_en.properties
tc_query_err_en.js
tc_query_err_es.js
tc_query_err_fr.js
tc_query_err_pt.js
tc_query_es.properties
tc_query_fr.properties
tc_query_pt.properties
tcadmin_soapservices_en.properties.jsp
tcadmin_soapservices_es.properties.jsp
tcadmin_soapservices_fr.properties.jsp
tcadmin_soapservices_pt.properties.jsp

META4OBJECT

SCO_FILTER_MAINTENANCE
SHCO_TD_MT_TH_TASK
SCO_MT_DEF_TAB_GRP
SCO_MT_PARAM_VALUES
SCO_TR_DEF_TAB_GRP
SHCO_GN_TR
SHCO_GN_TREE
SHCO_GN_VALUES
SHCO_TD_WKLIST_F
SHCO_TD_WZ_SWK_WORKLIST
SHCO_TL_MT_CHANGE_PASSWORD
SHCO_TL_MT_PARAM
SHCO_TL_SESSION_F
SHCO_JS_WZ_TASK_EXE
API_FILTER_HTML_CL
SHCO_GN_MT
SHCO_CH_MT_ORGANIZATION
SHCO_GN_WIZ
SHCO_GN_DYNFILTER
SHCO_GN_MT_GROUP_OBJS
SHCO_GN_ORG
SHCO_GN_ORG_FREE
SHCO_GN_PARAM_VALUES
SHCO_GN_PARAM_WIZ
SHCO_GN_PORTAL_TEC
SHCO_GN_ROOT
SHCO_FILTER_HTML
SHCO_TD_MT_SWK_WORKLIST
SHCO_GN_SPECIAL_CHARS
SHCO_PY_MT_PAYS
SHCO_PY_V_CONCEPTS
SHCO_PY_V_PAYROLLITEMS
SHCO_RP_MT_PUB_REPORTS
SHCO_RP_PUB_REPORTS
SHCO_TC_MENUS

SHCO_TD_MT_RMD_WKLIST
SHCO_JS_LIST_TASK_EXE
SHCO_TD_MT_ASG_WKLIST
SCH_CH_LIST_APPROL
SCH_CH_LIST_SCHED_TASKS
SCH_CH_LIST_TASK_EXE
SCH_CH_MT_CURRENCY
SCH_CH_MT_DEFINITION
SCH_CH_MT_LU_LANGUAGES
SCH_CH_MT_PRIORITY
SCH_CH_MT_SITE_DEF
SCH_CH_MT_TIPO_PERI
SCH_CH_MT_ZONEINFO_DST
SCH_CH_WIZARD_PLANIFICA
SHCO_AP_WZ_SCHED_TASK
SHCO_JS_LIST_TASK_EXE
SHCO_JS_WZ_TASK_EXE

BUSINESS PROCESS

SCO_GN_BP_FILTER_MAINTENANCE
M4JSPC
M4JSPSI
MN_DEF_GRP_TAB
SCO_GN_BP_DUPLICATE_FILTERS
TCADMIN
SCO_GN_BP_USR_PARAM
SHCO_ADV_TL
SHCO_G0
SHCO_RP
SHCO_S0
SHCO_TD
SHCO_TL

LOGICAL TABLE

SCO_FILTER_KEY
SCO_DEF_TAB_GRP
SCH_BO_ARG_TYPE
SCO_FILTER
SCO_FILTER_KEY_NM
SCO_HTML_CHARS
SCO_PARAM_VALUES
SCO_TMP_DAT_WIZ
SCO_USER_GROUPS
SCO_USR_GRP_COMP
SCO_FIELD_TYPE

APP_ROLE

SHCO_ADV_DEVELOPER

MENU OPTION

MN_TH_TOOLS_0_1_2
MN_Default_Grp_Value_Table
MN_GN_LIGHT_CLIENT_CONFIG
MN_PUB_CATEGORIES
MN_TC_MENUS
MN_TH_ADV_TOOLS
MN_TH_ADV_TOOLS_0_1
MN_TH_ADV_TOOLS_0_2
MN_TH_G0
MN_TH_MENUS_LEVEL_1
MN_TH_MENUS_LEVEL_6
MN_TH_TOOLS
MN_TH_TOOLS_2
SCO_GN_MN_USR_PARAM
SCO_GN_MN_DUPLICATE_FILTERS
MN_TH_TOOLS_7
MN_TH_TOOLS_6
MN_TH_TOOLS_0_1
MN_TH_TOOLS_3

MN_TH_TOOLS_0_1_1
MN_TH_TOOLS_1
MN_TH_TOOLS_0_1_6
MN_TH_TOOLS_0_1_5
MN_TH_TOOLS_0_1_4
MN_TH_TOOLS_0_1_3
SHCO_GN_MENU_CONFIG
MN_TH_TOOLS_4

PRESENTATION

SCO_MT_PARAM_VALUES
SCO_MT_DEF_TAB_GRP
SCO_FILTER_MAINTENANCE
SCO_DUPLICATE_FILTERS

APP PARAM

THIN_CLIENT/PORTAL_PARAM/DEBUG
THIN_CLIENT/PORTAL_PARAM/DEBUG_WIZARD
THIN_CLIENT/PORTAL_PARAM/_APP_MN
THIN_CLIENT/PORTAL_PARAM/DEBUG
THIN_CLIENT/PORTAL_PARAM/DEBUG

Apéndice: Recursos adicionales



Acerca de este capítulo

En este capítulo se enuncia una serie de recursos bibliográficos que puedan servir de apoyo para este manual.

Referencias disponibles en Internet

- Estándares del navegador cliente:
 - XHTML 1.0 (<http://www.w3.org/TR/xhtml1>)
 - ECMAScript 262 – estandarización de JavaScript/JScript por la Asociación europea de fabricantes de ordenadores (ECMA) (<http://www.ecma.ch/ecma1/stand/ecma-262.htm>)
 - JavaScript 1.5 (<http://www.mozilla.org/js/js15.html>)
 - CSS 1.0 (<http://www.w3.org/pub/WWW/TR/REC-CSS1>)
 - DOM 2.0 (<http://www.w3.org/TR/DOM-Level-2-Core>)
- Detección de navegadores no estándar:
 - Iniciativa de estándares Web (<http://www.webstandards.org/upgrade/tips.html>).
 - ¿Qué información puede obtener de un navegador sin ejecutar código en la parte del cliente? (<http://www.uiuc.edu/cgi-bin/info>).
- Convertir HTML a XHTML
 - W3C HTML Validator (<http://validator.w3.org>), incluidos HTML4 y XHTML1.
 - HTML Tidy (<http://www.w3.org/People/Ragget/tidy>)
 - Chamispace HTML Kit (<http://www.chami.com/html-kit>)
- Directivas meta
 - Diccionario de directivas meta (<http://www.vancouver-webpages.com/META>)
 - Comprobación del conjunto de caracteres ISO-8859-1 (<http://ppewww.ph.gla.ac.uk/~flavell/iso8859/isotable.html>)
 - Configuración de la caché de Internet Explorer: <http://www.jsifaq.com/SUBF/TIP2800/rh2895.htm>
- Herramientas de validación CSS
 - CSS 1 Validator (<http://www.htmlhelp.com/tools/csscheck>)
 - CSS 2 Validator (<http://jigsaw.w3.org/css-validator>)
 - CSS Style-O-Matic (<http://www.builder.com/Authoring/CSS/ss12.html>)
- Tecnología de Java Server Pages
 - FAQ JSP (<http://www.jguru.com/faq/home.jsp?topic=JSP>).
 - JSP Industry Momentum (<http://java.sun.com/products/jsp/industry.html>).

-
- Herramientas generales de otros fabricantes
 - Microsoft Visual SourceSafe (<http://msdn.microsoft.com/ssafe>)
 - EditPlus (<http://www.editplus.com>)
 - Microsoft Web Stressing Tool (<http://webtool.rte.microsoft.com>)
 - TRADOS (http://www.trados.com/products/team_edition.asp)
 - Lenguaje Java e IDE
 - Instrucciones de la codificación de Sun (<http://java.sun.com/docs/codeconv/html/CodeConvTOC.doc.html>)
 - Thinking in Java, de Bruce Eckel (<http://www.mindview.net/Books/TIJ>)
 - Jbuilder Enterprise Edition de Borland (<http://www.borland.com/jbuilder>)
 - Forte For Java Internet Edition de Sun (<http://www.sun.com/forte/ffj>)

Apéndice: Código de páginas JSP



Acerca de este apéndice

Este apéndice describe algunas páginas JSP que se genera para las funcionalidades Meta4.

El objetivo de estas páginas es explicar el código que es utilizada desde la mayor parte de las páginas de forma parecida.

Los formatos empleados tienen los significados siguientes:

- **Valores modificables:** Las partes modificables se indican en azul y negrita, y a veces con un literal Modificable.
- **Valores no modificables:** Las partes que no son modificables se indican en gris y con un literal No Modificable.
- Casos especiales: Las partes que tratan de casos especiales se marcan de este modo.
- Los *Includes* específicos se describen según el ejemplo de código

Tipos de páginas JSP

La generación de páginas JSP varía en función de los elementos que se incorporen a ella:

- Primero se explica la generación de las páginas asociadas al MT más sencillo, una tabla de mantenimiento sin claves primarias. Esto simplifica la generación, ya que no existen remotes ni listas asociadas.
- Después se explica la incorporación de estos elementos. Se explicarán los dos tipos de páginas: Tablas de mantenimiento y Listas.

Una vez creadas estas páginas relacionadas con el mantenimiento de datos, se detalla cómo se llaman desde las páginas jsp que las utilizan.

Por último, se detalla el proceso a seguir cuando se edita un registro y algunos campos se quieren deshabilitar.

Las páginas de mantenimiento siguen esta nomenclatura: shco_*Abrev.-módulo_mt_...*

Las páginas de listas siguen esta nomenclatura: shco_*Abrev.-módulo_list_...*

Páginas de mantenimiento

Se explica la página a partir del ejemplo del **Tipo interno del lugar de trabajo**.

```
<%@ include file="../../shco_g0/shco_gen_taglib.jsp"
%><html><head><title></title>
<%@ include file="../../shco_g0/shco_gen_bag.jsp" %><%@ include
file="../../shco_g0/shco_gen_css.jsp" %><%@ include file="../../
shco_g0/shco_gen_mt_arg.jsp" %>
<%
// Parámetros del M4Object:
```

Ver la explicación de los parámetros del M4Object.

```
String zsubsesion = "SHCO_OR_MT_WL_INT_TYPE";
```

Subsesiones

```
String zm4object = zsubsesion;
```

Si los nombres de los nodos no son los de por defecto (igual al ID del Meta4Object) hay que poner los correctos:

```
String znodoraiz = "SHCO_GN_ROOT";
String znodo = zm4object;
String znodo2 =znodo + "_FILTER";
```

```
znivelmenu = "2";
```

Nivel de menú.

```
zventanas = "20";/*MODIFICABLE
zvuelta = 5;/*MODIFICABLE
```

Declaración del tamaño de la ventana deseado.

```
String zdireccion = "shco_g1/shco_or_mt_wl_int_type.jsp";
/*MODIFICABLE
```

Declaración del nombre físico de la página.

```
// Items que vamos a utilizar (visualizar o requeridos en una acción):
```

Declaración de los campos. En este caso tendremos un id, un nombre y un comentario:

```
String zcampoID1 = "STD_ID_WL_INT_TYPE";/**MODIFICABLE  
String zcampoNombre1 = "STD_N_WL_INT_TYPE";  
String zcampoComentario = "STD_COMMENT";
```

```
// Indica el campo por el que ordenas en la estructura de nodo y si es  
asc o desc
```

Indicamos el campo por el que vamos a ordenar por defecto.

```
if ((zOrdenCampo==null)|| (zOrdenCampo.equals(""))){zOrdenCampo =  
zcampoID1;}
```

```
if ((zOrden==null)|| (zOrden.equals(""))){zOrden = "1";}
```

```
%><%@ include file="../../shco_g0/shco_gen_mt_preload.jsp" %><%
```

```
// campos usados
```

```
String zIdPk1 = zcomun + zcampoID1;
```

```
String zIdPk1 = zraiz + zcampoID1;
```

```
String zNPk1 = zcomun + zcampoNombre1;
```

```
String zNPk1 = zraiz + zcampoNombre1;
```

```
String zComment = zcomun + zcampoComentario;
```

```
String zLComment = zraiz + zcampoComentario;
```

```
%>
```

```
<%@ include file="../../shco_g0/shco_gen_mt_js.jsp" %>
```

```
<script type="text/javascript" language="Javascript1.5">
```

```
// En esta función se comprueba que los paramentro obligatorios estan  
reellenos antes
```

```
//de insertar un nuevo registro
```

```
var sIdPk="X"+"<%=zcampoID1%>";
```

```
var sNPk="<%=zcampoNombre1%>";
```

```
var sComment="<%=zcampoComentario%>";
```

```
var scampoant="<%=zOrdenCampo%>"
```

```
var sOrd="<%=zOrden%>";
```

```
</script></head><body>
```

```
<%@ include file="../../shco_g0/shco_gen_datadef.jsp" %><%@ include  
file="../../shco_g0/shco_gen_exec.jsp" %><%@ include file="../../  
shco_g0/shco_gen_mt_filter.jsp" %><%@ include file="../../shco_g0/  
shco_gen_mt_outputdef.jsp" %><%@ include file="../../shco_g0/  
shco_gen_mt_count.jsp" %><%
```

```

String zID = request.getParameter("zID");
String zNombre = request.getParameter("zNombre");
String zComentario = request.getParameter("zComentario");
if ((zID==null)|| (zID.equals(""))){zID = "";}
if ((zNombre==null)|| (zNombre.equals(""))){zNombre = "";}
if ((zComentario==null)|| (zComentario.equals(""))){zComentario = "";}
%>
<script type="text/javascript" language="Javascript1.5">
function comprobar(){
var sfunciones =
"m4valinput('_alfanum_oblig', 'NombreFormulario', sIdPk, 1, '<m4:label
m4name="<%=zIdPk1%>" jsafe="true"/>')"
sfunciones=sfunciones+"*m4valinput('_alfanum_oblig', 'NombreFormulario
', sNPk, 1, '<m4:label m4name="<%=zLNpk1%>" jsafe="true"/>')";
sfunciones=sfunciones+"*m4valinput('_comentario', 'NombreFormulario', '
'+sComment+'', 256, '<m4:label m4name="<%=zLComment%>" jsafe="true"/
>')";
var verr=m4valform(sfunciones);
if (verr == 1){ var vc=m4com2();if(vc ==
1){m4submit("NombreFormulario");}}
}

```

Función comprobar: (particular de la página).

Se usa para comprobar que todos los campos obligatorios están rellenos y que el campos comentario no tenga más de 256 caracteres. Antes del submit del formulario hay que ejecutar m4com2, y sólo ejecutar el submit si ésta nos devuelve un 1. Ver uso de funciones javaScript genéricas.

```

function valores(){
m4valor("oculto", "zID", m4valor("NombreFormulario", sIdPk, "", "get"), "set");
m4valor("oculto", "zNombre", m4valor("NombreFormulario", sNPk, "", "get"), "set");
m4valor("oculto", "zComentario", m4valor("NombreFormulario", sComment, "", "get"), "set");
m4submit("oculto");
}

```

Función valores : (particular de la página).

Se utiliza para enviar valores a un formulario (llamado 'oculto'). Es obligatoria en todas las paginas que tienen paginación u ordenación.

```

</script>
<form action="<%=zaccion%>" method="post" name="oculto" id="oculto" >
<input type="hidden" id="zPkA" name="zPkA" value="<%=zPkA%>" />
<input type="hidden" id="zinicio" name="zinicio" value="" />

```

```

<input type="hidden" id="zID" name="zID" value="" />
<input type="hidden" id="zNombre" name="zNombre" value="" />
<input type="hidden" id="zComentario" name="zComentario" value="" />
<input type="hidden" id="zOrdenCampo" name="zOrdenCampo"
value="<%=zOrdenCampo%>" />
<input type="hidden" id="zOrden" name="zOrden" value="<%=zOrden%>" />
</form>

<%@include file="../../shco_g0/shco_gen_title.jsp" %><%@ include
file="../../shco_g0/shco_gen_menusup.jsp" %>

<%
String zvalue = "";

```

Nombre de la pág. de ayuda, en caso de no tener poner un 0

```

String zhelp = "/servlet/CheckSecurity/JSP/shco_g0/shco_help.jsp";
%><%@ include file="../../shco_g0/shco_gen_cab.jsp" %>
<%if (ztipocarga!="ALL"){
if (zcounti2>0){
try {
M4Operations m = new M4Operations(request);
zID= m.getItem(znodo2,zm4object,znodo2,"",zcampoID1);
zNombre = m.getItem(znodo2,zm4object,znodo2,"",zcampoNombre1);
zComentario = m.getItem(znodo2,zm4object,znodo2,"",zcampoComentario);

}catch(Exception e) {}
}else{
int a=0;
String zTrozo="";
String zTrozo1="";
StringTokenizer T=new StringTokenizer(ztipocarga,"{");
while(T.hasMoreTokens()){
zTrozo=T.nextToken();
StringTokenizer S=new StringTokenizer(zTrozo,"=");
a=S.countTokens();
if (a==2){
zTrozo1=S.nextToken();
if ((zTrozo1==zcampoID1)|| (zTrozo1.equals(zcampoID1))){
zID=S.nextToken();
}else if ((zTrozo1==zcampoNombre1)|| (zTrozo1.equals(zcampoNombre1))){
zNombre=S.nextToken();
}
}
}
}
}
}

```

```

}
}
%>
<form action="/servlet/CheckSecurity/JSP/shco_g0/shco_gen_act.jsp"
method="post" name="NombreFormulario" id="NombreFormulario">
<input type="hidden" id="TAG" name="TAG" value="<%=zsubsesion%>" />
<input type="hidden" id="M40BJ_ID" name="M40BJ_ID"
value="<%=zm4object%>" />
<input type="hidden" id="ACC" name="ACC" value="<%=zUpd%>" />
<input type="hidden" id="zredireccion" name="zredireccion"
value="<%=zredireccion%>" />

```

Formulario del MT. En este formulario los input asociados a los campos de la pk tienen que tener como id y como name el 'X' + nombre del campo

```

<table class="form" width="100%" cellpadding="0">
<thead><tr class="titulo">
<th colspan="2" id="m4tit">&nbsp;</th>
<th><a title="<m4:label m4name="<%=zSHCOLBCLEAN%>" htmlsafe="true"/>"
href="javascript:m4clearform('NombreFormulario'); m4lim();">

```

Limpiamos el formulario

```

<img alt="<m4:label m4name="<%=zSHCOLBCLEAN%>" htmlsafe="true"/>" <%@
include file="../../files_gif/ic_des.jsp" %> /></a></th></tr></thead>
<tbody>
<tr>
<td class="campo"><label for="X<%=zcampoID1%>">*&nbsp;<m4:label
m4name="<%=zIdPk1%>" htmlsafe="true"/></label></td>
<td>&nbsp;<input tabindex="1" class="form" type="text"
id="X<%=zcampoID1%>" name="X<%=zcampoID1%>" size="4" maxlength="4"
title="<m4:label m4name="<%=zSHCOLBWRITE%>" htmlsafe="true"/>"
<m4:label m4name="<%=zIdPk1%>" htmlsafe="true"/>"
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zID
)%>" /></td>
<td class="campo"><label for="<%=zcampoNombre1%>">*&nbsp;<m4:label
m4name="<%=zLNPK1%>" htmlsafe="true"/>&nbsp;</label><input
tabindex="2" class="form" type="text" id="<%=zcampoNombre1%>"
name="<%=zcampoNombre1%>" size="40" maxlength="60" title="<m4:label
m4name="<%=zSHCOLBWRITE%>" htmlsafe="true"/>" <m4:label
m4name="<%=zLNPK1%>" htmlsafe="true"/>"
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zNo
mbre)%>" /></td>
</tr>
<tr>
<td class="campo"><label for="<%=zcampoComentario%>">&nbsp;<m4:label
m4name="<%=zLComment%>" htmlsafe="true"/></label></td>
<td colspan="3">&nbsp;<textarea tabindex="5" cols="" rows="3"
class="form" id="<%=zcampoComentario%>" name="<%=zcampoComentario%>"

```

```

title="<m4:label m4name="<%=zSHCOLBWRITE%>" htmlsafe="true"/>
<m4:label m4name="<%=zLComment%>" htmlsafe="true"/
>"><%=zComentario%></textarea>
</td>
</tr>
<%

```

//Escribe el nº de columnas que tiene la tabla.

```
int zCols =4;
```

//el próximo nº de tabulador

```

int zTab=4;
%><%@ include file="../../../shco_g0/ shco_gen_mt_bus.jsp" %>
</tbody></table>
</form>
<script type="text/javascript"
language="Javascript1.5">m4tabfocus('NombreFormulario',1);m4setm4tit(
);</script>
<%// Tabla principal de datos:
if (zcount>0) {%>
<table class="datos" width="100%" cellpadding="0" cellspacing="0">
<thead>

```

Cabecera de la tabla que muestra los datos. Se utilizan los nombre de los campos en el Meta4Object y para ordenar por ellos, en el onclick se llama a la función m4ordenar de javascript pasándole como argumento el nombre del campo.

```

<tr class="titulo">
<th>&nbsp;</th><th><a name="filter"></a><a>
<%if ((zOrdenCampo==zcampoID1)|| (zOrdenCampo.equals(zcampoID1))) {%>
<%if ((zOrden=="1")||(zOrden.equals("1"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zIdPk1%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoID1%>');" <%@ include file="../../../
files_gif/ic_ord_1.jsp" %> />
<%}else if ((zOrden=="2")||(zOrden.equals("2"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zIdPk1%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoID1%>');" <%@ include file="../../../
files_gif/ic_ord_2.jsp" %> />
<%}%>
<%} else{%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zIdPk1%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoID1%>');" <%@ include file="../../../
files_gif/ic_ord.jsp" %> />

```



```

m m4name="<%=zComment%>" jsafe="true" htmlsafe="true"/>', 'set');
m4edit();return false; "><img alt="<m4:label
m4name="<%=zSHCOLBEDIT%>" htmlsafe="true"/>" <%@ include file="../../../
files_gif/ic_mod.jsp" %> /></a></td>

<td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zIdPk1%>"
htmlsafe="true"/></td>

<td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zNPk1%>"
htmlsafe="true"/></td>

<td class="valor<%=zpos%>" colspan="3">&nbsp;<m4:item
m4name="<%=zComment%>" htmlsafe="true"/></td>

<td class="boton<%=zpos%>"><a title="<m4:label
m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/>"
href="javascript:m4eliminar('X<%=zcampoID1%>', '<m4:item
m4name="<%=zIdPk1%>" jsafe="true" htmlsafe="true"/>');"><img
alt="<m4:label m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/>" <%@
include file="../../../files_gif/ic_bor.jsp" %> /></a></td>

</tr>

</m4:loop>

</tbody></table>

<%@ include file="../../../shco_g0/shco_gen_vent_post.jsp" %><%}%><%@
include file="../../../shco_g0/ shco_gen_disclaimer.jsp" %>

</body></html>

```

Páginas de listas y de filtros tipo lista

Estas páginas tienes usos diferentes, sin embargo utilizan la misma página.

```

<%@ include file="../../../shco_g0/shco_gen_taglib.jsp" %><html
><head><title> </title>

<%@ include file="../../../shco_g0/shco_gen_bag.jsp" %><%@ include
file="../../../shco_g0/shco_gen_css.jsp" %>

<%@ include file="../../../shco_g0/shco_gen_list_arg.jsp" %><%

String zpag = request.getParameter("zpag");
if ((zpag==null)|| (zpag.equals(""))){zpag = "0";}

String znw = request.getParameter("znw");
if ((znw==null)|| (znw.equals(""))){if
((zpag=="0")||(zpag.equals("0"))){znw = "0";}else{znw = "1";}}

String znew = "shco\_g1/shco\_or\_mt\_wl\_int\_type.jsp";

//Valor del link nuevo en caso de existir, si no existe se añade una nueva línea
znw = "1";

if (!zpag.equals("0")){znew=zpag;}

znivelmenu = request.getParameter("znivelmenu");

// Escribe el nivel de menús por defecto

```

```

        if ((znivelmenu==null)|| (znivelmenu.equals(""))){znivelmenu = "2";}

// Parámetros del M4Object:

String zsubsesion = "SHCO_OR_MT_WL_INT_TYPE";
String zm4object = zsubsesion;
String znodo = zm4object;

zventanas = "20";//*MODIFICABLE
zvuelta = 5;//*MODIFICABLE
String zdireccion = "shco_g1_sse/shco_or_list_wl_int_type.jsp";

//Escribe el nombre de esta pág.

String zredireccion = "shco_or_list_wl_int_type.jsp";

// Items que vamos a utilizar (visualizar o requeridos en una acción):
String zcampoID= "STD_ID_WL_INT_TYPE";
String zcampoNombre = "STD_N_WL_INT_TYPE";
String zcampoC = "STD_COMMENT";

// Indica el campo por el que ordenas en la TI y si es asc o desc

if ((zOrdenCampo==null)|| (zOrdenCampo.equals(""))){zOrdenCampo =
zcampoID;}
if ((zOrden==null)|| (zOrden.equals(""))){zOrden = "1";}
%><%@ include file="../../../shco_g0/shco_gen_list_preload.jsp" %>
<script type="text/javascript" language="Javascript1.5">
var sIdPk="<%=zcampoID%>";
var sNPk="<%=zcampoNombre%>";
var scampoant="<%=zOrdenCampo%>";
var sOrd="<%=zOrden%>";
</script>
<%@ include file="../../../shco_g0/shco_gen_list_js.jsp" %>
<%

String zIdPk = zcomun + zcampoID;
String zlIdPk = zraiz+ zcampoID;

String zNPk = zcomun + zcampoNombre;
String zlNPk = zraiz + zcampoNombre;

String zCo = zcomun + zcampoC;
String zlCo = zraiz+ zcampoC;

```

```

%></head><body onload="javascript:m4setm4tit();"
<%@ include file="../../shco_g0/shco_gen_datadef.jsp" %><%@ include
file="../../shco_g0/shco_gen_exec.jsp" %><%@ include file="../../
shco_g0/shco_gen_list_filter.jsp" %><%@ include file="../../shco_g0/
shco_gen_list_outputdef.jsp" %><%@ include file="../../shco_g0/
shco_gen_list_count.jsp" %>
<%@include file="../../shco_g0/shco_gen_title.jsp" %>
<%if (zpag.equals("0")== false){%><%@ include file="../../shco_g0/
shco_gen_menusup.jsp" %><%}%>
<h2><m4:label m4name="<%=zSHCOLBTITLE%>" htmlsafe="true"/></h2>
<%@include file="../../shco_g0/shco_gen_list_filt.jsp" %>

```

Éste es el form. para pasar argumentos desde el tr.

```

<form action="/servlet/CheckSecurity/JSP/<%=zpag%>" method="post"
name="oculto2" id="oculto2" >
<input type="hidden" id="<%=zcampoID%>" name="<%=zcampoID%>" value=""
/>
<input type="hidden" id="<%=zcampoNombre%>" name="<%=zcampoNombre%>"
value="" />
</form>
<table class="datos" width="100%" cellpadding="0"
cellspacing="0"><thead>
<tr class="titulo"><th><a name="filter"></a>&nbsp;<a>
<%if ((zOrdenCampo==zcampoID)|| (zOrdenCampo.equals(zcampoID))){%>
<%if ((zOrden=="1")||(zOrden.equals("1"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zIdPk%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoID%>');" <%@ include file="../../
files_gif/ic_ord_1.jsp" %> />
<%}else if ((zOrden=="2")||(zOrden.equals("2"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zIdPk%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoID%>');" <%@ include file="../../
files_gif/ic_ord_2.jsp" %> />
<%}%>
<%} else{%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zIdPk%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoID%>');" <%@ include file="../../
files_gif/ic_ord.jsp" %> />
<%}%></a>&nbsp;<m4:label m4name="<%=zIdPk%>" htmlsafe="true"/></th>
<th>&nbsp;<a>
<%if
((zOrdenCampo==zcampoNombre)|| (zOrdenCampo.equals(zcampoNombre))){%>
<%if ((zOrden=="1")||(zOrden.equals("1"))){%>

```

```

<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zLNPK%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoNombre%>');" <%@ include file="../../../
files_gif/ic_ord_1.jsp" %> />
<%}else if ((zOrden=="2")||(zOrden.equals("2"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zLNPK%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoNombre%>');" <%@ include file="../../../
files_gif/ic_ord_2.jsp" %> />
<%}%>
<%} else{%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zLNPK%>" htmlsafe="true"/>"
onclick="m4ordenar('<%=zcampoNombre%>');" <%@ include file="../../../
files_gif/ic_ord.jsp" %> />
<%}%></a>&nbsp;<m4:label m4name="<%=zLNPK%>" htmlsafe="true"/></th>
<th>&nbsp;<m4:label m4name="<%=zLCo%>" htmlsafe="true"/></th>
<%@ include file="../../../shco_g0/shco_gen_pest.jsp" %>
</tr></thead><tbody><% if (zcount>0){%>
<m4:loop from="<%=zregistroiniciales%>" to="<%=zregistrofinales%>"><%@
include file="../../../shco_g0/shco_gen_loop.jsp" %>

```

Cuerpo de la tabla que muestra los datos. Contiene dos funcionalidades, dependiendo de si tenemos una lista o un TR:

Lista: En el campo de la P , en la propiedad onclick construimos un array con los valores que se envían a la otra página y se ejecuta la función m4returnvalues pasándole como argumento el array.

TR: En el campo de la PK, en la propiedad onclick se envía al formulario 'oculto2' los valores y se realiza el submit del formulario.

```

<tr>
<%if ((zpag=="0")||(zpag.equals("0"))){%>
<td class="valor<%=zpos%>">&nbsp;<a tabindex="8+zpos"title="<m4:label
m4name="<%=zSHCOLBLIST%>" htmlsafe="true"/>" href="" onclick="var
aval=new Array();aval[0]='<m4:item m4name="<%=zIdPk %>" jsafe="true"
htmlsafe="true"/>';aval[1]='<m4:item m4name="<%=zNPK%>" jsafe="true"
htmlsafe="true"/>';m4returnvalues(aval);return false;"><m4:item
m4name="<%=zIdPk %>" htmlsafe="true"/></a></td>
<%}else{%>
<td class="valor<%=zpos%>">&nbsp;<a tabindex="7+zpos"title="<m4:label
m4name="<%=zSHCOLBLIST%>" htmlsafe="true"/>" href=""
onclick="m4valor('oculto2', '<%=zcampoID%>', '<m4:item m4name="<%=zIdPk
%>" jsafe="true" htmlsafe="true"/
>', 'set');m4valor('oculto2', '<%=zcampoNombre%>', '<m4:item
m4name="<%=zNPK%>" jsafe="true" htmlsafe="true"/
>', 'set');m4submit('oculto2');return false;"><m4:item
m4name="<%=zIdPk %>" htmlsafe="true"/></a></td>
<%}%>

```

```

<td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zNPK%>"
htmlsafe="true"/></td><td class="valor<%=zpos%>">&nbsp;<m4:item
m4name="<%=zCo%>" htmlsafe="true"/></td>
<td class="valor<%=zpos%>"colspan="2">&nbsp;</td>
</tr>
</m4:loop>
</tbody></table>
<%@ include file="../../shco_g0/shco_gen_vent_post.jsp" %><%}else{%>
<tr><td class="fuentenodatos" colspan="5"><%@ include file="../../
shco_g0/ shco_gen_list_nodata.jsp" %></td></tr>
</tbody></table><%}%>
<%if (zpag.equals("0")== false){%><%@ include file="../../shco_g0/
shco_gen_disclaimer.jsp" %><%}else{%><m4:endpage/></div><%}%>
</body></html>

```

Modo de uso de estas páginas

Las llamadas a estas páginas se realizarán desde las páginas que las utilizan o desde las opciones de menú correspondientes. Más información, en el capítulo de *Gestión de menús*.

Para más información consulte también el apartado " Validaciones de código JavaScript " del capítulo *Metodología*.

Llamada a la página de la tabla de mantenimiento o a la lista desde otra página

Para acceder una página de mantenimiento se utiliza la función m4remonte. Para obtener más información y ejemplos sobre este tipo de páginas consulte el capítulo *Librerías API javascript*.

Para levantar una página tipo lista se utiliza la función m4filtro. Para obtener más información y ejemplos sobre este tipo de páginas consulte el capítulo *Librerías API javascript*.

Llamada a la página de filtro tipo lista

Para acceder una página de filtro tipo lista se utiliza la función de javascript m4navegar, con la que se navega a la página de lista, con los siguientes parámetros :

- zpag: (Obligatorio) Página destino.
- znw: (Opcional) Posibles valores:
 - 0 Hay un link de nuevo.
 - 1 No lo hay.

- **znivelmenu:** (Opcional) Si se desea ver un nivel de menú distinto al defecto.

```
href="javascript:var sparameters=['zpag','znw'];  
var svalues=['shco_ta/shco_ta_wz_week_model_p1.jsp','0'];  
m4navegar('shco_ta_sse/shco_ta_list_week_model.jsp', sparameters,  
svalues);"
```

En este ejemplo, se navegará a la página de filtro tipo lista `shco_ta_sse/shco_ta_list_week_model.jsp`, en esta página aparecerá el botón de nuevo.

Entonces al pulsar el botón de nuevo o seleccionar un registro del listado se navegará a la página `shco_ta/shco_ta_wz_week_model_p1.jsp` con un registro nuevo o uno editado.

Cómo la página llamada por el filtro tipo lista sabe qué registro fue elegido

Cuando se define la página de tipo filtro tipo lista se indican qué campos son los que se van a pasar al navegar a la página destino (`zpag`). Éstos campos se definen en el formulario `oculto2`.

Entonces para recuperar esos valores desde la página destino, ésta tiene que leerlos del request.

Dado ejemplo anterior, en la página `shco_ta_sse/shco_ta_list_week_model.jsp` se ha definido el formulario `oculto2`:

```
<form action="/servlet/CheckSecurity/JSP/<%=zpag%>" method="post"  
name="oculto2" id="oculto2" >  
  <input type="hidden" id="SCO_ID_WEEK_MDL" name="SCO_ID_WEEK_MDL"  
  value="" />  
  <input type="hidden" id="SCO_NM_WEEK_MDL" name="SCO_NM_WEEK_MDL"  
  value="" />  
</form>
```

Cuando se selecciona un elemento de la lista se hace el submit de este formulario, es decir, se navega a la página `zpag` (recordemos `shco_ta/shco_ta_wz_week_model_p1.jsp`) pasando dos parámetros.

En esta página podemos recoger esos valores haciendo

```
String zSHCO_ID_WEEK_MDL = request.getParameter("SCO_ID_WEEK_MDL");  
String zSHCO_NM_WEEK_MDL = request.getParameter("SCO_NM_WEEK_MDL");
```

Cómo volver a llamar al filtro tipo lista

Si desde una página se desea volver a su filtro tipo lista, se asigna a la variable `zcarril` (ya definida en la bolsa), la dirección del filtro tipo lista junto con sus parámetros. Si `zcarril` tiene valor en la página se habilita el botón de filtro del menú superior.

Dado ejemplo anterior, en la página shco_ta/shco_ta_wz_week_model_p1.jsp definimos la variable zcarril con el valor:

```
zcarril="/servlet/CheckSecurity/JSP/shco_ta_sse/  
shco_ta_list_week_model.jsp?zpag= shco_ta/  
shco_ta_wz_week_model_p1.jsp&znw=0";
```

Se habilitará el botón de filtro del menú superior y al pulsarlo volveremos a la página de tipo filtro tipo lista

Página lista multiselección

```
<%@ include file="../../shco_g0/shco_gen_taglib.jsp" %><html  
lang="es"><head><title> </title>  
<%@ include file="../../shco_g0/shco_gen_bag.jsp" %>  
<%@ include file="../../shco_g0/shco_gen_css.jsp" %>  
<%@ include file="../../shco_g0/shco_gen_list_arg.jsp" %><%  
String zpag = request.getParameter("zpag");  
if ((zpag==null)|| (zpag.equals(""))){zpag = "0";}  
String znw = request.getParameter("znw");  
if ((znw==null)|| (znw.equals(""))){if  
((zpag=="0")||(zpag.equals("0"))){znw = "0";}else{znw = "1";}}  
//En este caso znw siempre es 1  
znw="1";  
String znew = "shco_py/shco_or_list_ck_work_unit.jsp";  
if (!zpag.equals("0")){znew=zpag;}  
  
znivelmenu = request.getParameter("znivelmenu");  
// Escribe el nivel de menus por defecto  
if ((znivelmenu==null)|| (znivelmenu.equals(""))){znivelmenu = "2";}  
  
// Parámetros del M4Object:  
String zsubsesion = "SHCO_OR_MT_WORK_UNIT";  
String zm4object = zsubsesion;  
String znodo = zm4object;  
  
zventanas = "40";//*MODIFICABLE  
zvuelta = 5;//*MODIFICABLE  
String zdireccion = "shco_py/shco_or_list_ck_work_unit.jsp ";//  
*MODIFICABLE  
//escribe el nombre de esta pag.  
String zredireccion = "/shco_or_list_ck_work_unit.jsp ";
```

```
// Items que vamos a utilizar (visualizar o requeridos en una acción):
String zcampoID= "STD_ID_WORK_UNIT ";
String zcampoNombre = "STD_N_WORK_UNIT ";
```

```
// Indica el campo por el que ordenas en la TI y si es asc o desc
if ((zOrdenCampo==null)|| (zOrdenCampo.equals(""))){zOrdenCampo =
zcampoID;}
if ((zOrden==null)|| (zOrden.equals(""))){zOrden = "1";}
%><%@ include file="../../shco_g0/shco_gen_list_preload.jsp" %>
<script type="text/javascript" language="Javascript1.5">
var sIdPk="<%=zcampoID%>";
var sNPk="<%=zcampoNombre%>";
var scampoant="<%=zOrdenCampo%>";
var sOrd="<%=zOrden%>";
</script>
<%@ include file="../../shco_g0/shco_gen_list_ck_js.jsp" %>
<%
```

```
String zIdPk = zcomun + zcampoID;
String zIdPk = zraiz+ zcampoID;
```

```
String zNPk = zcomun + zcampoNombre;
String zNPk = zraiz + zcampoNombre;
```

```
%></head>
<body>
<%@ include file="../../shco_g0/shco_gen_datadef.jsp" %><%@ include
file="../../shco_g0/shco_gen_exec.jsp" %><%@ include file="../../
shco_g0/shco_gen_list_filter.jsp" %><%@ include file="../../shco_g0/
shco_gen_list_outputdef.jsp" %><%@ include file="../../shco_g0/
shco_gen_list_count.jsp" %>
<%@include file="../../shco_g0/shco_gen_title.jsp" %>
<%if (zpag.equals("0")== false){%><%@ include file="../../shco_g0/
shco_gen_menusup.jsp" %><%}%>
<h2><m4:label m4name="<%=zSHCOLBTITLE%>" htmlsafe="true"/></h2>
<%@include file="../../shco_g0/shco_gen_list_filt.jsp" %>
```

Función genérica que se ejecuta en el botón de aceptar

```
<script type="text/javascript" language="Javascript1.5">
function m4acep(){
var vpag="<%=zpag%>";
if (vpag == "0"){
m4returnvalues(m4retorno('<%=zcounti%>','<%=zcampoID%>'));
```

```

}else{

m4valor("oculto3","streampka",m4retorno('<%=zcounti%>','<%=zcampoID%>'),"set");;
    m4submit("oculto3");
}
}
</script>

<form action=" " method="post" name="oculto2" id="oculto2" >
<table class="datos" width="100%" cellpadding="0"
cellspacing="0"><thead>
<tr class="titulo"><th>&nbsp;</th><th><a name="filter"></a>&nbsp;<a>
<%if ((zOrdenCampo==zcampoID)|| (zOrdenCampo.equals(zcampoID))) {%>
<%if ((zOrden=="1")||(zOrden.equals("1"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlspecialchars="true"/>
<m4:label m4name="<%=zIdPk%>" htmlspecialchars="true"/>"
onclick="m4ordenar('<%=zcampoID%>');" <%@ include file="../../../
files_gif/ic_ord_1.jsp" %> />
<%}else if ((zOrden=="2")||(zOrden.equals("2"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlspecialchars="true"/>
<m4:label m4name="<%=zIdPk%>" htmlspecialchars="true"/>"
onclick="m4ordenar('<%=zcampoID%>');" <%@ include file="../../../
files_gif/ic_ord_2.jsp" %> />
<%}%>
<%} else{%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlspecialchars="true"/>
<m4:label m4name="<%=zIdPk%>" htmlspecialchars="true"/>"
onclick="m4ordenar('<%=zcampoID%>');" <%@ include file="../../../
files_gif/ic_ord.jsp" %> />
<%}%></a>&nbsp;<m4:label m4name="<%=zIdPk%>" htmlspecialchars="true"/></th>
<th>&nbsp;<a>
<%if
((zOrdenCampo==zcampoNombre)|| (zOrdenCampo.equals(zcampoNombre))) {%>
<%if ((zOrden=="1")||(zOrden.equals("1"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlspecialchars="true"/>
<m4:label m4name="<%=zLNPK%>" htmlspecialchars="true"/>"
onclick="m4ordenar('<%=zcampoNombre%>');" <%@ include file="../../../
files_gif/ic_ord_1.jsp" %> />
<%}else if ((zOrden=="2")||(zOrden.equals("2"))){%>
<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlspecialchars="true"/>
<m4:label m4name="<%=zLNPK%>" htmlspecialchars="true"/>"
onclick="m4ordenar('<%=zcampoNombre%>');" <%@ include file="../../../
files_gif/ic_ord_2.jsp" %> />
<%}%>
<%} else{%>

```

```

<img alt="<m4:label m4name="<%=zSHCOLBORD%>" htmlsafe="true"/>
<m4:label m4name="<%=zLNPK%>" htmlsafe="true"/>"
onclick="m4ordenar(' <%=zcampoNombre%>');" <%@ include file="../../
files_gif/ic_ord.jsp" %> />

<%}%></a>&nbsp;<m4:label m4name="<%=zLNPK%>" htmlsafe="true"/></th>

<%@ include file="../../shco_g0/shco_gen_pest.jsp" %>
</tr></thead><tbody>

<% if (zcount>0){
int zInnat=0;String zIn=String.valueOf(zInnat + 1);
%>

<m4:loop from="<%=zregistroinicial%>" to="<%=zregistrofinal%>"><%@
include file="../../shco_g0/shco_gen_loop.jsp" %>

<%zInnat=zInnat+1;zIn=String.valueOf(zInnat);
%>

<tr>

<td class="valor<%=zpos%>">&nbsp;<input tabIndex="1" class="form"
type="checkbox" name="CK<%=zIn%>" id="CK<%=zIn%>" title=""
maxlength="1" size="1" onclick="var
scam=m4valor('oculto2','CK<%=zIn%>',' ','get');if
(scam=='0'){scam='1';}else{scam='0';}m4valor('oculto2','CK<%=zIn%>','s
cam','set');" value="0" /></td>

<td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zIdPk %>"
htmlsafe="true"/></td>

<input type="hidden" id="<%=zcampoID%><%=zIn%>"
name="<%=zcampoID%><%=zIn%>" value="<m4:item m4name="<%=zIdPk %>"
htmlsafe="true"/>" />

<input type="hidden" id="<%=zcampoNombre%><%=zIn%>"
name="<%=zcampoNombre%><%=zIn%>" value="<m4:item m4name="<%=zLNPK%>"
htmlsafe="true"/>" />

<td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zLNPK%>"
htmlsafe="true"/></td>

<td class="valor<%=zpos%>" colspan="2">&nbsp;</td>

</tr> </m4:loop>

</tbody></table>

<%@ include file="../../shco_g0/shco_gen_vent_post.jsp" %><%}else{%>

<tr><td class="fuentenodatos" colspan="5"><%@ include file="../../
shco_g0 /shco_gen_list_nodata.jsp" %></td></tr>

</tbody></table><%}%>

</form>

<%if (zpag.equals("0")== false){%><%@ include file="../../shco_g0/
shco_gen_disclaimer.jsp" %> <%}else{%><m4:endpage/></div><%}%></
body></html>

```

Actualizaciones controladas en las páginas MT

En la página JSP se realizan los siguientes cambios:

```
<%@ include file="../../shco_g0/shco_gen_taglib.jsp"
%><html><head><title></title>

<%@ include file="../../shco_g0/shco_gen_bag.jsp" %><%@ include file
= "../../shco_g0/shco_gen_css.jsp" %> <%@ include file="../../
shco_g0/shco_gen_mt_arg.jsp" %>

<%
// Parámetros del M40bject:

String zsubsesion = "SHCO_TA_MT_DAY_TYPE";
String zm4object = zsubsesion;
String znodoraiz = "SHCO_GN_ROOT";
String znodo = zm4object;
String znodo2 =znodo + "_FILTER";
znivelmenu = "4";
zventanas = "50";
zvuelta = 5;
String zdireccion = "shco_ta/shco_ta_mt_day_type.jsp";

// Items que vamos a utilizar (visualizar o requeridos en una acción):
String zcampoID1 = "SCO_ID_DAY_TYPE";
String zcampoNombre1 = "SCO_NM_DAY_TYPE";
String zcampoID2 = "SCO_ID_ENTER_TYPE";
String zcampoNombre2 = "SCO_NM_ENTER_TYPE";
String zcampoID3 = "SCO_WKNG_HRS_DAY";
String zcampoComentario = "SCO_COMMENT";
String zID2h = "SHCO_TA_ID_ENTER_TYPE_H";
String zID2u = "SHCO_TA_ID_ENTER_TYPE_U";

// Indica el campo por el que ordenas en la TI y si es asc o desc
if ((zOrdenCampo==null)|| (zOrdenCampo.equals(""))){zOrdenCampo =
zcampoID1;}
if ((zOrden==null)|| (zOrden.equals(""))){zOrden = "1";}
%><%@ include file="../../shco_g0/shco_gen_mt_preload.jsp" %><%
// campos usados
String zIdPk1 = zcomun + zcampoID1;
String zIdPk1 = zraiz + zcampoID1;
String zNPk1 = zcomun + zcampoNombre1;
```

```

String zlNPK1 = zraiz + zcampoNombre1;

String zComment = zcomun + zcampoComentario;
String zlComment = zraiz + zcampoComentario;
String zIdPk2 = zcomun + zcampoID2;
String zlIdPk2= zraiz+ zcampoID2;
String zNPK2 = zcomun +zcampoNombre2;
String zlID2h= zraiz+ zID2h;
String zlID2u= zraiz+ zID2u;
String zIdPk3 = zcomun + zcampoID3;
String zlIdPk3= zraiz+ zcampoID3;
%><%@ include file="../../shco_g0/shco_gen_mt_js.jsp" %>
<script type="text/javascript" language="Javascript1.5">
// En esta función se comprueba que los parámetros obligatorios están
rellenos antes
//de insertar un nuevo registro
var sIdPk="X"+"<%=zcampoID1%>";
var sNPK="<%=zcampoNombre1%>";
var sId3="<%=zcampoID3%>";
var sComment="<%=zcampoComentario%>";
var sId2 = "<%=zcampoID2%>";
var sN2="<%=zcampoNombre2%>";
var scampoant="<%=zOrdenCampo%>"
var sOrd="<%=zOrden%>";
</script></head><body>
<%@ include file="../../shco_g0/shco_gen_datadef.jsp" %><%@ include
file="../../shco_g0/shco_gen_exec.jsp" %><%@ include file="../../
shco_g0/shco_gen_mt_filter.jsp" %>
<%@ include file="../../shco_g0/shco_gen_mt_outputdef.jsp" %><%@
include file="../../shco_g0/shco_gen_mt_count.jsp" %>
<%
String zID = request.getParameter("zID");
String zNombre = request.getParameter("zNombre");
String zID2 = request.getParameter("zID2");
String zNombre2 = request.getParameter("zNombre2");
String zID3 = request.getParameter("zID3");
String zComentario = request.getParameter("zComentario");
String zTipo = request.getParameter("zTipo");
if ((zID==null)|| (zID.equals(""))){zID = "";}
if ((zID2==null)|| (zID2.equals(""))){zID2 = "1";}
if ((zID3==null)|| (zID3.equals(""))){zID3 = "";}
if ((zNombre==null)|| (zNombre.equals(""))){zNombre = "";}

```



```

if ((zNombre2==null)|| (zNombre2.equals(""))){zNombre2 = "";}
if ((zComentario==null)|| (zComentario.equals(""))){zComentario = "";}
if ((zTipo==null)|| (zTipo.equals(""))){zTipo = "0";}
String zChh="checked=\\"checked\\"";
String zChu="";
if ((zID2=="2")|| (zID2.equals("2"))){zChh =
"";zChu="checked=\\"checked\\"";}
String zReOn="class=\\"form\\"";
String zDisOn="";
if ((zTipo=="1")|| (zTipo.equals("1"))){zReOn="readonly=\\"readonly\\"
class=\\"disabled\\"";zDisOn="disabled=\\"disabled\\"";}
String zUp = request.getParameter("zUp");
if ((zUp==null)|| (zUp.equals(""))){zUp = "0";}

```

A Hay que tener en cuenta que en la página, hay que bloquear los campos de la pk, en el modo de edición

```

%>
<script type="text/javascript" language="Javascript1.5">
A function comprobar(){
var sfunciones =
"m4valinput('_alfanum_oblig', 'NombreFormulario', sIdPk, 1, '<m4:label
m4name=<%=zIdPk1%>/>')\"
sfunciones=sfunciones+\"*m4valinput('_alfanum_oblig', 'NombreFormulario
', sNPk, 1, '<m4:label m4name=<%=zIdPk1%>/>')\";
var sTipo=m4valor("NombreFormulario", sId2, "", "get");
if
(sTipo=="1"){sfunciones=sfunciones+\"*m4valinput('_num_oblig', 'NombreF
ormulario', sId3, 1, '<m4:label m4name=<%=zIdPk3%>/
>')\";};else{sfunciones=sfunciones+\"*m4valinput('_decimal_oblig', 'Nombr
eFormulario', sId3, 3, 2, '<m4:label m4name=<%=zIdPk3%>/>', '3', '2')\";};
sfunciones=sfunciones+\"*m4valinput('_comentario', 'NombreFormulario', '
'+sComment+'', 256, '<m4:label m4name=<%=zIdComment%>/>')\";
var verr=m4valform(sfunciones);
if (verr == 1){
    var sTot=m4valor("NombreFormulario", sId3, "", "get");if
(sTipo=="1"){if
(sTot>24){m4setlog("_day_hour");return;}}m4submit("NombreFormulario")
;
}
}
function valores(){
m4valor("oculto", "zUp", m4valor("NombreFormulario", "SHCO_
UPDATING", "", "get"), "set");

```

```

m4valor("oculto","zID",m4valor("NombreFormulario",sIdPk,"","get"),"set");m4valor("oculto","zNombre",m4valor("NombreFormulario",sNPk,"","get"),"set");
m4valor("oculto","zID2",m4valor("NombreFormulario",sId2,"","get"),"set");m4valor("oculto","zID3",m4valor("NombreFormulario",sId3,"","get"),"set");m4valor("oculto","zComentario",m4valor("NombreFormulario",sComment,"","get"),"set");var
vTipo=m4prop('NombreFormulario',sIdPk,'readOnly','','get');
if
(vTipo==true){vTipo=1;}else{vTipo=0;}m4valor("oculto","zTipo",vTipo,"set");m4submit("oculto");
}

```

A En la limpieza del formulario, hay que poner el input con valor 0 y desbloquear los campos

```

functionlimp(){m4clearform('NombreFormulario','<%=zcampoID2%>');
m4valor('NombreFormulario','SHCO_TA_UPDATING','0','set');
m4lock('NombreFormulario','X<%=zcampoID1%>','readOnly','unlock','form');
m4lock('NombreFormulario','<%=zcampoID2%>','disabled','unlock');
m4lock('NombreFormulario','<%=zcampoID3%>','readOnly','unlock','form');}
</script>
<form action="<%=zaccion%>" method="post" name="oculto" id="oculto" >
<input type="hidden" id="zUp" name="zUp" value="" />
<input type="hidden" id="zinicio" name="zinicio" value="" /><input type="hidden" id="zID" name="zID" value="" /><input type="hidden" id="zID2" name="zID2" value="" />
<input type="hidden" id="zID3" name="zID3" value="" /><input type="hidden" id="zNombre" name="zNombre" value="" /><input type="hidden" id="zIDType" name="zIDType" value="" /><input type="hidden" id="zNombre2" name="zNombre2" value="" />
<input type="hidden" id="zComentario" name="zComentario" value="" /><input type="hidden" id="zTipo" name="zTipo" value="" /><input type="hidden" id="zOrdenCampo" name="zOrdenCampo" value="<%=zOrdenCampo%>" /><input type="hidden" id="zOrden" name="zOrden" value="<%=zOrden%>" />
</form>
<%@include file="../../shco_g0/shco_gen_title.jsp" %><%@ include file="../../shco_g0/shco_gen_menusup.jsp" %>
<%
String zvalue = "";
String zhelp = "/servlet/CheckSecurity/JSP/shco_g0/shco_help.jsp";
%><%@ include file="../../shco_g0/shco_gen_cab.jsp" %>
<%
if (ztipocarga!="ALL"){

```

```

if (zcounti2>0){
    try {
        M4Operations m = new M4Operations(request);
        zID= m.getItem(znodo2,zm4object,znodo2,"",zcampoID1);
        zNombre =
        m.getItem(znodo2,zm4object,znodo2,"",zcampoNombre1);
        zID2= m.getItem(znodo2,zm4object,znodo2,"",zcampoID2);
        zNombre2 =
        m.getItem(znodo2,zm4object,znodo2,"",zcampoNombre2);
        zID3= m.getItem(znodo2,zm4object,znodo2,"",zcampoID3);
        zComentario =
        m.getItem(znodo2,zm4object,znodo2,"",zcampoComentario);
    }catch(Exception e) {}
}else{
    int a=0;
    String zTrozo="";
    String zTrozo1="";
    StringTokenizer T=new StringTokenizer(ztipocarga,"{");
    while(T.hasMoreTokens()){
        zTrozo=T.nextToken();
        StringTokenizer S=new StringTokenizer(zTrozo,"=");
        a=S.countTokens();
        if (a==2){
            zTrozo1=S.nextToken();
            if ((zTrozo1==zcampoID1)|| (zTrozo1.equals(zcampoID1))){
                zID=S.nextToken();
            }else if
            ((zTrozo1==zcampoNombre1)|| (zTrozo1.equals(zcampoNombre1))){
                zNombre=S.nextToken();
            }
        }
    }
}
}
%>
<%
//escribe el n° de columnas que tiene la tabla
//el próximo n° de tabulador
    int zCols =5;
    int zTab=1;
%>

```

```

<form action="/servlet/CheckSecurity/JSP/shco_g0/shco_gen_act.jsp"
method="post" name="NombreFormulario" id="NombreFormulario">
<input type="hidden" id="TAG" name="TAG" value="%=zsubsesion%" />
<input type="hidden" id="ACC" name="ACC" value="INSERTAR" />
'Insertamos un nuevo input al formulario nombre formulario
<input type="hidden" id="SHCO_ UPDATING" name="SHCO_UPDATING"
value="%=zUp%" />
<input type="hidden" id="zredireccion" name="zredireccion"
value="%=zredireccion%" />
<table class="form" width="100%" cellpadding="2" border="2">
<thead><tr class="titulo"><th colspan="3">&nbsp;</th><th><a
title="m4:label m4name="%=zSHCOLBCLEAN%" />"
href="javascript:limp();"><img alt="m4:label
m4name="%=zSHCOLBCLEAN%" />" %> @ include file="../../files_gif/
ic_des.jsp" %> /></a></th></tr></thead>
<tbody>
<tr><td class="campo"><label for="X%=zcampoID1%">*&nbsp;<m4:label
m4name="%=zIdPk1%" /></label></td><td>&nbsp;<input
tabindex="%=(zTab + 1)%" %>=zReOn%> type="text"
id="X%=zcampoID1%" name="X%=zcampoID1%" size="9" maxlength="9"
title="m4:label m4name="%=zSHCOLBWRITE%" /> <m4:label
m4name="%=zIdPk1%" />" value="%=zID%" /></td><td
class="campo"><label for="%=zcampoNombre1%">*&nbsp;<m4:label
m4name="%=zLNPK1%" />&nbsp;</label></td><td>&nbsp;<input
tabindex="%=(zTab + 1)%" class="form" type="text"
id="%=zcampoNombre1%" name="%=zcampoNombre1%" size="40"
maxlength="62" title="m4:label m4name="%=zSHCOLBWRITE%" />
<m4:label m4name="%=zLNPK1%" />" value="%=zNombre%" /></td></tr>
<tr><td class="campo">&nbsp;<m4:label m4name="%=zIdPk2%" /></td>
<td><input tabindex="%=(zTab + 1)%" %>=zDisOn%> type="radio"
id="%=zcampoID2%" name="%=zcampoID2%" value="1" %>=zChh%> /
>&nbsp;<m4:label m4name="%=zID2h%" /></td><td colspan="2"><input
tabindex="%=(zTab + 1)%" %>=zDisOn%> type="radio"
id="%=zcampoID2%" name="%=zcampoID2%" value="2" %>=zChu%> /
>&nbsp;<m4:label m4name="%=zID2u%" /></td></tr>
<tr><td>&nbsp;&nbsp;&nbsp;<m4:label m4name="%=zIdPk3%" /></td><td
colspan="3">&nbsp;<input tabindex="%=(zTab + 1)%" %>=zReOn%>
type="text" id="%=zcampoID3%" name="%=zcampoID3%" size="6"
maxlength="6" title="m4:label m4name="%=zSHCOLBWRITE%" /> <m4:label
m4name="%=zIdPk3%" />" value="%=zID3%" /></td></tr>
<tr><td class="campo"><label
for="%=zcampoComentario%">&nbsp;<m4:label m4name="%=zLComment%" /
></label></td><td colspan="3">&nbsp;<textarea tabindex="%=(zTab +
1)%" cols="" rows="3" class="form" id="%=zcampoComentario%"
name="%=zcampoComentario%" title="m4:label
m4name="%=zSHCOLBWRITE%" /> <m4:label m4name="%=zLComment%" /
>">%=zComentario%</textarea></td></tr>
</tbody></table><%@ include file="../../shco_g0/shco_gen_mt_bus.jsp"
%>
</form>

```

```

<script type="text/javascript"
language="Javascript1.5">m4tabfocus('NombreFormulario',1);</script>
<%// Tabla principal de datos:
if (zcount>0) {%>
<table class="datos" width="100%" cellpadding="0"><thead><tr
class="titulo"><th>&nbsp;</th>
<th><a name="filter"></a><a><%if
((zOrdenCampo==zcampoID1)|| (zOrdenCampo.equals(zcampoID1))) {%><%if
((zOrden=="1")||(zOrden.equals("1"))) {%><img alt="<m4:label
m4name="<%=zSHCOLBORD%>" /> <m4:label m4name="<%=zIdPk1%>" />"
onclick="javascript:m4ordenar('<%=zcampoID1%>');" <%@ include
file="../../files_gif/ic_ord_1.jsp" %> /><%} else if
((zOrden=="2")||(zOrden.equals("2"))) {%><img alt="<m4:label
m4name="<%=zSHCOLBORD%>" /> <m4:label m4name="<%=zIdPk1%>" />"
onclick="javascript:m4ordenar('<%=zcampoID1%>');" <%@ include
file="../../files_gif/ic_ord_2.jsp" %> /><%}%><%} else {%><img
alt="<m4:label m4name="<%=zSHCOLBORD%>" /> <m4:label
m4name="<%=zIdPk1%>" />"
onclick="javascript:m4ordenar('<%=zcampoID1%>');" <%@ include
file="../../files_gif/ic_ord.jsp" %> /><%}%></a>&nbsp;<m4:label
m4name="<%=zIdPk1%>" /></th>
<th><a><%if
((zOrdenCampo==zcampoNombre1)|| (zOrdenCampo.equals(zcampoNombre1))) {i
f ((zOrden=="1")||(zOrden.equals("1"))) {%><img alt="<m4:label
m4name="<%=zSHCOLBORD%>" /> <m4:label m4name="<%=zLNPK1%>" />"
onclick="javascript:m4ordenar('<%=zcampoNombre1%>');" <%@ include
file="../../files_gif/ic_ord_1.jsp" %> /><%} else if
((zOrden=="2")||(zOrden.equals("2"))) {%><img alt="<m4:label
m4name="<%=zSHCOLBORD%>" /> <m4:label m4name="<%=zLNPK1%>" />"
onclick="javascript:m4ordenar('<%=zcampoNombre1%>');" <%@ include
file="../../files_gif/ic_ord_2.jsp" %> /><%}%><%} else {%><img
alt="<m4:label m4name="<%=zSHCOLBORD%>" /> <m4:label
m4name="<%=zLNPK1%>" />"
onclick="javascript:m4ordenar('<%=zcampoNombre1%>');" <%@ include
file="../../files_gif/ic_ord.jsp" %> /><%}%></a>&nbsp;<m4:label
m4name="<%=zLNPK1%>" /></th>
<th><a><%if
((zOrdenCampo==zcampoID2)|| (zOrdenCampo.equals(zcampoID2))) {%><%if
((zOrden=="1")||(zOrden.equals("1"))) {%><img alt="<m4:label
m4name="<%=zSHCOLBORD%>" /> <m4:label m4name="<%=zIdPk2%>" />"
onclick="javascript:m4ordenar('<%=zcampoID2%>');" <%@ include
file="../../files_gif/ic_ord_1.jsp" %> /><%} else if
((zOrden=="2")||(zOrden.equals("2"))) {%><img alt="<m4:label
m4name="<%=zSHCOLBORD%>" /> <m4:label m4name="<%=zIdPk2%>" />"
onclick="javascript:m4ordenar('<%=zcampoID2%>');" <%@ include
file="../../files_gif/ic_ord_2.jsp" %> /><%}%><%} else {%><img
alt="<m4:label m4name="<%=zSHCOLBORD%>" /> <m4:label
m4name="<%=zIdPk2%>" />"
onclick="javascript:m4ordenar('<%=zcampoID2%>');" <%@ include
file="../../files_gif/ic_ord.jsp" %> /><%}%></a>&nbsp;<m4:label
m4name="<%=zIdPk2%>" /></th>

```

```

<th>&nbsp;<m4:label m4name="<%=zIdPk3%>"/></th><th
colspan="2">&nbsp;<m4:label m4name="<%=zLComment%>"/></th><%@ include
file="../../shco_g0/shco_gen_pest.jsp" %>

</tr></thead><tbody><m4:loop from="<%=zregistroinicials%>"
to="<%=zregistrofinals%>"><%@ include file="../../shco_g0/
shco_gen_loop.jsp" %><tr>

<td class="boton<%=zpos%>"><a title="<m4:label
m4name="<%=zSHCOLBEDIT%>"/>" href="javascript:m4nothing();"

```

A En la edición hay que poner un 1 en la nuevo input y bloquear los campos no modificables

```

onclick="m4valor('NombreFormulario','SHCO_
UPDATING','1','set');m4lock('NombreFormulario','X<%=zcampoID1%>','rea
donly','lock','disabled');m4valor('NombreFormulario','X<%=zcampoID1%>
','<m4:item m4name="<%=zIdPk1%>"/
>','set');m4valor('NombreFormulario','<%=zcampoNombre1%>','<m4:item
m4name="<%=zNPK1%>"/
>','set');m4checkradio('NombreFormulario','<%=zcampoID2%>','<m4:item
m4name="<%=zIdPk2%>"/>);
m4lock('NombreFormulario','<%=zcampoID2%>','disabled','lock');m4lock(
'NombreFormulario','<%=zcampoID3%>','readonly','lock','disabled');m4v
alor('NombreFormulario','<%=zcampoID3%>','<m4:item
m4name="<%=zIdPk3%>"/
>','set');m4valor('NombreFormulario','<%=zcampoComentario%>','<m4:ite
m m4name="<%=zComment%>"/>','set');">

<img alt="<m4:label m4name="<%=zSHCOLBEDIT%>"/>" <%@ include
file="../../files_gif/ic_mod.jsp" %> /></a></td><td
class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zIdPk1%>"/></td>

<td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zNPK1%>"/></
td><td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zNPK2%>"/></
td><td class="valor<%=zpos%>">&nbsp;<m4:item m4name="<%=zIdPk3%>"/></
td><td class="valor<%=zpos%>"colspan="2">&nbsp;<m4:item
m4name="<%=zComment%>"/></td><td class="boton<%=zpos%>"><a
title="<m4:label m4name="<%=zSHCOLBDEL%>"/>"
href="javascript:m4eliminar('X<%=zcampoID1%>','<m4:item
m4name="<%=zIdPk1%>"/>');"><img alt="<m4:label
m4name="<%=zSHCOLBDEL%>"/>" <%@ include file="../../files_gif/
ic_bor.jsp" %> /></a></td>

</tr></m4:loop></tbody></table>

<%@ include file="../../shco_g0/shco_gen_vent_post.jsp" %><%}%><%@
include file="../../shco_g0/shco_gen_disclaimer.jsp" %>

<m4:endpage/></body></html>

```

Plantillas para asistentes: wizard

Duplicando la página más similar al caso a resolver se hacen los retoques necesarios. A continuación se describen cuáles son estos ajustes a realizar en las distintas páginas.

Se muestran dos ejemplos:

- Mantenimiento de 1 registros
- Mantenimiento de n registros

Mantenimiento de 1 registro

Se toma como ejemplo la página de la aplicación estándar:

shco_ta_wz_def_daycalen_p1.jsp.

Includes genéricos. NO MODIFICABLE

```
<%@ include file="../../shco_g0/shco_gen_taglib.jsp" %>
```

```
<html><head><title></title>
```

El título de la página puede ser un ítem creado en el nodo: [SHCO_GN_LABEL_N2](#)

```
<%@ include file="../../shco_g0/shco_gen_arg.jsp" %>
```

```
<%@ include file="../../shco_g0/shco_gen_bag.jsp" %>
```

```
<%@ include file="../../shco_g0/shco_gen_css.jsp" %>
```

Modificable. Descrito posteriormente.

```
<%//include particular del wizard%>
```

```
<%@ include file="../../shco_ta/shco_ta_wz_def_daycalen_m4def.jsp" %>
```

```
<%// Parámetros del M4Object:
```

```
zcarril="/servlet/CheckSecurity/JSP/shco_ta_sse/
```

```
shco_ta_list_def_daycalen.jsp?zpag=shco_ta/
```

```
shco_ta_wz_def_daycalen_p1.jsp&znw=0";
```

```
//*****
```

```
// *MODIFICABLE
```

```
zventanas = "10";
```

```
zvuelta = 5;
```

```
znivelmenu = "4";
```

```
String zdireccion = "shco_ta_wz_def_daycalen_p1.jsp";
```

```
String zredireccion = "shco_ta/shco_ta_wz_def_daycalen_p1.jsp";
```

```
//*****
```

Identificador obtenido de una lista.

```
String zSHCO_ID_DAY_CALENDAR =
```

```
request.getParameter("SCO_ID_DAY_CALENDAR");
```

```
// Parametro de la lista
```

```
//*****
```

```
//Recoge si es nuevo, para habilitar o no la Pk
```

```
String znw= request.getParameter("znw");
```

```

        if ((znw==null)|| (znw.equals(""))){znw = "0";}
        if
        ((zSHCO_ID_DAY_CALENDAR==null)|| (zSHCO_ID_DAY_CALENDAR.equals(""))
        ){zSHCO_ID_DAY_CALENDAR =
        request.getParameter("XSCO_ID_DAY_CALENDAR");}
        String zReOn="class=\"form\"";
        if ((znw=="0")|| (znw.equals("0"))){zReOn="readOnly=\"readOnly\"
        class=\"disabled\"";}

//*****
// *NO MODIFICABLE
int zLoadTypeStep = 1;           // Tipo de carga
int zIndexWizard = 0;           // Indice de control
//*****/

//*****/
// *MODIFICABLE
String znodoview = "SHCO_TA_WZ_DAY_CALENDAR_DEF";
/
//*****/

%>
*****

Include genérico
<%@ include file="../../shco_g0/shco_gen_wz_def.jsp" %>
<%@ include file="../../shco_g0/shco_gen_datadef.jsp" %>
*****

<%try {
    M4Operations m = new M4Operations(request);
    m.setItem(zm4object,znodoraiz,"","SHCO_NEW",znw);

    m.setItem(zm4object,znodoraiz,"","SHCO_WZ_FIRST_TIME","FIRST_TIME");
    if (znw=="1"){
    m.setItem(zm4object,znodoraiz,"","SHCO_ID_DAY_CALENDAR",zSHCO_ID_DAY_
    CALENDAR);
        }else if (zSHCO_ID_DAY_CALENDAR!=" " &&
        (! (zSHCO_ID_DAY_CALENDAR.equals("")))) &&
        zSHCO_ID_DAY_CALENDAR!=null){

    m.setItem(zm4object,znodoraiz,"","SHCO_ID_DAY_CALENDAR",zSHCO_ID_DAY_
    CALENDAR);
        }
    }catch(Exception e) {}
%>
*****

Include genérico
<%@ include file="../../shco_g0/shco_gen_wz_act.jsp" %>

```



```

<%@ include file="../../shco_ta/
shco_ta_wz_outputdef_def_daycalen.jsp" %>
*****

<%
// items
String zId= "SCO_ID_DAY_CALENDAR";
String zNm= "SCO_NM_DAY_CALENDAR";
String zDtIni= "DT_START";
String zDtFin= "DT_END";
String zId3= "SCO_COMMENT";

//Identificadores items
String zIdr= zraiz+zId;
String zNmr= zraiz+zNm;
String zDtInir= zraiz+zDtIni;
String zDtFinr= zraiz+zDtFin;
String zId3r= zraiz+zId3;
%>

<script type="text/javascript" language="Javascript1.5">
function val(){
var sfunciones =
"m4valinput('_alfanum_oblig','NombreFormulario','X<%=zId%>',1,'<m4:la
bel m4name="<%=zIdr%>" jsafe="true"/>')";
sfunciones =
sfunciones+"*"+"m4valinput('_alfanum_oblig','NombreFormulario','<%=zN
mr%>',1,'<m4:label m4name="<%=zNmr%>" jsafe="true"/>')";
sfunciones =
sfunciones+"*"+"m4valinput('_date_oblig','NombreFormulario','<%=zDtIn
i%>',1,'<m4:label m4name="<%=zDtInir%>" jsafe="true"/
>',sformatofechas)";
sfunciones =
sfunciones+"*"+"m4valinput('_date','NombreFormulario','<%=zDtFin%>',1
,'<m4:label m4name="<%=zDtInir%>" jsafe="true"/>',sformatofechas)";
sfunciones =
sfunciones+"*"+"m4valinput('_com','NombreFormulario','<%=zDtIni%>', 'N
ombreFormulario','<%=zDtFin%>', '<=','_date',0,'<m4:label
m4name="<%=zDtInir%>" jsafe="true"/>', '<m4:label
m4name="<%=zDtFinr%>" jsafe="true"/>')";
sfunciones=sfunciones+"*m4valinput('_comentario','NombreFormulario','
<%=zId3%>',256,'<m4:label m4name="<%=zId3r%>" jsafe="true"/>')";
var verr=m4valform(sfunciones);
if (verr == 1){ m4submit("NombreFormulario");}}
}

```

```

</script>
*****

Includes genéricos
<%@ include file="../../../shco_g0/shco_gen_wz_js.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_wz_nav.jsp" %>
*****

</head><body >
*****

Includes genéricos
<%@include file="../../../shco_g0/shco_gen_title.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_menusup.jsp" %>
<%@include file="../../../shco_g0/shco_gen_wz_error.jsp" %>
*****

Capa del menu superior
<div id="capa_link" style="position:absolute; left:0%; top:0%;
width:20%; height:0%; z-index:1">
<%@ include file="../../../shco_g0/shco_gen_wz_menu.jsp" %></div>
*****

Capa del cuerpo.
<div id="capa_cuerpo" style="position:relative; left:21%; top:0%;
width:78%; z-index:2">

<%
String zvalue = "";
String zhelp="SHCO_TA_WZ_DEF_DAYCALEN_P1.htm";
zCol = 4;
%>
<%@ include file="../../../shco_g0/shco_gen_cab.jsp" %>

*****

Formulario para la Inserción / Actualización de datos.
Cualquier imagen es un include. La imagen puede variar en función de
la parametrización de usuario.
<form action="<%=path%><%=links[zIndexWizard]%>" method="post"
name="NombreFormulario" id="NombreFormulario" >
<input type="hidden" id="TAG" name="TAG" value="<%=zsubsesion%>" />
<input type="hidden" id="M4OBJ_ID" name="M4OBJ_ID"
value="<%=zm4object%>" />
<input type="hidden" id="ACC" name="ACC" value="01" />
<input type="hidden" id="NOD" name="NOD" value="<%=znodoview%>" />
<input type="hidden" id="WZINDEX" name="WZINDEX"
value="<%=zIndexWizard%>" />

```

```

| &nbsp;<%=steps[zIndexWizard]%></th>  &nbsp;<a title="<m4:label m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/> <%=steps[zIndexWizard]%>" href="javascript:m4wzeliminar();"> <img alt="<m4:label m4name="<%=zSHCOLBDEL%>" htmlsafe="true"/> <%@ include file="../../ files_gif/ic_bor.jsp" %> /> </a></th>  <%=steps[zIndexWizard]%></th> | | | | |
| --- | --- | --- | --- | --- |
|  | | |
|  | | |
|  | | |

```

```

ni%>'));" ><img <%@ include file="../../../files_gif/ic_cal.jsp" %>
alt="<m4:label m4name="<%=zDtInir%>" htmlspecialchars="true"/>" /></a></td>

<td class="campo">&nbsp;<m4:label m4name="<%=zDtFinr%>"
htmlspecialchars="true"/></td>

<td class="valor"><input tabindex="<%= (zTab + 1)%>" class="form"
type="text" name="<%=zDtFin%>" id="<%=zDtFin%>" title="<m4:label
m4name="<%=zDtFinr%>" htmlspecialchars="true"/>" maxlength="10" size="10"
value="<m4:item m4name="<%=zDtFinr%>" htmlspecialchars="true"/>" />

<a tabindex="<%= (zTab +
1)%>" href="javascript:m4calendar(m4objeto('NombreFormulario', '<%=zDtF
in%>'));" ><img <%@ include file="../../../files_gif/ic_cal.jsp" %>
alt="<m4:label m4name="<%=zDtFinr%>" htmlspecialchars="true"/>" /></a></td>

</tr>

<tr>

<td class="campo">&nbsp;<m4:label m4name="<%=zId3r%>"
htmlspecialchars="true"/></td>

<td colspan="3">&nbsp;<textarea tabindex="<%= (zTab +
1)%>" tabindex="5" cols="" rows="3" class="form" id="<%=zId3%>"
name="<%=zId3%>" title="<m4:label m4name="<%=zSHCOLBWRITE%>"
htmlspecialchars="true"/> <m4:label m4name="<%=zId3r%>" htmlspecialchars="true"/
>"><m4:item m4name="<%=zId3r%>" htmlspecialchars="true"/></textarea></td>

</tr>

*****

Include generico. Botones de navegación.

<%@ include file="../../../shco_g0/shco_gen_wz_btt.jsp" %>

*****

</tbody></table></form>

Posicionamiento del foco al refresco de la página.

<script type="text/javascript" language="Javascript1.5">
m4tabfocus('NombreFormulario',1); m4setstatus();
</script>

*****

Includes genéricos.

*****

<%@ include file="../../../shco_g0/shco_gen_error.jsp" %><%}%>

<%@ include file="../../../shco_g0/shco_gen_disclaimer.jsp" %>

*****

</body></html>

```

Descripción del include particular

```

<%@ include file="../../../shco_ta/shco_ta_wz_def_daycalen_m4def.jsp" %>

<%

/*****

```

```

// *MODIFICABLE

String znodo1 = "SHCO_TA_WZ_DAY_CALENDAR_DEF";
String znodo2 = "SHCO_TA_WZ_DAYCALEN_DEF";
String znodo3 = "SHCO_TA_WZ_ASSIGN_CALENDAR";

String zoutputdef1 = zsubsesion + "!" + znodo1 + "[*]";
String zoutputdef2 = zsubsesion + "!" + znodo2 + "[0]";
String zoutputdef3 = zsubsesion + "!" + znodo3 + "[0]";

String zmove1 = znodo1 + ":" + znodo1 + "[0]";
String zmove2 = znodo2 + ":" + znodo2 + "[0]";
String zmove3 = znodo3 + ":" + znodo3 + "[0]";
switch(zLoadTypeStep)
{
case 1: zoutputdef1 = zsubsesion + "!" + znodo1 + "[*]";
case 2: zoutputdef2 = zsubsesion + "!" + znodo2 + "[" +
zregistroinicial + "-" + zregistrofinal + "]";
case 3: zoutputdef3 = zsubsesion + "!" + znodo3 + "[" +
zregistroinicial + "-" + zregistrofinal + "]";
}
//*****

%>

<m4:outputdef m4alias="<%=znodo1%>" ><m4:param name="m4name0"
value="<%=zoutputdef1%>" /></m4:outputdef>
<m4:outputdef m4alias="<%=znodo2%>" ><m4:param name="m4name0"
value="<%=zoutputdef2%>" /></m4:outputdef>
<m4:outputdef m4alias="<%=znodo3%>" ><m4:param name="m4name0"
value="<%=zoutputdef3%>" /></m4:outputdef>
<m4:outputdef m4alias="<%=znodocom%>" ><m4:param name="m4name0"
value="<%=zoutputdefcom%>" /></m4:outputdef>
<m4:outputdef m4alias="<%=znodolabel%>" ><m4:param name="m4name0"
value="<%=zoutputdeflab%>" /></m4:outputdef>
<m4:endjob/>

<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmove1%>" /></
m4:move>
<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmove2%>" /></
m4:move>
<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmove3%>" /></
m4:move>
<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmove1lab%>" /></
m4:move>

```

Mantenimiento de N registros

Se toma como ejemplo la página de la aplicación estándar:
shco_g2_p3_inf_person.jsp:

```
<%@ include file="../../shco_g0/shco_gen_taglib.jsp"
%><html><head><title></title>

<%@ include file="../../shco_g0/shco_gen_arg.jsp" %><%@ include
file="../../shco_g0/shco_gen_bag.jsp" %><%@ include file="../../
shco_g0/shco_gen_css.jsp" %>

<%// Parámetros del M4Object:
```

Página que se quiere utilizar de filtro tipo lista

```
zcarri="/servlet/CheckSecurity/JSP/shco_g2_sse/
shco_pa_list_person.jsp?zpag=shco_g2/
shco_g2_p1_inf_person.jsp&zpw=0";

//*****

// *MODIFICABLE
Nº de registros de la paginación
zventanas = "10";
zvuelta = 5;
String zdireccion = "shco_g2_p3_inf_person.jsp";
String zredireccion = "shco_g2/shco_g2_p3_inf_person.jsp";
```

Nivel de menú

```
znivelmenu = "3";
```

Valores por defecto de los input del formulario y recuperación de parámetros para la paginación

```
String zId = request.getParameter("zId");
String zNm = request.getParameter("zNm");
String zDtIni = request.getParameter("zDtIni");
String zDtFin = request.getParameter("zDtFin");
if ((zId==null)||zId.equals("")){zId = "";}
if ((zNm==null)||zNm.equals("")){zNm = "";}
if ((zDtIni==null)||zDtIni.equals("")){zDtIni = "";}
if ((zDtFin==null)||zDtFin.equals("")){zDtFin = "";}

//*****
/*****/

// *NO MODIFICABLE
int zLoadTypeStep = 0;           // Tipo de carga
int zIndexWizard = 0;           // Indice de control
/

*****/
```

```
%>
```

Modificable. Descrito en Mantenimiento de 1 registro.

```
<%@ include file="../../shco_g2/shco_pa_wz_subsesion_person.jsp" %>
<%
```

Nodo de visualización

```
String znodoview = "SHCO_PA_WZ_MAR_STATS";
/*****
%>
/
*****/

Include genérico
<%@ include file="../../shco_g0/shco_gen_wz_def.jsp" %>
<%@ include file="../../shco_g0/shco_gen_datadef.jsp" %>
<%@ include file="../../shco_g0/shco_gen_wz_act.jsp" %>
/
*****/

<%//include particular del wizard descrito en Mantenimiento de 1
registro %>
<%@ include file="../../shco_g2/shco_gen_wz_outputdef_pa.jsp" %>
<%
```

//Identificadores items

```
String zSTD_DT_START = zraiz + "STD_DT_START";
String zSTD_DT_END = zraiz + "STD_DT_END";
String zSTD_ID_MARITAL_STAT = zraiz + "STD_ID_MARITAL_STAT";
String zSTD_N_MARITAL_STAT = zraiz + "STD_N_MARITAL_STAT";
String zSTD_ID_HR = zraiz + "STD_ID_HR";
```

//Identificadores items LOOP

```
String zSTD_DT_STARTcm = zcomun + "STD_DT_START";
String zSTD_DT_ENDcm = zcomun + "STD_DT_END";
String zSTD_ID_MARITAL_STATcm = zcomun + "STD_ID_MARITAL_STAT";
String zSTD_N_MARITAL_STATcm = zcomun + "STD_N_MARITAL_STAT";
String zSTD_ID_HRcm = zcomun + "STD_ID_HR";
```

//Items

```
String zSTD_ID_HR_del = "STD_ID_HR";
String zSTD_DT_START_del = "STD_DT_START";
```

```

String zSTD_DT_END_del = "STD_DT_END";
String zSTD_ID_MARITAL_STAT_del = "STD_ID_MARITAL_STAT";
String zSTD_N_MARITAL_STAT_del = "STD_N_MARITAL_STAT";

%>
/*****
Include genérico
<%@ include file="../../shco_g0/shco_gen_wz_nav.jsp" %>
<%@ include file="../../shco_g0/shco_gen_wz_js.jsp" %>
*****/
<script type="text/javascript" language="Javascript1.5">

```

Función que se ejecuta antes de la inserción de datos

```
function val(){
```

En primer lugar validamos los formatos de los campos

```

var sfunciones =
"m4valinput('_date_oblig','NombreFormulario','XSTD_DT_START',1,'<m4:label m4name="<%=zSTD_DT_START%>" jsafe="true"/>',sformatofechas)"

sfunciones =
sfunciones+"*"+"m4valinput('_alfanum_oblig','NombreFormulario','STD_ID_MARITAL_STAT',1,'<m4:label m4name="<%=zSTD_ID_MARITAL_STAT%>" jsafe="true"/>',sformatofechas)";

sfunciones =
sfunciones+"*"+"m4valinput('_date','NombreFormulario','STD_DT_END',1,'<m4:label m4name="<%=zSTD_DT_END%>" jsafe="true"/>',sformatofechas)";

var verr=m4valform(sfunciones);
if (verr == 1){

```

Como ya sabemos que los campos fecha tienen el formato esperado , comparamos el mayor e igual

```

sfunciones
="m4valinput('_com','NombreFormulario','XSTD_DT_START','NombreFormulario','STD_DT_END','<=','_date',0,MSG_ERROR_ID+'_sl_61_0_1','<m4:label m4name="<%=zSTD_DT_START%>" jsafe="true"/>',<m4:label m4name="<%=zSTD_DT_END%>" jsafe="true"/>')";
verr=m4valform(sfunciones);
if (verr == 1){

```

No modificable en la función

```
var vc= m4comwz ();
```



```

        if(vc ==1 ){
            m4submit("NombreFormulario");
        }
    }
}
}

```

Función que se ejecuta antes de la paginación, ver la paginación

```

function valores(){
m4valor("oculto","zId",m4valor("NombreFormulario","STD_ID_MARITAL_STA
T","", "get"), "set");
m4valor("oculto","zNm",m4valor("NombreFormulario","STD_N_MARITAL_STAT
","", "get"), "set");
m4valor("oculto","zDtIni",m4valor("NombreFormulario","XSTD_DT_START",
","", "get"), "set");
m4valor("oculto","zDtFin",m4valor("NombreFormulario","STD_DT_END","",
"get"), "set");
m4submit('oculto');
}
</script>
/*****
No modificable
</head><body  onload="javascript:m4setm4titwz();">
/
*****/

<%
String zidperson = "";
String zgbyname = "";
try {
    M4Operations m1 = new M4Operations(request);
    zidperson =
    m1.getItem(znodocom,zsubsesion,znodocom,"","SHCO_STRING3");
    zgbyname =
    m1.getItem(znodocom,zsubsesion,znodocom,"","SHCO_STRING2");
}catch(Exception e) {}
%>
/*****/

No modificable
<%@include file="../../shco_g0/shco_gen_title.jsp" %>
<%@ include file="../../shco_g0/shco_gen_menusup.jsp" %>
<%@include file="../../shco_g0/shco_gen_wz_error.jsp" %>

```

```

<div id="capa_link" style="position:absolute; left:0%; top:0%;
width:20%; height:0%; z-index:1"><%@ include file="../../shco_g0/
shco_gen_wz_menu.jsp" %></div>

<div id="capa_cuerpo" style="position:relative; left:21%; top:0%;
width:78%; z-index:2">

/*****

/*include las cabeceras*/

<%String zhelp="SHCO_G2_P3_INF_PERSON.htm";%>
<%@ include file="../../shco_g2/shco_gen_cab_person.jsp" %>
/*****

Formulario para la Inserción / Actualización de datos.
Cualquier imagen es un include. La imagen puede variar en función de
la parametrización de usuario.

<form action="<%=path%><%=links[zIndexWizard]%>" method="post"
name="NombreFormulario" id="NombreFormulario" >
<input type="hidden" id="TAG" name="TAG" value="<%=zsubsesion%>" />
<input type="hidden" id="ACC" name="ACC" value="<%=zUpd%>" />
<input type="hidden" id="NOD" name="NOD" value="<%=znodoview%>" />
<input type="hidden" id="WZINDEX" name="WZINDEX"
value="<%=zIndexWizard%>" />
<input type="hidden" id="LOADTYPE" name="LOADTYPE"
value="<%=zLoadTypeStep%>" />
<input type="hidden" id="zredireccion" name="zredireccion"
value="<%=zredireccion%>" />
<input type="hidden" id="SHCO_ORDINAL" name="SHCO_ORDINAL" value="-1"
/>
<input type="hidden" id="SHCO_STATE" name="SHCO_STATE" value="N" />
<input type="hidden" id="XSTD_ID_HR" name="XSTD_ID_HR"
value="<%=zidperson%>" />
<table class="form" width="100%" cellpadding="2" border="2">
<thead><tr class="titulo">
<th id="m4tit" colspan="3">&nbsp;<%=steps[zIndexWizard]%></th>
<th><a title="<m4:label m4name="<%=zSHCOLBCLEAN%>" htmlsafe="true"/>"
href="javascript:m4clearform('NombreFormulario');m4limwz ();"><img
alt="<m4:label m4name="<%=zSHCOLBCLEAN%>" htmlsafe="true"/>" <%@
include file="../../files_gif/ic_des.jsp" %> /></a></th>
</tr>
</thead>
<tbody>
<tr><td class="campo">&nbsp;<%=zSTD_DT_START%>" htmlsafe="true"/></td><td>&nbsp;<input
tabindex="<%=zTab + 1)%>" class="form" type="text"
name="XSTD_DT_START" id="XSTD_DT_START" title="<m4:label

```

```

m4name="<%=zSTD_DT_START%>" htmlspecialchars="true"/>" maxlength="10"
size="10"
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zDt
Ini)%>" />&nbsp;<a tabindex="<%=zTab + 1)%>"
href="javascript:m4calendar(m4objeto('NombreFormulario','XSTD_DT_STAR
T'));" ><img <%@ include file="../../files_gif/ic_cal.jsp" %>
alt="<m4:label m4name="<%=zSTD_DT_START%>" htmlspecialchars="true"/>" /></
a></td>

<td class="campo">&nbsp;<m4:label m4name="<%=zSTD_DT_END%>"
htmlspecialchars="true"/></td>

<td><input tabindex="<%=zTab + 1)%>" class="form" type="text"
name="STD_DT_END" id="STD_DT_END" title="<m4:label
m4name="<%=zSTD_DT_END%>" htmlspecialchars="true"/>" maxlength="10" size="10"
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zDt
Fin)%>" /><a tabindex="<%=zTab + 1)%>"
href="javascript:m4calendar(m4objeto('NombreFormulario','STD_DT_END'
));" ><img <%@ include file="../../files_gif/ic_cal.jsp" %>
alt="<m4:label m4name="<%=zSTD_DT_END%>" htmlspecialchars="true"/>" /></a></
td>

</tr>

<tr>

<td>&nbsp;&nbsp;&nbsp;<a title="<m4:label m4name="<%=zSHCOLBREM2%>"
htmlspecialchars="true"/>" <m4:label m4name="<%=zSTD_ID_MARITAL_STAT%>"
htmlspecialchars="true"/>" href="javascript:m4remonte('shco_g2/
shco_pa_mt_mar_stat.jsp','<%=zSTD_ID_MARITAL_STAT_del%>');"><m4:label
m4name="<%=zSTD_ID_MARITAL_STAT%>" htmlspecialchars="true"/></a></td>

<td colspan="3">&nbsp;<input tabindex="<%=zTab + 1)%>" class="form"
type="text" id="<%=zSTD_ID_MARITAL_STAT_del%>"
name="<%=zSTD_ID_MARITAL_STAT_del%>" size="2" maxlength="2"
title="<m4:label m4name="<%=zSHCOLBWRITE%>" htmlspecialchars="true"/>"
<m4:label m4name="<%=zSTD_ID_MARITAL_STAT%>" htmlspecialchars="true"/>"
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zId
)%>" />&nbsp;<input class="disabled" type="text"
id="<%=zSTD_N_MARITAL_STAT_del%>" name="<%=zSTD_N_MARITAL_STAT_del%>"
tabindex="<%=zTab + 1)%>" size="30" maxlength="62" title="<m4:label
m4name="<%=zSTD_N_MARITAL_STAT%>" htmlspecialchars="true"/>"
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cookHTML(zNm
)%>" disabled="disabled" />&nbsp;<a tabindex="<%=zTab + 1)%>"
title="" href="javascript:m4filtro('shco_g2_sse/
shco_pa_list_mar_stat.jsp','<%=zSTD_ID_MARITAL_STAT_del%>','<%=zSTD_N
_MARITAL_STAT_del%>');" ><img alt="<m4:label m4name="<%=zSHCOLBLIST%>"
htmlspecialchars="true"/>" <m4:label m4name="<%=zSTD_N_MARITAL_STAT%>"
htmlspecialchars="true"/>"<%@ include file="../../files_gif/ic_list.jsp"%> /
></a></td>

</tr>

</table>

<%@ include file="../../shco_g0/shco_gen_wz_btt.jsp" %>

</form>

<script type="text/javascript"
language="Javascript1.5">m4tabfocus('NombreFormulario',1);m4setm4titw
z ();</script>

```

```

/*Formulario de paginación
<form action="<%=zdireccion%>" method="post" name="oculto"
id="oculto">
<input type="hidden" id="TAG" name="TAG" value="<%=zsubsesion%>" />
<input type="hidden" id="zPkA" name="zPkA" value="<%=zPkA%>" />
<input type="hidden" id="LOADTYPE" name="LOADTYPE"
value="<%=zLoadTypeStep%>" />
<input type="hidden" id="WZINDEX" name="WZINDEX"
value="<%=zIndexWizard%>" />
<input type="hidden" id="zinicio" name="zinicio" value="" />
<input type="hidden" id="zDtIni" name="zDtIni" value="" />
<input type="hidden" id="zDtFin" name="zDtFin" value="" />
<input type="hidden" id="zId" name="zId" value="" />
<input type="hidden" id="zNm" name="zNm" value="" />
</form>

```

Visualización de los n registros

```

<%if(zcounti>0){%>
<table class="datos" width="100%" cellpadding="0" cellspacing="0"
border="0">
<thead><tr class="titulo">
<th class="campo"></th>
<th class="campo">&nbsp;<m4:label m4name="<%=zSTD_DT_START%>"
htmlsafe="true"/> - <m4:label m4name="<%=zSTD_DT_END%>"
htmlsafe="true"/></th>
<th class="campo">&nbsp;<m4:label m4name="<%=zSTD_ID_MARITAL_STAT%>"
htmlsafe="true"/></th>
<th class="campo"><%=@ include file="../../shco_g0/shco_gen_pest.jsp"
%></th></tr>
</thead>
<tbody>
<m4:loop from="<%=zregistroinicial%>" to="<%=zregistrofinal%>">
<%=@include file="../../shco_g0/shco_gen_loop.jsp"%>
<tr>
<td class="boton<%=zpos%>"><a title="<m4:label
m4name="<%=zSHCOLBEDIT%>" htmlsafe="true"/>"
href="javascript:m4valor('NombreFormulario','X<%=zSTD_DT_START_del%>',
'<m4:item m4name="<%=zSTD_DT_STARTcm%>" jsafe="true"
htmlsafe="true"/
>', 'set');m4valor('NombreFormulario','<%=zSTD_DT_END_del%>', '<m4:item
m4name="<%=zSTD_DT_ENDcm%>" jsafe="true" htmlsafe="true"/
>', 'set');m4valor('NombreFormulario','<%=zSTD_ID_MARITAL_STAT_del%>',
'<m4:item m4name="<%=zSTD_ID_MARITAL_STATcm%>" jsafe="true"
htmlsafe="true"/
>', 'set');m4valor('NombreFormulario','<%=zSTD_N_MARITAL_STAT_del%>',
'<m4:item m4name="<%=zSTD_N_MARITAL_STATcm%>" jsafe="true"

```

```

htmlsafe="true"/>', 'set');m4editwz();"><img alt="m4:label
m4name="<%=zSHCOLBEDIT%" htmlsafe="true"/>" <%@ include file="../../
files_gif/ic_mod.jsp" %> /></a></td>

<td class="valor<%=zpos%">"><m4:item m4name="<%=zSTD_DT_STARTcm%">"
htmlsafe="true"/>-<m4:item m4name="<%=zSTD_DT_ENDcm%">"
htmlsafe="true"/></td>

<td class="valor<%=zpos%">" ><m4:item
m4name="<%=zSTD_ID_MARITAL_STATcm%">" htmlsafe="true"/>-<m4:item
m4name="<%=zSTD_N_MARITAL_STATcm%">" htmlsafe="true"/></td>

<td class="boton<%=zpos%">" colspan="3"><a title="m4:label
m4name="<%=zSHCOLBDEL%">" htmlsafe="true"/>"
href="javascript:m4wzeliminar('X<%=zSTD_ID_HR_del%">', '<m4:item
m4name="<%=zSTD_ID_HRcm%">" jsafe="true" htmlsafe="true"/
>', 'X<%=zSTD_DT_START_del%">', '<m4:item m4name="<%=zSTD_DT_STARTcm%">"
jsafe="true" htmlsafe="true"/>');"><img alt="m4:label
m4name="<%=zSHCOLBDEL%">" htmlsafe="true"/>" <%@ include file="../../
files_gif/ic_bor.jsp" %> /></a></td>

</tr>

</m4:loop>

</tbody></table>

<%}else{
//No hay datos...
}%>

<%@ include file="../../shco_g0/shco_gen_vent_post.jsp" %><%@ include
file="../../shco_g0/shco_gen_error.jsp" %><%@ include file="../../
shco_g0/shco_gen_disclaimer.jsp" %>

<%}%>

</body>

</html>

```

Plantilla del asistente de procesos

Para ilustrar la estructura de las páginas JSP asociadas al asistente de procesos, en este apartado se incluye un ejemplo de código de página con comentarios y texto en color que resalta las características de los códigos.

Ejemplo de código de página JSP para un paso del asistente de procesos

El nombre de esta página es shco_prueba1.jsp.

```

Includes genéricos. NO MODIFICABLE
*****

<%@ include file="../../shco_g0/shco_gen_taglib.jsp"
%><html><head><title></title>

```

```

<%@ include file="../../shco_g0/shco_gen_arg.jsp" %>
<%@ include file="../../shco_g0/shco_gen_bag.jsp" %>
<%@ include file="../../shco_g0/shco_gen_css.jsp" %>
*****

Modificable.
*****

<%
Directorio y página actuales
String zredireccion = "shco_g1/shco_prueba1.jsp";
*****

%>
*****

Modificable. Descrito según este ejemplo de código.
<%@ include file="../../shco_g1/shco_prueba_m4def.jsp" %>
<%//Nodo de visualización
String znodoview = "PRUEBA1";
%>
*****

<%@ include file="../../shco_g0/shco_gen_p_wz_def.jsp" %>
<%@ include file="../../shco_g0/shco_gen_p_wz_datadef.jsp" %>
<%@ include file="../../shco_g0/shco_gen_p_wz_act.jsp" %>
*****

Modificable. Descrito según este ejemplo de código.
<%@ include file="../../shco_g1/shco_outputdef_prueba.jsp" %>
<%
//Elementos mostrados en la página
String zPARAM_1 = zraiz + "PARAM_1";
String zPARAM_2 = zraiz + "PARAM_2";
%>

Includes genéricos. NO MODIFICABLE
*****

<%@ include file="../../shco_g0/shco_gen_p_wz_js.jsp" %>
<%@ include file="../../shco_g0/shco_gen_p_wz_nav.jsp" %>
*****

Funciones de validación.
<script type="text/javascript" language="Javascript1.5">
Función que se ejecuta antes de llevar a cabo el paso actual. Esta
función debe confirmar que se han rellenado todos los campos
obligatorios y que los demás elementos tienen el formato adecuado.
function val(){

```

```

var sfunciones =
"m4valinput('_alfanum_oblig','NombreFormulario','<%=zPARAM_1%>',1,'<m
4:label
m4name="<%=zPARAM_1%>" jsafe="true"/>')";
sfunciones = sfunciones
+"*"+"m4valinput('_date','NombreFormulario','<%=zPARAM_2%>',1,'<m4:la
bel m4name="<%=zPARAM_2%>" jsafe="true"/>',sformatofechas)";
var verr=m4valform(sfunciones);
return(verr);
}
</script>

```

Funciones de validación. Los input que forman parte de la PK deben tener la siguiente notación: 'X' + nombre del campo. Por ejemplo: 'X<%=zPARAM_1%>'.

Includes genéricos. NO MODIFICABLE

```

*****
</head><body >
<%@include file="../../../shco_g0/shco_gen_title.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_menusup.jsp" %>
<%@include file="../../../shco_g0/shco_gen_p_wz_error.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_p_wz_menu.jsp" %>

```

A Aquí se agregan las acciones al paso mediante un *include*.

```

<%@ include file="../../../shco_g0/shco_gen_p_wz_body.jsp" %>
<%
//Ayuda en línea y valores de encabezado.
// Para la ayuda en línea, defina zhelp con el nombre del archivo de
la página de ayuda
//Para agregar texto por debajo del título del encabezado, defina
zvalue con el texto que desea mostrar.
String zvalue = "";
String zhelp="";
%>
<%@ include file="../../../shco_g0/shco_gen_cab.jsp" %>
<%@ include file="../../../shco_g0/shco_gen_p_wz_form.jsp" %>
<table class="form" width="100%" cellpadding="2" border="2">
<thead><tr class="titulo">
<th>&nbsp;<%=steps[zIndexWizard]%></th>
<th><a title="m4:label m4name="<%=zSHCOLBCLEAN%>" htmlspecialchars="true"/>"
href="javascript:m4clearform('NombreFormulario');"><img
alt="m4:label m4name="<%=zSHCOLBCLEAN%>" htmlspecialchars="true"/>" <%@
include file="../../../files_gif/ic_des.jsp" %> /></a></th>

```

```

</tr></thead>
//Formulario que se muestra en la página
<tbody>
<tr>
<td class="campo">&nbsp;<m4:label m4name="<%=zPARAM_1%>"
htmlsafe="true"/></td>
<td>&nbsp;<input tabindex="<%= (zTab + 1)%>" class="form" type="text"
id="PARAM_1" size="9" maxlength="9" name="PARAM_1" value="<m4:item
m4name="<%=zPARAM_1%>" htmlsafe="true"/>" /></td>
</tr>
<tr>
<td class="campo">&nbsp;<m4:label m4name="<%=zPARAM_2%>"
htmlsafe="true"/></td>
<td>&nbsp;<input tabindex="<%= (zTab + 1)%>" class="form" type="text"
id="PARAM_2" size="10" maxlength="10" name="PARAM_2" value="<m4:item
m4name="<%=zPARAM_2%>" htmlsafe="true"/>" />&nbsp;<a
tabindex="<%= (zTab +
1)%>" href="javascript:m4calendar(m4objeto('NombreFormulario', 'PARAM_2
')); " ><img <%@ include file="../../files_gif/ic_cal.jsp" %>
alt="<m4:label m4name="<%=zPARAM_2%>" htmlsafe="true"/>" /></a></td>
</tr></tbody>

```

Los input que forman parte de la PK deben tener la siguiente notación:
'X' + nombre del campo. Por ejemplo: X<%=zPARAM_1%>.

```

</table>
A<%@ include file="../../shco_g0/shco_gen_p_wz_btt.jsp" %>
A(Si el paso sólo muestra los datos, utilice el include <%@ include
file="../../shco_g0/shco_gen_p_wz_btt_vis.jsp" %>)
</form>
//Lo que se indica a continuación coloca el cursor en el primer campo
de entrada del formulario
<script type="text/javascript"
language="Javascript1.5">m4tabfocus('NombreFormulario',1);</script>
<%@ include file="../../shco_g0/shco_gen_error.jsp" %><%}%><%@
include file="../../shco_g0/shco_gen_disclaimer.jsp" %>
</body></html>

```

Código de los *includes* especiales que se utilizan en el ejemplo

shco_prueba_m4def.jsp: define el Meta4Object que se utiliza.

```

<%//Nombre del Meta4Object
String zm4object = "SHCO_PARAM_TEST";
//Nivel del menú e información necesaria para la paginación
zventanas = "20";
zvuelta = 5;

```



```
znivelmenu = "3";  
%>
```

[shco_outputdef_prueba.jsp](#): ejemplo que consta de cuatro pasos, siendo el último una página de visualización de datos.

```
// *MODIFICABLE  
<%  
String znodo1 = "PRUEBA1";  
String znodo2 = "PRUEBA2";  
String znodo3 = "PRUEBA3";  
String znodo4 = "PRUEBA4";  
String zRecord4 = "0";  
  
String zregsmove1 = "0";  
String zregsmove2 = "0";  
String zregsmove3 = "0";  
String zregsmove4 = "0";  
switch(zLoadTypeStep)  
{  
case 1:  
zregsmove1 = String.valueOf(1);  
break;  
case 2:  
zregsmove2 = String.valueOf(1);  
break;  
case 3:  
zregsmove3 = String.valueOf(1);  
break;  
case 4:      zRecord4 = zregistroinicial + "-" + zregistrofinal;  
zregsmove4 = String.valueOf(zregistroinicial);  
break;  
}  
%>  
  
<m4:outputdef m4alias="<%=znodo1%>" m4object="<%=zm4object%>"  
node="<%=znodo1%>" records="*"/>  
  
<m4:outputdef m4alias="<%=znodo2%>" m4object="<%=zm4object%>"  
node="<%=znodo2%>" records="*"/>  
  
<m4:outputdef m4alias="<%=znodo3%>" m4object="<%=zm4object%>"  
node="<%=znodo3%>" records="*"/>  
  
<m4:outputdef m4alias="<%=znodo4%>" m4object="<%=zm4object%>"  
node="<%=znodo4%>" records="<%=zRecord4%>"/>
```

```

<m4:outputdef m4alias="<%=znodocom%>" m4object="<%=zm4object%>"
node="<%=znodocom%>" records="*" />
<m4:outputdef m4alias="<%=znodolabel%>" m4object="<%=zm4object%>"
node="<%=znodolabel%>" records="*" />
<m4:endjob/>
<%
try {
M4Operations t = new M4Operations(request);
    t.moveData (znodo1,zm4object,znodo1,zregsmove1);
    t.moveData (znodo2,zm4object,znodo2,zregsmove2);
    t.moveData (znodolabel,zm4object,znodolabel,"0");
}catch(Exception e) {}
try {
M4Operations t = new M4Operations(request);
    t.moveData (znodo3,zm4object,znodo3,zregsmove3);
}catch(Exception e) {}
try {
M4Operations t = new M4Operations(request);
    t.moveData (znodo4,zm4object,znodo4,zregsmove4);
}catch(Exception e) {}
%>

```